

File: action_config_placeholders.py**Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp**

```
from PyQt6.QtWidgets import QGroupBox, QVBoxLayout, QLabel
```

```
def _make_box(title: str, text: str) -> QGroupBox:
```

```
    box = QGroupBox(title)
    v = QVBoxLayout(box)
    lab = QLabel(text); lab.setWordWrap(True)
    v.addWidget(lab)
    v.addStretch()
    return box
```

```
# Place-holders – serão substituídos por UI completa depois
```

```
from barcode_config_widget import BarcodeConfigWidget
```

```
from inspection_config_widget import InspectionConfigWidget
```

```
from fiducial_config_widget import FiducialConfigWidget
```

File: inspection_engine.py**Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp**

```
# inspection_engine.py
```

```
"""
```

Módulo de inspeção visual – compara um ROI (posição mecânica) com suas imagens-referência e avalia as sub-janelas de inspeção.

Principais recursos

- Pipeline único de pré-processamento (blur, contraste, brilho, threshold, normalização) 100 % idempotente;
- Máscara opcional (x,y,w,h) normalizada à janela azul;
- Busca do melhor template via cv2.matchTemplate (TM_CCOEFF_NORMED) em versão downscaled para velocidade;
- Avaliação de múltiplas janelas de inspeção (percentual de pixels brancos ou pretos $\geq$ limiar do usuário).

Uso rápido

```
from inspection_engine import (
    MechanicalPosition, InspectionWindow, Inspector, default_preproc_cfg
)
```

```
# --- descreve a posição mecânica -----
```

```
pos = MechanicalPosition(
    roi=(100, 200, 450, 550),          # ROI ABSOLUTO (px)
    reference_dir="modelos/U1/referencia", # *.png da janela azul
    sim_threshold=0.975,                # similaridade mínima
    image_cfg=default_preproc_cfg(),    # pré-proc
    mask=dict(enabled=False, x=.4,y=.4,width=.2,height=.2) # opcional
)
```

```

# --- descreve janelas de inspeção dentro da ROI (normalizado) --
pos.windows = [
    InspectionWindow((.10,.12), (.30,.25), 128, 'white', 0.85),
    InspectionWindow((.60,.15), (.25,.30), 100, 'black', 0.90),
]

# --- inspeciona -----
insp = Inspector()
result = insp.inspect(cv2.imread("placa_atual.png"), pos)
print(result.ok, result.best_similarity, result.window_results)
"""

from __future__ import annotations

import os, cv2, numpy as np
from dataclasses import dataclass, field
from typing import List, Tuple, Dict, Any, Optional
from PIL import Image

# -----
# Estruturas de configuração pública
# -----
@dataclass
class InspectionWindow:
    """
    Define uma "janela vermelha" dentro da ROI:
    pos_norm – (x,y) posição normalizada (0-1) à ROI mecânica
    size_norm – (w,h) tamanho normalizado (0-1)
    threshold – valor de corte (0-255)
    pixel – 'white' ou 'black' → pixel alvo
    min_pct – percentual mínimo (0-1) para aprovar
    """
    pos_norm: Tuple[float, float]
    size_norm: Tuple[float, float]
    threshold: int = 128
    pixel: str = 'white'
    min_pct: float = .80
    enabled: bool = True

@dataclass
class MechanicalPosition:
    """
    Define a posição mecânica (janela amarela):
    roi – (x1,y1,x2,y2) absoluto na placa
    reference_dir – pasta *.png da janela azul
    sim_threshold – similaridade mínima (0-1) da janela azul
    image_cfg – dicionário de pré-proc (ver default_preproc_cfg)
    mask – dicionário opcional {enabled,x,y,width,height}
    """
    roi: Tuple[int,int,int,int]

```

```

reference_dir: str
sim_threshold: float = .975
image_cfg: Dict[str,Any] = field(default_factory=lambda: default_preproc_cfg())
mask: Dict[str,Any] = field(default_factory=dict)
windows: List[InspectionWindow] = field(default_factory=list)

```

@dataclass

class WindowResult:

```

    ok: bool
    pct: float

```

@dataclass

class InspectionResult:

```

    ok: bool
    best_similarity: float
    window_results: List[WindowResult]

```

Pré-processamento (adaptado de utils/image_processing.apply_preprocessing)

def default_preproc_cfg()->Dict[str,Any]:

```

    return dict(
        blur_enabled=False, blur=0,
        contrast_enabled=False, contrast=1.0,
        brightness_enabled=False, brightness=0,
        threshold_enabled=False, threshold=128,
        normalization_enabled=False
    )

```

def apply_preprocessing(bgr: np.ndarray, cfg: Dict[str,Any]) -> np.ndarray:

```

    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY) if bgr.ndim==3 else bgr.copy()
    gray = cv2.GaussianBlur(gray, (5,5), 0) # blur legado
    # blur custom
    if cfg.get("blur_enabled"):
        sigma = int(cfg.get("blur",0))
        if sigma>0:
            gray = cv2.GaussianBlur(gray,(5,5),sigma)
    # contraste / brilho
    if cfg.get("contrast_enabled") or cfg.get("brightness_enabled"):
        alpha = cfg.get("contrast",1.0) if cfg.get("contrast_enabled") else 1.0
        beta = cfg.get("brightness",0) if cfg.get("brightness_enabled") else 0
        gray = cv2.convertScaleAbs(gray, alpha=alpha, beta=beta)
    # threshold
    if cfg.get("threshold_enabled"):
        th = int(cfg.get("threshold",128))
        _, gray = cv2.threshold(gray, th, 255, cv2.THRESH_BINARY)
    # normalização
    if cfg.get("normalization_enabled"):
        gray = cv2.normalize(gray,None,0,255,cv2.NORM_MINMAX)
    return gray

```

```
def apply_mask(img: np.ndarray, mcfg: Dict[str,Any]) -> np.ndarray:
```

```
    if not mcfg or not mcfg.get("enabled", False):
```

```
        return img
```

```
    h,w = img.shape[:2]
```

```
    x1 = int(mcfg["x"]*w); y1 = int(mcfg["y"]*h)
```

```
    x2 = int((mcfg["x"]+mcfg["width"])*w)
```

```
    y2 = int((mcfg["y"]+mcfg["height"])*h)
```

```
    out = img.copy()
```

```
    out[y1:y2,x1:x2] = 0
```

```
    return out
```

```
# -----
```

```
# Núcleo de comparação (derivado de utils.image_comparator.ImageComparator)
```

```
# -----
```

```
class ROIComparator:
```

```
    def compare(self, roi_bgr: np.ndarray, ref_dir:str,
```

```
               cfg:Dict[str,Any]) -> float:
```

```
        """
```

```
        Percorre todos os *.png em ref_dir e devolve o maior score (0-1).
```

```
        """
```

```
    if not os.path.isdir(ref_dir):
```

```
        return 0.0
```

```
    proc_roi = apply_preprocessing(roi_bgr, cfg.get("image_cfg",{}))
```

```
    if cfg.get("mask",{}).get("enabled",False):
```

```
        proc_roi = apply_mask(proc_roi, cfg["mask"])
```

```
    ds = max(1,int(cfg.get("downscale",1)))
```

```
    small_roi = cv2.resize(proc_roi,(proc_roi.shape[1]//ds,  
                               proc_roi.shape[0]//ds)) if ds>1 else proc_roi
```

```
    best = 0.0
```

```
    for f in os.listdir(ref_dir):
```

```
        if not f.lower().endswith(".png"):
```

```
            continue
```

```
        tmpl = cv2.imread(os.path.join(ref_dir,f))
```

```
        if tmpl is None: continue
```

```
        if tmpl.ndim==3:
```

```
            tmpl = cv2.cvtColor(tmpl,cv2.COLOR_BGR2GRAY)
```

```
        tmpl = apply_preprocessing(tmpl, cfg.get("image_cfg",{}))
```

```
        if cfg.get("mask",{}).get("enabled",False):
```

```
            tmpl = apply_mask(tmpl, cfg["mask"])
```

```
        if ds>1:
```

```
            tmpl = cv2.resize(tmpl,(tmpl.shape[1]//ds, tmpl.shape[0]//ds))
```

```
        if (tmpl.shape[0]>small_roi.shape[0] or
```

```
            tmpl.shape[1]>small_roi.shape[1]):
```

```
            continue
```

```
        res = cv2.matchTemplate(small_roi, tmpl, cv2.TM_CCOEFF_NORMED)
```

```
        _, max_val, _, _ = cv2.minMaxLoc(res)
```

```
        best = max(best, float(max_val))
```

```
        if best>=cfg.get("sim_threshold",.975):
```

```
            break
```

```
    return best
```

```

# -----
# Lógica completa de inspeção
# -----
class Inspector:
    def __init__(self):
        self._cmp = ROIComparator()

    # ----- interface pública -----
    def inspect(self, img_bgr: np.ndarray|Image.Image,
                pos : MechanicalPosition) -> InspectionResult:
        if isinstance(img_bgr, Image.Image):
            img_bgr = cv2.cvtColor(np.array(img_bgr), cv2.COLOR_RGB2BGR)
        x1,y1,x2,y2 = map(int,pos.roi)
        roi_bgr = img_bgr[y1:y2, x1:x2]
        # 1) janela azul (similaridade global)
        sim = self._cmp.compare(roi_bgr, pos.reference_dir,
                                dict(image_cfg=pos.image_cfg,
                                     mask=pos.mask,
                                     sim_threshold=pos.sim_threshold))
        # 2) janelas vermelhas
        win_results : List[WindowResult]=[]
        roi_gray = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2GRAY)
        for w in pos.windows:
            if not w.enabled:
                win_results.append(WindowResult(True,0.0))
                continue
            wx,wy = w.pos_norm; ww,hh = w.size_norm
            rx1 = int(wx*roi_gray.shape[1]); ry1=int(wy*roi_gray.shape[0])
            rx2 = rx1+int(ww*roi_gray.shape[1]); ry2=ry1+int(hh*roi_gray.shape[0])
            sub = roi_gray[ry1:ry2, rx1:rx2]
            if sub.size==0:
                win_results.append(WindowResult(False,0.0)); continue
            if w.pixel.lower()=='white':
                pct = float((sub>=w.threshold).mean())
            else:
                pct = float((sub<w.threshold).mean())
            ok = pct >= w.min_pct
            win_results.append(WindowResult(ok,pct))
        overall_ok = (sim >= pos.sim_threshold and all(r.ok for r in win_results))
        return InspectionResult(overall_ok, sim, win_results)

```

File: fov_calibration_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

fov_calibration_dialog.py

Permite calibrar o campo-de-visão da câmera em dois pontos de Z
(ex.: Z=0 e Z=700). Os valores são gravados em settings.json.

"""

```
from PyQt6.QtWidgets import (
    QDialog, QGridLayout, QLabel, QSpinBox, QDoubleSpinBox,
    QPushButton, QHBoxLayout
)
from PyQt6.QtCore import Qt
```

```
class FOVCalibrationDialog(QDialog):
```

```
    def __init__(self, settings_mgr, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Calibração de Campo de Visão")
        self._s = settings_mgr
        self._orig = settings_mgr.camera_fov
        self._build_ui()
```

```
# ----- UI -----
```

```
def _build_ui(self):
```

```
    g = QGridLayout(self)
    # linha 0 – cabeçalhos
    g.addWidget(QLabel("Parâmetro"), 0, 0)
    g.addWidget(QLabel("Z-ref 0"), 0, 1)
    g.addWidget(QLabel("Z-ref 1"), 0, 2)
    # linha 1 – Z pulsos
    g.addWidget(QLabel("Z (pulsos):"), 1, 0)
    self.sp_z0 = QSpinBox(); self.sp_z0.setRange(-2_147_483_648, 2_147_483_647)
    self.sp_z1 = QSpinBox(); self.sp_z1.setRange(-2_147_483_648, 2_147_483_647)
    self.sp_z0.setValue(self._orig.get("z0_z_pulses", 0))
    self.sp_z1.setValue(self._orig.get("z1_z_pulses", 700))
    g.addWidget(self.sp_z0, 1, 1); g.addWidget(self.sp_z1, 1, 2)
    # linha 2 – largura
    g.addWidget(QLabel("Largura visível (mm):"), 2, 0)
    self.sp_w0 = QDoubleSpinBox(); self.sp_w0.setRange(0.01, 1000); self.sp_w0.setDecimals(2)
    self.sp_w1 = QDoubleSpinBox(); self.sp_w1.setRange(0.01, 1000); self.sp_w1.setDecimals(2)
    self.sp_w0.setValue(self._orig.get("z0_width_mm", 61.0))
    self.sp_w1.setValue(self._orig.get("z1_width_mm", 30.5))
    g.addWidget(self.sp_w0, 2, 1); g.addWidget(self.sp_w1, 2, 2)
    # linha 3 – altura
    g.addWidget(QLabel("Altura visível (mm):"), 3, 0)
    self.sp_h0 = QDoubleSpinBox(); self.sp_h0.setRange(0.01, 1000); self.sp_h0.setDecimals(2)
    self.sp_h1 = QDoubleSpinBox(); self.sp_h1.setRange(0.01, 1000); self.sp_h1.setDecimals(2)
    self.sp_h0.setValue(self._orig.get("z0_height_mm", 45.0))
    self.sp_h1.setValue(self._orig.get("z1_height_mm", 22.5))
    g.addWidget(self.sp_h0, 3, 1); g.addWidget(self.sp_h1, 3, 2)
    # botões
    h = QHBoxLayout()
    btn_ok = QPushButton("Salvar"); btn_ok.clicked.connect(self._save)
    btn_can = QPushButton("Cancelar"); btn_can.clicked.connect(self.reject)
    h.addStretch(); h.addWidget(btn_ok); h.addWidget(btn_can)
    g.addLayout(h, 4, 0, 1, 3)
```

```
# ----- salvar -----
```

```

def _save(self):
    fov = {
        "z0_z_pulses": self.sp_z0.value(),
        "z0_width_mm": self.sp_w0.value(),
        "z0_height_mm": self.sp_h0.value(),
        "z1_z_pulses": self.sp_z1.value(),
        "z1_width_mm": self.sp_w1.value(),
        "z1_height_mm": self.sp_h1.value()
    }
    self._s.camera_fov = fov
    self._s.save()
    self.accept()

```

File: virtual_home_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

"""

virtual_home_dialog.py

Diálogo para configurar o "homing virtual" (posição zero lógica)
de cada mesa. Salva em settings.json.

"""

```

from PyQt6.QtWidgets import (
    QDialog, QGridLayout, QLabel, QSpinBox, QPushButton, QHBoxLayout
)
from PyQt6.QtCore import Qt, pyqtSignal

```

```

class ConfigHomeDialog(QDialog):

```

"""

Janela NÃO modal (always-on-top) para configurar o zero virtual.

Sinal:

acceptedAndSaved() – emitido quando o utilizador clica OK.

"""

```

acceptedAndSaved = pyqtSignal()

```

```

def __init__(self, controller, parent=None):

```

```

    # NON-MODAL + sempre à frente

```

```

    super().__init__(parent, Qt.WindowType.Window)

```

```

    self.setWindowModality(Qt.WindowModality.NonModal)

```

```

    self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)

```

```

    self.setWindowTitle("Configuração de Homing Virtual")

```

```

    self._c = controller

```

```

    self._orig = controller.virtual_home

```

```

    self._build_ui()

```

```

# ----- UI -----

```

```

def _build_ui(self):

```

```

    g = QGridLayout(self)

```

```

    self._spins = {} # (mesa, coord) -> spin

```

```

    row = 0

```

```

    for mesa in (1, 2):

```

```

# título mesa
g.addWidget(QLabel(f"<b>Mesa {mesa}</b>"), row, 0, 1, 5)
row += 1
# linha X
g.addWidget(QLabel("X:"), row, 0)
spx = QSpinBox(); spx.setRange(-2_147_483_648, 2_147_483_647)
spx.setValue(self._orig[mesa]["x"])
g.addWidget(spx, row, 1)
# botões
btn_cap = QPushButton("Capturar posição atual")
btn_cap.clicked.connect(lambda _=False, m=mesa: self._capture(m))
btn_go = QPushButton("Ir para Home")
btn_go.clicked.connect(lambda _=False, m=mesa: self._goto(m))
g.addWidget(btn_cap, row, 2, 1, 2)
g.addWidget(btn_go, row, 4)
self._spins[(mesa, 'x')] = spx
row += 1
# linha Y
g.addWidget(QLabel("Y:"), row, 0)
spy = QSpinBox(); spy.setRange(-2_147_483_648, 2_147_483_647)
spy.setValue(self._orig[mesa]["y"])
g.addWidget(spy, row, 1)
self._spins[(mesa, 'y')] = spy
row += 1
# ----- OK / Cancel -----
h = QHBoxLayout()
ok = QPushButton("OK"); ok.clicked.connect(self._on_ok)
can = QPushButton("Cancelar"); can.clicked.connect(self.hide)
h.addStretch(); h.addWidget(ok); h.addWidget(can)
g.addLayout(h, row, 0, 1, 4)

# ----- handlers -----
def _on_ok(self):
    self.acceptedAndSaved.emit()
    self.hide()

# ----- helpers -----
def _capture(self, mesa:int):
    """Lê posição atual do controlador e copia para os campos."""
    x = self._c.current_positions['X']
    y = self._c.current_positions['Y1' if mesa == 1 else 'Y2']
    self._spins[(mesa, 'x')].setValue(x)
    self._spins[(mesa, 'y')].setValue(y)

def _goto(self, mesa:int):
    """Move para o ponto configurado nos spinBoxes atuais."""
    x = self._spins[(mesa, 'x')].value()
    y = self._spins[(mesa, 'y')].value()
    dx = x - self._c.current_positions['X']
    y_axis = 'Y1' if mesa == 1 else 'Y2'
    dy = y - self._c.current_positions[y_axis]

```

```

if dx: self._c.move_relative('X', dx)
if dy: self._c.move_relative('Y', dy)

```

```

# ----- saída -----
def get_home_dict(self) -> dict[int,dict[str,int]]:
    return {
        1: {"x": self._spins[(1,'x')].value(),
            "y": self._spins[(1,'y')].value()},
        2: {"x": self._spins[(2,'x')].value(),
            "y": self._spins[(2,'y')].value()}
    }

```

File: axis_calibration_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

"""

```

```

axis_calibration_dialog.py

```

```

-----

```

Diálogo modal que reutiliza CalibWidget (de cnc_calibration_widget.py)
e permite escolher qual eixo X/Y2/Y1/Z será calibrado.

```

"""

```

```

from PyQt6.QtWidgets import QDialog, QVBoxLayout, QComboBox, QLabel, QPushButton, \
    QHBoxLayout
from cnc_calibration_widget import CalibWidget

```

```

AXES = ("X", "Y2", "Y1", "Z")

```

```

class _BackendAdapter:

```

```

    """

```

Implementa CalibrationBackend e redireciona apply_steps_per_mm(...)
para o controlador principal, fixando o eixo escolhido.

```

    """

```

```

    def __init__(self, controller, axis: str):

```

```

        self._c = controller

```

```

        self._axis = axis

```

```

    # protocolo -----

```

```

    def apply_steps_per_mm(self, steps: float) -> bool:

```

```

        self._c._set_axis_steps(self._axis, steps)

```

```

        return True

```

```

class AxisCalibrationDialog(QDialog):

```

```

    def __init__(self, controller, parent=None):

```

```

        super().__init__(parent)

```

```

        self.setWindowTitle("Calibração de Eixos")

```

```

        self._c = controller

```

```

        self._build_ui()

```

```

    # ----- UI -----

```

```

    def _build_ui(self):

```

```

        v = QVBoxLayout(self)

```

```

# seleção de eixo
hsel = QHBoxLayout()
hsel.addWidget(QLabel("Eixo:"))
self.cmb_axis = QComboBox()
self.cmb_axis.addItem(Axes)
hsel.addWidget(self.cmb_axis); hsel.addStretch()
v.addLayout(hsel)

# widget de calibração (backend definido depois)
self.calib_widget = CalibWidget(None) # backend placeholder
v.addWidget(self.calib_widget)

# botões OK / Fechar
bok = QPushButton("Fechar")
bok.clicked.connect(self.accept)
h = QHBoxLayout(); h.addStretch(); h.addWidget(bok)
v.addLayout(h)

# sinais
self.cmb_axis.currentTextChanged.connect(self._on_axis_changed)
self._on_axis_changed(self.cmb_axis.currentText())

# ----- handlers -----
def _on_axis_changed(self, axis: str):
    # troca backend para refletir eixo actual
    self.calib_widget._backend = _BackendAdapter(self._c, axis)
    # mostra passos/mm atuais
    cur = self._c.steps_per_mm.get(axis, 1.0)
    self.calib_widget.lbl_result.setText(f"{cur:.3f} steps/mm (atual)")

```

File: cnc_calibration_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

cnc_calibration_widget.py

Widget genérico para calcular e aplicar a calibração de passos por milímetro (steps/mm) em sistemas CNC que usem motores de passo.

Fórmula adoptada

$$\text{steps_per_mm} = (\text{motor_full_steps_rev} \cdot \text{microsteps} \cdot \text{gear_ratio}) / \text{screw_pitch_mm}$$

onde

- motor_full_steps_rev – passos “inteiros” por volta (200 na maioria dos NEMA-17/23)
- microsteps – divisão electrónica do driver (1,2,4,8,16,32...)
- gear_ratio – relação de engrenagem/polia (se directo $\Rightarrow$ 1.0)
- screw_pitch_mm – avanço do fuso (mm por volta). Para correia dentada use pitch_mm = número_de_dentes_da_polia * passo_da_correa_mm.

O módulo expõe:

1. classe CalibrationCalculator (lógica pura → testável sem Qt)
2. protocolo CalibrationBackend (driver que receberá os novos steps/mm)
3. widget CalibWidget (interface PyQt 6)

Autor: 2025 – livre para uso sob MIT.

"""

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Protocol
```

----- 1) Lógica pura -----

@dataclass

class CalibrationParameters:

```
    motor_steps_rev: float = 200.0    # passos "cheios" por volta
    microsteps: float = 16.0         # divisão do driver
    gear_ratio: float = 1.0          # >1 se há redução; <1 se multiplicação
    screw_pitch_mm: float = 5.0      # avanço do fuso (mm)
```

class CalibrationCalculator:

"""Funções estáticas que podem ser unit-testadas sem Qt."""

@staticmethod

```
def steps_per_mm(p: CalibrationParameters) -> float:
    if p.screw_pitch_mm <= 0:
        raise ValueError("screw_pitch_mm deve ser > 0")
    return (p.motor_steps_rev * p.microsteps * p.gear_ratio) / p.screw_pitch_mm
```

----- 2) Backend para aplicação -----

class CalibrationBackend(Protocol):

"""

Qualquer classe fornecida à GUI deve implementar:

```
    apply_steps_per_mm(steps: float) -> bool
    (pode aplicar nos eixos X/Y/Z conforme necessidade interna)
```

"""

```
def apply_steps_per_mm(self, steps: float) -> bool: ...
```

----- 3) Interface PyQt 6 -----

from PyQt6.QtWidgets import (

```
    QWidget, QGroupBox, QFormLayout, QDoubleSpinBox, QHBoxLayout,
    QPushButton, QMessageBox, QLabel, QComboBox, QVBoxLayout
```

)

from PyQt6.QtCore import pyqtSignal

class CalibWidget(QWidget):

"""

Widget completo para o utilizador introduzir parâmetros,
ver o resultado em tempo-real e aplicar ao backend.

Sinais:

applied(float) – emitido quando o backend confirma aplicação

.....

```
applied = pyqtSignal(float)
```

```
def __init__(self,
              backend: CalibrationBackend | None = None,
              parent=None):
    super().__init__(parent)
    self._backend = backend
    self._build_ui()
```

```
# ----- pública -----
```

```
def parameters(self) -> CalibrationParameters:
    return CalibrationParameters(
        motor_steps_rev=self.spin_steps_rev.value(),
        microsteps=float(self.cmb_micro.currentText()),
        gear_ratio=self.spin_gear.value(),
        screw_pitch_mm=self.spin_pitch.value()
    )
```

```
# ----- UI -----
```

```
def _build_ui(self):
    box = QGroupBox("Calibração de Steps/mm")
    form = QFormLayout(box)

    # passos/volta – maioria é 200
    self.spin_steps_rev = QDoubleSpinBox()
    self.spin_steps_rev.setRange(1, 2000)
    self.spin_steps_rev.setValue(200)
    self.spin_steps_rev.setDecimals(0)
    form.addRow("Passos por volta (motor):", self.spin_steps_rev)
```

```
# micro-steps – drop-down comum
self.cmb_micro = QComboBox()
self.cmb_micro.addItems([str(v) for v in (1, 2, 4, 8, 16, 32, 64, 128, 256)])
self.cmb_micro.setCurrentText("16")
form.addRow("Microsteps do driver:", self.cmb_micro)
```

```
# relação de engrenagem
self.spin_gear = QDoubleSpinBox()
self.spin_gear.setRange(0.01, 100.0)
self.spin_gear.setSingleStep(0.01)
self.spin_gear.setValue(1.0)
form.addRow("Relação engrenagem (1=directo):", self.spin_gear)
```

```
# passo do fuso / avanço da correia
self.spin_pitch = QDoubleSpinBox()
self.spin_pitch.setRange(0.01, 100.0)
self.spin_pitch.setSingleStep(0.05)
self.spin_pitch.setValue(5.0)
form.addRow("Passo do fuso (mm/volta):", self.spin_pitch)
```

```

# resultado
self.lbl_result = QLabel("0.000 steps/mm")
form.addRow("Resultado:", self.lbl_result)

# botões
h = QHBoxLayout()
self.btn_apply = QPushButton("Aplicar ao Controlador")
h.addWidget(self.btn_apply)
form.addRow(h)

# layout externo
v = QVBoxLayout(self)
v.addWidget(box)
v.addStretch()

# conexões
for w in (self.spin_steps_rev, self.cmb_micro,
          self.spin_gear, self.spin_pitch):
    if hasattr(w, "valueChanged"):
        w.valueChanged.connect(self._update_result)
    else: # QComboBox
        w.currentTextChanged.connect(self._update_result)
self._update_result()
self.btn_apply.clicked.connect(self._on_apply_clicked)

# ----- slots -----
def _update_result(self):
    try:
        steps = CalibrationCalculator.steps_per_mm(self.parameters())
        self.lbl_result.setText(f"{steps:.3f} steps/mm")
    except Exception:
        self.lbl_result.setText("—")

def _on_apply_clicked(self):
    if self._backend is None:
        QMessageBox.warning(self, "Sem backend",
                            "Nenhum backend de CNC foi fornecido.")
        return
    steps = CalibrationCalculator.steps_per_mm(self.parameters())
    ok = False
    try:
        ok = self._backend.apply_steps_per_mm(steps)
    except Exception as exc:
        QMessageBox.critical(self, "Erro", f"Falha ao aplicar:\n{exc}")
        return
    if ok:
        QMessageBox.information(self, "Sucesso",
                                f"Parâmetro aplicado: {steps:.3f} steps/mm")
        self.applied.emit(steps)
    else:
        QMessageBox.critical(self, "Erro",

```

"Backend reportou falha ao aplicar o valor.")

```
# ----- Demo isolado -----
if __name__ == "__main__":
    from PyQt6.QtWidgets import QApplication
    import sys, logging
    logging.basicConfig(level=logging.INFO)

    # Backend fictício só para demonstração
    class _DummyBackend:
        def apply_steps_per_mm(self, steps: float) -> bool:
            print(f"[BACKEND] set $100=$101=$102 ← {steps:.3f}")
            return True

    app = QApplication(sys.argv)
    w = CalibWidget(_DummyBackend())
    w.setWindowTitle("Demo – Calibração CNC")
    w.resize(320, 220)
    w.show()
    sys.exit(app.exec())
```

File: movement_controls_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# movement_controls_widget.py
from PyQt6.QtCore import Qt, pyqtSignal
from PyQt6.QtGui import QFont, QDoubleValidator
from PyQt6.QtWidgets import (
    QWidget, QGroupBox, QGridLayout, QPushButton, QHBoxLayout,
    QLabel, QLineEdit, QVBoxLayout, QCheckBox, QMessageBox
)

class MovementControlsWidget(QWidget):
    """
    Painel com todos os controles de movimentação manual da CNC.
    A aplicação hospedeira conecta os sinais abaixo aos métodos do
    driver de movimento (GRBL, PLC, etc.).

    Sinais:
        stepMoveRequested(axis, distance, feed) – G90 (passo-a-passo)
        jogStart(axis, direction, feed) – G91 pressionado
        jogStop() – soltar botão em G91
        goToZeroRequested()
        emergencyStopToggled(engaged: bool) – STOP / RESET
    """

    stepMoveRequested = pyqtSignal(str, float, float)
    jogStart = pyqtSignal(str, int, float)
```

```

jogStop          = pyqtSignal()
goToZeroRequested = pyqtSignal()
emergencyStopToggled = pyqtSignal(bool)

def __init__(self, parent=None):
    super().__init__(parent)
    self._build_ui()

# -----
# interface pública
# -----
def feed_rate(self) -> float:
    """Retorna o feed-rate atual (mm/min)."""
    try:
        return float(self.ed_feed.text())
    except ValueError:
        return 1000.0

def step_size(self) -> float:
    """Retorna o step size atual (mm)."""
    try:
        return float(self.ed_step.text())
    except ValueError:
        return 1.0

# -----
# construção da UI
# -----
def _build_ui(self):
    outer = QVBoxLayout(self)

    grp = QGroupBox("Movement Controls")
    grid = QGridLayout(grp)
    self.grid = grid

    # ----- botões direcionais -----
    # Y2
    self.btn_up   = QPushButton("Y2+") # Y2 +
    self.btn_down = QPushButton("Y2-") # Y2 -
    # X
    self.btn_left = QPushButton("X-")  # X -
    self.btn_right = QPushButton("X+")  # X +
    # Y1
    self.btn_y1_up   = QPushButton("Y1+") # Y1 +
    self.btn_y1_down = QPushButton("Y1-") # Y1 -
    for b in (self.btn_up, self.btn_down, self.btn_left, self.btn_right, self.btn_y1_up, self.btn_y1_down):
        b.setMinimumSize(50, 50)
        f = QFont(); f.setPointSize(16); f.setBold(True); b.setFont(f)

    # LAYOUT
    # col0 col1 col2 col3 col4

```

```

# ■ Y2+ Y1+ ■ Z-
# X- STOP X+ (v) ( )
# ■ Y2- Y1- ■ Z+
grid.addWidget(self.btn_up, 0, 1) # Y2+
grid.addWidget(self.btn_y1_up, 0, 2) # Y1+

grid.addWidget(self.btn_left, 1, 0) # X-
grid.addWidget(self.btn_right, 1, 3) # X+

grid.addWidget(self.btn_down, 2, 1) # Y2-
grid.addWidget(self.btn_y1_down, 2, 2) # Y1-

# ----- Z +/- -----
# ----- Z +/- -----
# Origem em cima, positivo para baixo → Z- em cima, Z+ em baixo
self.btn_z_up = QPushButton("Z-") # sobe
self.btn_z_down = QPushButton("Z+") # desce
for b in (self.btn_z_up, self.btn_z_down):
    b.setMinimumSize(50, 50)
    f = QFont(); f.setPointSize(16); f.setBold(True); b.setFont(f)

grid.addWidget(self.btn_z_up, 0, 4) # Z- (coluna 4)
grid.addWidget(self.btn_z_down, 2, 4) # Z+ (coluna 4)

# ----- STOP / RESET -----
self.btn_stop = QPushButton("STOP")
self.btn_stop.setCheckable(True)
self.btn_stop.setMinimumSize(90, 40)
f = QFont(); f.setBold(True); self.btn_stop.setFont(f)
self._set_stop_style(False)
grid.addWidget(self.btn_stop, 1, 1, 1, 2, Qt.AlignmentFlag.AlignCenter)

# ----- step size & feed rate -----
self.ed_step = QLineEdit("10")
self.ed_feed = QLineEdit("1000")
self.ed_step.setValidator(QDoubleValidator(0.001, 100000, 4, self))
self.ed_feed.setValidator(QDoubleValidator(1, 30000, 0, self))

h_step = QHBoxLayout()
h_step.addWidget(QLabel("Step size:"))
h_step.addWidget(self.ed_step)
h_step.addWidget(QLabel("mm"))
grid.addLayout(h_step, 3, 0, 1, 3)

h_feed = QHBoxLayout()
h_feed.addWidget(QLabel("Feed rate:"))
h_feed.addWidget(self.ed_feed)
h_feed.addWidget(QLabel("mm/min"))
grid.addLayout(h_feed, 4, 0, 1, 3)

# ----- modos de distância -----

```

```

h_mode = QHBoxLayout()
self.btn_abs = QPushButton("(Passo)")
self.btn_rel = QPushButton("(Contínuo)")
self.btn_abs.setCheckable(True)
self.btn_rel.setCheckable(True)
self.btn_rel.setChecked(True)
h_mode.addWidget(self.btn_abs); h_mode.addWidget(self.btn_rel)
grid.addLayout(h_mode, 5, 0, 1, 3)

```

```

# ----- Go to Zero -----

```

```

self.btn_gotozero = QPushButton("Go to Zero")
self.btn_gotozero.setMinimumHeight(32)
fz = QFont(); fz.setBold(True)
self.btn_gotozero.setFont(fz)
grid.addWidget(self.btn_gotozero, 6, 0, 1, 3)

```

```

# -----

```

```

# NOVO: "Posição Atual" encaixado no espaço em branco (col-4)

```

```

# A grade fica: col-0..4 já usados; criamos col-5 só p/ leitura

```

```

# -----

```

```

self.lbl_pos_x = QLabel("0"); self.lbl_pos_y2 = QLabel("0")
self.lbl_pos_y1 = QLabel("0"); self.lbl_pos_z = QLabel("0")
for lab in (self.lbl_pos_x, self.lbl_pos_y2,
            self.lbl_pos_y1, self.lbl_pos_z):
    lab.setAlignment(Qt.AlignmentFlag.AlignRight)
    lab.setMinimumWidth(60)
    lab.setStyleSheet("QLabel { background:gray; padding:1px; }")

```

```

grid.setColumnStretch(5, 1)      # assegura 6.ª coluna

```

```

# grid.addWidget(QLabel("<b>Posição</b>"), 0, 5, Qt.AlignmentFlag.AlignCenter)
grid.addWidget(QLabel("X:"), 3, 3, Qt.AlignmentFlag.AlignRight)
grid.addWidget(self.lbl_pos_x, 3, 4, Qt.AlignmentFlag.AlignRight)
grid.addWidget(QLabel("Y2:"), 4, 3, Qt.AlignmentFlag.AlignRight)
grid.addWidget(self.lbl_pos_y2, 4, 4, Qt.AlignmentFlag.AlignRight)
grid.addWidget(QLabel("Y1:"), 5, 3, Qt.AlignmentFlag.AlignRight)
grid.addWidget(self.lbl_pos_y1, 5, 4, Qt.AlignmentFlag.AlignRight)
grid.addWidget(QLabel("Z:"), 6, 3, Qt.AlignmentFlag.AlignRight)
grid.addWidget(self.lbl_pos_z, 6, 4, Qt.AlignmentFlag.AlignRight)

```

```

# ----- finais -----

```

```

outer.addWidget(grp)
outer.addStretch()

```

```

# ligações
self._connect_signals()

```

```

# ----- API pública -----

```

```

def update_position(self, *, x: int, y2: int, y1: int, z: int):
    self.lbl_pos_x.setText(str(x))
    self.lbl_pos_y2.setText(str(y2))

```

```

self.lbl_pos_y1.setText(str(y1))
self.lbl_pos_z.setText(str(z))

# -----
# sinais internos → sinais públicos
# -----
def _connect_signals(self):

    # press / release – X e Y
    self.btn_up.pressed.connect( lambda: self._direction_press("Y2", +1))
    self.btn_down.pressed.connect( lambda: self._direction_press("Y2", -1))
    self.btn_left.pressed.connect( lambda: self._direction_press("X", -1))
    self.btn_right.pressed.connect(lambda: self._direction_press("X", +1))

    # Y1
    self.btn_y1_up.pressed.connect( lambda: self._direction_press("Y1", +1))
    self.btn_y1_down.pressed.connect(lambda: self._direction_press("Y1", -1))

    for b in (self.btn_up, self.btn_down, self.btn_left, self.btn_right, self.btn_y1_up, self.btn_y1_down):
        b.released.connect(self._direction_release)

    # press / release – Z (atenção: sentidos trocados)
    self.btn_z_up.pressed.connect( lambda: self._direction_press("Z", -1)) # Z-
    self.btn_z_down.pressed.connect(lambda: self._direction_press("Z", +1)) # Z+
    self.btn_z_up.released.connect(self._direction_release)
    self.btn_z_down.released.connect(self._direction_release)

    # STOP / RESET
    self.btn_stop.toggled.connect(self._on_stop_toggled)

    # Go to Zero
    self.btn_gotozero.clicked.connect(self.goToZeroRequested)

    # troca de modo
    self.btn_abs.clicked.connect(lambda: self._set_mode(True))
    self.btn_rel.clicked.connect(lambda: self._set_mode(False))

# -----
# handlers
# -----
def _direction_press(self, axis: str, direction: int):
    feed = self.feed_rate()
    if self.btn_abs.isChecked():      # G90 – passo / step
        distance = self.step_size() * direction
        self.stepMoveRequested.emit(axis, distance, feed)
    else:                            # G91 – jog contínuo
        self.jogStart.emit(axis, direction, feed)

def _direction_release(self):
    if self.btn_rel.isChecked():
        self.jogStop.emit()

```

```

def _on_stop_toggled(self, engaged: bool):
    self._set_stop_style(engaged)
    self.emergencyStopToggled.emit(engaged)

def _set_stop_style(self, engaged: bool):
    if engaged: # RESET
        self.btn_stop.setText("RESET")
        self.btn_stop.setStyleSheet("background: orange; color: black;")
    else: # STOP
        self.btn_stop.setText("STOP")
        self.btn_stop.setStyleSheet("background: red; color: white;")

def _set_mode(self, absolute: bool):
    self.btn_abs.setChecked(absolute)
    self.btn_rel.setChecked(not absolute)

# feedback optional (beep). Se desejar avisar o usuário
# QMessageBox.information(self, "Modo", "G90 selecionado" if absolute else "G91 selecionado")

```

```

# -----
# Demonstração isolada
# -----
if __name__ == "__main__":
    import sys
    from PyQt6.QtWidgets import QApplication

    app = QApplication(sys.argv)
    w = MovementControlsWidget()
    w.resize(350, 320)
    w.show()

    # Liga sinais a prints no console
    w.stepMoveRequested.connect(
        lambda ax, dist, f: print(f"[step] {ax} {dist}mm F{f}")
    )
    w.jogStart.connect(
        lambda ax, dir, f: print(f"[jog] {ax} dir={dir} F{f}")
    )
    w.jogStop.connect(lambda: print("[jog] stop"))
    w.emergencyStopToggled.connect(
        lambda engaged: print("STOP ->", "ENGAGED" if engaged else "released")
    )
    w.goToZeroRequested.connect(lambda: print("[cmd] G0 X0 Y0 Z0"))

    sys.exit(app.exec())

```

File: focus_calibration_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# focus_calibration_dialog.py
from PyQt6.QtWidgets import (QDialog, QVBoxLayout, QLabel, QSlider,
                             QPushButton, QHBoxLayout, QMessageBox,
                             QSpinBox)
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtGui import QImage, QPixmap, QPainter, QPen

class FocusCalibrationDialog(QDialog):
    """
    Passo-a-passo:
    1. mover Z ao topo, ajustar foco, 'Definir Foco Alto'
    2. mover Z ao fundo, ajustar foco, 'Definir Foco Baixo'
    """
    def __init__(self, ctrl, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Calibração de Foco (Z)")
        self.ctrl = ctrl      # MultiAxisMotorController
        self.a = 0.0; self.b = 0.0
        self._top_done = False
        self._build_ui()
        # ---- suspende auto-focus enquanto a janela estiver aberta ---
        self._prev_auto_state = ctrl.camera_manager.auto_enabled
        ctrl.camera_manager.enable_auto_focus(False)
        self._img = None

    def _build_ui(self):
        v = QVBoxLayout(self)

        # ----- foco manual -----
        self.slider = QSlider(Qt.Orientation.Horizontal)
        self.slider.setRange(0,1023)
        v.addWidget(self.slider)

        self.spin_focus = QSpinBox()
        self.spin_focus.setRange(0,1023)
        self.spin_focus.setButtonSymbols(QSpinBox.ButtonSymbols.NoButtons)
        self.spin_focus.setFixedWidth(70)

        h_focus = QHBoxLayout()
        h_focus.addWidget(QLabel("Foco:"))
        h_focus.addWidget(self.spin_focus)
        h_focus.addStretch()
        v.addLayout(h_focus)

        # ligações bidirecionais
        self.slider.valueChanged.connect(self.spin_focus.setValue)
```

```

self.spin_focus.valueChanged.connect(self.slider.setValue)

# aplica foco em cada mudança
self.slider.valueChanged.connect(self._apply_focus)
# garante aplicação depois de soltar o mouse
self.slider.sliderReleased.connect(lambda: self._apply_focus(self.slider.value()))
self.spin_focus.editingFinished.connect(
    lambda: self._apply_focus(self.spin_focus.value()))

# ----- posição Z atual -----
hZ = QHBoxLayout()
hZ.addWidget(QLabel("Z atual:"))
self.lbl_z = QLabel("0")
self.lbl_z.setMinimumWidth(80)
hZ.addWidget(self.lbl_z)
hZ.addStretch()
v.addLayout(hZ)

# timer interno p/ refrescar posição Z
self._timer = QTimer(self); self._timer.start(200)
self._timer.timeout.connect(
    lambda: self.lbl_z.setText(str(self.ctrl.current_positions['Z'])))

# ----- JOG Z (press / release) -----
hJog = QHBoxLayout()
btn_jog_up = QPushButton("JOG Z-")
btn_jog_down = QPushButton("JOG Z+")
btn_jog_up.pressed.connect( lambda: self.ctrl.jog_start('Z','-'))
btn_jog_up.released.connect( lambda: self.ctrl.jog_stop('Z'))
btn_jog_down.pressed.connect(lambda: self.ctrl.jog_start('Z','+'))
btn_jog_down.released.connect(lambda: self.ctrl.jog_stop('Z'))
hJog.addWidget(btn_jog_up); hJog.addWidget(btn_jog_down)
v.addLayout(hJog)

# ----- mover para valor absoluto -----
hAbs = QHBoxLayout()
self.spin_abs = QSpinBox()
self.spin_abs.setRange(-2_147_483_648, 2_147_483_647)
hAbs.addWidget(QLabel("Ir para Z ="))
hAbs.addWidget(self.spin_abs)
btn_go_abs = QPushButton("Mover")
btn_go_abs.clicked.connect(self._move_abs_z)
hAbs.addWidget(btn_go_abs)
v.addLayout(hAbs)

# ----- Go To LIMITS -----
hl = QHBoxLayout()
btn_top = QPushButton("Go to limit – (Z alto)")
btn_bot = QPushButton("Go to limit + (Z baixo)")
btn_top.clicked.connect(self._go_top)
btn_bot.clicked.connect(self._go_bottom)

```

```
hl.addWidget(btn_top); hl.addWidget(btn_bot)
v.addLayout(hl)
```

```
# define foco top / bottom
h2 = QHBoxLayout()
self.btn_top = QPushButton("Definir Foco Alto")
self.btn_bot = QPushButton("Definir Foco Baixo")
self.btn_bot.setEnabled(False)
self.btn_top.clicked.connect(self._set_top)
self.btn_bot.clicked.connect(self._set_bot)
h2.addWidget(self.btn_top); h2.addWidget(self.btn_bot)
v.addLayout(h2)
```

```
# OK/Cancelar
bok = QPushButton("Salvar"); bok.clicked.connect(self._save)
bcan= QPushButton("Cancelar"); bcan.clicked.connect(self.reject)
h3 = QHBoxLayout(); h3.addStretch(); h3.addWidget(bok); h3.addWidget(bcan)
v.addLayout(h3)
```

```
# ----- movimento Z até limites -----
```

```
def _go_top(self):
    z0 = self.ctrl.read_dword(self.ctrl.addresses['D23080_Z']) # limite-
    self.ctrl.pulsos_spin_Z.setValue(z0)
    self.ctrl.move_axis_absolute('Z')
```

```
def _go_bottom(self):
    zmax = self.ctrl.read_dword(self.ctrl.addresses['D23120']) # limite+
    if QMessageBox.question(self, "Atenção",
        "A cabeça irá DESCER até o limite POSITIVO de Z.\n"
        "Certifique-se de que não há obstáculos.\n\nContinuar?",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No) != QMessageBox.StandardButton.Yes:
        return
    self.ctrl.pulsos_spin_Z.setValue(zmax)
    self.ctrl.move_axis_absolute('Z')
```

```
def _set_top(self):
    z = self.ctrl.current_positions['Z']
    f = self.slider.value()
    self._z_top = z; self._f_top = f
    self._top_done = True
    self.btn_bot.setEnabled(True)
    QMessageBox.information(self, "Passo 1 concluído",
        "Agora baixe cuidadosamente o eixo Z até o limite inferior e ajuste o foco.\nDepois clique em 'Definir Foco Baixo'."
```

```
def _set_bot(self):
    z = self.ctrl.current_positions['Z']
    f = self.slider.value()
    if z == self._z_top:
        QMessageBox.warning(self, "Erro", "A posição Z deve ser diferente.")
    return
```

```

self._z_bot = z; self._f_bot = f
self.accept()          # encerra diálogo

def _save(self):
    if not self._top_done:
        QMessageBox.warning(self,"Incompleto","Defina os dois focos antes de salvar.")
    return

# ----- helper -----
def _apply_focus(self, val:int):
    self.ctrl.camera_manager.set_focus(val)

# ----- move para posição absoluta -----
def _move_abs_z(self):
    val = self.spin_abs.value()
    self.ctrl.pulsos_spin_Z.setValue(val)
    self.ctrl.move_axis_absolute('Z')

# -----
def closeEvent(self, ev):
    # restaura estado anterior do auto-focus
    self.ctrl.camera_manager.enable_auto_focus(self._prev_auto_state)
    super().closeEvent(ev)
    # encerra timer interno
    self._timer.stop()
    # garante que JOG foi desligado
    self.ctrl.jog_stop('Z')

```

File: position_status_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

# position_status_widget.py
from PyQt6.QtCore import pyqtSignal, Qt
from PyQt6.QtWidgets import (
    QWidget, QGroupBox, QLabel, QPushButton, QGridLayout, QVBoxLayout
)
from PyQt6.QtGui import QFont

class PositionStatusWidget(QWidget):
    """
    Widget que exibe a posição atual dos eixos, o estado da máquina
    e oferece botões para zerar X, Y, Z ou todos os eixos de uma vez.
    """

    # Sinais ■ a aplicação hospedeira deve conectá-los ao driver CNC
    zeroAllRequested = pyqtSignal()
    zeroXRequested  = pyqtSignal()
    zeroY2Requested = pyqtSignal()

```

```

zeroY1Requested = pyqtSignal()
zeroZRequested = pyqtSignal()

def __init__(self, parent=None):
    super().__init__(parent)
    self._build_ui()

# -----
# API pública para a aplicação hospedeira atualizar dados do widget
# -----
def update_position(self, x=0, y2=0, y1=0, z=0):
    """Atualiza as labels de posição (valores em pulsos)."""
    self.lbl_x.setText(str(x))
    self.lbl_y2.setText(str(y2))
    self.lbl_y1.setText(str(y1))
    self.lbl_z.setText(str(z))

def update_status(self, status: str):
    """Atualiza o label de status da máquina (Idle, Run, Jog...)."""
    self.lbl_status.setText(status)

# -----
# Construção da interface
# -----
def _build_ui(self):
    outer = QVBoxLayout(self)

    grp = QGroupBox("Posição Atual (pulsos)")
    grid = QGridLayout(grp)

    font_values = QFont()
    font_values.setBold(True)

    # ---- eixo X -----
    grid.addWidget(QLabel("X:"), 0, 0)
    self.lbl_x = QLabel("0")
    self.lbl_x.setFont(font_values)
    grid.addWidget(self.lbl_x, 0, 1)

    # ---- eixo Y2 -----
    grid.addWidget(QLabel("Y2:"), 1, 0)
    self.lbl_y2 = QLabel("0")
    self.lbl_y2.setFont(font_values)
    grid.addWidget(self.lbl_y2, 1, 1)

    # ---- eixo Y1 -----
    grid.addWidget(QLabel("Y1:"), 2, 0)
    self.lbl_y1 = QLabel("0")
    self.lbl_y1.setFont(font_values)
    grid.addWidget(self.lbl_y1, 2, 1)

```

```

# ---- eixo Z -----
grid.addWidget(QLabel("Z:"), 3, 0)
self.lbl_z = QLabel("0")
self.lbl_z.setFont(font_values)
grid.addWidget(self.lbl_z, 3, 1)

# ---- status -----
grid.addWidget(QLabel("Status:"), 4, 0)
self.lbl_status = QLabel("Desconectado")
grid.addWidget(self.lbl_status, 4, 1)

outer.addWidget(grp)
outer.addStretch()

# -----
# Demo rápido: executa o widget numa janela isolada
# -----
if __name__ == "__main__":
    from PyQt6.QtWidgets import QApplication
    import sys, random, time

    app = QApplication(sys.argv)
    w = PositionStatusWidget()
    w.resize(250, 200)
    w.show()

    # Simula atualização de posição/status a cada 500 ms
    from PyQt6.QtCore import QTimer
    def _simulate():
        w.update_position(
            x=random.uniform(0, 100),
            y=random.uniform(0, 100),
            z=random.uniform(0, 50)
        )
        w.update_status(random.choice(["Idle", "Run", "Jog"]))
    QTimer.singleShot(0, _simulate)
    timer = QTimer()
    timer.timeout.connect(_simulate)
    timer.start(500)

    sys.exit(app.exec())

```

File: config_dialogs.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

# config_dialogs.py
from PyQt6.QtWidgets import (
    QDialog, QGridLayout, QLabel, QSpinBox, QPushButton, QHBoxLayout
)

```

```

class WorkAreaConfigDialog(QDialog):
    """
    Diálogo modal para ajustar os limites de X e Y de cada mesa.
    Só permite valores positivos em pulsos.
    """

    def __init__(self, limits: dict[int, dict[str, tuple[int, int]]], parent=None):
        super().__init__(parent)
        self.setWindowTitle("Limites de Mesa")
        self._orig_limits = limits
        self._build_ui()

    # ----- UI -----
    def _build_ui(self):
        g = QGridLayout(self)

        row = 0
        self._spins = {}    # {(mesa, eixo, kind): QSpinBox}
        for mesa in (1, 2):
            g.addWidget(QLabel(f"Mesa {mesa}"), row, 0, 1, 3)
            row += 1
            for eixo in ('x', 'y'):
                lim_min, lim_max = self._orig_limits[mesa][eixo]

                g.addWidget(QLabel(f"{eixo.upper()} mín:"), row, 0)
                sp_min = QSpinBox(); sp_min.setRange(0, 2_147_483_647)
                sp_min.setValue(lim_min)
                g.addWidget(sp_min, row, 1)

                g.addWidget(QLabel("máx:"), row, 2)
                sp_max = QSpinBox(); sp_max.setRange(1, 2_147_483_647)
                sp_max.setValue(lim_max)
                g.addWidget(sp_max, row, 3)

                self._spins[(mesa, eixo, 'min')] = sp_min
                self._spins[(mesa, eixo, 'max')] = sp_max
                row += 1

        # botão OK / Cancelar
        h = QHBoxLayout()
        ok = QPushButton("OK"); ok.clicked.connect(self.accept)
        can = QPushButton("Cancelar"); can.clicked.connect(self.reject)
        h.addStretch(); h.addWidget(ok); h.addWidget(can)
        g.addLayout(h, row, 0, 1, 4)

    # ----- API -----
    def get_limits(self) -> dict[int, dict[str, tuple[int, int]]]:
        """Devolve novo dicionário igual ao recebido mas com máx alterado."""
        new = {1: {}, 2: {}}
        for mesa in (1, 2):

```

```

for eixo in ('x', 'y'):
    vmin = self._spins[(mesa, eixo, 'min')].value()
    vmax = self._spins[(mesa, eixo, 'max')].value()
    if vmax <= vmin:      # correção simples
        vmax = vmin + 1
    new[mesa][eixo] = (vmin, vmax)
return new

```

File: positions_backend.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

from __future__ import annotations
import json
from pathlib import Path
from typing import List

```

```

from program_io_widget import ProgramStorageBackend    # protocolo
from inspection_positions_widget import (
    InspectionPositionsWidget, InspectionPosition
)

```

```

class InspectionPositionsBackend(ProgramStorageBackend):

```

```

    """

```

Backend que salva/carrega a lista de InspectionPosition em JSON.

Estrutura gravada:

```

[
    {"name": "1", "x": 123, "y2": 456, "y1": 777, "z": 0},
    ...
]
"""

```

```

def __init__(self, widget: InspectionPositionsWidget):
    self._widget = widget

```

----- ProgramStorageBackend -----

```

def save_to_file(self, filename: str) -> bool:

```

```

    data = [
        {
            "name": p.name,
            "x": p.x,
            "y2": p.y2,
            "y1": p.y1,
            "z": p.z,
            "meta": p.meta or {}
        }
        for p in self._widget.positions()
    ]
    Path(filename).write_text(json.dumps(data, indent=2),
                              encoding="utf-8")

```

```
return True
```

```
def load_from_file(self, filename: str) -> bool:
    raw = json.loads(Path(filename).read_text(encoding="utf-8"))
    if not isinstance(raw, list):
        raise ValueError("formato inválido – era esperado uma lista JSON")

    self._widget.clear()
    for d in raw:
        pos = InspectionPosition(
            name=str(d.get("name", "")),
            x=float(d.get("x", 0)),
            y2=float(d.get("y2", 0)),
            y1=float(d.get("y1", 0)),
            z=float(d.get("z", 0)),
            meta=d.get("meta")
        )
        self._widget.add_position(pos)
    return True
```

File: plate_flow.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# plate_flow.py
from __future__ import annotations
import time
from enum import Enum, auto
from PyQt6.QtCore import QObject, QTimer
import datetime

class FlowState(Enum):
    IDLE      = auto()
    QUEUED    = auto()
    MOVING_IN = auto()
    PROCESSING = auto()
    MOVING_OUT = auto()

class PlateFlowManager(QObject):
    """
    Gerencia o fluxo de uma ÚNICA mesa:
    – monitora M20/M30 (botão do operador);
    – move a placa até o 1º fiducial;
    – dispara o SequenceControlWidget da aba;
    – ao terminar, devolve a placa ao operador;
    – sinaliza “done” em M21/M31 (pulso de 100 ms).
    """
    _POLL_MS = 100

    def __init__(self, ctrl, mesa_id: int, mesa_tab):
```

```
"""
```

```
ctrl    → MultiAxisMotorController
```

```
mesa_id → 1 ou 2
```

```
mesa_tab → TableProgramTab correspondente
```

```
"""
```

```
super().__init__(mesa_tab)
```

```
assert mesa_id in (1, 2)
```

```
self._c    = ctrl
```

```
# para registrar o último código de barras lido
```

```
self._last_barcode: str = ""
```

```
# para medir duração do ciclo desta mesa
```

```
self._start_time: float | None = None
```

```
self._tab    = mesa_tab
```

```
self._mesa   = mesa_id
```

```
self._state  = FlowState.IDLE
```

```
# True → berço já está sobre o 1.º fiducial e aguarda a cabeça
```

```
self._queued_ready = False
```

```
if mesa_id == 1:
```

```
    self._mem_btn = 'M20'
```

```
    self._mem_done = 'M21'
```

```
    self._y_axis   = 'Y1'
```

```
    self._y_limit  = ctrl.table_limits[1]['y'][1]
```

```
else:
```

```
    self._mem_btn = 'M30'
```

```
    self._mem_done = 'M31'
```

```
    self._y_axis   = 'Y2'
```

```
    self._y_limit  = ctrl.table_limits[2]['y'][1]
```

```
self._addr_btn = ctrl.addresses[self._mem_btn]
```

```
self._addr_done = ctrl.addresses[self._mem_done]
```

```
# estado anterior do botão físico (para borda ↑)
```

```
self._btn_prev = False
```

```
# pooler – evita mexer no thread de GUI da aplicação
```

```
self._timer = QTimer(self)
```

```
self._timer.timeout.connect(self._poll)
```

```
self._timer.start(self._POLL_MS)
```

```
# callback quando a sequência acaba
```

```
self._tab.seq_widget.sequenceFinished.connect(self._on_sequence_finished)
```

```
self._tab.seq_widget.sequenceError.connect(self._on_sequence_error)
```

```
# captura o último código de barras válido
```

```
self._tab.seq_widget.bcMatch.connect(self._record_last_barcode)
```

```
def _record_last_barcode(self, ok: bool, x:int, y:int, w:int, h:int, code:str):
```

```
    """
```

```
    Slot para armazenar o último código de barras lido
```

```
    durante a execução da sequência.
```

```
    """
```

```

if ok and code:
    self._last_barcode = code

# -----
# P U B L I C
# -----
def poll_now(self):      # pode ser chamado em testes
    self._poll()

# -----
# I N T E R N O S
# -----
def _poll(self):
    # -----
    # 1) Mesa "na fila" e já posicionada → inicia sequência assim
    #    que a cabeça ficar livre.
    # -----
    if (self._state is FlowState.QUEUED
        and self._queued_ready
        and self._c._head_busy is None):
        # ocupa mutex com o NÚMERO da mesa que assume a cabeça
        self._c._head_busy = self._mesa
        self._queued_ready = False
        self._start_sequence()

    # -----
    # 2) Condições normais de polling
    # -----
    if (not self._c.connected or
        self._state not in (FlowState.IDLE, FlowState.QUEUED) or
        not self._c._global_cycle_active):    # só depois do Start Geral
        return
    try:
        r = self._c.client.read_coils(self._addr_btn, count=1)
        if r.isError():
            return
        pressed = r.bits[0]
        if pressed and not self._btn_prev:      # Borda de subida
            self._start_send_cycle()
            self._btn_prev = pressed
    except Exception as exc:
        self._c.log(f"■ PlateFlow polling erro: {exc}")

# ----- S T A T E   M A C H I N E -----
def _start_send_cycle(self):
    """"Etapa 1 – leva a placa para dentro.""""

    # -----
    # LIMPA _targets de ciclos anteriores para que o próximo
    # wait_for_idle() considere apenas o movimento atual.
    # -----

```

```

# Removed manual clear of _targets here; move_to_absolute_position() now handles it
# self._c._plc_motion_backend._targets.clear()
fid_y = self._first_fiducial_y()
if fid_y is None:
    # sem fiducial → usa primeiro ponto ou mantém posição actual
    fid_y = self._first_point_y() or 0
    self._c.log(f"■ Mesa {self._mesa}: sem fiducial; usando Y={fid_y}")
self._state = FlowState.MOVING_IN
self._c.log(f"■ Mesa {self._mesa}: enviando placa (destino Y={fid_y})")
# dispara o movimento via PLCMotionBackend para manter o _targets correto
plc = self._c._plc_motion_backend
plc.set_offset_mode(False)      # sem offset extra
plc._targets.clear()           # limpa alvos antigos
# move apenas o eixo Y1 ou Y2 (passa None nos outros eixos)
ok = plc.move_to_absolute_position(
    x=None,
    y2=(fid_y if self._y_axis == "Y2" else None),
    y1=(fid_y if self._y_axis == "Y1" else None),
    z=None,
    feed_rate=None
)
if not ok:
    self._error("Falha ao enviar movimento para placa")
    return
if not plc.wait_for_idle():
    self._error("Timeout movendo placa para dentro")
    return

# -----
# Cabeça ocupada? → entra na fila já sobre o fiducial.
# -----
if self._c._head_busy is not None:
    self._state = FlowState.QUEUED
    self._queued_ready = True
    self._c.log(f"■ Mesa {self._mesa}: aguardando cabeça ficar livre...")
    return

# cabeça livre – ocupa mutex e começa a sequência
self._queued_ready = False
self._c._head_busy = self._mesa
self._start_sequence()

def _start_sequence(self):
    """Etapa 2 – roda SequenceControlWidget em modo APPLY."""
    self._state = FlowState.PROCESSING
    # marca início do ciclo

    self._start_time = time.time()
    # -----
    # 01-Ago-2025
    # Se o ciclo foi disparado pelo botão "Start Geral" (aba Controle

```

```

# de Eixos) usamos a opção VIEW/APPLY escolhida naquela aba e
# IGNORAMOS o que estiver marcado nas abas individuais.
#
# – execução global (PlateFlow) → herda da aba principal
# – execução local (botão “Executar Sequência” dentro da
# Mesa 1/Mesa 2) continua usando o próprio selector da mesa
# -----
global_selector = getattr(self._c.control_tab, "seq_widget", None)
if global_selector:
    # garante existência
    apply_global = global_selector.radio_apply.isChecked()
    # ■ força o mesmo estado no SequenceControl da mesa
    if apply_global:
        self._tab.seq_widget.radio_apply.setChecked(True)
    else:
        self._tab.seq_widget.radio_view.setChecked(True)
# Caso a execução tenha sido iniciada pela própria aba da mesa,
# o bloco acima não altera nada (o selector global não é usado).

# -----
# FIX – informa ao controlador qual eixo Y físico
# está vinculado à sequência que será iniciada.
# SequenceRunnerThread usará esse atributo em vez de
# depender da aba currently selected.
# -----
self._c._active_plate_y = self._y_axis # 'Y1' ou 'Y2'
self._tab.seq_widget._start()

def _on_sequence_finished(self):
    if self._state != FlowState.PROCESSING:
        return
    # Etapa 3 – devolve a placa usando o PLCMotionBackend (para que wait_for_idle
    # só retorne True após a mesa realmente atingir o limite)
    self._state = FlowState.MOVING_OUT
    limit = int(self._y_limit)
    self._c.log(f"■ Mesa {self._mesa}: devolvendo placa (Y→{limit}) via PLC backend")
    plc = self._c._plc_motion_backend
    # zera qualquer offset e alvos pendentes
    plc.apply_dynamic_offset(0, 0)
    plc._targets.clear()
    plc.set_offset_mode(False)
    # envia movimento de retorno apenas no eixo Y físico
    plc.move_to_absolute_position(
        x=None,
        y2=(limit if self._y_axis == "Y2" else None),
        y1=(limit if self._y_axis == "Y1" else None),
        z=None
    )
    if plc.wait_for_idle():
        # ■■ envia pulso DONE (M21 ou M31) somente após retorno completo ■■
        try:
            self._c.client.write_coil(self._addr_done, True)

```

```

        QTimer.singleShot(100,
            lambda a=self._addr_done: self._c.client.write_coil(a, False))
except Exception as exc:
    self._log(f"■ Erro ao pulsar {self._mem_done}: {exc}")
    self._finish_cycle()
else:
    self._error("Timeout devolvendo placa")

```

```

def _on_sequence_error(self, msg):
    if self._state == FlowState.PROCESSING:
        # sinaliza erro mas devolve placa ao operador
        self._error(f"Seq. erro: {msg}")
        self._return_after_error()

```

```

def _finish_cycle(self):
    """Etapa 4 – emite pulso DONE e regressa ao estado IDLE."""
    self._state = FlowState.IDLE
    self._queued_ready = False
    # libera a cabeça para outra mesa
    # libera a cabeça somente se ainda estivermos na posse dela
    if self._c._head_busy == self._mesa:
        self._c._head_busy = None
    try:
        # Zera offset dinâmico e targets (sem emitir pulso aqui)
        self._c._plc_motion_backend.apply_dynamic_offset(0, 0)
        self._c._plc_motion_backend._targets.clear()
    except Exception as exc:
        self._log(f"■ Erro ao limpar offset/targets: {exc}")
    # notifica a aba de controle o tempo decorrido
    try:
        import time
        if self._start_time is not None:
            elapsed = time.time() - self._start_time
            ctrl_tab = getattr(self._c, "control_tab", None)
            if ctrl_tab and hasattr(ctrl_tab, "_on_plate_time"):
                ctrl_tab._on_plate_time(self._mesa, elapsed)
    except Exception:
        self._log(f"■ Erro ao calcular tempo de ciclo da mesa")
    self._log(f"■ Mesa {self._mesa}: ciclo concluído ✓")
    # ——— Envia JSON simplificado ao SFCS (APPLY) respeitando Config. de Processo ———
    try:
        cfg = self._c.settings.get_config("process_config", {})
        # se Comunicação SFCS estiver desligada, pula o envio
        if not cfg.get("sfcs", True):
            self._log(f"■ Envio SFCS desativado pela Config. de Processo")
        else:
            ctrl_tab = getattr(self._c, 'control_tab', None)
            if ctrl_tab and ctrl_tab.seq_widget.radio_apply.isChecked():
                ts = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
                payload = {
                    "posto": self._c.inspection_logger.json_posto,

```

```

        "usuario":          self._c.inspection_logger.json_usuario,
        "data_hora":        ts,
        "quantidade_componentes": 1,
        "codigo_barras":    getattr(self, "_last_barcode", ""),
        "componentes":      {"aplicacao": 1}
    }
    if self._c.inspection_logger.send_sfcs_data_from_dict(payload):
        self._c.log(f"■ SFCS data enviado: {payload}")
    else:
        self._c.log("■ Falha ao enviar SFCS data (retorno False)")
except Exception as e:
    self._c.log(f"■ Exceção no envio de dados SFCS: {e}")

```

```
def _return_after_error(self):
```

```
    """
```

Falhou durante PROCESSING: devolve a placa e emite DONE
para que o ladder reset M20 / M30.

```
    """
```

1) Garante offset zerado

```
self._c._plc_motion_backend.apply_dynamic_offset(0, 0)
```

```
self._c._plc_motion_backend._targets.clear()
```

2) Eleva o eixo Z à origem (0) antes de devolver a placa, evitando colisões

```
self._c.log(f"■ Mesa {self._mesa}: elevando eixo Z antes de devolução de emergência")
```

ajusta spin para Z=0 e dispara o movimento

```
getattr(self._c, 'pulsos_spin_Z').setValue(0)
```

```
self._c.move_axis_absolute('Z')
```

```
self._c._plc_motion_backend.wait_for_idle()
```

3) agora devolve a placa no eixo Y

```
limit = int(self._y_limit)
```

```
getattr(self._c, f'pulsos_spin_{self._y_axis}').setValue(limit)
```

```
self._c.move_axis_absolute(self._y_axis)
```

```
self._c._plc_motion_backend.wait_for_idle()
```

pulso de conclusão mesmo em erro, no momento em que a mesa chega ao limite

try:

```
    self._c.client.write_coil(self._addr_done, True)
```

```
    QTimer.singleShot(100,
```

```
        lambda a=self._addr_done: self._c.client.write_coil(a, False))
```

except Exception as exc:

```
    self._c.log(f"■ Erro ao pulsar {self._mem_done} após erro: {exc}")
```

Mesmo em caso de erro devolvemos a placa e RESETAMOS o offset
dinâmico para garantir que o próximo ciclo comece "limpo".

```
self._c._plc_motion_backend.apply_dynamic_offset(0, 0)
```

limpa metas anteriores

```
self._c._plc_motion_backend._targets.clear()
```

```
self._state = FlowState.IDLE
```

```
self._queued_ready = False
```

```
if self._c._head_busy == self._mesa:
```

```

self._c._head_busy = None

# -----
def _first_fiducial_y(self):
    """Procura o 1º ponto fiducial e devolve coordenada Y em pulsos."""
    for p in self._tab.inspect_widget.positions():
        if (p.meta or {}).get('action') == 'fiducial':
            return p.y1 if self._mesa == 1 else p.y2
    return None

def _first_point_y(self):
    """Retorna Y do primeiro ponto do programa (qualquer ação)."""
    try:
        p0 = self._tab.inspect_widget.positions()[0]
        return p0.y1 if self._mesa == 1 else p0.y2
    except IndexError:
        self._c.log(f"■ Mesa {self._mesa}: sem pontos no programa")
        return None

def _error(self, msg):
    self._c.log(f"■ Mesa {self._mesa}: {msg}")
    self._state = FlowState.IDLE
    self._queued_ready = False
    # libera somente se continuamos "donos" da cabeça
    if self._c._head_busy == self._mesa:
        self._c._head_busy = None

```

File: axes_control_tab.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# axes_control_tab.py
```

```
"""
```

Widget completo de controle de eixos (quatro eixos + inspeção + salvar / carregar + sequência). Reaproveita toda a lógica já existente no MultiAxisMotorController; somente a parte visual fica aqui.

```
"""
```

```

from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QGridLayout, QFrame, QSizePolicy,
    QListWidget, QLabel, QGroupBox, QProgressBar,
    QPushButton, QTabWidget
)
from mesa_plot_widget import MesaPlotWidget
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtGui import QPixmap

from inspection_config_widget import AspectRatioLabel
from movement_controls_widget import MovementControlsWidget
from plate_flow import FlowState
from position_status_widget import PositionStatusWidget

```

```

from inspection_positions_widget import InspectionPositionsWidget
from program_io_widget import ProgramIOWidget
from positions_backend import InspectionPositionsBackend
from sequence_control import SequenceControlWidget
from sequence_control import InspectionPosition

```

```

class AxesControlTab(QWidget):

```

```

    """

```

Aba completa de controle. Recebe a instância do controlador principal (MultiAxisMotorController) para encaminhar sinais / slots.

Se `primary=True`, alguns atributos comuns (pos_widget, mov_widget ...) são expostos no controlador para que o código já existente continue funcionando sem alteração.

```

    """

```

```

def __init__(self, controller, *, primary: bool = False):
    super().__init__()
    self.ctrl = controller
    # initialize per-mesa production counters and timers
    self._count_m1 = 0
    self._count_m2 = 0
    self._times_m1: list[float] = []
    self._times_m2: list[float] = []
    self.primary = primary
    self._build_ui()

```

```

# -----

```

```

# CONSTRUÇÃO

```

```

# -----

```

```

def _build_ui(self):
    """Copia exatamente a UI usada anteriormente."""
    grid = QGridLayout(self)
    grid.setContentsMargins(0, 0, 0, 0)
    grid.setHorizontalSpacing(0)
    grid.setVerticalSpacing(0)

```

```

# ----- (COLUNA ESQUERDA REMOVIDA) -----

```

```

# Os controles auxiliares ficam apenas nas abas de Mesa 1/2.

```

```

# ----- COLUNA CENTRAL (85 %) -----

```

```

# ----- VISUALIZAÇÃO DAS MESAS -----

```

```

center_top = QFrame(objectName="centerTop")

```

```

grid.addWidget(center_top, 0, 0)

```

```

hplot = QHBoxLayout(center_top)

```

```

hplot.setContentsMargins(2,2,2,2)

```

```

hplot.setSpacing(4)

```

```

# Para cada "mesa" teremos agora um container com 2 colunas:

```

```

# • Coluna 0 (stretch=3): plotagem de pontos

```

```

# • Coluna 1 (stretch=2): área livre para novas funcionalidades

```

```

# Passa calibração steps/mm → evita "esticamento" no eixo X

```

```

self.plot_m1 = MesaPlotWidget(

```

[illegible]

```
)
group_m1_actions.setMaximumHeight(group_m1_actions.sizeHint().height())
# conecta botão "Abrir Projeto" de Mesa 1
btn_abrirproj_m1.clicked.connect(lambda: self._open_project_for_mesa(1))
# ■■■ GroupBox para exibir contagem e tempos (Mesa 1) ■■■
group_m1_stats = QGroupBox("Estatísticas de Produção")
stats_m1_layout = QHBoxLayout(group_m1_stats)
# – coluna esquerda: contador de placas
vcount_m1 = QVBoxLayout()
vcount_m1.setAlignment(Qt.AlignmentFlag.AlignCenter)
vcount_m1.addWidget(QLabel("Placas Concluídas:"), alignment=Qt.AlignmentFlag.AlignCenter)
self.lbl_count_m1 = QLabel("0")
self.lbl_count_m1.setAlignment(Qt.AlignmentFlag.AlignCenter)
vcount_m1.addWidget(self.lbl_count_m1)
# – coluna direita: tempos
vtime_m1 = QVBoxLayout()
vtime_m1.setAlignment(Qt.AlignmentFlag.AlignCenter)
vtime_m1.addWidget(QLabel("Tempo Total (s):"), alignment=Qt.AlignmentFlag.AlignCenter)
self.lbl_time_elapsed_m1 = QLabel("0 s")
self.lbl_time_elapsed_m1.setAlignment(Qt.AlignmentFlag.AlignCenter)
vtime_m1.addWidget(self.lbl_time_elapsed_m1)
vtime_m1.addWidget(QLabel("Tempo Médio (s):"), alignment=Qt.AlignmentFlag.AlignCenter)
self.lbl_time_avg_m1 = QLabel("0 s")
self.lbl_time_avg_m1.setAlignment(Qt.AlignmentFlag.AlignCenter)
vtime_m1.addWidget(self.lbl_time_avg_m1)
# adiciona as duas colunas ao layout principal
stats_m1_layout.addLayout(vcount_m1)
stats_m1_layout.addLayout(vtime_m1)
m1_btn_layout.addWidget(group_m1_stats)
m1_btn_layout.addStretch()
# proporções internas (3:2)
m1_layout.setStretch(0, 3)
m1_layout.setStretch(1, 2)
hplot.addWidget(m1_container)

# — Mesa 2
m2_container = QWidget()
# coluna de plotagem
m2_layout = QHBoxLayout(m2_container)
m2_layout.setContentsMargins(0,0,0,0)
m2_layout.setSpacing(2)
# coluna de plotagem
m2_layout.addWidget(self.plot_m2)
# coluna livre – use palette em vez de cor fixa
m2_blank = QFrame()
m2_blank.setAutoFillBackground(True)
m2_blank.setPalette(self.palette())
m2_blank.setSizePolicy(QSizePolicy.Policy.Expanding,
                      QSizePolicy.Policy.Expanding)
m2_layout.addWidget(m2_blank)
# ■■■ agrupa botões de Mesa 2 dentro de um QGroupBox ■■■■■■■■■■■■■■■■■■■■
```

```

m2_btn_layout = QVBoxLayout(m2_blank)
group_m2_actions = QGroupBox("Ações")
group_m2_layout = QVBoxLayout(group_m2_actions)
btn_enviar_m2 = QPushButton("Enviar")
# guarda para controle de habilitação
self.btn_enviar_m2 = btn_enviar_m2
# conecta "Enviar Mesa 2" → pulso M43
btn_enviar_m2.clicked.connect(lambda: self.ctrl._pulse_coil("M43"))
btn_retornar_m2 = QPushButton("Retornar")
# guarda para controlar estado
self.btn_retornar_m2 = btn_retornar_m2
btn_abrirproj_m2 = QPushButton("Abrir Projeto")
# guarda referência para habilitar/desabilitar conforme Start global
self.btn_abrirproj_m2 = btn_abrirproj_m2
group_m2_layout.addWidget(btn_enviar_m2)
group_m2_layout.addWidget(btn_retornar_m2)
btn_retornar_m2.clicked.connect(lambda: self.ctrl.flow_mesa2._return_after_error())
group_m2_layout.addWidget(btn_abrirproj_m2)
group_m2_layout.addStretch()
m2_btn_layout.addWidget(group_m2_actions)
# ■■ limita altura do groupbox 'Ações' para caber só os botões ■■
group_m2_actions.setSizePolicy(
    QSizePolicy.Policy.Expanding,
    QSizePolicy.Policy.Maximum
)
group_m2_actions.setMaximumHeight(group_m2_actions.sizeHint().height())
# conecta botão "Abrir Projeto" de Mesa 2
btn_abrirproj_m2.clicked.connect(lambda: self._open_project_for_mesa(2))
# ■■ GroupBox para exibir contagem e tempos (Mesa 2) ■■
group_m2_stats = QGroupBox("Estatísticas de Produção")
stats_m2_layout = QHBoxLayout(group_m2_stats)
# – coluna esquerda: contador de placas
vcount_m2 = QVBoxLayout()
vcount_m2.setAlignment(Qt.AlignmentFlag.AlignCenter)
vcount_m2.addWidget(QLabel("Placas Concluídas:"), alignment=Qt.AlignmentFlag.AlignCenter)
self.lbl_count_m2 = QLabel("0")
self.lbl_count_m2.setAlignment(Qt.AlignmentFlag.AlignCenter)
vcount_m2.addWidget(self.lbl_count_m2)
# – coluna direita: tempos
vtime_m2 = QVBoxLayout()
vtime_m2.setAlignment(Qt.AlignmentFlag.AlignCenter)
vtime_m2.addWidget(QLabel("Tempo Total (s):"), alignment=Qt.AlignmentFlag.AlignCenter)
self.lbl_time_elapsed_m2 = QLabel("0 s")
self.lbl_time_elapsed_m2.setAlignment(Qt.AlignmentFlag.AlignCenter)
vtime_m2.addWidget(self.lbl_time_elapsed_m2)
vtime_m2.addWidget(QLabel("Tempo Médio (s):"), alignment=Qt.AlignmentFlag.AlignCenter)
self.lbl_time_avg_m2 = QLabel("0 s")
self.lbl_time_avg_m2.setAlignment(Qt.AlignmentFlag.AlignCenter)
vtime_m2.addWidget(self.lbl_time_avg_m2)
# adiciona as duas colunas ao layout principal
stats_m2_layout.addLayout(vcount_m2)

```

```

stats_m2_layout.addLayout(vtime_m2)
m2_btn_layout.addWidget(group_m2_stats)
m2_btn_layout.addStretch()
# proporções internas (3:2)
m2_layout.setStretch(0, 3)
m2_layout.setStretch(1, 2)
hplot.addWidget(m2_container)

# ----- COLUNA DIREITA (15 %) -----
right_col = QVBoxLayout()

# --- Tabs para Câmera + Informações da Aplicadora ---
tabs = QTabWidget()
# Aba 1: Câmera
camera_group = QGroupBox()
cam_layout = QVBoxLayout(camera_group)
cam_layout.setContentsMargins(0, 0, 0, 0)
self.camera_label = AspectRatioLabel(aspect_ratio=4/3)
self.camera_label.setAutoFillBackground(True)
self.camera_label.setPalette(self.palette())
self.camera_label.setSizePolicy(
    QSizePolicy.Policy.Expanding,
    QSizePolicy.Policy.Preferred
)
cam_layout.addWidget(self.camera_label)
if hasattr(self.ctrl, "camera_manager"):
    self.ctrl.camera_manager.frameReady.connect(self._update_camera_frame)
tabs.addTab(camera_group, "Câmera")

# Aba 2: Leituras (antes chamada de "Temperaturas")
temp_group = QGroupBox() # remove título interno
tlay = QVBoxLayout(temp_group)
# Temperatura real e definida
self.lbl_temp_current = QLabel("Temperatura Atual: --°C")
self.lbl_temp_config = QLabel("Temperatura Configurada: --°C")
# Pressões
self.lbl_supply = QLabel("Alimentação Pneumática: --")
self.lbl_open = QLabel("Pressão do Fluido: --")
# Aquecimento
self.lbl_heater = QLabel("Aquecimento: Desligado")
for lbl in (self.lbl_temp_current,
            self.lbl_temp_config,
            self.lbl_supply,
            self.lbl_open,
            self.lbl_heater):
    lbl.setStyleSheet("padding:3px;")
    tlay.addWidget(lbl)
tlay.addStretch()
tabs.addTab(temp_group, "Leituras")

right_col.addWidget(tabs)

```

```

# Controles de movimento logo abaixo das abas
self.mov_widget = MovementControlsWidget()
right_col.addWidget(self.mov_widget)

# -----
# REMOVIDO O GROUPBOX "Posições de Inspeção" APENAS NESTA ABA
# – o widget continua existindo (para não quebrar callbacks
#   internos do SequenceControl), mas fica oculto da UI.
# -----
self.inspect_widget = InspectionPositionsWidget(
    get_current_position=self.ctrl._get_current_position_dict)
self.inspect_widget.hide()      # invisível na aba

# (não é adicionado ao layout right_col)

# Aba de controle global → utiliza extensão genérica .json
backend = InspectionPositionsBackend(self.inspect_widget)
file_filter = "Programas (*.json)"
self.prog_widget = ProgramIOWidget(
    backend,
    file_filter=file_filter,
    default_suffix=".json",
    default_dir=self.ctrl.projects_dir
)
right_col.addWidget(self.prog_widget)
# Esconde grupo de botões (passará a ser acessado via menu)
self.prog_widget.hide()

self.seq_widget = SequenceControlWidget(
    motion=self.ctrl._plc_motion_backend, # criado no controlador
    camera=None
)
# -----
# Libera e exibe os botões Pause / Stop SOMENTE nesta aba
# -----
self.seq_widget._btn_stop.show()
self.seq_widget._btn_pause.show()
# -----
# ABA "CONTROLE DE EIXOS"
# Oculta os elementos VISUAIS que já existem no espelho inferior:
#   – barra de progresso _progress
#   – label de status _status
#   – label do código _lbl_bc
# Eles continuam existindo (sinais/valores) e as demais abas
# permanecem inalteradas – apenas não aparecem aqui.
# -----
self.seq_widget._progress.setVisible(False)
self.seq_widget._status.setVisible(False)
self.seq_widget._lbl_bc.setVisible(False)
# -----

```

```

# AJUSTE "START"
# – renomeia o botão e mantém o _toggle_ interno com Stop/Pause.
# Após a lógica interna (_start) ser executada, também
# chamamos start_global_cycle() para preparar as mesas.
# -----
# -----
# BOTÕES Start / Pause / Stop (ciclo GERAL)
# • Start → ativa ciclo global + faz toggle dos botões
# • Pause → envia pause/resume + ajusta texto
# • Stop → encerra ciclo + refaz toggle
# -----
self.seq_widget._btn_execute.setText("Start")

# -----
# 1) Remove TODAS as conexões anteriores dos botões
# (PyQt6: disconnect() sem args → remove tudo)
# -----
for sig in (self.seq_widget._btn_execute.clicked,
            self.seq_widget._btn_stop.clicked,
            self.seq_widget._btn_pause.clicked):
    try:
        sig.disconnect() # limpa ligações antigas
    except TypeError:
        print("AxesControlTab: _build_ui() - "
              "erro ao desconectar sinal. ")
        # nenhum slot ligado → ignora
        pass

# 2) Conecta aos novos controladores locais
self.seq_widget._btn_execute.clicked.connect(self._on_global_start)
self.seq_widget._btn_stop.clicked.connect(self._on_global_stop)
self.seq_widget._btn_pause.clicked.connect(self._on_global_pause)

# 3) Estado inicial
# – Stop e Pause começam desabilitados
self.seq_widget._btn_stop.setEnabled(False)
self.seq_widget._btn_pause.setEnabled(False)
# Mantém o botão Stop VISÍVEL para permitir o toggle.
self.seq_widget._btn_stop.show()
right_col.addWidget(self.seq_widget)
right_col.addStretch()
# Aplica estilo inicial: Start habilitado (default),
# Stop desabilitado e pintado de vermelho
self._toggle_buttons(start_enabled=True)

# ■■ timer para habilitar/desabilitar os botões "Retornar" ■■
self._return_timer = QTimer(self)
self._return_timer.timeout.connect(self._update_return_buttons)
self._return_timer.start(200)

right_col.addStretch()

```

```

# direita – faz o right_wrap ocupar TODA a altura (duas linhas)
right_wrap = QWidget(); right_wrap.setLayout(right_col)
grid.addWidget(right_wrap, 0, 1, 2, 1)

# ----- LINHA INFERIOR -----
# só o bottom_placeholder na coluna 0
bottom_placeholder = QFrame(objectName="centerBottom")
# use palette em vez de cor fixa
bottom_placeholder.setAutoFillBackground(True)
bottom_placeholder.setPalette(self.palette())
bottom_placeholder.setSizePolicy(
    QSizePolicy.Policy.Expanding,
    QSizePolicy.Policy.Expanding)

bottom_lay = QHBoxLayout(bottom_placeholder)
bottom_lay.setContentsMargins(0, 0, 0, 0)
bottom_lay.setSpacing(0)

# -----
# ESPELHO DAS POSIÇÕES – MESA 1
# -----
def _make_half() -> QFrame:
    half = QFrame()
    # adapta o background ao tema do sistema em vez de cor fixa
    half.setAutoFillBackground(True)
    half.setPalette(self.palette())
    half.setSizePolicy(QSizePolicy.Policy.Expanding,
        QSizePolicy.Policy.Expanding)
    return half

left_half = _make_half()
right_half = _make_half()

# -----
# ESPELHO DAS POSIÇÕES – MESA 1
# -----
left_half_lay = QVBoxLayout(left_half)
left_half_lay.setContentsMargins(4, 4, 4, 4)

self.mirror_m1_label = QLabel("Posições de Inspeção – Mesa 1")
# apenas negrito; deixa cor de texto do sistema
font = self.mirror_m1_label.font()
font.setBold(True)
self.mirror_m1_label.setFont(font)

# --- lista + espaço vazio (60 % | 40 %) -----
self.mirror_m1_list = QListWidget()
# use palette para fundo e texto
self.mirror_m1_list.setAutoFillBackground(True)
self.mirror_m1_list.setPalette(self.palette())
self.mirror_m1_list.setMinimumHeight(140)

```

```

m1_row = QHBoxLayout()
m1_row.setContentsMargins(0, 0, 0, 0)
m1_row.addWidget(self.mirror_m1_list)
self._m1_blank = QFrame()          # área vazia 40 %
# adapta o background ao tema do sistema
self._m1_blank.setAutoFillBackground(True)
self._m1_blank.setPalette(self.palette())
m1_row.addWidget(self._m1_blank)
m1_row.setStretchFactor(self.mirror_m1_list, 3) # ≈60 %
m1_row.setStretchFactor(self._m1_blank, 2) # ≈40 %
# ===== B L O C O – CONTROLE DE SEQUÊNCIA (M1) =====
self._seq_m1_box = QGroupBox("Controle de Sequência")
vseq1 = QVBoxLayout(self._seq_m1_box)
self.lbl_m1_fid = QLabel("Fiducial: —")
self.pb_m1 = QProgressBar(); self.pb_m1.setValue(0)
self.lbl_m1_stat = QLabel("Status: —")
self.lbl_m1_code = QLabel("Código: —")
for w in (self.lbl_m1_fid, self.pb_m1, self.lbl_m1_stat, self.lbl_m1_code):
    vseq1.addWidget(w)
vseq1.addStretch()
self._m1_blank_layout = QVBoxLayout(self._m1_blank);
self._m1_blank_layout.setContentsMargins(0,0,0,0)
self._m1_blank_layout.addWidget(self._seq_m1_box)
left_half_lay.addWidget(self.mirror_m1_label)
left_half_lay.addLayout(m1_row, 1)

# -----
# ESPELHO DAS POSIÇÕES – MESA 2 (lado direito)
# -----

right_half_lay = QVBoxLayout(right_half)
right_half_lay.setContentsMargins(4, 4, 4, 4)

self.mirror_m2_label = QLabel("Posições de Inspeção – Mesa 2")
# apenas negrito; deixa cor de texto do sistema
font = self.mirror_m2_label.font()
font.setBold(True)
self.mirror_m2_label.setFont(font)

self.mirror_m2_list = QListWidget()
# use palette para fundo e texto
self.mirror_m2_list.setAutoFillBackground(True)
self.mirror_m2_list.setPalette(self.palette())
self.mirror_m2_list.setMinimumHeight(140)

m2_row = QHBoxLayout()
m2_row.setContentsMargins(0, 0, 0, 0)
m2_row.addWidget(self.mirror_m2_list)
self._m2_blank = QFrame()
# adapta o background ao tema do sistema
self._m2_blank.setAutoFillBackground(True)

```

```

self._m2_blank.setPalette(self.palette())
m2_row.addWidget(self._m2_blank)
m2_row.setStretchFactor(self.mirror_m2_list, 3)
m2_row.setStretchFactor(self._m2_blank, 2)
# ===== B L O C O – CONTROLE DE SEQUÊNCIA (M2) =====
self._seq_m2_box = QGroupBox("Controle de Sequência")
vseq2 = QVBoxLayout(self._seq_m2_box)
self.lbl_m2_fid = QLabel("Fiducial: —")
self.pb_m2 = QProgressBar(); self.pb_m2.setValue(0)
self.lbl_m2_stat = QLabel("Status: —")
self.lbl_m2_code = QLabel("Código: —")
for w in (self.lbl_m2_fid, self.pb_m2, self.lbl_m2_stat, self.lbl_m2_code):
    vseq2.addWidget(w)
vseq2.addStretch()
self._m2_blank_layout = QVBoxLayout(self._m2_blank);
self._m2_blank_layout.setContentsMargins(0,0,0,0)
self._m2_blank_layout.addWidget(self._seq_m2_box)

right_half_lay.addWidget(self.mirror_m2_label)
right_half_lay.addLayout(m2_row, 1)

# As conexões dependem de controller.mesa_tabs,
# que só estará disponível APÓS o controlador criar
# as abas Mesa-1 / Mesa-2. Programamos para o próximo
# ciclo do event-loop.
QTimer.singleShot(0, self._setup_mirror_connections)

# divisor visual (1 px) para evidenciar as duas colunas
divider = QFrame()
# usa forma nativa de linha em vez de cor fixa
divider.setFrameShape(QFrame.Shape.VLine)
divider.setFrameShadow(QFrame.Shadow.Sunken)
divider.setLineWidth(1)

bottom_lay.addWidget(left_half, 1)
bottom_lay.addWidget(divider)
bottom_lay.addWidget(right_half, 1)

# ----- LINHA INFERIOR -----
# agora a cena inferior central ocupa 100% da coluna 0;
# o spacer da direita permanece na coluna 1.
grid.addWidget(bottom_placeholder, 1, 0)
# ----- proporções (colunas 85|15, linhas 70|30) -----
grid.setColumnStretch(0, 85)
grid.setColumnStretch(1, 15)
grid.setRowStretch(0, 7)
grid.setRowStretch(1, 3)

# ----- CONEXÕES DE SINAIS -----
self.mov_widget.stepMoveRequested.connect(
    self.ctrl._on_step_move_requested)

```

```

self.mov_widget.jogStart.connect(self.ctrl._on_widget_jog_start)
self.mov_widget.jogStop.connect(self.ctrl._on_widget_jog_stop)
self.mov_widget.goToZeroRequested.connect(self.ctrl._on_go_to_zero)
self.mov_widget.emergencyStopToggled.connect(
    lambda e: self.ctrl.emergency_stop() if e else None)

```

```

# Como o widget está oculto, as conexões abaixo só são
# necessárias caso outro código o utilize programaticamente.
self.inspect_widget.positionAdded.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))
self.inspect_widget.positionRemoved.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))
self.prog_widget.fileLoaded.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))

```

```

# logs salvar / carregar
self.prog_widget.fileLoaded.connect(
    lambda f: self.ctrl.log(f"Programa carregado de: {f}"))
self.prog_widget.fileSaved.connect(
    lambda f: self.ctrl.log(f"Programa salvo em: {f}"))

```

```

# ----- expõe widgets principais no controlador -----
if self.primary:
    self.ctrl.mov_widget = self.mov_widget
    self.ctrl.inspect_widget = self.inspect_widget
    self.ctrl.prog_io_widget = self.prog_widget
    self.ctrl.seq_widget = self.seq_widget
    # feedback ciclo global → garante reset dos botões
    if hasattr(self.ctrl, "globalCycleFinished"):
        self.ctrl.globalCycleFinished.connect(
            lambda: self._toggle_buttons(start_enabled=True))
    # expõe gráficos
    self.ctrl.mesa_plot_widgets = {1: self.plot_m1,
                                   2: self.plot_m2}

```

```

# -----
# S L O T S   p/   c i c l o   g e r a l
# -----

```

```

def _toggle_buttons(self, *, start_enabled: bool):

```

```

    """Liga/desliga Start x Pause/Stop."""

```

```

    # referências locais

```

```

    btn_start = self.seq_widget._btn_execute

```

```

    btn_stop = self.seq_widget._btn_stop

```

```

    btn_pause = self.seq_widget._btn_pause

```

```

# ■ Start: se estiver "pressionado" (disabled), pinta de verde

```

```

    btn_start.setEnabled(start_enabled)

```

```

    if not start_enabled:

```

```

        btn_start.setStyleSheet(

```

```

            "QPushButton { background-color: #4CAF50; color: white; }"

```

```

    )
else:
    btn_start.setStyleSheet("")

# ■ Parar: se estiver disabled (ou seja, start_enabled=True), pinta de vermelho
btn_stop.setEnabled(not start_enabled)
if not btn_stop.isEnabled():
    btn_stop.setStyleSheet(
        "QPushButton { background-color: #F44336; color: white; }"
    )
else:
    btn_stop.setStyleSheet("")

# ■ Pause: só toggla texto/estado, mantém cor padrão
btn_pause.setEnabled(not start_enabled)
if start_enabled:
    btn_pause.setText("Pause")

def _on_global_start(self):
    """Aciona ciclo geral + desabilita botão Start."""
    # 1) Chama start_global_cycle (exibe warning se não houver programa)
    if hasattr(self.ctrl, "start_global_cycle"):
        self.ctrl.start_global_cycle()
    # 2) Só se o ciclo realmente foi ativado é que desabilitamos Start
    # e enviamos o pulso M40; caso contrário, nada muda na UI nem no CLP.
    if getattr(self.ctrl, "_global_cycle_active", False):
        # Desabilita Start e ativa Stop/Pause
        self._toggle_buttons(start_enabled=False)
        # Pulso momentâneo em M40 para sinalizar Start global
        if hasattr(self.ctrl, "_pulse_coil"):
            self.ctrl._pulse_coil("M40")
        # Sincroniza RANGE das barras de progresso externas com widgets internos
        m1_tab = getattr(self.ctrl, "mesa_tabs", {}).get(1)
        if m1_tab:
            self._sync_progress_range(self.pb_m1, m1_tab.seq_widget)
        m2_tab = getattr(self.ctrl, "mesa_tabs", {}).get(2)
        if m2_tab:
            self._sync_progress_range(self.pb_m2, m2_tab.seq_widget)
        # Durante o ciclo global, desabilita "Abrir Projeto"
        self.btn_abrirproj_m1.setEnabled(False)
        self.btn_abrirproj_m2.setEnabled(False)
        # ■■ Desabilita todo o grupo de Movement Controls ■■
        self.mov_widget.setEnabled(False)
        # ■■ Desabilita mudança de abas (exceto esta) ■■
        tabs = getattr(self.ctrl, "tab_widget", None)
        if tabs:
            me_idx = tabs.indexOf(self)
            for idx in range(tabs.count()):
                tabs.setTabEnabled(idx, idx == me_idx)

# ■■ Desabilita toda a barra de menus enquanto o Start estiver ativo ■■

```

```

try:
    menubar = self.ctrl.menuBar()
    menubar.setEnabled(False)
except Exception:
    print("AxesControlTab: _on_global_start() - "
          "erro ao desabilitar menuBar().")

```

```

def _on_global_stop(self):
    """Pede parada do ciclo geral e restabelece botão Start."""
    # Pulso momentâneo em M41 para sinalizar Stop global
    if hasattr(self.ctrl, "_pulse_coil"):
        self.ctrl._pulse_coil("M41")
    # 1) Garante que o ciclo geral seja desativado
    self.ctrl._global_cycle_active = False
    # 2) Se existir um método stop_global_cycle(), também o chama
    if hasattr(self.ctrl, "stop_global_cycle"):
        self.ctrl.stop_global_cycle()
    # ■■ solicita parada dos runners em execução nas mesas ■■■■■■■■■■
    # quem já estiver no meio de uma sequência terminará o ponto atual
    # e abortará o restante da sequência.
    for tab in getattr(self.ctrl, "mesa_tabs", {}).values():
        seqw = getattr(tab, "seq_widget", None)
        if seqw and getattr(seqw, "_runner", None) and seqw._runner.isRunning():
            seqw._stop()
    # ■■ estrela: retorna todas as mesas ao operador ■■■■■■■■■■
    # Depois de parar o ciclo global, força cada PlateFlowManager
    # a devolver a placa (movendo Y→limite+ e limpando estado).
    for mgr in (getattr(self.ctrl, 'flow_mesa1', None),
               getattr(self.ctrl, 'flow_mesa2', None)):
        if mgr:
            try:
                mgr._return_after_error()
            except Exception as exc:
                # registra falha sem interromper a UI
                self.ctrl.log(f"■■ Falha ao devolver Mesa {getattr(mgr, '_mesa', '?')}: {exc}")
    # 3) Reativa Start e desabilita Stop/Pause
    self._toggle_buttons(start_enabled=True)
    # 4) E desabilita imediatamente os botões "Enviar"
    # 5) Reativa o grupo de Movement Controls
    self.mov_widget.setEnabled(True)
    self.btn_enviar_m1.setEnabled(False)
    # 6) Após parar, reabilita os botões "Abrir Projeto"
    self.btn_abrirproj_m1.setEnabled(True)
    self.btn_abrirproj_m2.setEnabled(True)
    # 7) Reabilita mudança de abas
    tabs = getattr(self.ctrl, "tab_widget", None)
    if tabs:
        for idx in range(tabs.count()):
            tabs.setTabEnabled(idx, True)
    # ■■ Reabilita toda a barra de menus após o Stop ■■

```

```

try:
    menubar = self.ctrl.menuBar()
    menubar.setEnabled(True)
except Exception:
    print("AxesControlTab: _on_global_stop() - "
          "erro ao reabilitar menuBar().")

def _on_global_pause(self):
    """Alterna pausa/continuação do ciclo geral."""
    if hasattr(self.ctrl, "pause_global_cycle"):
        self.ctrl.pause_global_cycle()
    # toggling do próprio texto/estado

    if self.seq_widget._btn_pause.text() == "Pause":
        self.seq_widget._btn_pause.setText("Continuar")
        self.seq_widget._status.setText("Pausado")
    else:
        self.seq_widget._btn_pause.setText("Pause")
        self.seq_widget._status.setText("Executando...")

def _open_project_for_mesa(self, mesa_id: int):
    """
    Abre o diálogo 'Abrir Projeto...' para a mesa especificada,
    exatamente como se você fosse na aba Mesa <n> e clicasse
    em Arquivo → Abrir Projeto....
    """
    tab = getattr(self.ctrl, "mesa_tabs", {}).get(mesa_id)
    if not tab or not hasattr(tab, "prog_widget"):
        return
    # abre o diálogo de Abrir Projeto na própria aba, sem trocar de aba
    tab.prog_widget._on_load_clicked()

# ----- helper: converte lista p/ SequenceControl -----
def _to_model(self):
    lst = []
    for p in self.inspect_widget.positions():
        lst.append(
            InspectionPosition(
                name=p.name,
                x=p.x, y2=p.y2, y1=p.y1, z=p.z
            )
        )
    return lst

def _update_camera_frame(self, img):
    """Receive QImage frames and display scaled in camera_label."""
    pix = QPixmap.fromImage(img).scaled(
        self.camera_label.size(),
        Qt.AspectRatioMode.KeepAspectRatio,
        Qt.TransformationMode.SmoothTransformation
    )

```

```
self.camera_label.setPixmap(pix)
```

```
def _update_return_buttons(self):
```

```
    """
```

```
    Ativa ou desativa os botões "Retornar":
```

```
    – habilita enquanto o fluxo da mesa não estiver em IDLE,
```

```
    – desabilita em IDLE (mesa no limite positivo).
```

```
    """
```

```
    # Mesa 1 – habilita "Retornar" apenas enquanto estiver em fila
```

```
    mgr1 = getattr(self.ctrl, 'flow_mesa1', None)
```

```
    if mgr1 and mgr1._state is FlowState.QUEUED:
```

```
        self.btn_retornar_m1.setEnabled(True)
```

```
    else:
```

```
        self.btn_retornar_m1.setEnabled(False)
```

```
    # estilo: Retornar em laranja quando habilitado
```

```
    if self.btn_retornar_m1.isEnabled():
```

```
        self.btn_retornar_m1.setStyleSheet(
```

```
            "QPushButton { background-color: #FF9800; color: white; }"
```

```
        )
```

```
    else:
```

```
        self.btn_retornar_m1.setStyleSheet("")
```

```
    # Mesa 2 – habilita "Retornar" apenas enquanto estiver em fila
```

```
    mgr2 = getattr(self.ctrl, 'flow_mesa2', None)
```

```
    if mgr2 and mgr2._state is FlowState.QUEUED:
```

```
        self.btn_retornar_m2.setEnabled(True)
```

```
    else:
```

```
        self.btn_retornar_m2.setEnabled(False)
```

```
    # estilo: Retornar em laranja quando habilitado (Mesa 2)
```

```
    if self.btn_retornar_m2.isEnabled():
```

```
        self.btn_retornar_m2.setStyleSheet(
```

```
            "QPushButton { background-color: #FF9800; color: white; }"
```

```
        )
```

```
    else:
```

```
        self.btn_retornar_m2.setStyleSheet("")
```

```
    # ■■ Enviar só se: programa carregado + Start ativo + mesa em IDLE ■■
```

```
    # Mesa 1
```

```
    en1 = (getattr(self.ctrl, "_global_cycle_active", False)
```

```
           and self.ctrl._mesa_has_program(1)
```

```
           and mgr1 and mgr1._state is FlowState.IDLE)
```

```
    self.btn_enviar_m1.setEnabled(bool(en1))
```

```
    # estilo: Enviar em verde quando habilitado
```

```
    if self.btn_enviar_m1.isEnabled():
```

```
        self.btn_enviar_m1.setStyleSheet(
```

```
            "QPushButton { background-color: #4CAF50; color: white; }"
```

```
        )
```

```
    else:
```

```
        self.btn_enviar_m1.setStyleSheet("")
```

```
    # Mesa 2
```

```
    en2 = (getattr(self.ctrl, "_global_cycle_active", False)
```

```
           and self.ctrl._mesa_has_program(2)
```

```

        and mgr2 and mgr2._state is FlowState.IDLE)
self.btn_enviar_m2.setEnabled(bool(en2))
# estilo: Enviar em verde quando habilitado (Mesa 2)
if self.btn_enviar_m2.isEnabled():
    self.btn_enviar_m2.setStyleSheet(
        "QPushButton { background-color: #4CAF50; color: white; }"
    )
else:
    self.btn_enviar_m2.setStyleSheet("")

# -----
#      S I N C ■ M e s a  1  →  E s p e l h o
# -----
def _sync_mesa1_positions(self):
    """
    Copia as posições da aba “Mesa 1” para a lista espelhada
    exibida na parte inferior da aba “Controle de Eixos”.
    """
    try:
        pts = self.ctrl.mesa_tabs[1].inspect_widget.positions()
    except Exception:
        pts = []
    self.mirror_m1_list.blockSignals(True)
    self.mirror_m1_list.clear()
    for p in pts:
        self.mirror_m1_list.addItem(
            f"{p.name}. X={p.x:.0f} Y={p.y1:.0f} Z={p.z:.0f}"
        )
    # mantém seleção actual
    row_sel = self.ctrl.mesa_tabs[1].inspect_widget.list_widget.currentRow()
    if row_sel >= 0 and row_sel < self.mirror_m1_list.count():
        self.mirror_m1_list.setCurrentRow(row_sel)
    self.mirror_m1_list.blockSignals(False)

# -----
# Conecta-se aos sinais da aba Mesa-1 depois que ela existir
# -----
def _setup_mirror_connections(self):
    mesa1_tab = getattr(self.ctrl, "mesa_tabs", {}).get(1)
    if mesa1_tab is None:      # ainda não criado
        return
    m1w = mesa1_tab.inspect_widget
    for sig in (m1w.positionAdded,
                m1w.positionRemoved,
                m1w.positionInserted):
        sig.connect(lambda _=None: self._sync_mesa1_positions())
    mesa1_tab.prog_widget.fileLoaded.connect(
        lambda _=None: self._sync_mesa1_positions())
# -----
# SINCRONIZAÇÃO DO INDICADOR DE SELEÇÃO
# – sempre que o usuário (ou o SequenceRunner) mudar a linha

```

```

# selecionada na QListWidget da Mesa 1, o espelho na aba
# "Controle de Eixos" realça a mesma linha.
# -----
mesa1_tab.inspect_widget.list_widget.currentRowChanged.connect(
    self._mirror_select_row)
# ----- conexões de feedback (Mesa 1) -----
m1_seq = mesa1_tab.seq_widget
m1_seq.fidMatch.connect(lambda sim,ok,x,y,w,h: self.lbl_m1_fid.setText(
    f"Fiducial: {'OK' if ok else 'Fail'} ({sim:.1f}%))")
m1_seq.fidClear.connect(lambda: self.lbl_m1_fid.setText("Fiducial: —"))
# ■■■■■■ SINCRONIZAÇÃO DA BARRA DE PROGRESSO (Mesa-1) ■■■■■■
# Usa EXACTAMENTE o mesmo QProgressBar interno do SequenceControlWidget
# – qualquer alteração de faixa ou valor será espelhada.
# ■■■■■■ SINCRONIZAÇÃO DA BARRA DE PROGRESSO (Mesa-1) ■■■■■■
# QProgressBar não possui 'rangeChanged' → copiamos o intervalo
# imediatamente antes de cada execução e sempre que as posições
# forem alteradas.
m1_seq._progress.valueChanged.connect(self.pb_m1.setValue)
# antes de iniciar
m1_seq._btn_execute.clicked.connect(
    lambda _=False, s=m1_seq: self._sync_progress_range(self.pb_m1, s))
# quando a lista de pontos muda
mesa1_tab.inspect_widget.positionAdded.connect(
    lambda _=None, s=m1_seq: self._sync_progress_range(self.pb_m1, s))
mesa1_tab.inspect_widget.positionRemoved.connect(
    lambda _=None, s=m1_seq: self._sync_progress_range(self.pb_m1, s))
# seq encerrada → garante barra completa
m1_seq.sequenceFinished.connect(lambda: self.pb_m1.setValue(self.pb_m1.maximum()))
m1_seq.sequenceError.connect( lambda _msg: self.pb_m1.setValue(self.pb_m1.maximum()))
m1_seq.sequenceFinished.connect(lambda: self.lbl_m1_stat.setText("Status: Concluído"))
# increment Mesa 1 count only on successful finish
m1_seq.sequenceFinished.connect(self._on_plate_done_m1)
m1_seq.sequenceError.connect(lambda msg: self.lbl_m1_stat.setText(f"Status: Erro - {msg}"))
m1_seq.bcMatch.connect(lambda ok,x,y,w,h,txt:
    self.lbl_m1_code.setText(f"Código: {txt if ok else '—'}"))
# ----- Mesa 2 -----
mesa2_tab = getattr(self.ctrl, "mesa_tabs", {}).get(2)
if mesa2_tab:
    m2w = mesa2_tab.inspect_widget
    for sig in (m2w.positionAdded,
                m2w.positionRemoved,
                m2w.positionInserted):
        sig.connect(lambda _=None: self._sync_mesa2_positions())
    mesa2_tab.prog_widget.fileLoaded.connect(
        lambda _=None: self._sync_mesa2_positions())
    mesa2_tab.inspect_widget.list_widget.currentRowChanged.connect(
        self._mirror_select_row_m2)
# ----- conexões de feedback (Mesa 2) -----
m2_seq = mesa2_tab.seq_widget
m2_seq.fidMatch.connect(lambda sim,ok,x,y,w,h: self.lbl_m2_fid.setText(
    f"Fiducial: {'OK' if ok else 'Fail'} ({sim:.1f}%))")

```

```

m2_seq.fidClear.connect(lambda: self.lbl_m2_fid.setText("Fiducial: —"))
# ■■■■■■■■ SINCRONIZAÇÃO DA BARRA DE PROGRESSO (Mesa-2) ■■■■■■■■
# ■■■■■■■■ SINCRONIZAÇÃO DA BARRA DE PROGRESSO (Mesa-2) ■■■■■■■■
m2_seq._progress.valueChanged.connect(self.pb_m2.setValue)
m2_seq._btn_execute.clicked.connect(
    lambda _=False, s=m2_seq: self._sync_progress_range(self.pb_m2, s))
mesa2_tab.inspect_widget.positionAdded.connect(
    lambda _=None, s=m2_seq: self._sync_progress_range(self.pb_m2, s))
mesa2_tab.inspect_widget.positionRemoved.connect(
    lambda _=None, s=m2_seq: self._sync_progress_range(self.pb_m2, s))
m2_seq.sequenceFinished.connect(lambda: self.pb_m2.setValue(self.pb_m2.maximum()))
m2_seq.sequenceError.connect( lambda _msg: self.pb_m2.setValue(self.pb_m2.maximum()))
m2_seq.sequenceFinished.connect(lambda: self.lbl_m2_stat.setText("Status: Concluído"))
# increment Mesa 2 count only on successful finish
m2_seq.sequenceFinished.connect(self._on_plate_done_m2)
m2_seq.sequenceError.connect(lambda msg: self.lbl_m2_stat.setText(f"Status: Erro - {msg}"))
m2_seq.bcMatch.connect(lambda ok,x,y,w,h,txt:
    self.lbl_m2_code.setText(f"Código: {txt if ok else '—'}"))
# primeira sincronização
self._sync_mesa1_positions()
self._sync_mesa2_positions()
# -----
# Slots para exibir temperatura da aplicadora nesta aba
# -----
def update_applicator_current_temp(self, temp: float):
    """Atualiza o label de temperatura real."""
    self.lbl_temp_current.setText(f"Temperatura Atual: {temp:.1f}°C")

def update_applicator_config_temp(self, temp: float):
    """Atualiza o label de temperatura configurada."""
    self.lbl_temp_config.setText(f"Temperatura Configurada: {temp:.1f}°C")

def update_applicator_supply(self, p: float):
    """Atualiza o label de pressão de supply."""
    # Escolhe a unidade atual (PSI ou KPA) da controladora de aplicação
    unit = (self.ctrl.applicator_window.cmb_unit.currentText()
            if hasattr(self.ctrl, "applicator_window") else "")
    # Converte para bar: 1 psi = 0.0689476 bar, 1 kPa = 0.01 bar
    if unit.upper() == "PSI":
        bar = p * 0.0689476
    elif unit.upper() == "KPA":
        bar = p * 0.01
    else:
        bar = None
    # Exibe "Supply: X.Y UNI (Z.W bar)" ou sem conversão se não souber a unidade
    if bar is not None:
        self.lbl_supply.setText(
            f"Alimentação Pneumática: {p:.1f} {unit} ({bar:.2f} bar)")
    else:
        self.lbl_supply.setText(f"Alimentação Pneumática: {p:.1f} {unit}")
def update_applicator_open(self, p: float):

```

```

"""Atualiza o label de pressão de open."""
unit = (self.ctrl.applicator_window.cmb_unit.currentText()
        if hasattr(self.ctrl, "applicator_window") else "")
if unit.upper() == "PSI":
    bar = p * 0.0689476
elif unit.upper() == "KPA":
    bar = p * 0.01
else:
    bar = None
if bar is not None:
    self.lbl_open.setText(
        f"Pressão do Fluido: {p:.1f} {unit} ({bar:.2f} bar)")
else:
    self.lbl_open.setText(f"Pressão do Fluido: {p:.1f} {unit}")
def update_applicator_heater(self, val: int):
    """
    Atualiza o label do heater:
    – Ligado (quando val==0)
    – Desligado (quando val==1)
    """
    state = "Ligado" if val == 0 else "Desligado"
    self.lbl_heater.setText(f"Aquecimento: {state}")

# ----- helper interno -----
@staticmethod
def _sync_progress_range(ext_bar, seq_widget):
    """
    Ajusta o range da barra externa `ext_bar` para [0, N] onde N é o
    número de posições definidas em seq_widget, garantindo que
    cada passo corresponda a 1/N·100%.
    """
    # tenta usar o tamanho da lista de posições (mais confiável)
    total = None
    if hasattr(seq_widget, '_positions'):
        total = len(seq_widget._positions)
    if total is None:
        # fallback: range do progress interno (0...100 por padrão)
        lo = seq_widget._progress.minimum()
        hi = seq_widget._progress.maximum()
        ext_bar.setRange(lo, hi)
    else:
        ext_bar.setRange(0, total)
    ext_bar.setValue(seq_widget._progress.value())

def _sync_mesa2_positions(self):
    """Reflete Mesa 2 na lista da direita."""
    try:
        pts = self.ctrl.mesa_tabs[2].inspect_widget.positions()
    except Exception:
        pts = []
    self.mirror_m2_list.blockSignals(True)

```

```

self.mirror_m2_list.clear()
for p in pts:
    self.mirror_m2_list.addItem(
        f"{p.name}. X={p.x:.0f} Y={p.y2:.0f} Z={p.z:.0f}"
    )
# mantém seleção
row_sel = self.ctrl.mesa_tabs[2].inspect_widget.list_widget.currentRow()
if 0 <= row_sel < self.mirror_m2_list.count():
    self.mirror_m2_list.setCurrentRow(row_sel)
self.mirror_m2_list.blockSignals(False)

# -----
# Seleção vinda da Mesa 1 → aplica no espelho
# -----
def _mirror_select_row(self, row: int):
    """
    Recebe índice da linha seleccionada na QListWidget original
    (Mesa 1) e replica a selecção no QListWidget espelhado.
    """
    if row < 0 or row >= self.mirror_m1_list.count():
        self.mirror_m1_list.clearSelection()
        return
    self.mirror_m1_list.blockSignals(True)
    self.mirror_m1_list.setCurrentRow(row)
    self.mirror_m1_list.blockSignals(False)

# ----- Mesa 2 -----
def _mirror_select_row_m2(self, row: int):
    if row < 0 or row >= self.mirror_m2_list.count():
        self.mirror_m2_list.clearSelection()
        return
    self.mirror_m2_list.blockSignals(True)
    self.mirror_m2_list.setCurrentRow(row)
    self.mirror_m2_list.blockSignals(False)

# -----
# Slots to count completed plates (only on successful sequence end)
# -----
def _on_plate_done_m1(self):
    """Increment and display Mesa 1 production count (only in APPLY mode)."""
    # only count if Mesa 1's sequence widget is in APPLY
    tab1 = getattr(self.ctrl, "mesa_tabs", {}).get(1)
    seq1 = getattr(tab1, "seq_widget", None)
    if not (seq1 and seq1.radio_apply.isChecked()):
        return
    self._count_m1 += 1
    self.lbl_count_m1.setText(str(self._count_m1))

def _on_plate_done_m2(self):
    """Increment and display Mesa 2 production count (only in APPLY mode)."""
    # only count if Mesa 2's sequence widget is in APPLY

```

```

tab2 = getattr(self.ctrl, "mesa_tabs", {}).get(2)
seq2 = getattr(tab2, "seq_widget", None)
if not (seq2 and seq2.radio_apply.isChecked()):
    return
self._count_m2 += 1
self.lbl_count_m2.setText(str(self._count_m2))

# -----
# Slot chamado por PlateFlowManager quando termina cada ciclo
# -----
def _on_plate_time(self, mesa_id: int, elapsed: float):
    """
    Atualiza:
    – lbl_time_elapsed_m<n>: tempo (s) do último ciclo com 2 decimais;
    – lbl_time_avg_m<n>: média de todos os tempos registrados.
    """

    # Só processa tempos quando estamos no modo APPLY
    if not self.seq_widget.radio_apply.isChecked():
        return
    if mesa_id == 1:
        self._times_m1.append(elapsed)
        self.lbl_time_elapsed_m1.setText(f"{elapsed:.2f} s")
        avg = sum(self._times_m1) / len(self._times_m1)
        self.lbl_time_avg_m1.setText(f"{avg:.2f} s")
    elif mesa_id == 2:
        self._times_m2.append(elapsed)
        self.lbl_time_elapsed_m2.setText(f"{elapsed:.2f} s")
        avg = sum(self._times_m2) / len(self._times_m2)
        self.lbl_time_avg_m2.setText(f"{avg:.2f} s")

```

File: aplicadora_tab.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

import time
import serial
import serial.tools.list_ports
from typing import Callable
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QGroupBox, QGridLayout, QLabel, QComboBox,
    QPushButton, QSpinBox, QMessageBox
)
from PyQt6.QtCore import Qt, QTimer, pyqtSignal

# Configurações seriais e Modbus
COM_SETTINGS = dict(
    baudrate=115200, bytesize=8, parity='N',
    stopbits=1, timeout=0.5, rtscts=False, dsrdtr=False
)

```

```
)  
MONITOR_INTERVAL_MS = 3000  
AUTO_TEST_DELAY_MS = 500
```

```
DEVICE_ID      = 0x01  
FUNC_READ      = 0x03  
FUNC_WRITE_SINGLE = 0x06  
FUNC_WRITE_MULTI  = 0x10
```

```
class AdhesiveApplicatorTab(QWidget):
```

```
    """
```

```
    Aba de controle da aplicadora de adesivo em Modbus RTU via serial.  
    main_ctrl: instância de MultiAxisMotorController (para log e settings).
```

```
    """
```

```
    # novos sinais para supply, open e valor de Canal1 (heater)  
    supplyPressureUpdated = pyqtSignal(float)  
    openPressureUpdated   = pyqtSignal(float)  
    heatStateChanged      = pyqtSignal(int)  
    # sinaliza quando a temperatura real é atualizada (°C)  
    temperatureUpdated = pyqtSignal(float)  
    # sinaliza quando a temperatura configurada (spin_t1) é atualizada (°C)  
    configTempUpdated = pyqtSignal(float)
```

```
    def __init__(self, main_ctrl):
```

```
        super().__init__()  
        self.ctrl = main_ctrl  
        self.serial_connection = None  
        # flag para suprimir repetição de erros de permissão no COM  
        self._perm_error_logged = False  
        self._extended_controls = []  
        self._build_ui()  
        # 1) Preenche combo com portas e seleciona a última porta salva (se existir)  
        self.refresh_ports()  
        last = self.ctrl.settings.data.get('applicator_last_port')  
        if last:  
            idx = self.port_combo.findData(last)  
            if idx >= 0:  
                self.port_combo.setCurrentIndex(idx)  
                # 2) Conecta automaticamente na última porta  
                self.connect_serial()
```

```
    def _build_ui(self):
```

```
        self.setLayout(QVBoxLayout())  
        # — Conexão Serial —  
        conn_box = QGroupBox("Conexão Serial")  
        g = QGridLayout(conn_box)  
        self.port_combo = QComboBox(); self.port_combo.setMinimumWidth(300)  
        self.btn_refresh = QPushButton("Atualizar Portas")  
        self.btn_connect = QPushButton("Conectar")  
        self.lbl_status = QLabel("Desconectado")
```

```

self.lbl_status.setStyleSheet("color:red; font-weight:bold;")
g.addWidget(QLabel("Porta:"), 0,0); g.addWidget(self.port_combo,0,1)
g.addWidget(self.btn_refresh,0,2)
g.addWidget(QLabel("Status:"),1,0); g.addWidget(self.lbl_status,1,1)
g.addWidget(self.btn_connect,1,2)
self.layout().addWidget(conn_box)

# — Monitoramento —
mon_box = QGroupBox("Monitoramento")
mg = QGridLayout(mon_box)
self.lbl_temp = QLabel("Temp: --°C")
self.lbl_press1 = QLabel("Supply: --")
self.lbl_press2 = QLabel("Open: --")
for w in (self.lbl_temp, self.lbl_press1, self.lbl_press2):
    w.setStyleSheet("font-weight:bold;")
mg.addWidget(self.lbl_temp,0,0)
mg.addWidget(self.lbl_press1,1,0)
mg.addWidget(self.lbl_press2,1,1)
self.layout().addWidget(mon_box)

# — Configurações Operacionais —
op_box = QGroupBox("Configurações")
og = QGridLayout(op_box)
# Modo
og.addWidget(QLabel("Modo:"),0,0)
self.cmb_mode = QComboBox(); self.cmb_mode.addItem(
    ["Defined", "Infinite", "Group", "Purge"])
self.btn_set_mode = QPushButton("Definir")
og.addWidget(self.cmb_mode,0,1); og.addWidget(self.btn_set_mode,0,2)
# Unidade
og.addWidget(QLabel("Unidade:"),1,0)
self.cmb_unit = QComboBox(); self.cmb_unit.addItem(["PSI", "KPA"])
self.btn_set_unit = QPushButton("Definir")
og.addWidget(self.cmb_unit,1,1); og.addWidget(self.btn_set_unit,1,2)
# Heat1
og.addWidget(QLabel("Canal1:"),2,0)
self.cmb_heat1 = QComboBox(); self.cmb_heat1.addItem(["ON", "OFF"])
self.btn_set_heat1 = QPushButton("Definir")
og.addWidget(self.cmb_heat1,2,1); og.addWidget(self.btn_set_heat1,2,2)
# Tempos Open/Close
og.addWidget(QLabel("Open time (0.1ms):"),3,0)
self.spin_open = QSpinBox(); self.spin_open.setRange(0,9999)
self.btn_set_open = QPushButton("Definir")
og.addWidget(self.spin_open,3,1); og.addWidget(self.btn_set_open,3,2)
og.addWidget(QLabel("Close time (0.1ms):"),4,0)
self.spin_close = QSpinBox(); self.spin_close.setRange(0,9999)
self.btn_set_close = QPushButton("Definir")
og.addWidget(self.spin_close,4,1); og.addWidget(self.btn_set_close,4,2)
# Temp1
og.addWidget(QLabel("Temp1 (0.1°C):"),5,0)
self.spin_t1 = QSpinBox(); self.spin_t1.setRange(200,800)

```

```
self.btn_set_t1 = QPushButton("Definir")
og.addWidget(self.spin_t1,5,1); og.addWidget(self.btn_set_t1,5,2)
```

```
self.layout().addWidget(op_box)
```

```
# Timer de monitoramento
self.monitor_timer = QTimer(self)
self.monitor_timer.timeout.connect(self.monitor_device)
```

```
# Conecta sinais
self.btn_refresh.clicked.connect(self.refresh_ports)
self.btn_connect.clicked.connect(self.toggle_connection)
self.btn_set_mode.clicked.connect(self.set_mode)
self.btn_set_unit.clicked.connect(self.set_unit)
self.btn_set_heat1.clicked.connect(self.set_heat1)
self.btn_set_open.clicked.connect(self.set_open_time)
self.btn_set_close.clicked.connect(self.set_close_time)
self.btn_set_t1.clicked.connect(self.set_temp1)
```

```
# Registra controles para habilitar/desabilitar
self._extended_controls += [
    self.cmb_mode, self.btn_set_mode,
    self.cmb_unit, self.btn_set_unit,
    self.cmb_heat1, self.btn_set_heat1,
    self.spin_open, self.btn_set_open,
    self.spin_close, self.btn_set_close,
    self.spin_t1, self.btn_set_t1
]
```

```
def log(self, msg: str):
    # redireciona para o log geral
    if hasattr(self.ctrl, "log"):
        self.ctrl.log(f"[Aplicadora] {msg}")
    else:
        print(f"[Aplicadora] {msg}")
```

```
def refresh_ports(self):
    self.port_combo.clear()
    ports = serial.tools.list_ports.comports()
    if not ports:
        self.port_combo.addItem("— nenhuma —")
        self.log("nenhuma porta serial encontrada")
        return
    for p in ports:
        txt = f"{p.device} ({p.description})"
        self.port_combo.addItem(txt, p.device)
    self.log(f"{len(ports)} porta(s) detectada(s)")
    # Se existir porta salva, tenta selecioná-la
    last = self.ctrl.settings.data.get('applicator_last_port')
    if last:
        idx = self.port_combo.findData(last)
```

```

        if idx >= 0:
            self.port_combo.setCurrentIndex(idx)

def get_selected_port(self):
    d = self.port_combo.currentData()
    if d: return d
    txt = self.port_combo.currentText()
    return txt.split(' ')[0]

def toggle_connection(self):
    if not self.serial_connection:
        self.connect_serial()
    else:
        self.disconnect_serial()

def connect_serial(self):
    # evita reconectar na mesma porta se já estiver conectado
    if self.serial_connection:
        self.log("porta serial já conectada, pulando reconnect")
        return
    port = self.get_selected_port()
    if "nenhuma" in port.lower():
        QMessageBox.warning(self, "Erro", "Selecione uma porta válida")
        return
    try:
        self.serial_connection = serial.Serial(port=port, **COM_SETTINGS)
        time.sleep(0.2)
        self.lbl_status.setText("Conectado")
        self.lbl_status.setStyleSheet("color:green;")
        self.btn_connect.setText("Desconectar")
        self._set_controls_enabled(True)
        self.log(f"conectado em {port}")
        # Persiste porta usada
        try:
            self.ctrl.settings.data['applicator_last_port'] = port
            self.ctrl.settings.save()
        except Exception:
            print("Erro ao salvar porta no settings")
        QTimer.singleShot(AUTO_TEST_DELAY_MS, self.auto_test_connection)
        # liberamos o aviso de permissão após (re)conectar
        self._perm_error_logged = False
    except Exception as e:
        QMessageBox.critical(self, "Erro", str(e))
        self.log(f"falha na conexão: {e}")

def disconnect_serial(self):
    if self.serial_connection:
        self.monitor_timer.stop()
        self.serial_connection.close()
        self.serial_connection = None
        self.lbl_status.setText("Desconectado")

```

```

self.lbl_status.setStyleSheet("color:red;")
self.btn_connect.setText("Conectar")
self._set_controls_enabled(False)
self.log("desconectado")

```

```

def _set_controls_enabled(self, en: bool):
    for w in self._extended_controls:
        w.setEnabled(en)

```

```

def auto_test_connection(self):
    self.log("teste comunicação...")
    # comando de leitura temperatura
    base = f'{DEVICE_ID:02X} {FUNC_READ:02X} 0048 00 01'
    cmd = self.build_modbus_command(base)
    r = self.send_command(cmd, "Teste Temp")
    if r and len(r)>=7:
        self.log("comunicação OK")
        self.parse_temperature_response(r)
        self.read_initial_parameters()
        self.monitor_timer.start(MONITOR_INTERVAL_MS)
    else:
        QMessageBox.warning(self, "Falha", "não foi possível comunicar")

```

```

def send_command(self, hexstr: str, desc: str='') -> bytes | None:
    if not self.serial_connection:
        return None
    try:
        data = bytes.fromhex(hexstr.replace(" ", ""))
        self.serial_connection.reset_input_buffer()
        self.serial_connection.reset_output_buffer()
        # Suprime log de transmissão para reduzir poluição de terminal
        # self.log(f'TX {desc}: {hexstr}')
        self.serial_connection.write(data)
        resp = b''
        start = time.time()
        while time.time()-start < self.serial_connection.timeout:
            n = self.serial_connection.in_waiting
            if n:
                resp += self.serial_connection.read(n)
            else:
                QTimer.singleShot(1, lambda: None)
                time.sleep(0.01)
            if len(resp)>=5: break
        if resp:
            # reduzir logs
            # self.log(f'RX ({len(resp)}): " +
            #     ' '.join(f'{b:02X}' for b in resp))
            return resp
        self.log("sem resposta")
        return None
    except PermissionError as e:

```

```

# só loga a primeira vez que ocorrer falha de permissão
if not self._perm_error_logged:
    self.log(f"AVISO: falha de permissão ao enviar comando: {e}")
    self._perm_error_logged = True
# interrompe sem desconectar para não poluir log
return None
except Exception as e:
    self.log(f"erro: {e}")
    return None

def build_modbus_command(self, base_hex: str) -> str:
    data = bytes.fromhex(base_hex.replace(" ", ""))
    crc = self.calculate_crc(data)
    return base_hex + f" {crc[0]:02X} {crc[1]:02X}"

@staticmethod
def calculate_crc(data: bytes) -> bytes:
    crc = 0xFFFF
    for b in data:
        crc ^= b
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
    return crc.to_bytes(2, 'little')

def send_and_parse(self, base_hex: str,
                    parser: Callable[[bytes], None],
                    desc: str):
    cmd = self.build_modbus_command(base_hex)
    r = self.send_command(cmd, desc)
    if r:
        try: parser(r)
        except Exception as e: self.log(f"parser '{desc}' erro: {e}")
    return r

# — Helpers Modbus —
def write_register(self, func: int, reg: int,
                  val: int, desc: str) -> bool:
    hi, lo = (val >> 8) & 0xFF, val & 0xFF
    base = f"{DEVICE_ID:02X} {func:02X} {reg:04X} 00 01 02 {hi:02X} {lo:02X}"
    return bool(self.send_command(self.build_modbus_command(base), desc))

def write_single_register(self, reg: int, val: int, desc: str) -> bool:
    base = f"{DEVICE_ID:02X} {FUNC_WRITE_SINGLE:02X} {reg:04X} 00 {val:02X}"
    return bool(self.send_command(self.build_modbus_command(base), desc))

def read_register(self, reg: int, parser: Callable, desc: str):
    base = f"{DEVICE_ID:02X} {FUNC_READ:02X} {reg:04X} 00 01"
    return self.send_and_parse(base, parser, desc)

```

— Setters e Parsers —

```
def set_mode(self):
    idx = self.cmb_mode.currentIndex()
    if self.write_single_register(0x0040, idx, "Modo"):
        QMessageBox.information(self, "OK", "Modo definido")
        # se quiser notificar alguma aba, pode emitir sinal aqui
        # ex: self.modeChanged.emit(idx)

def set_unit(self):
    idx = self.cmb_unit.currentIndex()
    if self.write_single_register(0x0041, idx, "Unidade"):
        QMessageBox.information(self, "OK", "Unidade definida")
        # ex: self.unitChanged.emit(idx)

def set_heat1(self):
    idx = self.cmb_heat1.currentIndex()
    if self.write_single_register(0x0042, idx, "Heat1"):
        QMessageBox.information(self, "OK", "Canal1 definido")
        # atualização imediata na aba Controle de Eixos
        try:
            self.heatStateChanged.emit(idx)
        except Exception:
            print("Erro ao emitir sinal heatStateChanged")

def set_open_time(self):
    v = self.spin_open.value()
    if self.write_register(FUNC_WRITE_MULTI, 0x0043, v, "Open"):
        QMessageBox.information(self, "OK", "Open time definido")
        # ex: self.openTimeChanged.emit(v)

def set_close_time(self):
    v = self.spin_close.value()
    if self.write_register(FUNC_WRITE_MULTI, 0x0044, v, "Close"):
        QMessageBox.information(self, "OK", "Close time definido")
        # ex: self.closeTimeChanged.emit(v)

def set_temp1(self):
    v = self.spin_t1.value()
    if self.write_register(FUNC_WRITE_MULTI, 0x0047, v, "Temp1"):
        QMessageBox.information(self, "OK", "T1 definido")
        # atualiza imediatamente o label de temperatura configurada
        try:
            self.configTempUpdated.emit(v/10)
        except Exception:
            print("Erro ao emitir sinal configTempUpdated")

def parse_temperature_response(self, r: bytes):
    if len(r) >= 7:
        raw = (r[3] << 8) | r[4]
        t = raw / 10
```

```

self.lbl_temp.setText(f"Temp: {t:.1f}°C")
color = "red" if t>100 else "green"
self.lbl_temp.setStyleSheet(f"color:{color};")
# emite sinal de temperatura atual
try:
    self.temperatureUpdated.emit(t)
except Exception as e:
    print(f"Erro ao emitir sinal de temperatura: {e}")

```

```

def parse_supply_pressure(self, r: bytes):
    if len(r)>=5:
        raw = (r[3]<<8)|r[4]
        p = raw/10
        self.lbl_press1.setText(f"Supply: {p:.1f}")
        # emite sinal de pressão supply
        self.supplyPressureUpdated.emit(p)

```

```

def parse_open_pressure(self, r: bytes):
    if len(r)>=5:
        raw = (r[3]<<8)|r[4]
        p = raw/10
        self.lbl_press2.setText(f"Open: {p:.1f}")
        # emite sinal de pressão open
        self.openPressureUpdated.emit(p)

```

```

def read_initial_parameters(self):
    """
    Lê os registradores de configuração e popula:
    – modo (0x0040)
    – unidade (0x0041)
    – heat1 (0x0042)
    – open (0x0043)
    – close (0x0044)
    – temp1 (0x0047)
    """
    for reg, parser in [
        (0x0040, self.parse_mode_response),
        (0x0041, self.parse_unit_response),
        (0x0042, self.parse_heat1_response),
        (0x0043, lambda r: self.parse_time_response(r, 'open')),
        (0x0044, lambda r: self.parse_time_response(r, 'close')),
        (0x0047, lambda r: self.parse_tempset_response(r, 1)),
    ]:
        self.read_register(reg, parser, "Leitura Config")

```

```

def monitor_device(self):
    for reg, parser, desc in [
        (0x0048, self.parse_temperature_response, "Temp"),
        (0x0049, self.parse_supply_pressure, "Supply"),
        (0x004A, self.parse_open_pressure, "Open"),
    ]:

```

```

        self.read_register(reg, parser, desc)

# -----
# Parsers para atualização dos campos de configuração
# -----
def parse_mode_response(self, r: bytes):
    if len(r) >= 5:
        val = (r[3] << 8) | r[4]
        if 0 <= val < self.cmb_mode.count():
            self.cmb_mode.setCurrentIndex(val)

def parse_unit_response(self, r: bytes):
    if len(r) >= 5:
        val = (r[3] << 8) | r[4]
        if 0 <= val < self.cmb_unit.count():
            self.cmb_unit.setCurrentIndex(val)

def parse_heat1_response(self, r: bytes):
    if len(r) >= 5:
        val = (r[3] << 8) | r[4]
        if 0 <= val < self.cmb_heat1.count():
            self.cmb_heat1.setCurrentIndex(val)
        # emite sinal com valor inteiro de Canal1 (0 ou 1)
        self.heatStateChanged.emit(val)

def parse_time_response(self, r: bytes, which: str):
    if len(r) >= 5:
        val = (r[3] << 8) | r[4]
        if which == 'open':
            self.spin_open.setValue(val)
        elif which == 'close':
            self.spin_close.setValue(val)

def parse_tempset_response(self, r: bytes, idx: int):
    if len(r) >= 5:
        val = (r[3] << 8) | r[4]
        self.spin_t1.setValue(val)
        # emite sinal de temperatura configurada (°C)
        try:
            self.configTempUpdated.emit(val/10)
        except Exception as e:
            print(f"Erro ao emitir sinal de temperatura configurada: {e}")

# -----
# Sempre que a janela for exibida, relê configurações
# -----
def showEvent(self, event):
    super().showEvent(event)
    if self.serial_connection:
        self.read_initial_parameters()

```

File: testes_comunicacao_e_comandos.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
import sys
import time
import json
import cv2
from pathlib import Path
from plate_flow import PlateFlowManager
from inspection_logger import InspectionLogger
from PyQt6.QtWidgets import QSystemTrayIcon, QSplashScreen
from PyQt6.QtGui import QIcon, QPixmap
from virtual_home_dialog import ConfigHomeDialog
from axis_calibration_dialog import AxisCalibrationDialog
from settings_manager import SettingsManager
from camera_manager import CameraManager
from alignment_dialog import AlignmentDialog
# widgets externos importados
from table_program_tab import TableProgramTab
from axes_control_tab import AxesControlTab
from movement_controls_widget import MovementControlsWidget
from position_status_widget import PositionStatusWidget
from inspection_positions_widget import InspectionPositionsWidget
from program_io_widget import ProgramIOWidget
from positions_backend import InspectionPositionsBackend
from sequence_control import (
    SequenceControlWidget, InspectionPosition, MotionBackend
)

from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QLabel, QSpinBox, QPushButton,
                             QGroupBox, QGridLayout, QTabWidget, QTextEditor,
                             QFrame, QSizePolicy, QCheckBox, QScrollArea,
                             QMessageBox)
from PyQt6.QtCore import Qt, QTimer, pyqtSignal, QObject, QThread, QSize
from PyQt6.QtCore import pyqtSlot
from PyQt6.QtGui import QFont
from pymodbus.client import ModbusTcpClient
from PyQt6.QtCore import QCoreApplication

#####
# QDialog para configurar URL / Posto / Usuário da API do InspectionLogger
#####
from PyQt6.QtWidgets import (
    QDialog, QFormLayout, QLineEdit, QPushButton, QMessageBox,
    QRadioButton
)
from PyQt6.QtCore import Qt

class ProcessConfigDialog(QDialog):
    """
```

Diálogo não-modal e always-on-top para ligar/desligar módulos:

- Fiducial
- Leitura de Código
- Inspeção
- Temperatura
- Comunicação SFCS

.....

```
def __init__(self, parent=None):
    super().__init__(parent, flags=Qt.WindowType.Window)
    self.setWindowTitle("Configuração de Processo")
    self.setWindowModality(Qt.WindowModality.NonModal)
    self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)
    self._ctrl = parent
    self._build_ui()
```

```
def _build_ui(self):
    layout = QVBoxLayout(self)
    # Carrega configurações persistidas em settings.json
    proc_cfg = self._ctrl.settings.get_config("process_config", {})
    # Definição dos cinco processos
    itens = [
        ("Fiducial", "fiducial"),
        ("Leitura de Código", "barcode"),
        ("Inspeção", "inspect"),
        ("Temperatura", "temperature"),
        ("Comunicação SFCS", "sfcs")
    ]
    self._groups = {}
    for título, chave in itens:
        box = QGroupBox(título)
        row = QHBoxLayout(box)
        rb_on = QRadioButton("Ligado")
        rb_off = QRadioButton("Desligado")
        # Estado inicial conforme valor salvo (default = ligado)
        if proc_cfg.get(chave, True):
            rb_on.setChecked(True)
        else:
            rb_off.setChecked(True)
        row.addWidget(rb_on)
        row.addWidget(rb_off)
        layout.addWidget(box)
        self._groups[chave] = (rb_on, rb_off)

    # Botões Salvar/Cancelar
    btns = QHBoxLayout()
    btn_save = QPushButton("Salvar")
    btn_cancel = QPushButton("Cancelar")
    btns.addStretch()
    btns.addWidget(btn_save)
    btns.addWidget(btn_cancel)
    layout.addLayout(btns)
```

```
btn_save.clicked.connect(self._on_save)
btn_cancel.clicked.connect(self.close)
```

```
def _on_save(self):
    # Lê estados e persiste
    cfg = self._ctrl.settings.get_config("process_config", {})
    for chave, (rb_on, rb_off) in self._groups.items():
        cfg[chave] = rb_on.isChecked()
    self._ctrl.settings.set_config("process_config", cfg)
    self._ctrl.log("■ Config. de Processo salva")
    self.close()
```

```
# Override exec_ para não bloquear
```

```
def exec_(self):
    self.show()
    self.raise_()
    self.activateWindow()
```

```
class ApiConfigDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent, flags=Qt.WindowType.Window)
        self.setWindowModality(Qt.WindowModality.NonModal)
        self.setWindowTitle("Configuração da API")
        self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)
        self._ctrl = parent
        self._build_ui()
```

```
def _build_ui(self):
    form = QFormLayout(self)
    # URL
    url = self._ctrl.settings.data.get('api_url', "")
    self.edit_url = QLineEdit(url)
    form.addRow("URL da API:", self.edit_url)
    # Posto
    posto = self._ctrl.settings.data.get('json_posto', "")
    self.edit_posto = QLineEdit(posto)
    form.addRow("Posto SFCS:", self.edit_posto)
    # Usuário
    usuario = self._ctrl.settings.data.get('json_usuario', "")
    self.edit_usuario = QLineEdit(usuario)
    form.addRow("Usuário SFCS:", self.edit_usuario)
    # Botão Salvar
    btn_save = QPushButton("Salvar")
    btn_save.clicked.connect(self._on_save)
    form.addRow(btn_save)
```

```
def _on_save(self):
    url = self.edit_url.text().strip()
    posto = self.edit_posto.text().strip()
    usuario = self.edit_usuario.text().strip()
```

```

# persiste no SettingsManager
s = self._ctrl.settings
s.data['api_url'] = url
s.data['json_posto'] = posto
s.data['json_usuario'] = usuario
s.save()
# atualiza InspectionLogger em tempo real
log = self._ctrl.inspection_logger
log.api_url = url
log.json_posto = posto
log.json_usuario = usuario
QMessageBox.information(self, "Configuração API",
                        "Configurações salvas com sucesso.")
self.close()

```

class MultiAxisMotorController(QMainWindow):

```

# -----
# S I G N A L (garante acesso aos widgets só no thread GUI)
# -----
logRequested = pyqtSignal(str) # emitido por qualquer thread
def __init__(self):
    super().__init__()
    self.client = None
    self.connected = False
    # configurações persistentes
    self.settings = SettingsManager()
    # logger de inspeção, injeta config manager para api_url, posto, usuário
    self.inspection_logger = InspectionLogger(
        parent=self,
        config_manager=self.settings
    )

# Endereços Modbus CORRIGIDOS
self.addresses = {
    # Memórias M - Endereços corrigidos conforme ladder real
    # EIXO Y2
    'M0_Y2': 0, # ATRIBUI ZERO A POSIÇÃO ATUAL (Y2)
    'M50_Y2': 50, # INICIA O MOVIMENTO ABSOLUTO (Y2)
    'M100_Y2': 100, # ACIONA LÓGICA DE COMPARAÇÃO (Y2)
    # EIXO Y1
    'M500_Y1': 500, # ATRIBUI ZERO A POSIÇÃO ATUAL (Y1)
    'M550_Y1': 550, # INICIA O MOVIMENTO ABSOLUTO (Y1)
    'M600_Y1': 600, # ACIONA LÓGICA DE COMPARAÇÃO (Y1)
    # EIXO Z
    'M1500_Z': 1500, # ATRIBUI ZERO A POSIÇÃO ATUAL (Z)
    'M1550_Z': 1550, # INICIA O MOVIMENTO ABSOLUTO (Z)
    'M1600_Z': 1600, # ACIONA LÓGICA DE COMPARAÇÃO (Z)
    # EIXO X
    'M1000_X': 1000, # ATRIBUI ZERO A POSIÇÃO ATUAL (X)
    'M1050_X': 1050, # INICIA O MOVIMENTO ABSOLUTO (X)
    'M1100_X': 1100, # ACIONA LÓGICA DE COMPARAÇÃO (X)
}

```

APLICADORA DE ADESIVO

'M5000': 5000, # Trigger aplicadora de adesivo

'M5001': 5001, # Auxiliar aplicadora

HOMING - Busca pelo HOME

'M350': 350, # GO TO HOME Y2

'M850': 850, # GO TO HOME Y1

'M1350': 1350, # GO TO HOME X

'M1850': 1850, # GO TO HOME Z

HOMING REALIZADO - Status do homing

'M300': 300, # HOMING REALIZADO Y2

'M800': 800, # HOMING REALIZADO Y1

'M1300': 1300, # HOMING REALIZADO X

'M1800': 1800, # HOMING REALIZADO Z

SISTEMA JOG TODOS OS EIXOS (conforme nova estrutura)

Y2

'M70_Y2': 70, # JOG POSITIVO Y2

'M80_Y2': 80, # JOG NEGATIVO Y2

'M10_Y2': 10, # INTERROMPE JOG LIMITE POSITIVO Y2

'M11_Y2': 11, # INTERROMPE JOG LIMITE NEGATIVO Y2

Y1

'M570_Y1': 570, # JOG POSITIVO Y1

'M580_Y1': 580, # JOG NEGATIVO Y1

'M510_Y1': 510, # INTERROMPE JOG LIMITE POSITIVO Y1

'M511_Y1': 511, # INTERROMPE JOG LIMITE NEGATIVO Y1

X

'M1070_X': 1070, # JOG POSITIVO X

'M1080_X': 1080, # JOG NEGATIVO X

'M1010_X': 1010, # INTERROMPE JOG LIMITE POSITIVO X

'M1011_X': 1011, # INTERROMPE JOG LIMITE NEGATIVO X

Z

'M1570_Z': 1570, # JOG POSITIVO Z

'M1580_Z': 1580, # JOG NEGATIVO Z

'M1510_Z': 1510, # INTERROMPE JOG LIMITE POSITIVO Z

'M1511_Z': 1511, # INTERROMPE JOG LIMITE NEGATIVO Z

botões / sinal de conclusão do fluxo de placa

'M20': 20, # botão Mesa 1

'M21': 21, # done Mesa 1

'M22': 22, # reset homing realizado (desativa M300/M800/M1300/M1800)

'M30': 30, # botão Mesa 2

'M31': 31, # done Mesa 2

Pulso para Start / Stop global via aba Controle de Eixos

'M40': 40, # Start global cycle

'M41': 41, # Stop global cycle

Pulsos para botões "Enviar" nas mesas

'M42': 42, # Enviar – Mesa 1

'M43': 43, # Enviar – Mesa 2

MEMÓRIAS AUXILIARES HOMING (conforme ladder real)

Y2

'M311': 311, 'M312': 312, 'M313': 313, 'M314': 314, 'M315': 315,
'M360': 360,

Y1

'M811': 811, 'M812': 812, 'M813': 813, 'M814': 814, 'M815': 815,
'M860': 860,

X

'M1311': 1311, 'M1312': 1312, 'M1313': 1313, 'M1314': 1314, 'M1315': 1315,
'M1360': 1360,

Z

'M1811': 1811, 'M1812': 1812, 'M1813': 1813, 'M1814': 1814, 'M1815': 1815,
'M1860': 1860,

TEMPORIZADORES HOMING

'T0': 0, 'T1': 1, 'T2': 2, 'T3': 3, 'T4': 4, 'T5': 5, 'T6': 6, 'T7': 7,

VELOCIDADES JOG (nova estrutura D22000 série)

'D22000': 22000, # VELOCIDADE JOG POSITIVO Y2
'D22010': 22010, # VELOCIDADE JOG POSITIVO Y1
'D22020': 22020, # VELOCIDADE JOG POSITIVO X
'D22030': 22030, # VELOCIDADE JOG POSITIVO Z
'D22050': 22050, # VELOCIDADE JOG NEGATIVO Y2
'D22060': 22060, # VELOCIDADE JOG NEGATIVO Y1
'D22070': 22070, # VELOCIDADE JOG NEGATIVO X
'D22080': 22080, # VELOCIDADE JOG NEGATIVO Z
'D22100': 22100, # TEMPO DE ACELERAÇÃO JOG
'D22110': 22110, # TEMPO DE DESACELERAÇÃO JOG

LIMITES SALVOS EM ROM (novos conforme planilha)

'D23050': 23050, # LIMITE NEGATIVO Y2 (ROM)
'D23060': 23060, # LIMITE NEGATIVO Y1 (ROM)
'D23070': 23070, # LIMITE NEGATIVO X (ROM)
'D23080': 23080, # LIMITE NEGATIVO Z (ROM)
'D23090': 23090, # LIMITE POSITIVO Y2 (ROM)
'D23100': 23100, # LIMITE POSITIVO Y1 (ROM)
'D23110': 23110, # LIMITE POSITIVO X (ROM)
'D23120': 23120, # LIMITE POSITIVO Z (ROM)

VELOCIDADES HOMING (conforme planilha)

'D23000': 23000, # VELOCIDADE HOMING Y2 (ROM)
'D23010': 23010, # VELOCIDADE HOMING Y1 (ROM)
'D23020': 23020, # VELOCIDADE HOMING X (ROM)
'D23030': 23030, # VELOCIDADE HOMING Z (ROM)
'D450': 450, # VELOCIDADE HOMING Y2 (aplicada)
'D950': 950, # VELOCIDADE HOMING Y1 (aplicada)
'D1450': 1450, # VELOCIDADE HOMING X (aplicada)
'D1950': 1950, # VELOCIDADE HOMING Z (aplicada)

Registradores D - Corrigidos conforme ladder real

```

# EIXO Y2
'D0_Y2': 0,      # MOVE TO ABS (Y2)
'D50_Y2': 50,    # LIMITE NEGATIVO Y2 (trabalho)
'D23050_Y2': 23050, # LIMITE NEGATIVO Y2 (ROM)
'D100_Y2': 100,   # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (Y2) - ENTRADA
'D150_Y2': 150,   # POSIÇÃO ABSOLUTA DEFINIDA (Y2)
'D20000_Y2': 20000, # VELOCIDADE DE DESLOCAMENTO (Y2)
# EIXO Y1 - NOVO
'D500_Y1': 500,   # MOVE TO ABS (Y1)
'D550_Y1': 550,   # LIMITE NEGATIVO Y1 (trabalho)
'D23060_Y1': 23060, # LIMITE NEGATIVO Y1 (ROM)
'D600_Y1': 600,   # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (Y1) - ENTRADA
'D650_Y1': 650,   # POSIÇÃO ABSOLUTA DEFINIDA (Y1)
'D20500_Y1': 20500, # VELOCIDADE DE DESLOCAMENTO (Y1)
# EIXO Z
'D1500_Z': 1500,  # MOVE TO ABS (Z)
'D1550_Z': 1550,  # LIMITE NEGATIVO Z (trabalho)
'D23080_Z': 23080, # LIMITE NEGATIVO Z (ROM)
'D1600_Z': 1600,  # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (Z) - ENTRADA
'D1650_Z': 1650,  # POSIÇÃO ABSOLUTA DEFINIDA (Z)
'D21500_Z': 21500, # VELOCIDADE DE DESLOCAMENTO (Z)
# EIXO X
'D1000_X': 1000,  # MOVE TO ABS (X)
'D1050_X': 1050,  # LIMITE NEGATIVO X (trabalho)
'D23070_X': 23070, # LIMITE NEGATIVO X (ROM)
'D1100_X': 1100,  # POSIÇÃO ABSOLUTA DEFINIDA PELO USUÁRIO (X) - ENTRADA
'D1150_X': 1150,  # POSIÇÃO ABSOLUTA DEFINIDA (X)
'D21000_X': 21000, # VELOCIDADE DE DESLOCAMENTO (X)

# ----- DOT CONTROL -----
# D-registradores para frequência e quantidade de aplicação
# (mudaram de D4000 / D4100 → D24000 / D24100 no ladder)
'D24000_FREQ': 24000, # Frequência (Hz) dos dots
'D24100_QTY': 24100,  # Quantidade de dots

# Registradores de status (leitura)
'D3100_Y2': 3100,  # POSIÇÃO ATUAL MOTOR Y2 (cópia do SR460)
'D3400_Y1': 3400,  # POSIÇÃO ATUAL MOTOR Y1 (cópia do SR500)
'D3200_Z': 3200,   # POSIÇÃO ATUAL MOTOR Z (cópia do SR480)
'D3000_X': 3000,   # POSIÇÃO ATUAL MOTOR X (cópia do SR520)

# Saídas Y (coils) – endereços Modbus oficiais (AS-Series Manual, Tabela 7-3)
'Y00': 40960, 'Y01': 40961,  # Y0.0 / Y0.1 eixo-Y2
'Y02': 40962, 'Y03': 40963,  # Y0.2 / Y0.3 eixo-Z
'Y04': 40964, 'Y05': 40965,  # Y0.4 / Y0.5 eixo-Y1
'Y06': 40966, 'Y07': 40967,  # Y0.6 / Y0.7 eixo-X
'Y010': 40970,                # Y0.10 (0xA00A) aplicadora de adesivo <<< FIX

# Entradas X (sensores)
'X04_Y2': 8196,  # X0.4 sensor homing Y2
'X06_Y1': 8198,  # X0.6 sensor homing Y1

```

```

        'X08_X': 8200,    # X0.8 sensor homing X
        'X09_Z': 8201,    # X0.9 sensor homing Z
    }
    # Posição atual dos eixos
    self.current_positions = {
        'Y2': 0,
        'Y1': 0,
        'Z' : 0,
        'X' : 0
    }

    # ----- CONTEXTO DO PROJETO -----
    self.prog_mgr      = None # AdhesiveProgramManager ou None
    self.current_proj_name = None # string ou None

    # -----
    # SPINBOX "FANTASMA" PARA CADA EIXO (necessário para o backend PLC)
    # -----
    from PyQt6.QtWidgets import QSpinBox
    for _axis in ('Y2', 'Y1', 'X', 'Z'):
        if not hasattr(self, f'pulsos_spin_{_axis}'):
            sb = QSpinBox()
            sb.setRange(-2_147_483_648, 2_147_483_647)
            sb.setValue(0)
            setattr(self, f'pulsos_spin_{_axis}', sb)

    # ----- NOVO: lista de registradores configuráveis -----
    # (qualquer outro pode ser acrescentado facilmente depois)
    self.configurable_registers = [
        # ---- LIMITES SALVOS EM ROM ----
        ('D23050', 'Limite- Y2 (ROM)'), ('D23090', 'Limite+ Y2 (ROM)'),
        ('D23060', 'Limite- Y1 (ROM)'), ('D23100', 'Limite+ Y1 (ROM)'),
        ('D23070', 'Limite- X (ROM)'), ('D23110', 'Limite+ X (ROM)'),
        ('D23080', 'Limite- Z (ROM)'), ('D23120', 'Limite+ Z (ROM)'),
        # ---- VELOCIDADE DE DESLOCAMENTO ----
        ('D20000_Y2', 'Vel. Desloc. Y2'),
        ('D20500_Y1', 'Vel. Desloc. Y1'),
        ('D21000_X', 'Vel. Desloc. X'),
        ('D21500_Z', 'Vel. Desloc. Z'),
        # ---- VELOCIDADE DE HOMING (ROM) ----
        ('D23000', 'Vel. Homing Y2'),
        ('D23010', 'Vel. Homing Y1'),
        ('D23020', 'Vel. Homing X'),
        ('D23030', 'Vel. Homing Z'),
        # ---- VELOCIDADE JOG ----
        ('D22000', 'Jog+ Y2'), ('D22050', 'Jog- Y2'),
        ('D22010', 'Jog+ Y1'), ('D22060', 'Jog- Y1'),
        ('D22020', 'Jog+ X'), ('D22070', 'Jog- X'),
        ('D22030', 'Jog+ Z'), ('D22080', 'Jog- Z'),
        ('D22100', 'Tempo Acel. Jog'),
        ('D22110', 'Tempo Desac. Jog'),
    ]

```

```

# ----- DOT -----
('D24000_FREQ', 'Frequência dot (Hz)'),
('D24100_QTY', 'Quantidade dot')
]

# ----- limites de trabalho (padrão) + persistência ---
# ----- carrega configurações persistentes -----
self.settings = SettingsManager()
self.table_limits = self.settings.table_limits

# -----
# OFFSET CÂMERA ↔ NOZZLE deve existir ANTES do backend PLC
# -----
self.camera_nozzle_offset = self.settings.cam_noz_offset

# -----
# Diretório-base dos projetos (...\\Projetos\\modelos\\...)
# -----
self.projects_dir = self.settings.projects_dir
self.projects_dir.mkdir(parents=True, exist_ok=True)

# ----- câmera -----
self.camera_manager = CameraManager(self.settings.camera_index)

# aplica calibração se existir
a,b = self.settings.focus_coeffs
self.camera_manager.load_calibration(a,b)

# backend de movimento criado uma única vez e reutilizado
self._plc_motion_backend = PLCMotionBackend(self)
# ----- STEPS/MM -----
self.steps_per_mm: dict[str,float] = self.settings.axis_steps
# ----- escala da câmera (mm/pixel) -----
self.mm_per_pixel: float = self.settings.camera_mm_per_pixel

# ----- campo-de-visão (dois pontos) -----
self.camera_fov: dict = self.settings.camera_fov
# coeficientes lineares largura =  $aX + b$  (idem altura)
self._fov_coeffs = self._calc_fov_coeffs()

# ---- homing virtual -----
self.virtual_home: dict[int,dict[str,int]] = self.settings.virtual_home
self.init_ui()

# -----
# M U T E X   D A   C A B E Ç A
# None = cabeça livre
# 1/2 = número da mesa que detém a cabeça
# -----
self._head_busy: int | None = None

```

```

# Ao iniciar, criação/edição de programa fica BLOQUEADA
QTimer.singleShot(0, lambda: self._set_creation_controls_enabled(False))
self.connect_plc()
# -----
# Inicializa controladora de aplicação de adesivo em segundo plano
# para leitura de temperatura sem exibir janela.
try:
    from aplicadora_tab import AdhesiveApplicatorTab
    # instancia o tab (já conecta serial se houver porta salva),
    # mas NÃO chama .show()
    self.applicator_window = AdhesiveApplicatorTab(self)
    # conecta os sinais de temperatura e pressões/heater à aba Controle de Eixos
    self.applicator_window.temperatureUpdated.connect(
        self.control_tab.update_applicator_current_temp)
    self.applicator_window.configTempUpdated.connect(
        self.control_tab.update_applicator_config_temp)
    self.applicator_window.supplyPressureUpdated.connect(
        self.control_tab.update_applicator_supply)
    self.applicator_window.openPressureUpdated.connect(
        self.control_tab.update_applicator_open)
    self.applicator_window.heatStateChanged.connect(
        self.control_tab.update_applicator_heater)
except Exception as e:
    self.log(f"■ Falha ao inicializar controladora de aplicação: {e}")

# Lê velocidades salvas na ROM após conectar
QTimer.singleShot(1500, self.read_initial_jog_velocities)
QTimer.singleShot(2500, self.read_initial_limits)
QTimer.singleShot(3000, self.read_current_positions)
QTimer.singleShot(3500, self.read_initial_homing_status)

# Controle de teclas do teclado
self.keyboard_jog_active = {
    'X': False,
    'Y2': False,
    'Y1': False,
    'Z': False
} # Flags para controlar se JOG está ativo via teclado para cada eixo
self.setFocusPolicy(Qt.FocusPolicy.StrongFocus) # Permite capturar teclas

# Timer para atualização
# ----- logger seguro -----
self.logRequested.connect(self._append_log) # slot GUI

self.timer = QTimer()
self.timer.timeout.connect(self.update_status)
self.timer.start(500)

# --- inicializa timer de blink do botão Go to Zero ---
self._gozero_blink_timer = QTimer(self)

```

```

self._gozero_blink_timer.setInterval(500) # ms
self._gozero_blink_timer.timeout.connect(self._toggle_go_to_zero_alert)
self._gozero_alert_on = False

# ===== LOG thread-safe =====
def log(self, msg: str):
    """
    Pode ser chamado de QUALQUER thread.
    No thread GUI a mensagem é escrita imediatamente;
    nos demais threads emite-se logRequested.
    """
    ts = time.strftime("%H:%M:%S")
    full = f"[{ts}] {msg}"
    print(full, flush=True)
    if QThread.currentThread() is self.thread(): # GUI?
        self._append_log(full)
    else:
        self.logRequested.emit(full)

def _append_log(self, text: str):
    """Slot executado SEMPRE no thread GUI"""
    if not hasattr(self, "log_text"):
        return
    self.log_text.append(text)
    # mantém no máx. 100 linhas
    if self.log_text.document().lineCount() > 100:
        cursor = self.log_text.textCursor()
        cursor.movePosition(cursor.MoveOperation.Start)
        cursor.select(cursor.SelectionType.LineUnderCursor)
        cursor.removeSelectedText()

def init_ui(self):
    self.setWindowTitle("Controle Multi-Eixos - Delta AS (ENDEREÇOS LADDER REAIS)")
    self.setMinimumSize(900, 700)

    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    layout = QVBoxLayout(central_widget)

    # ----- MENU -----
    menubar = self.menuBar()
    # ----- MENU "ARQUIVO" -----
    menu_file = menubar.addMenu("Arquivo")
    act_new = menu_file.addAction("Novo Projeto")
    act_open = menu_file.addAction("Abrir Projeto...")
    act_save = menu_file.addAction("Salvar Projeto")
    act_saveas = menu_file.addAction("Salvar Como...")
    menu_file.addSeparator()
    act_exit = menu_file.addAction("Sair")

    act_new.triggered.connect(self._new_project)

```

```
act_open.triggered.connect(self._open_project)
act_save.triggered.connect(self._save_project)
act_saveas.triggered.connect(self._save_as_project)
act_exit.triggered.connect(self.close)
menu_cfg = menubar.addMenu("Configurações")

# ===== NOVO ■ PREFERÊNCIAS =====
menu_pref = menubar.addMenu("Preferências")
act_setdir = menu_pref.addAction("Definir caminho Modelos...")
act_setdir.triggered.connect(self._set_models_path)
menu_prog = menubar.addMenu("Programa")
act_limits = menu_cfg.addAction("Limites de mesa...")
act_limits.triggered.connect(self._open_limits_dialog)

# ---- submenu câmera -----
menu_cam = menu_cfg.addMenu("Câmera")
act_select = menu_cam.addAction("Selecionar câmera...")
act_reopen = menu_cam.addAction("Reconectar")
act_focus = menu_cam.addAction("Calibrar foco...")
act_fov = menu_cam.addAction("Calibração campo de visão...")
act_select.triggered.connect(self._select_camera_dialog)
act_reopen.triggered.connect(lambda: self.camera_manager.open(self.settings.camera_index))
act_focus.triggered.connect(self._open_focus_dialog)
act_fov.triggered.connect(self._open_fov_dialog)

# ----- CALIBRAÇÃO EIXOS -----
act_calib = menu_cfg.addAction("Calibração dos Eixos...")
act_calib.triggered.connect(self._open_axis_calib_dialog)

# ----- HOMING VIRTUAL -----
act_homecfg = menu_cfg.addAction("Homing Virtual...")
act_homecfg.triggered.connect(self._open_virtual_home_dialog)

# ----- ALINHAMENTO NOZZLE/CÂMERA -----
act_align = menu_cfg.addAction("Alinhamento Nozzle↔Câmera...")
act_align.triggered.connect(self._open_alignment_dialog)

# ação para abrir controladora de aplicação ---
act_applicator = menu_cfg.addAction("Controladora de aplicação")
act_applicator.triggered.connect(self._open_applicator_controller)
# >>> NOVO: ação de configuração da API de inspeção
act_api = menu_cfg.addAction("config. API")
act_api.triggered.connect(self._open_api_config)

# >>> NOVO: ação de configuração de processo
act_proc_cfg = menu_cfg.addAction("Config. de Processo")
act_proc_cfg.triggered.connect(self._open_process_config)
# ■■■ Diálogo: Altura de Visualização Dot ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
act_dot_view = menu_cfg.addAction("Altura de Visualização Dot...")
act_dot_view.triggered.connect(self._open_dot_view_height_dialog)
```

```

# ----- DOT PATTERNS -----
act_dots = menu_prog.addAction("Padrões de Dots...")
act_dots.triggered.connect(self._open_dot_dialog)

# ----- NOVO – CONVERSOR DE MESA -----
act_conv = menu_prog.addAction("Converter programa da outra mesa...")
act_conv.triggered.connect(self._open_converter_dialog)

# ----- BANNER DE CONEXÃO (rodapé) -----
# Usa o status-bar nativo do QMainWindow (aparece no rodapé).
self.status_bar = self.statusBar()      # QStatusBar
self.status_bar.setSizeGripEnabled(False)
self.status_bar.showMessage("Desconectado")
# Adapt to system theme
self.status_bar.setAutoFillBackground(True)
self.status_bar.setPalette(self.palette())

# Abas para organizar
# -----
# TAB WIDGET principal fica em self.tab_widget para acesso global
# -----
self.tab_widget = QTabWidget()

# === ABA 1: CONTROLE DOS EIXOS (principal) ===
self.control_tab = AxesControlTab(self, primary=True)
self.tab_widget.addTab(self.control_tab, "Controle de Eixos")

# === ABA 2: STATUS ===
status_tab = QWidget()
status_layout = QVBoxLayout(status_tab)

# Status das saídas
outputs_group = self.create_outputs_status()
status_layout.addWidget(outputs_group)

# Status das memórias
memories_group = self.create_memories_status()
status_layout.addWidget(memories_group)

# Sensores de HOMING (entradas X0.4 / X0.6 / X0.8 / X0.9)
sensors_group = self.create_homing_sensors_status()
status_layout.addWidget(sensors_group)

# Status dos registradores
registers_group = self.create_registers_status()
status_layout.addWidget(registers_group)

self.tab_widget.addTab(status_tab, "Status do Sistema")

# === ABA 3: CONFIGURAÇÃO DE REGISTRADORES (NOVA) ===
config_tab = self.create_config_tab()

```

```

self.tab_widget.addTab(config_tab, "Config. Registradores")

# === ABA 4: LOG ===
log_tab = QWidget()
log_layout = QVBoxLayout(log_tab)

log_label = QLabel("Log de Eventos:")
log_label.setFont(QFont("Arial", 10, QFont.Weight.Bold))
log_layout.addWidget(log_label)

self.log_text = QTextEdit()
self.log_text.setReadOnly(True)
self.log_text.setFont(QFont("Consolas", 9))
log_layout.addWidget(self.log_text)

clear_log_btn = QPushButton("Limpar Log")
clear_log_btn.clicked.connect(self.log_text.clear)
log_layout.addWidget(clear_log_btn)

self.tab_widget.addTab(log_tab, "Log")

# === ABA 5: MESA 1 =====
self.mesa_tabs = {}
mesa1_tab = TableProgramTab(self, mesa_id=1)
self.mesa_tabs[1] = mesa1_tab
self.tab_widget.addTab(mesa1_tab, "Mesa 1")

# === ABA 6: MESA 2 =====
mesa2_tab = TableProgramTab(self, mesa_id=2)
self.mesa_tabs[2] = mesa2_tab
self.tab_widget.addTab(mesa2_tab, "Mesa 2")

layout.addWidget(self.tab_widget)

# --- gerentes de fluxo de placa -----

self.flow_mesa1 = PlateFlowManager(self, 1, self.mesa_tabs[1])
self.flow_mesa2 = PlateFlowManager(self, 2, self.mesa_tabs[2])
# flag de execução geral
self._global_cycle_active = False

# ----- rastreia arquivo atual do projeto -----
self._current_project_file = None
# ----- registra todos os ProgramIOWidgets -----
self._register_prog_widget(self.prog_io_widget)      # controle de eixos
self._register_prog_widget(mesa1_tab.prog_widget)
self._register_prog_widget(mesa2_tab.prog_widget)

# =====
# CONVERSOR MESA1 ■ MESA2
# =====

```

```

def _open_converter_dialog(self):
    """Abre (ou traz à frente) a janela de conversão de programa."""
    w = self.tab_widget.currentWidget()
    if w not in self.mesa_tabs.values():
        QMessageBox.warning(self, "Converter",
                            "Selecione primeiro a aba Mesa 1 ou Mesa 2.")
        return
    if not hasattr(self, "_conv_dialog"):
        from program_converter_dialog import ProgramConverterDialog
        self._conv_dialog = ProgramConverterDialog(self, w)
    else:
        # actualiza referência da aba actual
        self._conv_dialog.mesa_tab = w
        self._conv_dialog.setWindowTitle(
            f"Conversor de Programa – Mesa {w.mesa}")
    self._conv_dialog.show()
    self._conv_dialog.raise_()
    self._conv_dialog.activateWindow()

# -----
# Helpers para ProgramIOWidget por aba
# -----

def _register_prog_widget(self, widget):
    """Acopla sinais de load/save para lembrar o último arquivo usado."""
    widget._last_file = None
    widget.fileSaved.connect(lambda f, w=widget: setattr(w, "_last_file", f))
    # ■■■ NOVO ■■■
    # sempre que um programa for CARREGADO por este widget
    # (botão "Carregar Programa" nas abas), ajusta estado global
    widget.fileLoaded.connect(
        lambda f, w=widget: self._on_program_loaded(f, w))
    # mantém lista para futura atualização do default_dir
    if not hasattr(self, "_prog_widgets"):
        self._prog_widgets = []
    self._prog_widgets.append(widget)

# =====
# handler único para qualquer ProgramIOWidget.fileLoaded
# =====

def _on_program_loaded(self, filepath: str, widget):
    """
    • cria (ou re-usa) AdhesiveProgramManager para a pasta do arquivo;
    • grava current_proj_name;
    • habilita imediatamente todos os botões de criação/edição.
    """
    try:
        from adhesive_program_manager import AdhesiveProgramManager
        path = Path(filepath).resolve()
        prog_name = path.stem
        self.prog_mgr = AdhesiveProgramManager(path.parent)
    try:

```

```

        # tenta criar; se já existir apenas re-usa
        self.prog_mgr.create_program(prog_name, overwrite=False)
except FileExistsError:
    # programa já existe → apenas aponta o diretório
    self.prog_mgr.model_dir = (
        self.prog_mgr.base_dir / "modelos" / prog_name)
self.current_proj_name = prog_name

# memoriza no widget para futuros “Salvar”
widget._last_file = str(path)

# libera botões de criação/edição
self._set_creation_controls_enabled(True)

self.log(f"■ Projeto “{prog_name}” carregado: {path.name}")
except Exception as exc:
    self.log(f"■ Erro ao processar programa carregado: {exc}")

# =====
#   ALINHAMENTO NOZZLE ↔ CÂMERA   (offset X,Y)
# =====
def _open_alignment_dialog(self):

    dlg = AlignmentDialog(self, self)      # não modal
    dlg.offsetSaved.connect(self._save_nozzle_offset)
    dlg.show()

def _open_applicator_controller(self):
    """Abre janela flutuante da controladora de aplicação de adesivo."""
    try:
        from aplicadora_tab import AdhesiveApplicatorTab
        # instancia apenas uma vez
        if not hasattr(self, 'applicator_window'):
            self.applicator_window = AdhesiveApplicatorTab(self)
            # conecta sinais de temperatura à aba Controle de Eixos
            try:
                self.applicator_window.temperatureUpdated.connect(
                    self.control_tab.update_applicator_current_temp)
                self.applicator_window.configTempUpdated.connect(
                    self.control_tab.update_applicator_config_temp)
            except Exception:
                pass
            self.applicator_window.setWindowTitle("Controladora de aplicação")
            # faz com que a janela fique sempre acima da principal
            self.applicator_window.setWindowFlags(
                self.applicator_window.windowFlags()
                | Qt.WindowType.Window
                | Qt.WindowType.WindowStaysOnTopHint
            )
        # mostra sem bloquear (não modal), traz para frente
        self.applicator_window.show()

```

```

self.applicator_window.raise_()
self.applicator_window.activateWindow()
# auto■seleciona a última porta e conecta sem precisar clicar
last = self.settings.data.get('applicator_last_port')
if last:
    # recarrega portas e tenta posicionar no último valor
    self.applicator_window.refresh_ports()
    idx = self.applicator_window.port_combo.findData(last)
    if idx >= 0:
        self.applicator_window.port_combo.setCurrentIndex(idx)
    # reconecta só se ainda não estiver conectado
    if not self.applicator_window.serial_connection:
        self.applicator_window.connect_serial()
except Exception as e:
    QMessageBox.critical(
        self,
        "Erro",
        f"Não foi possível abrir a controladora de aplicação:\n{e}"
    )

```

```

def _save_nozzle_offset(self, off: dict[str, int]):
    """Salvo em memória e disco."""
    self.camera_nozzle_offset = off
    self.settings.cam_noz_offset = off
    self.settings.save()
    self.log(f"■ Offset câmera↔nozzle salvo: ΔX={off['x']} ΔY={off['y']}")

    # informa backend caso já exista
    if hasattr(self, "_plc_motion_backend"):
        self._plc_motion_backend._off_x = off.get("x", 0)
        self._plc_motion_backend._off_y = off.get("y", 0)

```

```

def _current_prog_widget(self):
    """Devolve o ProgramIOWidget associado à aba visível."""
    w = self.tab_widget.currentWidget()
    if w is self.control_tab:
        return self.prog_io_widget
    for mesa_id, tab in self.mesa_tabs.items():
        if w is tab:
            return tab.prog_widget
    # fallback
    return self.prog_io_widget

```

```

# =====
# Conversões pulsos ↔ milímetros
# =====

def pulses_from_mm(self, axis: str, mm: float) -> int:
    """Converte mm → pulsos utilizando a calibração do eixo."""
    return int(round(mm * self.steps_per_mm.get(axis, 1.0)))

# ----- NOVO: pixels → pulsos -----

```

```

def pulses_from_pixels(self, dx_pix: float, dy_pix: float, z_pos: int,
                        y_axis: str = "Y1") -> tuple[int,int]:
    """
    Converte deslocamento em pixels (no frame da câmera) em pulsos
    dos eixos X e Y dados o z_pos (pulsos Z).
    """

    # calcula mm/pixel a partir dos coef. do FOV
    c = self._calc_fov_coeffs()
    w_mm = c["aX"]*z_pos + c["bX"]
    h_mm = c["aY"]*z_pos + c["bY"]
    mmpp_x = w_mm / self.camera_manager._cap.get(cv2.CAP_PROP_FRAME_WIDTH)
    mmpp_y = h_mm / self.camera_manager._cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
    dx_mm = dx_pix * mmpp_x
    dy_mm = dy_pix * mmpp_y
    dx_p = self.pulses_from_mm("X", dx_mm)
    dy_p = self.pulses_from_mm(y_axis, dy_mm)
    return dx_p, dy_p

def _set_axis_steps(self, axis: str, steps: float):
    """Atualiza fator steps/mm de um eixo e persiste em settings."""
    self.steps_per_mm[axis] = float(steps)
    self.settings.axis_steps = self.steps_per_mm
    self.settings.save()
    self.log(f"■ Calibração {axis}: {steps:.3f} steps/mm salva")

# =====
# HOMING VIRTUAL
# =====

def _open_virtual_home_dialog(self):
    # cria uma única instância reutilizável
    if not hasattr(self, '_vh_dialog'):
        from virtual_home_dialog import ConfigHomeDialog
        self._vh_dialog = ConfigHomeDialog(self, self)
        self._vh_dialog.acceptedAndSaved.connect(self._save_virtual_home)
    # mostra janela não modal, mantendo-a na frente
    self._vh_dialog.show()
    self._vh_dialog.raise_()
    self._vh_dialog.activateWindow()

# ----- DOTS -----

def _open_dot_dialog(self):
    if not hasattr(self, "_dot_dialog"):
        from dot_patterns_dialog import DotPatternsDialog
        self._dot_dialog = DotPatternsDialog(self)
    self._dot_dialog.show()
    self._dot_dialog.raise_(); self._dot_dialog.activateWindow()

def _save_virtual_home(self):
    self.virtual_home = self._vh_dialog.get_home_dict()
    self.settings.virtual_home = self.virtual_home
    self.settings.save()

```

```

self.log("■ Homing virtual atualizado")

def goto_virtual_home(self, mesa: int):
    """Usado pelas abas Mesa 1/Mesa 2."""
    home = self.virtual_home.get(mesa, {"x":0,"y":0})
    x_tgt = home["x"]
    y_tgt = home["y"]
    # calcula diferenças
    dx = x_tgt - self.current_positions["X"]
    y_axis = 'Y1' if mesa == 1 else 'Y2'
    dy = y_tgt - self.current_positions[y_axis]
    if abs(dx) > 0:
        self.move_relative('X', dx)
    if abs(dy) > 0:
        self.move_relative(y_axis, dy)
    self.log(f"■ Mesa {mesa}: cabeça movida para Home virtual")

# =====
# AÇÕES DO MENU "ARQUIVO"
# =====

def _new_project(self):
    """
    Inicia um NOVO programa.
    Passos:
    1. usuário já está na aba Mesa 1 ou Mesa 2;
    2. pergunta nome do programa;
    3. cria imediatamente modelos/<nome>/... (estrutura base);
    4. limpa listas e libera botões de criação.
    """
    from PyQt6.QtWidgets import QInputDialog
    mesa_tab = self.tab_widget.currentWidget()
    if mesa_tab not in self.mesa_tabs.values():
        QMessageBox.warning(self, "Novo projeto",
                            "Antes de criar um projeto, selecione a aba Mesa 1 ou Mesa 2.")
        return

    proj_name, ok = QInputDialog.getText(
        self, "Novo Projeto",
        "Nome do novo programa:",
        text=""
    )
    if not ok or not proj_name.strip():
        return # cancelado
    proj_name = proj_name.strip()

    # Cria a pasta-raiz imediatamente
    try:
        from adhesive_program_manager import AdhesiveProgramManager
        # garante .../<Projetos>/ existente
        self.projects_dir.mkdir(parents=True, exist_ok=True)
        self.prog_mgr = AdhesiveProgramManager(self.projects_dir)

```

```

# overwrite=True → recria estrutura completa sempre
self.proj_root = self.proj_mgr.create_program(proj_name, overwrite=True)
self.current_proj_name = proj_name
self.log(f"■■■ Estrutura base criada: {self.proj_root}")
except FileExistsError:
    QMessageBox.warning(self, "Projeto existente",
                        f"O programa \"{proj_name}\" já existe.")
    return
except Exception as exc:
    self.log(f"■ Erro ao criar estrutura do projeto: {exc}")
    QMessageBox.critical(self, "Erro", str(exc))
    return

```

```

# Limpa listas das abas
for tab in self.mesa_tabs.values():
    tab.inspect_widget.clear()

```

```

# Libera botões de criação / ação
self._set_creation_controls_enabled(True)
self._current_project_file = None
self.log(f"■■■ Novo projeto \"{proj_name}\" iniciado")

```

```

# =====
# DEFINIR DIRETÓRIO BASE (Preferências ■ Definir caminho Modelos)
# =====
def _set_models_path(self):
    from PyQt6.QtWidgets import QFileDialog
    sel = QFileDialog.getExistingDirectory(
        self, "Escolha a pasta onde será criado \"Projetos\"")
    if not sel:
        return
    base = Path(sel).resolve()
    projetos_path = base / "Projetos"
    try:
        projetos_path.mkdir(parents=True, exist_ok=True)
    except Exception as exc:
        QMessageBox.critical(self, "Erro",
                            f"Não foi possível criar \"Projetos\":\n{exc}")
        return
    # persiste
    self.projects_dir = projetos_path
    self.settings.projects_dir = projetos_path
    self.settings.save()
    # atualiza diretório-padrão nos ProgramIOWidgets já existentes
    for w in getattr(self, "_prog_widgets", []):
        if hasattr(w, "_default_dir"):
            w._default_dir = str(projetos_path)
    self.log(f"■ Diretório de projetos definido para: {projetos_path}")
    QMessageBox.information(self, "Preferências",
                            f"Caminho configurado:\n{projetos_path}")

```

```

# --- salvar / abrir utilizando ProgramIOWidget já existente -----
def _save_project(self):
    """Salva arquivo referente à aba ativa."""
    if not self.current_proj_name or not self.projects_dir:
        QMessageBox.warning(self, "Salvar",
                             "Nenhum projeto em edição ou pasta padrão indefinida.")
    return
    widget = self._current_prog_widget()
    # sufixo .m1 / .m2 conforme backend
    suffix = getattr(widget, "_default_suf", ".json")
    mesa_id = 1 if suffix.endswith("1") else 2
    dest = self.projects_dir / f"{self.current_proj_name}{suffix}"
    try:
        ok = widget._backend.save_to_file(str(dest))
        if ok:
            widget._last_file = str(dest)
            self.log(f"Projeto salvo em {dest}")
            QMessageBox.information(self, "Salvar",
                                   f"Projeto salvo com sucesso em:\n{dest}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o projeto.")
    except Exception as exc:
        QMessageBox.critical(self, "Erro", str(exc))

def _save_as_project(self):
    """Abre diálogo de 'Salvar Como...' reaproveitando ProgramIOWidget."""
    widget = self._current_prog_widget()
    widget._on_save_clicked()

def _open_project(self):
    """Abre projeto (diálogo de arquivo) via ProgramIOWidget oculto."""
    widget = self._current_prog_widget()
    old_enabled = widget.isEnabled()
    widget._on_load_clicked()
    # se carregou com sucesso (_last_file setado), habilita botões
    if getattr(widget, "_last_file", None):
        # garante ponteiro do gerente de programa
        try:
            from adhesive_program_manager import AdhesiveProgramManager
            proj_path = Path(widget._last_file).resolve()
            proj_name = proj_path.stem
            self.prog_mgr = AdhesiveProgramManager(proj_path.parent)
            try:
                self.prog_mgr.create_program(proj_name, overwrite=False)
            except FileExistsError:
                # estrutura já criada anteriormente → ok
                self.prog_mgr.model_dir = (
                    self.prog_mgr.base_dir / "modelos" / proj_name)
                self.current_proj_name = proj_name
        except Exception:
            QMessageBox.critical(self, "Erro",

```

```

        "Não foi possível abrir o programa selecionado.")
        pass
        self._set_creation_controls_enabled(True)

# -----
# Habilita / desabilita botões de criação (ActionSelector + Add)
# -----
def _set_creation_controls_enabled(self, enabled: bool):
    """
    Ativa ou bloqueia os botões que só devem ser usados quando um
    projeto está em edição (novo ou aberto).
    """
    for mesa_tab in self.mesa_tabs.values():
        # selector de ação
        mesa_tab.action_selector.setEnabled(enabled)
        # botão "Adicionar posição atual"
        mesa_tab.inspect_widget.btn_add.setEnabled(enabled)
    # movimento / salvar posição da aba principal
    if hasattr(self, 'inspect_widget'):
        self.inspect_widget.btn_add.setEnabled(enabled)

def _open_focus_dialog(self):
    from focus_calibration_dialog import FocusCalibrationDialog
    dlg = FocusCalibrationDialog(self, self)
    if dlg.exec():
        # pega coeficientes
        z0, f0, z1, f1 = dlg._z_top, dlg._f_top, dlg._z_bot, dlg._f_bot
        a = (f1 - f0) / (z1 - z0)
        b = f0 - a*z0
        self.camera_manager.load_calibration(a, b)
        self.settings.focus_coeffs = (a, b)
        self.settings.save()
        self.log("Calibração de foco salva")
        # agora que existe calibração, liga auto-focus
        self.camera_manager.enable_auto_focus(True)

# ----- diálogo FOV -----
def _open_fov_dialog(self):
    from fov_calibration_dialog import FOVCalibrationDialog
    dlg = FOVCalibrationDialog(self.settings, self)
    if dlg.exec():
        # recarrega as constantes na instância
        self.camera_fov = self.settings.camera_fov
        self._fov_coeffs = self._calc_fov_coeffs()
        self.log("Calibração de campo-de-visão salva")

# -----
# Calcula coeficientes lineares campo_mm(z)=a*z+b
# -----
def _calc_fov_coeffs(self) -> dict[str, float]:
    f = self.camera_fov

```

```

z0 = float(f.get("z0_z_pulses", 0))
z1 = float(f.get("z1_z_pulses", 700))
w0 = float(f.get("z0_width_mm", 61.0))
w1 = float(f.get("z1_width_mm", 30.5))
h0 = float(f.get("z0_height_mm", 45.0))
h1 = float(f.get("z1_height_mm", 22.5))
if z1 == z0:      # garante denom.
    z1 += 1
aX = (w1 - w0) / (z1 - z0)
bX = w0 - aX * z0
aY = (h1 - h0) / (z1 - z0)
bY = h0 - aY * z0
return {"aX":aX, "bX":bX, "aY":aY, "bY":bY}

```

----- diálogo calibração eixos -----

```
def _open_axis_calib_dialog(self):
```

```

    dlg = AxisCalibrationDialog(self, self)
    dlg.exec()

```

----- diálogo de limites -----

```
def _open_limits_dialog(self):
```

```

    from config_dialogs import WorkAreaConfigDialog
    dlg = WorkAreaConfigDialog(self.table_limits, self)
    if dlg.exec():
        self.table_limits = dlg.get_limits()
        # envia às abas
        for mesa, tab in self.mesa_tabs.items():
            tab.set_limits(self.table_limits[mesa])
        self.settings.table_limits = self.table_limits
        self.settings.save()
        self.log("Limites de mesa atualizados pelo usuário")

```

----- câmera -----

```
def _select_camera_dialog(self):
```

```

    from PyQt6.QtWidgets import QInputDialog
    idx, ok = QInputDialog.getInt(self, "Selecionar câmera",
                                  "Índice da câmera (0 = padrão):",
                                  value=self.settings.camera_index, min=0, max=10)
    if ok:
        self.settings.camera_index = idx
        self.settings.save()
        self.camera_manager.open(idx)
        self.log(f"Câmera {idx} selecionada")

```

----- persistência em JSON -----

```
def _load_limits_from_disk(self):
```

```

    try:
        if self._limits_file.exists():
            data = json.loads(self._limits_file.read_text(encoding="utf-8"))
            # valida estrutura mínima

```

```

        if all(str(k) in ("1", "2") for k in data):
            self.table_limits = {int(k): v for k, v in data.items()}
            self.log("Limites de mesa carregados do disco")
    except Exception as exc:
        self.log(f"Erro ao ler limites salvos: {exc}")

```

```

def _save_limits_to_disk(self):
    try:
        self._limits_file.write_text(
            json.dumps(self.table_limits, indent=2), encoding="utf-8")
        self.log("Limites de mesa gravados em table_limits.json")
    except Exception as exc:
        self.log(f"Erro ao salvar limites: {exc}")

```

```

def create_homing_sensors_status(self):
    """Exibe o estado ON/OFF dos sensores físicos de HOME"""
    group = QGroupBox("SENSORES DE HOME (Entradas X)")
    layout = QGridLayout()

    sensors = [
        ('X04_Y2', 'Sensor HOME Y2 (X0.4)'),
        ('X06_Y1', 'Sensor HOME Y1 (X0.6)'),
        ('X08_X', 'Sensor HOME X (X0.8)'),
        ('X09_Z', 'Sensor HOME Z (X0.9)')
    ]

    for i, (addr_key, desc) in enumerate(sensors):
        layout.addWidget(QLabel(f"{desc}:"), i, 0)
        lbl = QLabel("OFF")
        lbl.setStyleSheet("QLabel { background-color: gray; color: white; padding: 5px; }")
        layout.addWidget(lbl, i, 1)
        setattr(self, f'{addr_key}_status', lbl)

    group.setLayout(layout)
    return group

```

```

def create_config_tab(self):
    """Aba que exibe TODOS os registradores configuráveis em formato editável"""
    tab = QWidget()
    vbox = QVBoxLayout(tab)

    # Scroll para caber em qualquer resolução
    scroll = QScrollArea()
    scroll.setWidgetResizable(True)
    scroll_content = QWidget()
    grid = QGridLayout(scroll_content)

    # Cria um spinBox 32 bits para cada registrador listado em self.configurable_registers
    for row, (reg_key, desc) in enumerate(self.configurable_registers):
        addr = self.addresses.get(reg_key)
        if addr is None:
            continue # pula se endereço ainda não mapeado

```

```

# Coluna 0 – label descritivo
grid.addWidget(QLabel(f"{reg_key} ({desc})"), row, 0)

# Coluna 1 – spinBox (-2 147 483 648 a 2 147 483 647)
spin = QSpinBox()
spin.setRange(-2_147_483_648, 2_147_483_647)
spin.setObjectName(f"cfg_spin_{reg_key}")
grid.addWidget(spin, row, 1)

# Coluna 2 – botão “Gravar”
btn = QPushButton("Gravar")
btn.clicked.connect(lambda _=False, rk=reg_key: self.write_single_config(rk))
grid.addWidget(btn, row, 2)

```

```

scroll_content.setLayout(grid)
scroll.addWidget(scroll_content)
vbox.addWidget(scroll)

```

```

# Botões globais
h = QHBoxLayout()
ler_btn = QPushButton("Ler Todos")
gravar_btn = QPushButton("Gravar Todos")
ler_btn.clicked.connect(self.read_all_config)
gravar_btn.clicked.connect(self.write_all_config)
h.addStretch()
h.addWidget(ler_btn)
h.addWidget(gravar_btn)
vbox.addLayout(h)

```

```

# Leitura inicial automática quando a aba é criada
QTimer.singleShot(200, self.read_all_config)
return tab

```

```

# -----
# Funções auxiliares da aba de configuração
# -----
def read_all_config(self):
    """Lê todos os registradores configuráveis e atualiza os spinBoxes"""
    if not self.connected:
        return
    for reg_key, _ in self.configurable_registers:
        addr = self.addresses.get(reg_key)
        if addr is None:
            continue
        try:
            value = self.read_dword(addr)
            spin: QSpinBox = self.findChild(QSpinBox, f"cfg_spin_{reg_key}")
            if spin:
                spin.blockSignals(True)
                spin.setValue(value)

```

```

        spin.blockSignals(False)
    except Exception as exc:
        self.log(f"ð Falha ao ler {reg_key}: {exc}")
        continue
    self.log("üÖ Configuração – leitura concluída")

```

```

def write_single_config(self, reg_key):
    """Grava somente o registrador indicado"""
    if not self.connected:
        self.log("ü CLP não conectado")
        return
    addr = self.addresses.get(reg_key)
    if addr is None:
        return
    spin: QSpinBox = self.findChild(QSpinBox, f'cfg_spin_{reg_key}')
    if spin is None:
        return
    value = spin.value()
    try:
        res = self.write_dword(addr, value)
        if not res.isError():
            self.log(f"üÖ {reg_key} gravado: {value}")
        else:
            self.log(f"ð Falha ao gravar {reg_key}")
    except Exception as e:
        self.log(f"ð Erro ao gravar {reg_key}: {e}")

```

```

def write_all_config(self):
    """Grava TODOS os registradores configuráveis"""
    for reg_key, _ in self.configurable_registers:
        self.write_single_config(reg_key)
    self.log("üÖ Configuração – gravação concluída")

```

```

def create_axis_control(self, title, axis_name):
    """Cria controle para um eixo """
    group = QGroupBox(title)
    # use native font instead of stylesheet
    font = group.font()
    font.setBold(True)
    font.setPointSize(12)
    group.setFont(font)
    layout = QVBoxLayout()

    # Parâmetros
    params_frame = QFrame()
    params_layout = QGridLayout(params_frame)

    # Mapeamento de endereços corrigido
    addr_map = {
        'Y2': (f'D100_{axis_name}', f'D20000_{axis_name}'), # Entrada usuário, Velocidade
        'Y1': (f'D600_{axis_name}', f'D20500_{axis_name}'), # Entrada usuário, Velocidade
    }

```

```

        'Z': (f'D1600_{axis_name}', f'D21500_{axis_name}'), # Entrada usuário, Velocidade
        'X': (f'D1100_{axis_name}', f'D21000_{axis_name}') # Entrada usuário, Velocidade
    }
    pulsos_addr, vel_addr = addr_map[axis_name]

```

```

# Apenas posição absoluta (velocidade passou para aba de configuração)
pulsos_label = f"{pulsos_addr} - Posição Absoluta:"
pulsos_tooltip = "Coordenada absoluta de destino"
default_pulsos = 0

```

```

# Posição absoluta – 32 bits (aceita valores negativos)
params_layout.addWidget(QLabel(pulsos_label), 0, 0)
pulsos_spin = QSpinBox()
pulsos_spin.setRange(-2_000_000_000, 2_000_000_000)
pulsos_spin.setValue(default_pulsos)
pulsos_spin.setToolTip(pulsos_tooltip)
params_layout.addWidget(pulsos_spin, 0, 1)
setattr(self, f'pulsos_spin_{axis_name}', pulsos_spin)

```

```

# Display da posição atual
sr_addr_map = {'Y2': 'D3100_Y2', 'Y1': 'D3400_Y1', 'Z': 'D3200_Z', 'X': 'D3000_X'}

```

```

params_layout.addWidget(QLabel(f"{sr_addr_map[axis_name]} - Posição Atual:"), 1, 0)
current_pos_label = QLabel("0")
current_pos_label.setStyleSheet("QLabel { background-color: lightblue; padding: 2px; font-weight: bold; max-height: 20px; }")
params_layout.addWidget(current_pos_label, 1, 1)

```

```

setattr(self, f'current_pos_label_{axis_name}', current_pos_label)

```

```

# Indicador LED de HOMING REALIZADO
params_layout.addWidget(QLabel("Status Homing:"), 2, 0)
homing_led = QLabel("●")
homing_led.setStyleSheet("QLabel { background-color: gray; color: gray; padding: 2px; font-size: 16px; font-weight: bold; }")
homing_led.setAlignment(Qt.AlignmentFlag.AlignCenter)
homing_led.setToolTip("Cinza: Homing não realizado | Verde: Homing realizado")
params_layout.addWidget(homing_led, 2, 1)
setattr(self, f'homing_led_{axis_name}', homing_led)

```

```

layout.addWidget(params_frame)

```

```

# Botões de escrita
write_btn = QPushButton("Escrever Parâmetros")
write_btn.clicked.connect(lambda: self.write_axis_parameters(axis_name))
layout.addWidget(write_btn)

```

```

# Botões de movimento
move_frame = QFrame()
move_layout = QGridLayout(move_frame)

```

```

# Movimento absoluto principal
move_abs_btn = QPushButton("■ MOVER PARA POSIÇÃO ABSOLUTA")

```

```

# adapt to system theme
move_abs_btn.setAutoFillBackground(True)
move_abs_btn.setPalette(self.palette())
fb = move_abs_btn.font(); fb.setBold(True); move_abs_btn.setFont(fb)
move_abs_btn.clicked.connect(lambda: self.move_axis_absolute(axis_name))
move_layout.addWidget(move_abs_btn, 0, 0, 1, 2)

# Controles JOG para TODOS OS EIXOS (novo sistema)
if axis_name in ['Y2', 'Y1', 'X', 'Z']: # Todos os eixos têm JOG agora
    # Separador visual
    separator = QFrame()
    separator.setFrameStyle(QFrame.Shape.HLine | QFrame.Shadow.Sunken)
    move_layout.addWidget(separator, 1, 0, 1, 2)

    # Frame JOG
    jog_frame = QFrame()
    # use native styled panel
    jog_frame.setFrameShape(QFrame.Shape.StyledPanel)
    jog_frame.setFrameShadow(QFrame.Shadow.Raised)
    jog_frame.setLineWidth(2)
    jog_layout = QVBoxLayout(jog_frame)

    # Botões JOG com pressionar/soltar
    jog_buttons_layout = QHBoxLayout()

    # Botão JOG Negativo (à esquerda)
    jog_minus_btn = QPushButton(f"■ JOG {axis_name}-")
    jog_minus_btn.setAutoFillBackground(True)
    jog_minus_btn.setPalette(self.palette())
    fj = jog_minus_btn.font(); fj.setBold(True); jog_minus_btn.setFont(fj)
    jog_minus_btn.pressed.connect(lambda: self.jog_start(axis_name, '-'))
    jog_minus_btn.released.connect(lambda: self.jog_stop(axis_name))
    jog_buttons_layout.addWidget(jog_minus_btn)
    setattr(self, f'jog_minus_btn_{axis_name}', jog_minus_btn)

    # Botão JOG Positivo (à direita)
    jog_plus_btn = QPushButton(f"■ JOG {axis_name}+")
    jog_plus_btn.setAutoFillBackground(True)
    jog_plus_btn.setPalette(self.palette())
    fp = jog_plus_btn.font(); fp.setBold(True); jog_plus_btn.setFont(fp)
    jog_plus_btn.pressed.connect(lambda: self.jog_start(axis_name, '+'))
    jog_plus_btn.released.connect(lambda: self.jog_stop(axis_name))
    jog_buttons_layout.addWidget(jog_plus_btn)
    setattr(self, f'jog_plus_btn_{axis_name}', jog_plus_btn)

    jog_layout.addLayout(jog_buttons_layout)

move_layout.addWidget(jog_frame, 2, 0, 1, 2)

layout.addWidget(move_frame)

```

```
group.setLayout(layout)
return group
```

```
def create_auxiliary_controls(self):
    """Cria controles auxiliares - Y0.10 CORRIGIDO"""
    group = QGroupBox("CONTROLES AUXILIARES")
    group.setStyleSheet("QGroupBox { font-weight: bold; font-size: 12px; }")
    layout = QVBoxLayout()

    # Saída Y0.10 ■ comando tipo "falling-edge trigger"
    y010_btn = QPushButton("SHOT (M5000)")
    y010_btn.setCheckable(True)
    y010_btn.clicked.connect(self.pulse_y010)
    # adapt to system theme
    y010_btn.setAutoFillBackground(True)
    y010_btn.setPalette(self.palette())
    layout.addWidget(y010_btn)
    setattr(self, 'y010_btn', y010_btn)

    group.setLayout(layout)
    return group
```

```
def _open_api_config(self):
    """
    Abre (ou traz à frente) a janela de configuração da API:
    - URL
    - Posto SFCS
    - Usuário SFCS
    """
    if not hasattr(self, '_api_config_dialog'):
        self._api_config_dialog = ApiConfigDialog(self)
    dlg = self._api_config_dialog
    dlg.show()
    dlg.raise_()
    dlg.activateWindow()
```

```
# -----
```

```
def _open_dot_view_height_dialog(self):
    """
    Abre (ou traz à frente) o diálogo de altura de visualização para dots.
    """
    if not hasattr(self, '_dot_view_dialog'):
        from dot_view_height_dialog import DotViewHeightDialog
        self._dot_view_dialog = DotViewHeightDialog(self)
    dlg = self._dot_view_dialog
    # carrega valor salvo (0 se não existir)
    val = self.settings.data.get("dot_view_height", 0)
    dlg.spin.setValue(int(val))
    dlg.show()
    dlg.raise_()
    dlg.activateWindow()
```

```

def _open_process_config(self):
    """
    Abre (ou traz à frente) a janela de Config. de Processo.
    """
    if not hasattr(self, '_process_config_dialog'):
        self._process_config_dialog = ProcessConfigDialog(self)
    dlg = self._process_config_dialog
    dlg.show()
    dlg.raise_()
    dlg.activateWindow()

# =====
#          SEQUÊNCIA DE HOMING GERAL (Z → (Y2,Y1,X))
# =====

def home_all_axes(self):
    """
    1) Aciona homing do eixo Z (M1850) e aguarda 3 s.
    2) Depois aciona simultaneamente os homings de Y2 (M350), Y1 (M850) e X (M1350).
    Todos os coils são soltos 200 ms após o disparo (pulso momentâneo).
    """
    if not self.connected:
        self.log("■ CLP não conectado")
        return

    # Etapa 1 – homing apenas do Z
    self.log("■■ HOMING GERAL: acionando homing do eixo Z (M1850)...")
    self._pulse_coil('M1850')

    # Agenda etapa 2 para daqui a 3 s
    QTimer.singleShot(3000, self._home_remaining_axes)

def _home_remaining_axes(self):
    """Dispara homing de Y2, Y1 e X em paralelo e solta todos os coils."""
    try:
        self.log("■■ Acionando homing simultâneo de Y2, Y1 e X...")
        for mem in ['M350', 'M850', 'M1350']:
            self._pulse_coil(mem)

    except Exception as e:
        self.log(f"■ Erro na etapa 2 do homing geral: {e}")

# -----
# Helper interno – envia pulso de 20 ms em uma memória M
# -----

def _pulse_coil(self, mem_key):
    """Liga o coil por 20 ms e depois desliga (edge-trigger)."""
    try:
        addr = self.addresses[mem_key]
        self.client.write_coil(addr, True)
    
```

```

# Mantém ON por apenas 20 ms e depois desliga (falling-edge)
QTimer.singleShot(20,
    lambda a=addr: self.client.write_coil(a, False))
self.log(f"■ Pulso {mem_key} enviado (20 ms)")
except Exception as e:
    self.log(f"■ Falha ao pulsar {mem_key}: {e}")

```

```
def create_outputs_status(self):
```

```
    """Cria status das saídas"""
```

```
    group = QGroupBox("STATUS DAS SAÍDAS")
```

```
    layout = QGridLayout()
```

```
    outputs = [
```

```
        ('Y0.0', 'Pulso Eixo 1'),
```

```
        ('Y0.1', 'Dir Eixo 1'),
```

```
        ('Y0.2', 'Pulso Eixo 2'),
```

```
        ('Y0.3', 'Dir Eixo 2'),
```

```
        ('Y0.4', 'Pulso Eixo 3'),
```

```
        ('Y0.5', 'Dir Eixo 3'),
```

```
        ('Y0.6', 'Pulso Eixo 4'),
```

```
        ('Y0.7', 'Dir Eixo 4'),
```

```
        ('Y0.10', 'Y0.10 (SHOT)')
```

```
    ]
```

```
    for i, (name, desc) in enumerate(outputs):
```

```
        layout.addWidget(QLabel(f"{name}:"), i//2, (i%2)*2)
```

```
        status_label = QLabel("OFF")
```

```
        # adapt to system theme
```

```
        status_label.setAutoFillBackground(True)
```

```
        status_label.setPalette(self.palette())
```

```
        layout.addWidget(status_label, i//2, (i%2)*2 + 1)
```

```
        setattr(self, f'{name.replace(".", "_")}__status', status_label)
```

```
    layout.addWidget(QLabel(f"{desc}"), i//2, (i%2)*2 + 2)
```

```
    group.setLayout(layout)
```

```
    return group
```

```
def create_memories_status(self):
```

```
    """Cria status das memórias"""
```

```
    group = QGroupBox("STATUS DAS MEMÓRIAS")
```

```
    layout = QGridLayout()
```

```
    memories = ['M0_Y2', 'M50_Y2', 'M100_Y2', 'M500_Y1', 'M550_Y1', 'M600_Y1',
```

```
                'M1500_Z', 'M1550_Z', 'M1600_Z', 'M1000_X', 'M1050_X', 'M1100_X',
```

```
                'M5000', 'M5001',
```

```
                'M350', 'M850', 'M1350', 'M1850',
```

```
                'M300', 'M800', 'M1300', 'M1800',
```

```
                'M70_Y2', 'M570_Y1', 'M1070_X', 'M1570_Z']
```

```
    for i, mem in enumerate(memories):
```

```

layout.addWidget(QLabel(f'{mem}:'), i//4, (i%4)*2)
status_label = QLabel("OFF")
status_label.setAutoFillBackground(True)
status_label.setPalette(self.palette())
layout.addWidget(status_label, i//4, (i%4)*2 + 1)
setattr(self, f'{mem}_status', status_label)

```

```

group.setLayout(layout)
return group

```

```
def create_registers_status(self):
```

```
    """Cria status dos registradores"""
```

```
    group = QGroupBox("STATUS DOS REGISTRADORES")
```

```
    layout = QGridLayout()
```

```
    registers = [
```

```

        'D0_Y2', 'D100_Y2', 'D20000_Y2',    # Y2 (entrada usuário)
        'D500_Y1', 'D600_Y1', 'D20500_Y1',  # Y1 (entrada usuário)
        'D1500_Z', 'D1600_Z', 'D21500_Z',   # Z (entrada usuário)
        'D1000_X', 'D1100_X', 'D21000_X',   # X (entrada usuário)
        'D3100_Y2', 'D3400_Y1', 'D3200_Z', 'D3000_X', # Posições atuais - CORRIGIDO
        'D22000', 'D22010', 'D22020', 'D22030',    # Velocidades JOG
        'D23000', 'D23010', 'D23020', 'D23030',    # Velocidades Homing ROM
        'D23050', 'D23060', 'D23070', 'D23080',    # Limites Negativos ROM
        'D23090', 'D23100', 'D23110', 'D23120'    # Limites Positivos ROM

```

```
]
```

```
for i, reg in enumerate(registers):
```

```

    layout.addWidget(QLabel(f'{reg}:'), i//4, (i%4)*2)
    status_label = QLabel("0")
    status_label.setAutoFillBackground(True)
    status_label.setPalette(self.palette())
    layout.addWidget(status_label, i//4, (i%4)*2 + 1)
    setattr(self, f'{reg}_status', status_label)

```

```

group.setLayout(layout)
return group

```

```
def connect_plc(self):
```

```
    """Conecta ao CLP"""
```

```
    try:
```

```

        self.log("Conectando ao CLP...")
        self.client = ModbusTcpClient('192.168.1.5', port=502)

```

```
    if self.client.connect():
```

```

        self.connected = True
        self.status_bar.showMessage("■ CONECTADO: 192.168.1.5:502")
        self.status_bar.setPalette(self.palette())
        self.log("■ Conectado com sucesso!")
        self.log("■ Endereços corrigidos conforme ladder real")

```

```
    else:
```

```

self.connected = False
self.status_bar.showMessage("Desconectado")
self.status_bar.setPalette(self.palette())
self.log("■ Falha na conexão")

```

```

except Exception as e:

```

```

    self.connected = False
    self.status_bar.showMessage("Desconectado")
    self.status_bar.setStyleSheet(
        "QStatusBar { background-color:red; color:white; font-weight:bold; }")
    self.log(f"■ Erro na conexão: {e}")

```

```

def read_initial_jog_velocities(self):

```

```

    """Lê velocidades JOG atuais dos registradores D22000 série"""

```

```

    if not self.connected:

```

```

        return

```

```

    try:

```

```

        self.log("■ Lendo velocidades JOG atuais...")

```

```

        # Mapeamento dos registradores JOG

```

```

        jog_velocity_map = {
            'Y2': ('D22000', 'jog_velocity_spin_Y2'),
            'Y1': ('D22010', 'jog_velocity_spin_Y1'),
            'X': ('D22020', 'jog_velocity_spin_X'),
            'Z': ('D22030', 'jog_velocity_spin_Z')
        }

```

```

        for axis, (reg_key, spin_attr) in jog_velocity_map.items():

```

```

            try:

```

```

                jog_velocity = self.read_dword(self.addresses[reg_key])

```

```

            # Atualiza interface se spinbox existir

```

```

                if hasattr(self, spin_attr):
                    jog_spin = getattr(self, spin_attr)
                    jog_spin.setValue(abs(jog_velocity))
                    self.log(f"■ Velocidade JOG {axis}: {abs(jog_velocity)} Hz")

```

```

            except Exception as e:

```

```

                self.log(f"■ Erro ao ler velocidade JOG {axis}: {e}")
                continue

```

```

        except Exception as e:

```

```

            self.log(f"■ Erro geral na leitura das velocidades JOG: {e}")

```

```

def read_initial_limits(self):

```

```

    """Lê limites salvos na ROM"""

```

```

    if not self.connected:

```

```

        return

```

```

    try:

```

```

        self.log("■ Lendo limites da ROM...")

```

```
limits_map = {
    'Y2': ('D23050', 'D23090'), # Negativo, Positivo
    'Y1': ('D23060', 'D23100'), # Negativo, Positivo
    'X': ('D23070', 'D23110'), # Negativo, Positivo
    'Z': ('D23080', 'D23120') # Negativo, Positivo
}
```

```
for axis, (neg_reg, pos_reg) in limits_map.items():
    try:
        neg_limit = self.read_dword(self.addresses[neg_reg])
        pos_limit = self.read_dword(self.addresses[pos_reg])
        self.log(f"■ Limites {axis}: {neg_limit} a {pos_limit}")
```

```
    except Exception as e:
        self.log(f"■ Erro ao ler limites {axis}: {e}")
        continue
```

```
except Exception as e:
    self.log(f"■ Erro geral na leitura dos limites: {e}")
```

```
def read_initial_homing_status(self):
    """Lê status inicial das memórias de homing na inicialização"""
    if not self.connected:
        return
```

```
    try:
        self.log("■ Lendo status inicial das memórias de homing...")
```

```
    homing_memories = {
        'M300': 'Y2',
        'M800': 'Y1',
        'M1300': 'X',
        'M1800': 'Z'
    }
```

```
    for mem_name, axis in homing_memories.items():
        try:
            addr = self.addresses[mem_name]
            result = self.client.read_coils(addr, count=1)
```

```
            if not result.isError():
                state = result.bits[0]
```

```
            # Atualiza LED diretamente
            homing_led = getattr(self, f'homing_led_{axis}', None)
            if homing_led:
                self.update_homing_led(homing_led, state)
```

```
            # Log do status inicial
            status_text = "REALIZADO" if state else "NÃO REALIZADO"
```

```

self.log(f"■ Status inicial Homing {axis}: {status_text}")

# Inicializa variável de controle de mudança
setattr(self, f'_last_homing_status_M{mem_name[1:]}', state)

# FORÇA atualização do LED imediatamente
homing_led = getattr(self, f'homing_led_{axis}', None)
if homing_led:
    self.update_homing_led(homing_led, state)

except Exception as e:
    self.log(f"■ Erro ao ler status inicial {mem_name}: {e}")
    continue

self.log("■ Leitura inicial de homing concluída")

except Exception as e:
    self.log(f"■ Erro geral na leitura inicial de homing: {e}")

# =====
# SISTEMA DE CACHE PARA PRESERVAR JANELAS DE COMPARAÇÃO
# =====

def _save_current_comparison_windows(self, parent_item):
    """
    Salva o estado atual das janelas de comparação no cache
    associado à posição mecânica especificada
    """
    if parent_item is None or not self.roi_editor:
        return

    try:
        # Coleta todas as janelas atualmente no visor ROI
        current_windows = []
        for item in self.roi_editor.windows:
            if hasattr(item, 'comparison_name'):
                # Salva dados essenciais da janela
                window_data = {
                    'name': item.comparison_name,
                    'rect': {
                        'x': item.rect().x(),
                        'y': item.rect().y(),
                        'width': item.rect().width(),
                        'height': item.rect().height()
                    },
                    'scene_pos': {
                        'x': item.scenePos().x(),
                        'y': item.scenePos().y()
                    },
                    'tooltip': item.toolTip()
                }
        }

```

```

        current_windows.append(window_data)

    # Salva no cache
    self._comparison_windows_cache[parent_item] = current_windows

    if current_windows:
        self.log(f"■ {len(current_windows)} janelas de comparação salvas no cache para posição mecânica")

    except Exception as e:
        self.log(f"■■ Erro ao salvar janelas no cache: {e}")

def _load_comparison_windows_from_cache(self, parent_item):
    """
    Carrega as janelas de comparação do cache para a posição mecânica especificada
    """
    if parent_item is None or not self.roi_editor:
        return

    try:
        # Verifica se há janelas salvas para esta posição
        cached_windows = self._comparison_windows_cache.get(parent_item, [])

        if not cached_windows:
            return

        # Limpa janelas atuais do ROI editor (sem emitir sinais)
        self._clear_roi_editor_silently()

        # Recria cada janela salva
        for window_data in cached_windows:
            try:
                # Cria nova janela com os mesmos parâmetros
                rect_data = window_data['rect']
                new_item = self.roi_editor.add_window(
                    x=rect_data['x'],
                    y=rect_data['y'],
                    w=rect_data['width'],
                    h=rect_data['height'],
                    deletable=True
                )

                # Restaura propriedades
                new_item.comparison_name = window_data['name']
                new_item.setToolTip(window_data.get('tooltip', ''))

                # Adiciona à TreeView
                self._add_comparison_window_to_tree(parent_item, new_item)

            except Exception as e:
                self.log(f"■■ Erro ao restaurar janela {window_data.get('name', '?')}: {e}")
                continue

```

```

self.log(f"■ {len(cached_windows)} janelas de comparação restauradas do cache")

except Exception as e:
    self.log(f"■■ Erro ao carregar janelas do cache: {e}")

def _clear_roi_editor_silently(self):
    """Remove todas as janelas do ROI editor sem emitir sinais"""
    if not self.roi_editor:
        return

    try:
        # Remove itens da cena silenciosamente
        for item in self.roi_editor.windows[:]: # cópia para evitar modificação durante iteração
            try:
                if item.scene():
                    self.roi_editor.view.scene().removeItem(item)
                    self.roi_editor.windows.remove(item)
            except (RuntimeError, ValueError):
                # Item já foi removido ou não existe mais
                if item in self.roi_editor.windows:
                    self.roi_editor.windows.remove(item)

    except Exception as e:
        self.log(f"■■ Erro ao limpar ROI editor: {e}")

def _clear_cache_for_position(self, parent_item):
    """Remove do cache todas as janelas associadas a uma posição mecânica"""
    if parent_item in self._comparison_windows_cache:
        del self._comparison_windows_cache[parent_item]
        self.log(f"■■ Cache de janelas limpo para posição mecânica removida")

def read_current_positions(self):
    """Lê posições atuais dos motores (função dedicada)"""
    if not self.connected:
        return

    try:
        position_registers = {
            'D3100_Y2': 'Y2',
            'D3400_Y1': 'Y1',
            'D3200_Z': 'Z',
            'D3000_X': 'X'
        }

        for reg_name, axis in position_registers.items():
            try:
                addr = self.addresses[reg_name]
                value = self.read_dword(addr)

                # Atualiza posição e interface

```

```

self.current_positions[axis] = value
label = getattr(self, f'current_pos_label_{axis}', None)
if label:
    label.setText(str(value))

except Exception as e:
    self.log(f"■■■ Erro ao ler posição {axis} ({reg_name}): {e}")
    continue

```

```

except Exception as e:
    # Falha silenciosa para não afetar outras operações
    self.log(f"■■■ Erro ao ler posições: {e}")

```

```

def monitor_homing_status(self):
    """Função dedicada APENAS para monitorar status de homing em tempo real"""
    if not self.connected:
        return

    try:
        homing_memories = {
            'M300': 'Y2',
            'M800': 'Y1',
            'M1300': 'X',
            'M1800': 'Z'
        }

        for mem_name, axis in homing_memories.items():
            try:
                addr = self.addresses[mem_name]
                result = self.client.read_coils(addr, count=1)

                if not result.isError():
                    state = result.bits[0]

                    # Atualiza LED SEMPRE (sem verificação de mudança)
                    homing_led = getattr(self, f'homing_led_{axis}', None)
                    if homing_led:
                        current_led_state = "lime" in homing_led.styleSheet()

                        # Só atualiza se estado for diferente do LED atual
                        if (state and not current_led_state) or (not state and current_led_state):
                            self.update_homing_led(homing_led, state)
                            self.log(f"■■■ LED Homing {axis} atualizado: {'VERDE' if state else 'CINZA'}")

            except Exception as e:
                self.log(f"■■■ Erro ao ler status de homing {axis}: {e}")
                continue

    except Exception as e:
        self.log(f"■■■ Erro ao monitorar status de homing: {e}")
        pass

```

```

def test_homing_status(self):
    """Função de teste específica para verificar status das memórias de homing"""
    if not self.connected:
        return

    self.log("■ Testando status das memórias de homing...")

    homing_memories = {
        'M300': 'Y2',
        'M800': 'Y1',
        'M1300': 'X',
        'M1800': 'Z'
    }

    for mem_name, axis in homing_memories.items():
        try:
            addr = self.addresses[mem_name]
            result = self.client.read_coils(addr, count=1)

            if not result.isError():
                state = result.bits[0]
                status_text = "REALIZADO" if state else "NÃO REALIZADO"
                self.log(f"■ Homing {axis} ({mem_name}): {status_text}")

                # Força atualização do LED
                homing_led = getattr(self, f'homing_led_{axis}', None)
                if homing_led:
                    self.update_homing_led(homing_led, state)
                    self.log(f"■ LED {axis} forçado para: {'VERDE' if state else 'CINZA'}")
                else:
                    self.log(f"■ Erro ao ler {mem_name}: {result}")

            # Força uma atualização completa dos LEDs após o teste
            QTimer.singleShot(100, self.force_homing_leds_update)

            # Também força verificação dedicada
            QTimer.singleShot(200, self.force_homing_status_check)

        except Exception as e:
            self.log(f"■ Erro ao testar {mem_name}: {e}")

def force_homing_leds_update(self):
    """Força atualização imediata de todos os LEDs de homing"""
    try:
        # Reset das variáveis de controle para forçar atualização
        for axis in ['Y2', 'Y1', 'X', 'Z']:
            if hasattr(self, f'_last_led_status_{axis}'):
                delattr(self, f'_last_led_status_{axis}')
            if hasattr(self, f'_last_homing_status_M{axis}'):
                delattr(self, f'_last_homing_status_M{axis}')
    
```

```

self.log("■ Forçando atualização completa dos LEDs...")

# Força leitura imediata
QTimer.singleShot(50, self.monitor_homing_status)

except Exception as e:
    self.log(f"■ Erro ao forçar atualização: {e}")

def force_update_homing_leds(self):
    """Força atualização de todos os LEDs de homing"""
    self.log("■ Forçando atualização dos LEDs de homing...")
    self.test_homing_status()

# ----- helpers de 32 bits -----
def write_dword(self, address, value):
    """Escreve INT32 em dois registradores Modbus (little-endian)."""
    u32 = value & 0xFFFFFFFF
    lo = u32 & 0xFFFF
    hi = (u32 >> 16) & 0xFFFF
    return self.client.write_registers(address, [lo, hi])

def read_dword(self, address):
    """Lê INT32 assinado de dois registradores."""
    res = self.client.read_holding_registers(address, count=2)
    if res.isError():
        raise ValueError(res)
    lo, hi = res.registers
    u32 = (hi << 16) | lo
    return u32 if u32 < 0x8000_0000 else u32 - 0x1_0000_0000

# -----
def write_axis_parameters(self, axis_name):
    """Escreve parâmetros de um eixo"""
    if not self.connected:
        self.log("■ CLP não conectado")
        return

    try:
        # Obtém valores da interface
        p_spin = getattr(self, f'pulsos_spin_{axis_name}')
        pulsos_value = p_spin.value()

        # Mapeamento de endereços
        addr_map = {
            'Y2': (f'D100_{axis_name}', f'D20000_{axis_name}'),
            'Y1': (f'D600_{axis_name}', f'D20500_{axis_name}'),
            'Z': (f'D1600_{axis_name}', f'D21500_{axis_name}'),
            'X': (f'D1100_{axis_name}', f'D21000_{axis_name}')
        }
        p_addr_key, v_addr_key = addr_map[axis_name]

```

```

pulsos_addr = self.addresses[p_addr_key]
vel_addr = self.addresses[v_addr_key]

self.log(f"■ Escrevendo parâmetros Eixo {axis_name}: Posição={pulsos_value}")

# -----
# 1) POSIÇÃO ABSOLUTA (sempre grava)
# -----
if self.write_dword(pulsos_addr, pulsos_value).isError():
    self.log(f"■ Erro ao escrever posição {axis_name}")
    return

# -----
# 2) VELOCIDADE (novo) – pega do spinBox da aba
# Config. Registradores ou usa default 20000 Hz
# -----
try:
    vel_spin = self.findChild(QSpinBox, f"cfg_spin_{v_addr_key}")
    vel_value = vel_spin.value() if vel_spin else 20000
except Exception:
    vel_value = 20000

if self.write_dword(vel_addr, vel_value).isError():
    self.log(f"■ Erro ao escrever velocidade {axis_name}")
    return

# LOG
self.log(f"■ Eixo {axis_name}: pos={pulsos_value} vel={vel_value}")

# Verifica se posição ficou realmente gravada
self.verify_write(pulsos_addr, pulsos_value, p_addr_key)

except Exception as e:
    self.log(f"■ Erro ao escrever parâmetros eixo {axis_name}: {e}")

def verify_write(self, address, expected_value, name):
    """Verifica DWORD"""
    try:
        actual_value = self.read_dword(address)
        if actual_value == expected_value:
            self.log(f"✓ {name} verificado: {actual_value}")
        else:
            self.log(f"■ {name} divergente: esperado {expected_value}, lido {actual_value}")
    except Exception as e:
        self.log(f"■ Erro verificação {name}: {e}")

def move_axis_absolute(self, axis_name):
    """Move eixo para posição absoluta"""
    if not self.connected:
        self.log("■ CLP não conectado")
    return

```

```

try:
    # Escreve parâmetros primeiro
    self.write_axis_parameters(axis_name)

    # Comando de movimento
    cmd_map = {
        'Y2': f'M50_{axis_name}', # INICIA MOVIMENTO ABSOLUTO
        'Y1': f'M550_{axis_name}', # INICIA MOVIMENTO ABSOLUTO
        'Z': f'M1550_{axis_name}', # INICIA MOVIMENTO ABSOLUTO
        'X': f'M1050_{axis_name}' # INICIA MOVIMENTO ABSOLUTO
    }
    cmd_addr = self.addresses[cmd_map[axis_name]]

    target_position = getattr(self, f'pulsos_spin_{axis_name}').value()
    self.log(f"■ Movendo eixo {axis_name} para posição {target_position}")

    # envia pulso de 20 ms no coil → garante borda de subida
    self._pulse_coil(cmd_map[axis_name])
    self.log(f"■ Pulso {cmd_map[axis_name]} enviado – alvo {target_position}")

except Exception as e:
    self.log(f"■ Erro ao mover eixo {axis_name}: {e}")

def move_axis_home(self, axis_name):
    """Move eixo para HOME (zera posição)"""
    if not self.connected:
        self.log("■ CLP não conectado")
        return
    try:
        self.log(f"■ Zerando posição eixo {axis_name}")

        # Comando de busca pelo HOME - endereços corrigidos conforme planilha
        cmd_map = {
            'Y2': 'M350', # GO TO HOME Y2
            'Y1': 'M850', # GO TO HOME Y1
            'Z': 'M1850', # GO TO HOME Z
            'X': 'M1350' # GO TO HOME X
        }
        cmd_addr = self.addresses[cmd_map[axis_name]]
        result = self.client.write_coil(cmd_addr, True)
        if not result.isError():
            self.log(f"■ Comando HOME enviado para eixo {axis_name}")

    except Exception as e:
        self.log(f"■ Erro movimento HOME eixo {axis_name}: {e}")

def move_relative(self, axis_name, offset):
    """Movimento relativo a partir da posição atual"""

    if not self.connected:

```

```
self.log("■ CLP não conectado")
return
```

```
try:
```

```
    current_pos = self.current_positions[axis_name]
    new_position = current_pos + offset
```

```
    # Atualiza interface e executa movimento
    p_spin = getattr(self, f'pulsos_spin_{axis_name}')
    p_spin.setValue(new_position)
    self.move_axis_absolute(axis_name)
```

```
    self.log(f"■ Movimento relativo {axis_name}: {current_pos} + {offset} = {new_position}")
```

```
except Exception as e:
```

```
    self.log(f"■ Erro movimento relativo {axis_name}: {e}")
```

```
# SISTEMA JOG TODOS OS EIXOS
```

```
def set_jog_velocity(self, axis_name):
```

```
    """Define velocidade do JOG para qualquer eixo"""
```

```
    if not self.connected:
```

```
        self.log("■ CLP não conectado")
        return
```

```
    try:
```

```
        # Obtém velocidade: se spinbox existir usa-o; caso contrário
```

```
        # lê o valor já programado no CLP (mantém UI enxuta).
```

```
        if hasattr(self, f'jog_velocity_spin_{axis_name}')
```

```
            velocity = getattr(self, f'jog_velocity_spin_{axis_name}').value()
```

```
        else:
```

```
            pos_addr_key = {
                'Y2': 'D22000', 'Y1': 'D22010', 'X': 'D22020', 'Z': 'D22030'
            }[axis_name]
            velocity = abs(self.read_dword(self.addresses[pos_addr_key]))
```

```
    # Mapeamento dos registradores JOG por eixo
```

```
    jog_addr_map = {
        'Y2': ('D22000', 'D22050'), # Positivo, Negativo
        'Y1': ('D22010', 'D22060'), # Positivo, Negativo
        'X': ('D22020', 'D22070'), # Positivo, Negativo
        'Z': ('D22030', 'D22080') # Positivo, Negativo
    }
```

```
    pos_addr_key, neg_addr_key = jog_addr_map[axis_name]
```

```
    # Verifica se mudou antes de escrever
```

```
    current_velocity = self.read_dword(self.addresses[pos_addr_key])
```

```
    if current_velocity == velocity:
```

```
        self.log(f"■ Velocidade JOG {axis_name} já está em {velocity} Hz - não alterada")
        return
```

```

# Escreve velocidades positiva e negativa
result1 = self.write_dword(self.addresses[pos_addr_key], velocity) # Positiva
result2 = self.write_dword(self.addresses[neg_addr_key], -velocity) # Negativa

# Escreve tempos de aceleração/desaceleração (comuns para todos os eixos)
result3 = self.write_dword(self.addresses['D22100'], 1) # Tempo aceleração
result4 = self.write_dword(self.addresses['D22110'], 1) # Tempo desaceleração

if not result1.isError() and not result2.isError() and not result3.isError() and not result4.isError():
    self.log(f"■ Velocidade JOG {axis_name} definida: ±{velocity} Hz")
    # Verifica se foi escrito corretamente
    QTimer.singleShot(200, lambda: self.verify_jog_velocity(axis_name, velocity))
else:
    self.log(f"■ Erro ao definir velocidade JOG {axis_name}")

except Exception as e:
    self.log(f"■ Erro ao configurar velocidade JOG {axis_name}: {e}")

def verify_jog_velocity(self, axis_name, expected_velocity):
    """Verifica se a velocidade JOG foi escrita corretamente"""
    try:
        jog_addr_map = {
            'Y2': ('D22000', 'D22050'),
            'Y1': ('D22010', 'D22060'),
            'X': ('D22020', 'D22070'),
            'Z': ('D22030', 'D22080')
        }
        pos_addr_key, neg_addr_key = jog_addr_map[axis_name]
        actual_positive = self.read_dword(self.addresses[pos_addr_key])
        actual_negative = self.read_dword(self.addresses[neg_addr_key])

        if actual_positive == expected_velocity and actual_negative == -expected_velocity:
            self.log(f"✓ Velocidade JOG {axis_name} verificada: {pos_addr_key}={actual_positive}, {neg_addr_key}={actual_negative}")
        else:
            self.log(f"■ Velocidade JOG {axis_name} divergente: Esperado ±{expected_velocity}, Lido {actual_positive}/{actual_negative}")

    except Exception as e:
        self.log(f"■ Erro verificação velocidade JOG {axis_name}: {e}")

def jog_start(self, axis_name, direction):
    """Inicia movimento JOG contínuo para qualquer eixo"""
    if not self.connected:
        self.log("■ CLP não conectado")
        return

    try:
        # Configura velocidade automaticamente
        self.set_jog_velocity(axis_name)

        # Mapeamento de comandos JOG por eixo (conforme ladder real)

```

```

jog_cmd_map = {
    'Y2': ('M70_Y2', 'M80_Y2'), # JOG+, JOG-
    'Y1': ('M570_Y1', 'M580_Y1'), # JOG+, JOG-
    'X': ('M1070_X', 'M1080_X'), # JOG+, JOG-
    'Z': ('M1570_Z', 'M1580_Z') # JOG+, JOG-
}

pos_cmd, neg_cmd = jog_cmd_map[axis_name]

if direction == '+':
    result = self.client.write_coil(self.addresses[pos_cmd], True)
    direction_text = f"POSITIVO ({pos_cmd})"
    self.log(f"■ Acionando {pos_cmd} para JOG {axis_name}+")
else:
    result = self.client.write_coil(self.addresses[neg_cmd], True)
    direction_text = f"NEGATIVO ({neg_cmd})"
    self.log(f"■ Acionando {neg_cmd} para JOG {axis_name}-")

if not result.isError():
    self.log(f"■ JOG {axis_name} {direction_text} INICIADO")
    self.update_jog_status(axis_name, f"ATIVO - {direction_text}", "lime")
    # Verifica segurança apenas para informar o usuário (não bloqueia)
    self.check_jog_safety(axis_name)
else:
    self.log(f"■ Erro ao iniciar JOG {axis_name} {direction_text}")

except Exception as e:
    self.log(f"■ Erro JOG start {axis_name}: {e}")

def jog_stop(self, axis_name):
    """Para movimento JOG de qualquer eixo"""
    if not self.connected:
        return

    try:
        # Mapeamento de comandos JOG por eixo
        jog_cmd_map = {
            'Y2': ('M70_Y2', 'M80_Y2'),
            'Y1': ('M570_Y1', 'M580_Y1'),
            'X': ('M1070_X', 'M1080_X'),
            'Z': ('M1570_Z', 'M1580_Z')
        }

        pos_cmd, neg_cmd = jog_cmd_map[axis_name]

        # Para ambos os comandos JOG do eixo
        result1 = self.client.write_coil(self.addresses[pos_cmd], False)
        result2 = self.client.write_coil(self.addresses[neg_cmd], False)

        self.log(f"■ Desligando {pos_cmd} e {neg_cmd}")

```

```
self.log(f"■ JOG {axis_name} PARADO")
self.update_jog_status(axis_name, "PARADO", "gray")
```

except Exception as e:

```
self.log(f"■ Erro JOG stop {axis_name}: {e}")
```

```
def check_jog_safety(self, axis_name):
```

```
    """Monitora status de segurança JOG (apenas informativo - não bloqueia)"""
```

```
    try:
```

```
        # Mapeamento de flags de segurança por eixo
```

```
        safety_map = {
```

```
            'Y2': ('M10_Y2', 'M11_Y2'),    # Limite positivo, negativo
```

```
            'Y1': ('M510_Y1', 'M511_Y1'),  # Limite positivo, negativo
```

```
            'X': ('M1010_X', 'M1011_X'),   # Limite positivo, negativo
```

```
            'Z': ('M1510_Z', 'M1511_Z')   # Limite positivo, negativo
```

```
        }
```

```
        pos_limit_mem, neg_limit_mem = safety_map[axis_name]
```

```
        result1 = self.client.read_coils(self.addresses[pos_limit_mem], count=1)
```

```
        result2 = self.client.read_coils(self.addresses[neg_limit_mem], count=1)
```

```
        if not result1.isError() and not result2.isError():
```

```
            positive_ok = not result1.bits[0] # Invertido: flag ativa = bloqueado
```

```
            negative_ok = not result2.bits[0] # Invertido: flag ativa = bloqueado
```

```
            safety_ok = positive_ok or negative_ok # Pelo menos uma direção liberada
```

```
            safety_indicator = getattr(self, f'safety_indicator_{axis_name}', None)
```

```
            if not safety_indicator:
```

```
                return safety_ok
```

```
            # Atualiza indicador visual
```

```
            if positive_ok and negative_ok:
```

```
                status_text = "■ Ambas direções OK"
```

```
                color = "green"
```

```
            elif positive_ok:
```

```
                status_text = "■ Apenas JOG+ permitido"
```

```
                color = "orange"
```

```
            elif negative_ok:
```

```
                status_text = "■ Apenas JOG- permitido"
```

```
                color = "orange"
```

```
            else:
```

```
                status_text = "■ Fora dos limites"
```

```
                color = "red"
```

```
            safety_indicator.setText(status_text)
```

```
            safety_indicator.setStyleSheet(f"QLabel {{ background-color: {color}; color: white; padding: 3px; font-size: 10px; b
```

```
            # Log detalhado para debug
```

```
            status_key = f'_last_safety_status_{axis_name}'
```

```
            current_status = (positive_ok, negative_ok)
```

```
            if hasattr(self, status_key) and getattr(self, status_key) != current_status:
```

```
                if not safety_ok:
```

```

        self.log(f"■■■ {axis_name} fora dos limites - {pos_limit_mem}={not positive_ok}, {neg_limit_mem}={not negative_ok}")
    else:
        self.log(f"■ {axis_name} dentro dos limites - {pos_limit_mem}={not positive_ok}, {neg_limit_mem}={not negative_ok}")

    setattr(self, status_key, current_status)

    return safety_ok
else:
    return False

except Exception as e:
    self.log(f"■■■ Erro ao verificar segurança JOG {axis_name}: {e}")
    return False

def update_jog_status(self, axis_name, status_text, color):
    """Atualiza status visual do JOG"""
    status_label = getattr(self, f'jog_status_label_{axis_name}', None)
    if status_label:
        status_label.setText(f"Status: {status_text}")
        status_label.setStyleSheet(f"QLabel {{ background-color: {color}; color: white; padding: 5px; font-weight: bold; border: 1px solid black; }}")

# Funções específicas para eixo X (compatibilidade com teclado)
def jog_start_x(self, direction): self.jog_start('X', direction)
def jog_stop_x(self): self.jog_stop('X')

# =====
#          CONTROLE VIA TECLADO
# =====

def on_keyboard_jog_toggle(self, state):
    """Callback quando checkbox global de controle via teclado é alterado"""
    enabled = state == Qt.CheckState.Checked.value

    if enabled:
        self.log("■■■ Controle via teclado HABILITADO para todos os eixos")
        self.log("■ Teclas: X(← →) | Y2(↑ ↓) | Y1(W S) | Z(Q E)")
        self.keyboard_jog_checkbox.setStyleSheet("QCheckBox { font-weight: bold; color: #4CAF50; }")
    else:
        self.log("■■■ Controle via teclado DESABILITADO para todos os eixos")
        self.keyboard_jog_checkbox.setStyleSheet("QCheckBox { font-weight: bold; color: #E65100; }")
    # Para qualquer movimento JOG ativo se desabilitar
    for axis in self.keyboard_jog_active:
        if self.keyboard_jog_active[axis]:
            self.jog_stop(axis)
            self.keyboard_jog_active[axis] = False

def keyPressEvent(self, event):
    """Captura teclas pressionadas"""
    # Só processa se checkbox estiver habilitado e conectado
    if not hasattr(self, 'keyboard_jog_checkbox') or not self.keyboard_jog_checkbox.isChecked() or not self.connected:
        super().keyPressEvent(event)

```

```
return
```

```
key = event.key()
axis_to_start = None
direction = None
```

```
# Mapeamento de teclas para eixos e direções
```

```
if key == Qt.Key.Key_Left:
    axis_to_start, direction = 'X', '-'
elif key == Qt.Key.Key_Right:
    axis_to_start, direction = 'X', '+'
elif key == Qt.Key.Key_Up:
    axis_to_start, direction = 'Y2', '+'
elif key == Qt.Key.Key_Down:
    axis_to_start, direction = 'Y2', '-'
elif key == Qt.Key.Key_W:
    axis_to_start, direction = 'Y1', '+'
elif key == Qt.Key.Key_S:
    axis_to_start, direction = 'Y1', '-'
elif key == Qt.Key.Key_Q:
    axis_to_start, direction = 'Z', '+'
elif key == Qt.Key.Key_E:
    axis_to_start, direction = 'Z', '-'
```

```
# Inicia JOG se tecla válida e eixo não estiver ativo
```

```
if axis_to_start and not self.keyboard_jog_active[axis_to_start]:
    self.keyboard_jog_active[axis_to_start] = True
    self.jog_start(axis_to_start, direction)
```

```
# Mapeamento de símbolos para log
```

```
key_symbols = {
    'X': {'+': "→", '-': "←"},
    'Y2': {'+': "↑", '-': "↓"},
    'Y1': {'+': "W", '-': "S"},
    'Z': {'+': "Q", '-': "E"}
}
symbol = key_symbols[axis_to_start][direction]
self.log(f"■■■ JOG {axis_to_start}{direction} iniciado via teclado ({symbol})")
event.accept()
return
```

```
# Outras teclas passam para o comportamento padrão
```

```
super().keyPressEvent(event)
```

```
def keyReleaseEvent(self, event):
```

```
    """Captura teclas soltas"""
```

```
    # Só processa se checkbox estiver habilitado e conectado
```

```
    if not hasattr(self, 'keyboard_jog_checkbox') or not self.keyboard_jog_checkbox.isChecked() or not self.connected:
        super().keyReleaseEvent(event)
    return
```

```
key = event.key()
axis_to_stop = None
direction = None
```

```
# Mapeamento de teclas para eixos
```

```
if key == Qt.Key.Key_Left:
    axis_to_stop, direction = 'X', '-'
elif key == Qt.Key.Key_Right:
    axis_to_stop, direction = 'X', '+'
elif key == Qt.Key.Key_Up:
    axis_to_stop, direction = 'Y2', '+'
elif key == Qt.Key.Key_Down:
    axis_to_stop, direction = 'Y2', '-'
elif key == Qt.Key.Key_W:
    axis_to_stop, direction = 'Y1', '+'
elif key == Qt.Key.Key_S:
    axis_to_stop, direction = 'Y1', '-'
elif key == Qt.Key.Key_Q:
    axis_to_stop, direction = 'Z', '+'
elif key == Qt.Key.Key_E:
    axis_to_stop, direction = 'Z', '-'
```

```
# Para JOG se tecla válida e eixo estiver ativo
```

```
if axis_to_stop and self.keyboard_jog_active[axis_to_stop]:
    self.keyboard_jog_active[axis_to_stop] = False
    self.jog_stop(axis_to_stop)
```

```
# Mapeamento de símbolos para log
```

```
key_symbols = {
    'X': {'+': "→", '-': "←"},
    'Y2': {'+': "↑", '-': "↓"},
    'Y1': {'+': "W", '-': "S"},
    'Z': {'+': "Q", '-': "E"}
}
symbol = key_symbols[axis_to_stop][direction]
self.log(f"■■■ JOG {axis_to_stop}{direction} parado via teclado ({symbol})")
event.accept()
return
```

```
# Outras teclas passam para o comportamento padrão
```

```
super().keyPressEvent(event)
```

```
# -----
```

```
# NOVO: pulso momentâneo em M5000 (Y0.10) – “falling-edge trigger”
```

```
# -----
```

```
def pulse_y010(self):
```

```
    """Dispara um pulso de 3 ms de borda de subida em M5000 (rising-edge trigger)."""
```

```
    if not self.connected:
```

```
        self.log("■ CLP não conectado")
```

```
        return
```

```

try:
    # Aciona o coil
    self.log("■ ð Pulso M5000 (SHOT)")
    result = self.client.write_coil(self.addresses['M5000'], True)
    if result.isError():
        self.log(f"ð Erro ao escrever M5000: {result}")
    else:
        # Mantém ON por apenas 1 ms e depois desliga (falling-edge)
        QTimer.singleShot(1,
            lambda: self.client.write_coil(self.addresses['M5000'], False))
        # Libera o botão na interface logo após (~2 ms)
        QTimer.singleShot(2,
            lambda: self.y010_btn.setChecked(False))
        self.log("ð Pulso M5000 enviado (1 ms)")

```

```

except Exception as e:
    self.log(f"■ Erro Y0.10: {e}")

```

```

def pulse_m100(self):
    """Pulso em M100"""
    if not self.connected:
        return

```

```

try:
    self.client.write_coil(self.addresses['M100'], True)
    QTimer.singleShot(100, lambda: self.client.write_coil(self.addresses['M100'], False))
    self.log("M100 pulsado")
except Exception as e:
    self.log(f"■ Erro M100: {e}")

```

```

def emergency_stop(self):
    """Parada de emergência"""
    if not self.connected:
        return

```

```

try:
    self.log("■ PARADA DE EMERGÊNCIA!")

    # Desliga todas as memórias importantes
    emergency_memories = [
        'M0_Y2', 'M50_Y2', 'M100_Y2', 'M500_Y1', 'M550_Y1', 'M600_Y1',
        'M1500_Z', 'M1550_Z', 'M1600_Z', 'M1000_X', 'M1050_X', 'M1100_X',
        'M5000', 'M350', 'M850', 'M1350', 'M1850',
        'M70_Y2', 'M80_Y2', 'M570_Y1', 'M580_Y1', 'M1070_X', 'M1080_X', 'M1570_Z', 'M1580_Z'
    ]
    for mem in emergency_memories:
        self.client.write_coil(self.addresses[mem], False)

    # Para JOG de todos os eixos
    for axis in ['Y2', 'Y1', 'X', 'Z']:
        self.jog_stop(axis)

```

```

# Lê posições uma última vez antes de parar
self.read_current_positions()

# Para JOG via teclado se estiver ativo para qualquer eixo
if hasattr(self, 'keyboard_jog_active'):
    for axis in self.keyboard_jog_active:
        if self.keyboard_jog_active[axis]:
            self.keyboard_jog_active[axis] = False
            self.jog_stop(axis)
    self.log("■■■ JOG via teclado interrompido por emergência")

# Desliga o botão Y0.10 na interface
self.y010_btn.setChecked(False)

self.log("■ Todos os comandos desligados")

except Exception as e:
    self.log(f"■ Erro na parada: {e}")

# =====
# callbacks vindos do MovementControlsWidget
# =====
def _on_step_move_requested(self, axis: str, distance: float, feed: float):
    """Step = movimento incremental curto, em milímetros."""
    pulses = self.pulses_from_mm(axis, distance)
    self.move_relative(axis, pulses)

def _on_widget_jog_start(self, axis: str, direction: int, feed: float):
    self._active_widget_axis = axis
    self.jog_start(axis, '+' if direction > 0 else '-')

def _on_widget_jog_stop(self):
    if hasattr(self, '_active_widget_axis'):
        self.jog_stop(self._active_widget_axis)

def _on_go_to_zero(self):
    # usa homing (já implementado) para Z primeiro e depois Y2,Y1,X
    self.home_all_axes()

# =====
# S T A R T   G E R A L
# =====
def start_global_cycle(self):
    """
    1. Verifica se existe pelo menos UM programa carregado
    (Mesa 1 e/ou Mesa 2).
    2. Para cada mesa com programa:
        - move o berço (eixo Y físico da mesa) até o limite
        POSITIVO configurado em settings.table_limits;
        - aguarda o eixo ficar idle.
    """

```

3. Exibe erro se nenhuma mesa possui programa.
4. Deixa `_global_cycle_active = True` para indicar que a máquina está pronta e o fluxo PlateFlow pode iniciar quando o operador pressionar M20 / M30.

"""

```
from PyQt6.QtWidgets import QMessageBox
```

```
if self._global_cycle_active:
```

```
    self.log("■ Start Geral já está ativo – comando ignorado")
```

```
    return
```

```
# ----- 1. detecção de programas -----
```

```
mesas_com_prog = []
```

```
for mesa_id, tab in self.mesa_tabs.items():
```

```
    if self._mesa_has_program(mesa_id):
```

```
        mesas_com_prog.append(mesa_id)
```

```
if not mesas_com_prog:
```

```
    QMessageBox.warning(
```

```
        self, "Start Geral",
```

```
        "Nenhum programa está carregado em Mesa 1 ou Mesa 2.")
```

```
    self.log("■ Start Geral cancelado – nenhuma mesa tem programa")
```

```
    return
```

```
# ----- 2. prepara mesas -----
```

```
for mesa_id in mesas_com_prog:
```

```
    try:
```

```
        self._prepare_table_for_cycle(mesa_id)
```

```
    except Exception as exc:
```

```
        self.log(f"■ Erro preparando Mesa {mesa_id}: {exc}")
```

```
self._global_cycle_active = True
```

```
self.log(f"■■ START GERAL – pronto. Mesas ativas: {mesas_com_prog}")
```

```
# Aciona polling inicial dos PlateFlowManagers das mesas ativas
```

```
for mid, mgr in ((1, getattr(self, "flow_mesa1", None)),
```

```
                (2, getattr(self, "flow_mesa2", None))):
```

```
    if mgr and mid in mesas_com_prog:
```

```
        mgr.poll_now()
```

```
# -----
```

```
# H E L P E R S
```

```
# -----
```

```
def _mesa_has_program(self, mesa_id: int) -> bool:
```

```
    """
```

```
    Retorna True se a aba da mesa possui pelo menos um ponto  
    carregado (positions != []) OU se o ProgramIOWidget já tem  
    um arquivo carregado (_last_file).
```

```
    """
```

```
    tab = self.mesa_tabs.get(mesa_id)
```

```
    if tab is None:
```

```

        return False
    if tab.inspect_widget.positions():
        return True
    return bool(getattr(tab.prog_widget, "_last_file", None))

def _prepare_table_for_cycle(self, mesa_id: int):
    """
    Move o eixo Y físico da mesa até o limite POSITIVO configurado
    e aguarda a conclusão do movimento.
    """
    axis = 'Y1' if mesa_id == 1 else 'Y2'
    lim = int(self.table_limits[mesa_id][axis][1])
    self.log(f"■ Mesa {mesa_id}: levando berço ao limite +Y ({lim})")
    getattr(self, f'pulsos_spin_{axis}').setValue(lim)
    self.move_axis_absolute(axis)
    if not self._plc_motion_backend.wait_for_idle():
        raise RuntimeError("timeout aguardando eixo chegar ao limite")

# =====
# zerar posição individual (M0/M500/...)
# =====
def zero_axis(self, axis: str):
    """Pulsa a memória M0/M500/M1000/M1500 conforme eixo."""
    mem_map = {'Y2': 'M0_Y2', 'Y1': 'M500_Y1', 'X': 'M1000_X', 'Z': 'M1500_Z'}
    if axis in mem_map:
        self._pulse_coil(mem_map[axis])
        self.log(f"■ Zero absoluto do eixo {axis} solicitado")

def update_status(self):
    """Atualiza status em tempo real"""
    if not self.connected:
        return

    # Lê posições atuais e status de homing (prioridade)
    self.read_current_positions()
    # ----- atualiza MovementControls principal -----
    if hasattr(self, 'mov_widget'):
        self.mov_widget.update_position(
            x=self.current_positions['X'],
            y2=self.current_positions['Y2'],
            y1=self.current_positions['Y1'],
            z=self.current_positions['Z'])

    # ----- atualiza widgets das mesas -----
    for mesa_id, tab in getattr(self, 'mesa_tabs', {}).items():
        # envia posição para cada mov_widget da mesa
        if tab.y_axis == 'Y1':
            tab.mov_widget.update_position(
                x=self.current_positions['X'],
                y2=0,
                y1=self.current_positions['Y1'],

```

```

        z=self.current_positions['Z'])
    else:
        tab.mov_widget.update_position(
            x=self.current_positions['X'],
            y2=self.current_positions['Y2'],
            y1=0,
            z=self.current_positions['Z'])
    self.monitor_homing_status()
    # ----- atualiza gráficos das mesas -----
    if hasattr(self, "mesa_plot_widgets"):
        self.mesa_plot_widgets[1].update_head_position(
            self.current_positions['X'],
            self.current_positions['Y1'])
        self.mesa_plot_widgets[2].update_head_position(
            self.current_positions['X'],
            self.current_positions['Y2'])
    # ajusta foco conforme Z
    self.camera_manager.auto_focus(self.current_positions['Z'])

    # --- Atualiza visual do botão "Go to Zero" ---
    try:
        all_homed = True
        # verifica cada memória de homing realizado
        for mem in ('M300','M800','M1300','M1800'):
            addr = self.addresses.get(mem)
            if addr is None:
                continue
            result = self.client.read_coils(addr, count=1)
            # se leitura falhar ou estiver OFF, não está homed
            if result.isError() or not result.bits[0]:
                all_homed = False
                break
        self._update_go_to_zero_button(all_homed)
    except Exception:
        print("Erro ao atualizar botão Go to Zero")

    try:
        # Atualiza saídas
        outputs_map = {
            'Y0_0': ('Y00', self.Y0_0_status),
            'Y0_1': ('Y01', self.Y0_1_status),
            'Y0_4': ('Y04', self.Y0_4_status),
            'Y0_5': ('Y05', self.Y0_5_status),
            'Y0_2': ('Y02', self.Y0_2_status),
            'Y0_3': ('Y03', self.Y0_3_status),
            'Y0_6': ('Y06', self.Y0_6_status),
            'Y0_7': ('Y07', self.Y0_7_status),
            'Y0_10': ('Y010', self.Y0_10_status)
        }
    }

```

```

for name, (addr_key, label) in outputs_map.items():
    # Y = coils → FC01 (read_coils)
    result = self.client.read_coils(self.addresses[addr_key], count=1)
    if not result.isError():
        state = result.bits[0]
        label.setText("ON" if state else "OFF")
        label.setStyleSheet(
            "QLabel { background-color: lime; color: black; padding: 5px; }" if state
            else "QLabel { background-color: gray; color: white; padding: 5px; }"
        )

    # Sincroniza botão Y0.10 com o estado real
    if name == 'Y0_10':
        if self.y010_btn.isChecked() != state:
            self.y010_btn.setChecked(state)

# Atualiza memórias
memories_map = {
    # Atualizado conforme novos endereços
    'M0_Y2': (self.addresses['M0_Y2'], getattr(self, 'M0_Y2_status', None)),
    'M50_Y2': (self.addresses['M50_Y2'], getattr(self, 'M50_Y2_status', None)),
    'M500_Y1': (self.addresses['M500_Y1'], getattr(self, 'M500_Y1_status', None)),
    'M550_Y1': (self.addresses['M550_Y1'], getattr(self, 'M550_Y1_status', None)),
    # Disparo e status da aplicadora
    'M5000': (self.addresses['M5000'], getattr(self, 'M5000_status', None)),
    'M5001': (self.addresses['M5001'], getattr(self, 'M5001_status', None)),

    'M350': (self.addresses['M350'], getattr(self, 'M350_status', None)),
    'M850': (self.addresses['M850'], getattr(self, 'M850_status', None)),
    'M1350': (self.addresses['M1350'], getattr(self, 'M1350_status', None)),
    'M1850': (self.addresses['M1850'], getattr(self, 'M1850_status', None)),
    # MEMÓRIAS DE HOMING REALIZADO - PRIORITÁRIAS
    'M300': (self.addresses['M300'], getattr(self, 'M300_status', None)),
    'M800': (self.addresses['M800'], getattr(self, 'M800_status', None)),
    'M1300': (self.addresses['M1300'], getattr(self, 'M1300_status', None)),
    'M1800': (self.addresses['M1800'], getattr(self, 'M1800_status', None)),
    # Monitoramento JOG (apenas alguns principais)
    'M70_Y2': (self.addresses['M70_Y2'], getattr(self, 'M70_Y2_status', None)),
    'M570_Y1': (self.addresses['M570_Y1'], getattr(self, 'M570_Y1_status', None)),
    'M1070_X': (self.addresses['M1070_X'], getattr(self, 'M1070_X_status', None)),
    'M1570_Z': (self.addresses['M1570_Z'], getattr(self, 'M1570_Z_status', None))
}

for name, (addr, label) in memories_map.items():
    if name in ['M300', 'M800', 'M1300', 'M1800']:
        continue # Pula memórias de homing (tratadas em função dedicada)

    if label is None:
        continue
    result = self.client.read_coils(addr, count=1)
    if not result.isError():

```

```

state = result.bits[0]
# Atualiza label se existir
if label is not None:
    label.setText("ON" if state else "OFF")
    # Para M5001 usamos cor laranja enquanto ativo
    if name == 'M5001':
        label.setStyleSheet(
            "QLabel { background-color: orange; color: black; padding: 5px; }" if state
            else "QLabel { background-color: gray; color: white; padding: 5px; }"
        )
    else:
        label.setStyleSheet(
            "QLabel { background-color: red; color: white; padding: 5px; }" if state
            else "QLabel { background-color: gray; color: white; padding: 5px; }"
        )

# Atualiza status JOG ativo para todos os eixos
if name in ['M70_Y2', 'M570_Y1', 'M1070_X', 'M1570_Z']:
    jog_status_map = {
        'M70_Y2': ('Y2', 'POSITIVO'),
        'M570_Y1': ('Y1', 'POSITIVO'),
        'M1070_X': ('X', 'POSITIVO'),
        'M1570_Z': ('Z', 'POSITIVO')
    }

    if name in jog_status_map and not result.isError():
        axis, direction = jog_status_map[name]
        status_label = getattr(self, f'jog_status_label_{axis}', None)
        if status_label:
            if state:
                self.update_jog_status(axis, f"ATIVO - {direction}", "lime")
            elif not state and direction in status_label.text():
                self.update_jog_status(axis, "Inativo", "gray")

# Atualiza segurança JOG em tempo real apenas quando necessário
if name in ['M10_Y2', 'M11_Y2', 'M510_Y1', 'M511_Y1', 'M1010_X', 'M1011_X', 'M1510_Z', 'M1511_Z']:
    axis_map = {
        'M10_Y2': 'Y2', 'M11_Y2': 'Y2',
        'M510_Y1': 'Y1', 'M511_Y1': 'Y1',
        'M1010_X': 'X', 'M1011_X': 'X',
        'M1510_Z': 'Z', 'M1511_Z': 'Z'
    }

    if name in axis_map:
        axis = axis_map[name]
        QTimer.singleShot(50, lambda a=axis: self.check_jog_safety(a))

# Atualiza registradores
registers_map = {
    # Registradores dos eixos
    'D100_Y2': (self.addresses['D100_Y2'], getattr(self, 'D100_Y2_status', None)),
    'D20000_Y2': (self.addresses['D20000_Y2'], getattr(self, 'D20000_Y2_status', None)),

```

```

'D1600_Z': (self.addresses['D1600_Z'], getattr(self, 'D1600_Z_status', None)),
'D21500_Z': (self.addresses['D21500_Z'], getattr(self, 'D21500_Z_status', None)),
'D1100_X': (self.addresses['D1100_X'], getattr(self, 'D1100_X_status', None)),
'D21000_X': (self.addresses['D21000_X'], getattr(self, 'D21000_X_status', None))
}

```

```

# ----- SENSORES HOME -----

```

```

sensors_map = {
    'X04_Y2': ('X04_Y2_status', self.addresses['X04_Y2']),
    'X06_Y1': ('X06_Y1_status', self.addresses['X06_Y1']),
    'X08_X': ('X08_X_status', self.addresses['X08_X']),
    'X09_Z': ('X09_Z_status', self.addresses['X09_Z'])
}

```

```

for sensor_key, (label_attr, addr) in sensors_map.items():

```

```

    lbl = getattr(self, label_attr, None)

```

```

    if lbl is None:

```

```

        continue

```

```

    # X = discrete inputs → FC02 (read_discrete_inputs)

```

```

    res = self.client.read_discrete_inputs(addr, count=1)

```

```

    if not res.isError():

```

```

        state = res.bits[0]

```

```

        lbl.setText("ON" if state else "OFF")

```

```

        lbl.setStyleSheet(

```

```

            "QLabel { background-color: lime; color: black; padding: 5px; }" if state

```

```

            else "QLabel { background-color: gray; color: white; padding: 5px; }"

```

```

        )

```

```

for name, (addr, label) in registers_map.items():

```

```

    try:

```

```

        # Todos agora são registradores D normais

```

```

        signed = self.read_dword(addr)

```

```

        if label is not None:

```

```

            label.setText(str(signed))

```

```

            # cor laranja se negativo

```

```

            label.setStyleSheet(

```

```

                "QLabel { background-color: orange; padding: 5px; }"

```

```

                if signed < 0 else

```

```

                "QLabel { background-color: lightgray; padding: 5px; }"

```

```

            )

```

```

        except Exception as e:

```

```

            self.log(f"■■■ Erro ao ler registrador {name}: {e}")

```

```

except Exception as e:

```

```

    self.log(f"■■■ Erro ao atualizar status: {e}")

```

```

# -----

```

```

# Fornece posição atual para o widget de inspeção

```

```

# -----

```

```

def _get_current_position_dict(self) -> dict:

```

```

    """

```

Retorna dict {x,y2,y1,z} em pulsos – usado pelo
InspectionPositionsWidget para 'Adicionar posição atual'.

"""

```
return {
    "x": self.current_positions.get("X", 0),
    "y2": self.current_positions.get("Y2", 0),
    "y1": self.current_positions.get("Y1", 0),
    "z": self.current_positions.get("Z", 0),
}
```

=====

Métodos adicionados para controlar o piscar e o estilo do botão

=====

```
def _toggle_go_to_zero_alert(self):
```

"""Pisca o botão Go to Zero alternando vermelho e estilo padrão."""

```
    btn = getattr(self, 'mov_widget', None)
```

```
    if not btn:
```

```
        return
```

```
    w = btn.btn_gotozero
```

```
    if self._gozero_alert_on:
```

```
        w.setStyleSheet("") # estilo padrão
```

```
    else:
```

```
        # alerta em vermelho piscante
```

```
        w.setStyleSheet("QPushButton { background-color: red; color: white; }")
```

```
    self._gozero_alert_on = not self._gozero_alert_on
```

```
def _update_go_to_zero_button(self, all_ok: bool):
```

"""

Se todas as memórias de homing estiverem OK, para o blink e mantém verde.

Caso contrário, inicia o piscar em vermelho.

"""

```
    btn = getattr(self, 'mov_widget', None)
```

```
    if not btn:
```

```
        return
```

```
    w = btn.btn_gotozero
```

```
    if all_ok:
```

```
        # homing OK → para blink e fixa verde
```

```
        if self._gozero_blink_timer.isActive():
```

```
            self._gozero_blink_timer.stop()
```

```
        w.setStyleSheet("QPushButton { background-color: green; color: white; }")
```

```
    else:
```

```
        # homing perdido → inicia blink vermelho
```

```
        if not self._gozero_blink_timer.isActive():
```

```
            self._gozero_alert_on = False
```

```
            self._gozero_blink_timer.start()
```

converte lista do widget para InspectionPosition do sequence_control

```
def _positions_to_model(self):
```

```
    lst = []
```

```
    for p in self.inspect_widget.positions():
```

```

lst.append(
    InspectionPosition(
        name=p.name,
        x=p.x, y2=p.y2, y1=p.y1, z=p.z
    )
)
return lst

```

```

def force_homing_status_check(self):
    """Força uma verificação imediata do status de homing"""
    try:
        self.log("■ Verificação forçada do status de homing...")
        self.monitor_homing_status()
    except Exception as e:
        self.log(f"■ Erro na verificação forçada: {e}")

```

```

def update_homing_led(self, led_widget, homing_status):
    """Atualiza o indicador LED de homing"""
    if homing_status:
        # Homing realizado - LED verde
        led_widget.setStyleSheet("QLabel { background-color: lime; color: darkgreen; padding: 2px; font-size: 16px; font-weight: bold; }")
        led_widget.setText("●")
    else:
        # Homing não realizado - LED cinza
        led_widget.setStyleSheet("QLabel { background-color: gray; color: darkgray; padding: 2px; font-size: 16px; font-weight: bold; }")
        led_widget.setText("●")

```

```

def test_position_reading(self):
    """Função de teste para verificar leitura das posições"""
    if not self.connected:
        return

```

```

    self.log("■ Testando leitura das posições atuais...")

```

```

    # Usa a função dedicada e exibe resultados
    self.read_current_positions()

```

```

    # Log das posições lidas
    for axis, position in self.current_positions.items():
        self.log(f"■ Posição {axis}: {position}")
    self.test_homing_status()

```

```

def closeEvent(self, event):
    """Fecha conexão ao sair"""
    if self.client:
        # reset homing■done flags so next start must re-home all axes
        try:
            self.log("■ Resetando homing statuses: acionando M22")
            self._pulse_coil('M22')
        except Exception as e:

```

```

        self.log(f"■ Falha ao pulsar M22: {e}")
    # then do emergency stop and close
    self.emergency_stop()
    self.client.close()
    self.status_bar.showMessage("Desconectado")
    self.status_bar.setStyleSheet(
        "QStatusBar { background-color:red; color:white; font-weight:bold; }")
    self.log("Desconectado do CLP")
    event.accept()

```

```

# -----
# BACK-END DE MOVIMENTO para o SequenceControlWidget
# -----

```

```

class PLCMotionBackend(QObject):

```

```

    """

```

Adaptador simples que converte as chamadas do SequenceControlWidget para os métodos já existentes do MultiAxisMotorController.

```

    """

```

```

# sinal para agendar movimento de eixo no GUI thread
axis_move = pyqtSignal(str, int)

```

```

def __init__(self, ctrl: MultiAxisMotorController):

```

```

    # inicializa QObject antes de qualquer uso de signals

```

```

    QObject.__init__(self)

```

```

    self._c = ctrl

```

```

    self._off_x = ctrl.camera_nozzle_offset.get("x", 0)

```

```

    self._off_y = ctrl.camera_nozzle_offset.get("y", 0)

```

```

    self._feed = 1000

```

```

    self._apply_offset = True    # default = APPLY

```

```

    # ----- deslocamento dinâmico calculado por fiducial -----

```

```

    self._dx_dyn = 0            # em pulsos

```

```

    self._dy_dyn = 0

```

```

    # guarda último  $\Delta X/\Delta Y \neq 0$  para leitura pós-runner

```

```

    self._last_offset: tuple[int,int] = (0, 0)

```

```

    self._targets: dict[str,int] = {}    # destino mais recente por eixo

```

```

    # registrador de QUANTIDADE de dots

```

```

    self._addr_qty = ctrl.addresses.get('D24100_QTY')

```

```

    # coils para dot

```

```

    self._addr_trig = ctrl.addresses.get('M5000')

```

```

    self._addr_stat = ctrl.addresses.get('M5001')

```

```

    # conecta axis_move para despachar setValue/move no GUI thread

```

```

    self.axis_move.connect(self._on_axis_move, Qt.ConnectionType.QueuedConnection)

```

```

# -----

```

```

def move_to_absolute_position(self,

```

```

    x: float | None,

```

```

    y2: float | None,

```

```

    y1: float | None,

```

```

    z: float | None,

```

```

    feed_rate: float = 1000) -> bool:

```

```

self._feed = feed_rate
# -----
# LIMPA destinos da execução anterior ← BUG FIX travamento 2ª run
# -----
self._targets.clear()
# DEBUG ■■ cleared any previous targets before sending new moves
self._c.log(f"[PLCBackend] move_to_abs: cleared targets, apply_offset={self._apply_offset}")
# DEBUG ■■ log requested move parameters
self._c.log(f"[PLCBackend] move_to_abs: x={x}, y2={y2}, y1={y1}, z={z}, feed_rate={feed_rate}")
try:
    # Escreve apenas eixos cujo valor não é None
    dx = self._off_x if self._apply_offset else 0
    dy = self._off_y if self._apply_offset else 0
    if x is not None: self._move_axis('X', int(x) - dx)
    if y2 is not None: self._move_axis('Y2', int(y2) - dy)
    if y1 is not None: self._move_axis('Y1', int(y1) - dy)
    if z is not None: self._move_axis('Z', int(z))
    return True
except Exception as exc:
    # DEBUG ■■ failure in move_to_abs
    self._c.log(f"[PLCBackend] move_to_abs ERROR: {exc}")
    print(f"Erro move_to_absolute_position: {exc}")
    self._c.log(f"Erro move_to_abs: {exc}")
    return False

# -----
# NOVO: muda modo de offset (True = APPLY, False = VIEW)
# -----
def set_offset_mode(self, apply_offset: bool):
    self._apply_offset = bool(apply_offset)

# =====
# recebe ΔX / ΔY (pulsos) calculado pelo fiducial
# =====
def apply_dynamic_offset(self, dx_pulses: int, dy_pulses: int):
    self._dx_dyn = dx_pulses
    self._dy_dyn = dy_pulses
    if dx_pulses or dy_pulses:
        self._last_offset = (dx_pulses, dy_pulses)
    self._c.log(f"■ Offset dinâmico aplicado: ΔX={dx_pulses} ΔY={dy_pulses}")

def _move_axis(self, axis: str, pulses: int):
    # acrescenta deslocamento dinâmico a X/Y apenas
    # DEBUG ■■ about to queue axis move
    self._c.log(f"[PLCBackend] _move_axis -> axis={axis}, raw_target={pulses}, dx_dyn={self._dx_dyn}, dy_dyn={self._dy_dyn}")
    if axis == "X":
        pulses += self._dx_dyn
    elif axis in ("Y1", "Y2"):
        pulses += self._dy_dyn

```

```
# registra o alvo e dispara sinal para execução no GUI thread
self._targets[axis] = pulses
self.axis_move.emit(axis, pulses)
# DEBUG ■■ queued target for wait_for_idle
self._c.log(f"[PLCBackend] _move_axis: queued {axis} → target={pulses}")
# Não bloqueia aqui – wait_for_idle fará polling
```

```
# -----
```

```
def wait_for_idle(self) -> bool:
```

```
    """Bloqueia até que cada eixo alcance o pulso alvo  $\pm 1$ ."""
```

```
    t0 = time.time()
```

```
    timeout = 10      # s – eixo pode percorrer longas distâncias
```

```
    ok_axes = set()
```

```
    pos_regs = {      # registradores D que espelham SR
```

```
        'Y2': self._c.addresses['D3100_Y2'],
```

```
        'Y1': self._c.addresses['D3400_Y1'],
```

```
        'X' : self._c.addresses['D3000_X'],
```

```
        'Z' : self._c.addresses['D3200_Z'],
```

```
    }
```

```
    while time.time() - t0 < timeout:
```

```
        elapsed = time.time() - t0
```

```
        # DEBUG ■■ iteration status
```

```
        self._c.log(f"[PLCBackend] wait_for_idle: elapsed={elapsed:.1f}s, "
                    f"targets={self._targets}, ok_axes={ok_axes}")
```

```
        all_reached = True
```

```
        # copia para evitar “dictionary changed size”
```

```
        for axis, target in list(self._targets.items()):
```

```
            if axis in ok_axes:
```

```
                continue
```

```
            try:
```

```
                pos = self._c.read_dword(pos_regs[axis])
```

```
            except Exception as exc:
```

```
                self._c.log(f"Erro ao ler posição do eixo {axis}: {exc}")
```

```
                pos = None
```

```
            if pos is None or abs(pos - target) > 1:
```

```
                all_reached = False
```

```
            else:
```

```
                ok_axes.add(axis)
```

```
    if all_reached:
```

```
        return True
```

```
    # deixa o Qt processar o slot _on_axis_move agendado,
```

```
    # que dispara de fato move_axis_absolute():
```

```
    QCoreApplication.processEvents()
```

```
    # depois aguarda um tiquinho para não poluir demais o CLP
```

```
    time.sleep(0.05)
```

```
    # DEBUG ■■ timeout reached, log which axes never met target
```

```

remaining = [ax for ax in self._targets if ax not in ok_axes]
self._c.log(f"[PLCBackend] wait_for_idle: TIMEOUT after {timeout}s, remaining_axes={remaining}")
# existing generic timeout log
self._c.log("wait_for_idle: timeout atingido")

```

```

return False

```

```

@pyqtSlot(str, int)

```

```

def _on_axis_move(self, axis: str, pulses: int):

```

```

    """

```

```

    Slot executado no GUI thread:

```

- atualiza o spin-box
- dispara o comando de movimento ao CLP

```

    """

```

```

    spin = getattr(self._c, f"pulsos_spin_{axis}")

```

```

    spin.setValue(int(pulses))

```

```

    self._c.move_axis_absolute(axis)

```

```

# DEBUG (opcional): confirmar que o move foi disparado

```

```

self._c.log(f"[PLCBackend] _on_axis_move: executed move {axis} → {pulses}")

```

```

# -----

```

```

def set_feed_rate(self, fr: float):

```

```

    self._feed = fr

```

```

# -----

```

```

# DOT – grava SOMENTE a quantidade (D24100)

```

```

# -----

```

```

def apply_dot_qty(self, qty: int):

```

```

    if not self._c.connected:

```

```

        return

```

```

    try:

```

```

        if self._addr_qty is not None:

```

```

            self._c.write_dword(self._addr_qty, int(qty))

```

```

            self._c.log(f"■ DOT qty → D24100={qty}")

```

```

    except Exception as exc:

```

```

        self._c.log(f"■ Erro ao escrever DOT cfg: {exc}")

```

```

# -----

```

```

# DISPARO DO DOT (pulso M5000) + espera M5001

```

```

# -----

```

```

def trigger_dot(self, pulse_ms: int = 1) -> bool:

```

```

    """

```

```

    Gera um pulso no coil M5000. Retorna True se escrito com sucesso.

```

```

    """

```

```

    if not self._c.connected or self._addr_trig is None:

```

```

        return False

```

```

    try:

```

```

        # pulso de duração fixa de 1 ms para M5000

```

```

        self._c.client.write_coil(self._addr_trig, True)

```

```

        time.sleep(pulse_ms / 1000.0)

```

```

        self._c.client.write_coil(self._addr_trig, False)
        self._c.log(f"■ Pulso M5000 enviado ({pulse_ms} ms)")
        return True
    except Exception as exc:
        self._c.log(f"■ Erro no pulso M5000: {exc}")
        return False

```

```

def wait_dot_complete(self, timeout: float = 10.0) -> bool:
    """
    Aguarda M5001 ligar (start) e desligar (fim) dentro do timeout.
    """

```

```

    if not self._c.connected or self._addr_stat is None:
        return True # nada a esperar – não bloqueia
    t0 = time.time()
    # espera LIGAR
    while time.time() - t0 < timeout:
        try:
            r = self._c.client.read_coils(self._addr_stat, count=1)
            if not r.isError() and r.bits[0]:
                break
        except Exception as exc:
            self._c.log(f"Erro ao esperar M5001=ON: {exc}")
            pass
        time.sleep(0.05)
    else:
        self._c.log("■ Timeout esperando M5001=ON")
        return False

```

```

    # espera DESLIGAR
    while time.time() - t0 < timeout:
        try:
            r = self._c.client.read_coils(self._addr_stat, count=1)
            if not r.isError() and not r.bits[0]:
                self._c.log("■ Dot concluído (M5001=OFF)")
                return True
        except Exception as exc:
            self._c.log(f"Erro ao esperar M5001=OFF: {exc}")
            time.sleep(0.05)
    self._c.log("■ Timeout esperando M5001=OFF")
    return False

```

```

def main():
    app = QApplication(sys.argv)
    app.setStyle('Fusion')

    # Exibe splash screen com o app_icon.png
    splash_pix = QPixmap(str(Path(__file__).resolve().parent / 'resources' / 'app_icon.png'))
    splash = QSplashScreen(splash_pix)
    splash.show()
    app.processEvents()

```

```

# carrega o .ico e embute várias resoluções para rodar ok no tray do Windows
res_dir = Path(__file__).resolve().parent / 'resources'
ico_file = res_dir / 'app_icon.ico'
if not ico_file.exists():
    raise FileNotFoundError(f"resources/app_icon.ico não encontrado em {res_dir}")
icon = QIcon()
# adiciona as resoluções mais comuns

for size in (16, 32, 48, 256):
    icon.addFile(str(ico_file), QSize(size, size))
# aplica globalmente
app.setWindowIcon(icon)

# janela principal
window = MultiAxisMotorController()
window.setWindowIcon(icon)
# system tray icon (keep reference so it's not GC'd)
tray = QSystemTrayIcon(parent=window)
tray.setIcon(icon)
tray.setToolTip("Controle Multi-Eixos - Delta AS")
tray.show()
window.tray_icon = tray

# 1) start homing sequence (same as "Go to Zero")
window.home_all_axes()

# 2) poll every 500 ms for completion of all four homing coils:
# M300 (Y2), M800 (Y1), M1300 (X), M1800 (Z)
homing_timer = QTimer()
# Timeout de 30 segundos para homing
homing_start = time.time()
homing_timer.setInterval(500)
def check_homing():
    mems = ['M300', 'M800', 'M1300', 'M1800']
    all_done = True
    for mem in mems:
        try:
            result = window.client.read_coils(window.addresses[mem], count=1)
            if result.isError() or not result.bits[0]:
                all_done = False
                break
        except Exception:
            all_done = False
            break
    if all_done:
        homing_timer.stop()
        # 1) Exibe janela no estado RESTAURADO para calcular layout corretamente
        window.showNormal()
        # 2) Logo após (100 ms), maximize e deixe o layout "assentar"
        QTimer.singleShot(100, lambda: window.showMaximized())
        # 3) Feche o splash logo depois que a janela estiver maximizada

```

```

        QTimer.singleShot(110, lambda: splash.finish(window))
# Se timeout de 30s atingido sem homing completo, ignora falha e prossegue
elif time.time() - homing_start >= 30:
    homing_timer.stop()
    # Exibe janela mesmo sem homing realizado
    window.showNormal()
    QTimer.singleShot(100, lambda: window.showMaximized())
    QTimer.singleShot(110, lambda: splash.finish(window))
    # Informa o usuário sobre o timeout de homing
    QMessageBox.warning(window, "Timeout Homing",
                        "Homing não foi realizado. " \
                        "Verifique as conexões e tente novamente.")
homing_timer.timeout.connect(check_homing)
homing_timer.start()

print("="*60)
print("CONTROLE MULTI-EIXOS - DELTA AS (ENDEREÇOS LADDER REAIS)")
print("="*60)
print("■ MELHORIAS IMPLEMENTADAS:")
print("• Endereços corrigidos conforme planilha do ladder real")
print("• Controle de 4 eixos: Y2, Y1, Z, X")
print("• Botões HOME corrigidos: M350, M1350, M1850")
print("• Posições atuais via D3000, D3100, D3200")
print("• Indicadores LED de homing e leitura automática de velocidades")
print("• Sistema JOG integrado para TODOS os eixos com segurança")
print("✓ Controle via teclado para TODOS os eixos:")
print("• X: ← → • Y2: ↑ ↓ • Y1: W S • Z: Q E")
print("• Movimentos absolutos com verificação de limites")
print("• Configurações de homing e limites salvos em ROM")
print("• Interface redesenhada para melhor usabilidade")
print("• Leitura em tempo real das posições dos motores corrigida")
print("• Status inicial e atualização automática dos LEDs de homing")
print("="*60)

sys.exit(app.exec())

if __name__ == '__main__':
    main()

```

File: sequence_control.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

sequence_control.py

Módulo plug-and-play para executar uma sequência de posições (X,Y,Z)
num sistema CNC + câmera.

Dependências:

PyQt6>=6.5

(Opcional) o dataclass InspectionPosition já usado nos módulos anteriores.

Caso não exista no projeto hospedeiro basta
substituir por dicts {'x':..., 'y':..., 'z':...}.

"""

```
from __future__ import annotations
import time                # ← NECESSÁRIO para wait_for_idle / trigger_dot
from typing import List, Protocol, Callable, Dict, Any
from dataclasses import dataclass
from PyQt6.QtCore import QThread, pyqtSignal, QObject, Qt
from pathlib import Path
from barcode_scanner import BarcodeScanner
from datetime import datetime
from PyQt6.QtWidgets import (
    QWidget, QGroupBox, QVBoxLayout, QHBoxLayout, QPushButton,
    QProgressBar, QLabel, QFileDialog, QMessageBox, QSpinBox
)
```

1) Modelo de dados (reaproveitável)

@dataclass

class InspectionPosition:

```
    name: str
    x: float
    y2: float
    y1: float
    z: float = 0.0
    camera_params: Dict[str, Any] | None = None
```

2) Protocolos (interfaces) para back-ends

class MotionBackend(Protocol):

"""Funções mínimas exigidas do sistema de movimento (GRBL, PLC etc.)."""

```
    def move_to_absolute_position(self,
        x: float | None,
        y2: float | None,
        y1: float | None,
        z: float | None,
        feed_rate: float = 1000) -> bool: ...
```

```
    def wait_for_idle(self) -> bool: ...
```

opcional, mas ajuda:

```
    def set_feed_rate(self, feed_rate: float): ...
```

novo: habilita ou não o uso do offset câmera↔nozzle

```
    def set_offset_mode(self, apply_offset: bool): ...
```

opcional – grava SOMENTE a quantidade de dots

```
    def apply_dot_qty(self, qty: int): ...
```

```

class CameraBackend(Protocol):
    """Funções mínimas exigidas da câmera (pode ser dummy)."""
    def capture(self, params: Dict[str, Any] | None = None): ...

# -----
# 3) Thread que percorre a sequência
# -----

class SequenceRunnerThread(QThread):
    progress      = pyqtSignal(int, int) # idx_atual, total
    imageCaptured = pyqtSignal(object)  # imagem capturada
    finished      = pyqtSignal()
    error         = pyqtSignal(str)
    # ----- feedback visual do fiducial -----
    # mesmo formato usado pelo FiducialConfigWidget
    # similarity(%) , ok? , x , y , w , h
    fidMatch      = pyqtSignal(float, bool, int, int, int, int)
    fidClear      = pyqtSignal()
    # ---- overlay + código lido -----
    bcMatch       = pyqtSignal(bool, int, int, int, int, str) # ok,x,y,w,h,data
    bcClear       = pyqtSignal()

    def __init__(self,
        motion: MotionBackend,
        camera: CameraBackend | None,
        positions: List[InspectionPosition],
        *,
        apply_mode: bool,          # True = APPLY / False = VIEW
        feed_rate: float = 1000.0):
        super().__init__()
        self._motion    = motion
        self._camera     = camera
        self._positions = positions
        self._feed_rate = feed_rate
        self._apply_mode = apply_mode # guarda selecção do usuário
        self._stop_flag = False
        self._pause_flag = False
        self.is_paused   = False
        # deslocamento acumulado por fiducial (pulsos)
        self._dx_acc = 0
        self._dy_acc = 0
        self._prev_was_fid = False
        self._prev_was_bc  = False

        self._scanner = BarcodeScanner()
        self._session_dir: Path | None = None # pasta logs/<...>

    def request_stop(self):
        self._stop_flag = True

```

```

# ----- NOVO: pausa/continua -----
def toggle_pause(self):
    self._pause_flag = not self._pause_flag
    self.is_paused = self._pause_flag

def run(self):
    # -----
    # A partir daqui qualquer saída (normal ou por exceção) deverá
    # restaurar o offset dinâmico para zero, evitando que  $\Delta X/\Delta Y$ 
    # "vaze" para a execução seguinte.
    # -----
    # PATCH 01-Ago-2025: clear dynamic offset and leftover targets before starting
    try:
        # clear any applied dynamic offset
        if hasattr(self._motion, "apply_dynamic_offset"):
            self._motion.apply_dynamic_offset(0, 0)
        # reset internal dynamic accumulators if present
        if hasattr(self._motion, "_dx_dyn"):
            self._motion._dx_dyn = 0
        if hasattr(self._motion, "_dy_dyn"):
            self._motion._dy_dyn = 0
        # clear any leftover movement targets
        if hasattr(self._motion, "_targets"):
            self._motion._targets.clear()
    except Exception:
        print("[seq] Falha ao limpar alvos de movimento")
        pass
    try:
        total = len(self._positions)
        if total == 0:
            self.error.emit("Sequência vazia")
            return
        # feed opcional
        if hasattr(self._motion, "set_feed_rate"):
            self._motion.set_feed_rate(self._feed_rate)

        for idx, pos in enumerate(self._positions, 1):
            # ----- PAUSA -----
            while self._pause_flag and not self._stop_flag:
                self.msleep(100)
            if self._stop_flag:
                self.error.emit("Execução interrompida pelo usuário")
                return

            # --- IGNORA QUALQUER AÇÃO DESLIGADA EM Config. de Processo ---
            action = (pos.camera_params or {}).get("action")
            proc_cfg = self._motion._c.settings.get_config("process_config", {})
            if action and not proc_cfg.get(action, True):
                # loga e avança o progresso sem mover nem processar
                self._motion._c.log(f"■ Ação '{action}' ignorada pela Config. de Processo")

```

```

        self.progress.emit(idx, total)
        continue
# -----

# -----
# DECIDE SE OFFSET É APLICADO NESTE PONTO
# • VIEW → nunca aplica
# • APPLY → depende do tipo da ação
# -----
if not self._apply_mode:      # modo VIEW
    apply_off = False
else:                        # modo APPLY
    a = (pos.camera_params or {}).get("action")
    apply_off = a not in ("barcode", "fiducial", "inspect")
if hasattr(self._motion, "set_offset_mode"):
    try:
        self._motion.set_offset_mode(apply_off)
    except Exception:
        print("[seq] Falha ao definir modo de offset")
        pass

# ■■■■ Override Z em modo VIEW para ação 'dot':
#   usa a altura fixa de visualização configurada
z_move = pos.z
if (not self._apply_mode
    and (pos.camera_params or {}).get("action") == "dot"):
    try:
        z_move = self._motion._c.settings.data.get(
            "dot_view_height", pos.z)
    except Exception:
        z_move = pos.z
ok = self._motion.move_to_absolute_position(
    pos.x, pos.y2, pos.y1, z_move,
    feed_rate=self._feed_rate)
if not ok:
    self.error.emit(f"Falha ao enviar movimento para {pos.name}")
    return
if not self._motion.wait_for_idle():
    self.error.emit("Timeout de movimento")
    return

# ----- dwell 100 ms entre pontos -----
# evita que o próximo comando seja enfileirado antes
# do CLP realmente liberar todos os eixos.
self.msleep(100)      # (== 0,1 s)
# -----
# Após a CHEGADA ao ponto atual.....
# se o ponto ANTERIOR era um fiducial
# ⇒ limpamos o overlay agora (o retângulo ficou
# visível durante todo o deslocamento).
# -----

```

```

if self._prev_was_fid:
    self.fidClear.emit()

# -----
# limpa overlay do BARCODE
# -----

if self._prev_was_bc:
    self.bcClear.emit()

# -----
# -----
# AÇÃO "dot"
# A frequência (D24000) permanece a que o operador
# definiu na aba "Config. Registradores".
# -----

action = (pos.camera_params or {}).get("action")

# =====
#      FIDUCIAL
# =====

if action == "fiducial":
    err = self._process_fiducial(pos)
    if err:
        self.error.emit(err)
        return
    self._prev_was_fid = True
    self.progress.emit(idx, total)
    continue
else:
    self._prev_was_fid = False

# =====
#      BARCODE
# =====

if action == "barcode":
    err = self._process_barcode(pos)
    if err:
        self.error.emit(err)
        return
    self._prev_was_bc = True
    self.progress.emit(idx, total)
    continue
else:
    self._prev_was_bc = False

if action == "dot" and self._apply_mode: # << APPLY apenas
    # aplica configurações de dot (freq e qty)
    qty = int((pos.camera_params or {}).get("dot_qty", 1))
    # aplica configuração de quantidade no CLP
    if hasattr(self._motion, "apply_dot_qty"):
        try:

```

```

        self._motion.apply_dot_qty(qty)
    except Exception:
        print("[seq] Falha ao aplicar dot settings")
        pass

    # 1. envia PULSO em M5000 com duração adaptada a qty/freq
    # obtém frequência de aplicação (Hz) do meta ou do spinBox de D24000_FREQ
    freq = int((pos.camera_params or {}).get(
        "dot_freq_hz",
        self._motion._c.findChild(QSpinBox, "cfg_spin_D24000_FREQ").value()
    ))
    # calcula tempo de pulso em ms = qty/freq*1000 + folga de 2 ms
    pulse_ms = max(1, int(qty * 1000 / freq) + 2)
    if hasattr(self._motion, "trigger_dot"):
        ok = self._motion.trigger_dot(3)
        if not ok:
            self.error.emit("Falha ao acionar dot – M5000")
            return

    # 2. espera conclusão via M5001
    if hasattr(self._motion, "wait_dot_complete"):
        ok = self._motion.wait_dot_complete(timeout=10.0)
        if not ok:
            self.error.emit("Timeout aguardando fim do dot (M5001)")
            return

    # Se modo VIEW, apenas registra que o ponto DOT foi visitado
    # (sem aplicar cola).
    if self._camera is not None:
        try:
            img = self._camera.capture(pos.camera_params or None)
            self.imageCaptured.emit(img)
        except Exception as exc:
            print(f"[seq] Erro ao capturar imagem: {exc}")
            self.error.emit(f"Erro na captura: {exc}")
            return

    self.progress.emit(idx, total)

    self.finished.emit()

except Exception as exc:
    print(f"[seq] Erro na execução da sequência: {exc}")
    self.error.emit(str(exc))
finally:
    # -----
    # PATCH 01-Ago-2025
    # Independentemente do desfecho, zera o offset dinâmico gerado
    # pelos fiduciais antes de retornar ao chamador. Isso evita que
    # o próximo programa (na mesma ou noutra mesa) receba destinos
    # contaminados e fique preso no wait_for_idle().
    # -----

```

```

if hasattr(self._motion, "apply_dynamic_offset"):
    try:
        self._motion.apply_dynamic_offset(0, 0)
    except Exception:
        # não podemos disparar erro daqui; apenas registra no log
        print("[seq] Falha ao zerar offset dinâmico no finally")

```

```

# -----
# Processamento do ponto "barcode"
# -----

```

```

def _process_barcode(self, pos) -> str | None:

```

```

    meta = pos.camera_params or {}
    w_roi = int(meta.get("width", 400))
    h_roi = int(meta.get("height", 150))

```

```

    ctrl = getattr(self._motion, "_c")
    cam = ctrl.camera_manager

```

```

# ----- calcula ROI central -----
# (basta fazer 1x; w_img/h_img só são precisos para cx/cy)
ok, frame0 = cam._cap.read()
if not ok:
    return "Frame indisponível"
h_img, w_img, _ = frame0.shape
cx, cy = w_img // 2, h_img // 2
x0 = max(0, cx - w_roi // 2)
y0 = max(0, cy - h_roi // 2)
roi = (x0, y0, w_roi, h_roi)

```

```

# =====
# DELAYs:
#   • 0,1 s para estabilizar depois do movimento
#   • se falhar, +0,2 s e tenta novamente
# =====
self.msleep(100)      # 0,1 s

```

```

code = None
for attempt in (1, 2):
    ok, frame = cam._cap.read()
    if not ok:
        return "Frame indisponível"
    results = self._scanner.scan(frame, roi=roi)
    if results:
        code = results[0].data.strip()
        break
# primeira tentativa falhou → feedback vermelho + espera
if attempt == 1:
    self.bcMatch.emit(False, x0, y0, w_roi, h_roi, "")
    self.msleep(200)    # +0,2 s

```

```

if code is None:

```

```
# segunda falha → erro
self.bcMatch.emit(False, x0, y0, w_roi, h_roi, "")
return "Leitura de código de barras falhou (2 tentativas)"
```

```
# ----- sucesso -----
self.bcMatch.emit(True, x0, y0, w_roi, h_roi, code)
```

```
# ----- grava no histórico -----
ctrl = getattr(self._motion, "_c")
pm = getattr(ctrl, "prog_mgr", None)
if pm:
    try:
        if self._session_dir is None:
            self._session_dir = pm.new_log_session(code)
        pm.append_json_log(self._session_dir, {
            "evento": "barcode",
            "codigo": code,
            "t": datetime.now().isoformat()
        })
    except Exception as exc:
        ctrl.log(f"■ Falha ao registrar barcode: {exc}")

ctrl.log(f"★ Barcode lido: {code}")
return None
```

```
# -----
# Processamento do ponto "fiducial"
# -----
def _process_fiducial(self, pos) -> str | None:
    """
    Executa template matching, move a cabeça para o centro
    encontrado e calcula novo offset para o backend.
    Retorna msg de erro ou None em caso de sucesso.
    """
    meta = pos.camera_params or {}
    b64 = meta.get("template_png_b64")
    if not b64:
        return "Fiducial sem template"
    import base64, cv2, numpy as np, math, time

    # ----- converte template b64 → gray -----
    try:
        tmp_png = base64.b64decode(b64)
        tmp = cv2.imdecode(np.frombuffer(tmp_png, np.uint8),
                           cv2.IMREAD_GRAYSCALE)
    except Exception as exc:
        return f"Template inválido: {exc}"

    # captura frame atual (RGB numpy)
    cam = getattr(self._motion, "_c").camera_manager
    ok, frame_bgr = cam._cap.read()
```

```

if not ok:
    return "Frame indisponível"
gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)

# template-matching
res = cv2.matchTemplate(gray, tmp, cv2.TM_CCOEFF_NORMED)
_, max_val, _, max_loc = cv2.minMaxLoc(res)
if max_val*100 < meta.get("threshold", 70):
    return "Fiducial não encontrado (similaridade baixa)"

# coordenadas do centro encontrado
t_h, t_w = tmp.shape
tx, ty = max_loc
cx_t = tx + t_w//2
cy_t = ty + t_h//2

# centro da imagem
h, w = gray.shape
cx_i = w//2
cy_i = h//2

similarity = max_val * 100
dx_pix = cx_t - cx_i
dy_pix = cy_t - cy_i

# ----- converte para pulsos -----
ctrl = getattr(self._motion, "_c")

z_cur = ctrl.current_positions['Z']
# -----
# Determina qual eixo Y físico está vinculado a ESTA execução.
# – PlateFlowManager define ctrl._active_plate_y antes de iniciar
# o SequenceRunnerThread (valor 'Y1' ou 'Y2').
# – Execuções manuais (botão “Executar Sequência” na aba Mesa-1/2
# ou na aba Controle de Eixos) não passam por PlateFlowManager.
# • Se vier da Mesa-1 → y_axis = 'Y1'
# • Mesa-2 → y_axis = 'Y2'
# • Qualquer outra → fallback 'Y1'
# -----
active_y = getattr(ctrl, "_active_plate_y", None)
if active_y not in ("Y1", "Y2"):
    # chamada directa (edição / preview) – deduz da própria posição
    active_y = "Y1" if pos.y1 is not None else "Y2"

dx_p, dy_p = ctrl.pulses_from_pixels(
    dx_pix, dy_pix, z_cur,
    y_axis=active_y
)

# -----
# NÃO movemos a cabeça agora; usamos apenas offset dinâmico.

```

```
# -----
self._dx_acc += dx_p
self._dy_acc += dy_p
if hasattr(self._motion, "apply_dynamic_offset"):
    self._motion.apply_dynamic_offset(self._dx_acc, self._dy_acc)

# ----- FEEDBACK VISUAL -----
# janela encontrada (verde se OK, vermelho se não passou no thresh)
ok_match = similarity >= meta.get("threshold", 70)
self.fidMatch.emit(similarity, ok_match, tx, ty, t_w, t_h)

ctrl.log(f"★ Fiducial OK ΔX={dx_p} ΔY={dy_p} pulsos "
        f"(offset acumulado X={self._dx_acc} Y={self._dy_acc})")
return None
```

```
# -----
# 4) Widget de controle da sequência
# -----
```

```
class SequenceControlWidget(QWidget):
```

```
    """
```

Widget genérico com:

- Botão Executar
- Botão Parar
- Barra de progresso
- Status

Sinais principais do runner são replicados para facilitar
ligação com outros componentes da GUI.

```
    """
```

```
imageCaptured = pyqtSignal(object)
sequenceFinished = pyqtSignal()
sequenceError = pyqtSignal(str)
# encaminha feedback do fiducial aos interessados (TableProgramTab)
fidMatch = pyqtSignal(float, bool, int, int, int, int)
fidClear = pyqtSignal()
bcMatch = pyqtSignal(bool, int, int, int, int, str)
bcClear = pyqtSignal()
progressIdx = pyqtSignal(int)
```

```
def __init__(self,
              motion: MotionBackend,
              camera: CameraBackend | None = None,
              parent=None):
    super().__init__(parent)
    self._motion = motion
    self._camera = camera
    self._positions: List[InspectionPosition] = []
    self._runner: SequenceRunnerThread | None = None
    self._build_ui()
```

```

# ----- API pública -----
def set_positions(self, positions: List[InspectionPosition]):
    """Define/atualiza a lista de posições da sequência."""
    self._positions = positions
    self._status.setText(f'{len(positions)} posições prontas')

def load_positions_from_callable(self, loader: Callable[[], List[InspectionPosition]]):
    """Permite ligar um botão 'Carregar...' externo a esta função."""
    try:
        self.set_positions(loader())
    except Exception as exc:
        print(f'[seq] Falha ao carregar posições: {exc}')
        QMessageBox.critical(self, "Erro", f'Falha ao carregar posições:\n{exc}')

# ----- UI -----
def _build_ui(self):
    grp = QGroupBox("Controle de Sequência")
    v = QVBoxLayout(grp)

    self._btn_execute = QPushButton("Executar Sequência")
    self._btn_stop = QPushButton("Parar")
    self._btn_stop.setEnabled(False)

    # ----- NOVO BOTÃO PAUSE -----
    # Mantém oculto por padrão – apenas a aba “Controle de Eixos”
    # o tornará visível posteriormente.
    self._btn_pause = QPushButton("Pause")
    self._btn_pause.setEnabled(False)
    self._btn_pause.hide()

    self._progress = QProgressBar()
    self._progress.setValue(0)
    self._progress.setAlignment(Qt.AlignmentFlag.AlignCenter)

    self._status = QLabel("Sem sequência")

    # ----- FEEDBACK BARCODE -----
    self._lbl_bc = QLabel("Código: —")
    self._lbl_bc.setStyleSheet("QLabel { background:gray; padding:3px; }")

    v.addWidget(self._btn_execute)
    v.addWidget(self._btn_pause)
    v.addWidget(self._btn_stop)

    # ----- NOVO SELETOR VIEW / APPLY -----
    hmode = QHBoxLayout()
    from PyQt6.QtWidgets import QRadioButton, QLabel as _QLabel
    self.radio_view = QRadioButton("View")
    self.radio_apply = QRadioButton("Apply")
    self.radio_apply.setChecked(True) # default = aplicar offset
    hmode.addWidget(_QLabel("Offset:"))

```

```

hmode.addWidget(self.radio_view)
hmode.addWidget(self.radio_apply)
hmode.addStretch()
v.addLayout(hmode)

v.addWidget(self._progress)
v.addWidget(self._status)

v.addWidget(self._lbl_bc)

main = QVBoxLayout(self)
main.addWidget(grp)
main.addStretch()

# ligações
self._btn_execute.clicked.connect(self._start)
self._btn_stop.clicked.connect(self._stop)
self._btn_pause.clicked.connect(self._pause_toggle)

# troca de modo view/apply
self.radio_view.toggled.connect(
    lambda checked: self._on_mode_changed(checked))

# ----- modo offset -----
def _on_mode_changed(self, view_checked: bool):
    """
    view_checked = True → modo VIEW (não aplica offset)
    False           → radio_apply ativo (aplica offset)
    """
    apply_off = not view_checked
    if hasattr(self._motion, "set_offset_mode"):
        try:
            self._motion.set_offset_mode(apply_off)
        except Exception:
            print("[seq] Falha ao definir modo de offset")
            pass
    self._status.setText("Modo: VIEW (somente visualização)"
                        if view_checked else
                        "Modo: APPLY (produção)")

# ----- handlers -----
def _start(self):
    # ----- verifica posições -----
    if not self._positions:
        QMessageBox.warning(self, "Aviso", "Nenhuma posição definida")
        return
    # ----- encerra runner anterior -----
    if self._runner is not None:
        if self._runner.isRunning():

```

```

        QMessageBox.warning(self, "Execução",
                             "A sequência ainda está em andamento.")
        return
# desconecta e destrói completamente
try:
    self._runner.finished.disconnect()
    self._runner.error.disconnect()
except Exception:
    pass
self._runner.deleteLater()
self._runner = None

self._progress.setRange(0, len(self._positions))
self._progress.setValue(0)
self._status.setText("Executando...")
self._lbl_bc.setText("Código: —")
self._btn_execute.setEnabled(False)
self._btn_stop.setEnabled(True)
self._btn_pause.setEnabled(True)

self._runner = SequenceRunnerThread(
    self._motion,
    self._camera,
    self._positions,
    apply_mode=self.radio_apply.isChecked() # passa modo global
)
self._runner.progress.connect(self._on_progress)
self._runner.imageCaptured.connect(self.imageCaptured)
self._runner.finished.connect(self._on_finished)
self._runner.error.connect(self._on_error)
self._runner.fidMatch.connect(self.fidMatch)
self._runner.fidClear.connect(self.fidClear)
self._runner.bcMatch.connect(self.bcMatch)
self._runner.bcClear.connect(self.bcClear)

# ----- exibe código lido na própria UI -----
self._runner.bcMatch.connect(self._on_bc_match)

self._runner.start()

def _stop(self):
    if self._runner and self._runner.isRunning():
        self._runner.request_stop()

def _on_progress(self, idx: int, total: int):
    self._progress.setValue(idx)
    self._status.setText(f"{idx}/{total} concluídos")
    self.progressIdx.emit(idx)

def _on_finished(self):
    self._status.setText("Concluída")

```

```

self._btn_execute.setEnabled(True)
self._btn_stop.setEnabled(False)
self._btn_pause.setEnabled(False)
self.sequenceFinished.emit()
# ----- encerra thread e limpa ponteiro -----
if self._runner:
    self._runner.wait(1000)
    self._runner.deleteLater()
    self._runner = None

# -----
# P A U S E / R E S U M E (implementação simples)
# -----
def _pause_toggle(self):
    """
    Primeira pressão → pede pausa.
    Segunda pressão → retoma sequencia.
    Implementação minimalista: enquanto pausado o thread
    permanece num loop dormindo.
    """
    if self._runner is None or not self._runner.isRunning():
        return
    # sinaliza flag no runner
    self._runner.toggle_pause()
    if self._runner.is_paused:
        self._status.setText("Pausado")
        self._btn_pause.setText("Continuar")
    else:
        self._status.setText("Executando...")
        self._btn_pause.setText("Pause")

# -----
# B A R C O D E   s l o t
# -----
def _on_bc_match(self, ok: bool, x:int, y:int, w:int, h:int, text:str):
    """
    Atualiza label "Código:" logo abaixo do status.
    • ok = True → mostra texto do código
    • ok = False → indica falha
    A label não é limpa pelo bcClear; permanece visível
    até nova leitura ou até o operador iniciar outra execução.
    """
    if ok and text:
        self._lbl_bc.setText(f"Código: {text}")
    else:
        self._lbl_bc.setText("Código: — (não lido)")

def _on_error(self, msg: str):
    self._status.setText(f"Erro: {msg}")
    self._btn_execute.setEnabled(True)
    self._btn_stop.setEnabled(False)

```

```

self.sequenceError.emit(msg)
if self._runner:
    self._runner.wait(1000)
    self._runner.deleteLater()
    self._runner = None

# -----
# 5) Demonstração stand-alone
# -----
if __name__ == "__main__":
    import sys
    from random import random
    from PyQt6.QtWidgets import QApplication

    # --- Back-ends fictícios -----
    class _DummyMotion:
        def move_to_absolute_position(self, x, y2, y1, z, feed_rate=1000):
            print(f"[motion] X={x:.2f} Y2={y2:.2f} Y1={y1:.2f} Z={z:.2f} F{feed_rate}")
            return True
        def wait_for_idle(self): pass
        def set_feed_rate(self, fr): print("[motion] Feed =", fr)

    class _DummyCamera:
        def capture(self, params=None):
            import numpy as np
            # gera imagem RGB aleatória 320x240
            return (np.random.rand(240, 320, 3) * 255).astype("uint8")

    # --- dados de exemplo -----
    demo_positions = [
        InspectionPosition(f"P{i+1}", random()*100, random()*80, 0.0)
        for i in range(5)
    ]

    # --- aplica ao widget -----
    app = QApplication(sys.argv)
    widget = SequenceControlWidget(_DummyMotion(), _DummyCamera())
    widget.set_positions(demo_positions)
    widget.resize(280, 200)
    widget.show()

    widget.imageCaptured.connect(lambda img: print("[seq] imagem capturada", type(img)))

    sys.exit(app.exec())

```

File: table_program_tab.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# table_program_tab.py
```

```
"""
```

Aba de programação de uma mesa (Mesa 1 ou Mesa 2).

Mostra controles para X, Y (lógico) e Z apenas.

```
"""
```

```
from PyQt6.QtWidgets import QWidget, QGridLayout, QVBoxLayout, QFrame, QSizePolicy, QLabel, QSpinBox, QPushButton
```

```
from PyQt6.QtCore import Qt, QTimer
```

```
from PyQt6.QtGui import QPixmap
```

```
import base64, cv2
```

```
import numpy as np
```

```
from PIL import Image
```

```
import time
```

```
from helpers import read_fiducials
```

```
from sequence_control import SequenceRunnerThread
```

```
from inspection_config_widget import InspectionConfigWidget
```

```
from movement_controls_widget import MovementControlsWidget
```

```
from position_status_widget import PositionStatusWidget
```

```
from inspection_positions_widget import InspectionPositionsWidget
```

```
from program_io_widget import ProgramIOWidget
```

```
from table_positions_backend import TablePositionsBackend
```

```
from sequence_control import SequenceControlWidget, InspectionPosition
```

```
from dot_patterns_widget import DotPatternsWidget
```

```
from dot_action_selector_widget import DotActionSelectorWidget
```

```
from action_config_placeholders import (
```

```
    BarcodeConfigWidget, FiducialConfigWidget, InspectionConfigWidget
```

```
)
```

```
from PyQt6.QtWidgets import QStackedWidget
```

```
from PyQt6.QtCore import pyqtSignal
```

```
# -----
```

```
# QLabel que emite sinal ao clicar – para mover a cabeça
```

```
# -----
```

```
class ClickableLabel(QLabel):
```

```
    # usa float porque ev.position().x()/y() devolvem qreal (float)
```

```
    clicked = pyqtSignal(float, float) # x,y em pixels dentro do label
```

```
    def mousePressEvent(self, ev):
```

```
        if ev.button() == Qt.MouseButton.LeftButton:
```

```
            self.clicked.emit(ev.position().x(), ev.position().y())
```

```
            super().mousePressEvent(ev)
```

```
class TableProgramTab(QWidget):
```

```
    """
```

```
    Mesa 1 → usa eixo físico Y1
```

```
    Mesa 2 → usa eixo físico Y2
```

```
    """
```

```
    def __init__(self, controller, mesa_id: int):
```

```

super().__init__()
assert mesa_id in (1, 2)
self.ctrl = controller
self.mesa = mesa_id      # 1 ou 2
self.y_axis = 'Y1' if mesa_id == 1 else 'Y2'
# orientação do Y: +1 → clicar acima do centro move Y+
#                -1 → clicar abaixo move Y-
# sua câmera está "invertida", portanto usamos +1
self._PIXEL_TO_Y_SIGN = +1
self._build_ui()
# ----- estado da PRÉ-VISUALIZAÇÃO -----
self._preview_runner = None
self._jog_active_axis: str | None = None
# ajuste único inicial (antes da primeira exibição)
QTimer.singleShot(0, self._fix_video_size)
self.ctrl.camera_manager.frameReady.connect(self._update_frame)

# -----
# TAMANHO FIXO DO VÍDEO
# -----
def _fix_video_size(self):
    """Define dimensões fixas
    (máx. 50 % da janela) para o lbl_video mantendo aspecto 4:3."""
    win = self.window()
    if not win:
        return
    # limita a 50% da largura e da altura da janela principal
    max_w = int(win.width() * 0.7)
    max_h = int(win.height() * 0.7)
    # calcula w,h mantendo razão 4:3
    if max_h * 4 < max_w * 3:
        w = int(max_h * 4 / 3)
        h = max_h
    else:
        w = max_w
        h = int(max_w * 3 / 4)
    # aplica tamanho fixo
    self.lbl_video.setSizePolicy(QSizePolicy.Policy.Fixed,
                                QSizePolicy.Policy.Fixed)
    self.lbl_video.setFixedSize(w, h)

def resizeEvent(self, event):
    """
    A cada redimensionamento do widget (ex.: maximize/restore),
    recalcula o tamanho do preview para manter a proporção.
    """
    super().resizeEvent(event)
    self._fix_video_size()

# ----- ATUALIZAÇÃO DO QUADRO -----
def _update_frame(self, img):

```

```

# escala mantendo proporção
# guarda tamanho original para coordenadas
self._last_frame_sz = (img.width(), img.height())
pix = QPixmap.fromImage(img).scaled(
    self.lbl_video.size(), Qt.AspectRatioMode.KeepAspectRatio,
    Qt.TransformationMode.SmoothTransformation)

# ---- desenha cruz -----
p = QPixmap(pix.size())
p.fill(Qt.GlobalColor.transparent)
from PyQt6.QtGui import QPainter, QPen
pa = QPainter(p)
pen = QPen(Qt.GlobalColor.red, 4, Qt.PenStyle.SolidLine)
pa.setPen(pen)
cx = p.width()//2; cy = p.height()//2
pa.drawLine(cx-100, cy, cx+100, cy)
pa.drawLine(cx, cy-100, cx, cy+100)
# -----
# OVERLAY 1 – seletor em “fiducial” (edição)
# -----
if self.action_selector.current_action() == 'fiducial':
    win = self._fid_window
    rad = self._fid_radius
    sx = pix.width() / self._last_frame_sz[0]
    sy = pix.height() / self._last_frame_sz[1]
    # ROI (amarelo cheio)
    pen_y = QPen(Qt.GlobalColor.yellow, 2, Qt.PenStyle.SolidLine)
    pa.setPen(pen_y)
    rw = int(win * sx); rh = int(win * sy)
    pa.drawRect(cx - rw//2, cy - rh//2, rw, rh)
    # ÁREA DE BUSCA (amarelo tracejado)
    pen_d = QPen(Qt.GlobalColor.yellow, 1, Qt.PenStyle.DashLine)
    pa.setPen(pen_d)
    r2 = win + 2*rad
    rw2 = int(r2 * sx); rh2 = int(r2 * sy)
    pa.drawRect(cx - rw2//2, cy - rh2//2, rw2, rh2)
    # Resultado do último teste (2 s)
    if self._match_info:
        rect, color, t0 = self._match_info
        if time.time() - t0 < 2.0:
            pen_m = QPen(color, 2)
            pa.setPen(pen_m)
            x, y, w, h = rect
            pa.drawRect(int(x*sx), int(y*sy), int(w*sx), int(h*sy))
        else:
            self._match_info = None
# -----
# OVERLAY 2 – execução automática
# • Se _match_info existir, desenhamos o retângulo SEM
# limite de tempo; ele será limpo pelo sinal fidClear
# quando o cabeçote chegar ao próximo ponto.

```

```

# -----
elif self._match_info and self.action_selector.current_action() != 'fiducial':
    sx = pix.width() / self._last_frame_sz[0]
    sy = pix.height() / self._last_frame_sz[1]
    rect, color, t0 = self._match_info
    # mantém no máximo 2 s
    if time.time() - t0 > 2.0:
        self._match_info = None
    else:
        x, y, w0, h0 = rect
        pen_m = QPen(color, 2)
        pa.setPen(pen_m)
        pa.drawRect(int(x*sx), int(y*sy), int(w0*sx), int(h0*sy))

elif (self.action_selector.current_action() == 'barcode' or self._bc_match):
    bw, bh = self._bc_w, self._bc_h
    sx = pix.width() / self._last_frame_sz[0]
    sy = pix.height() / self._last_frame_sz[1]
    pen_b = QPen(Qt.GlobalColor.yellow, 2)
    pa.setPen(pen_b)
    # retângulo da ROI (amarelo) – converte para int
    rx = int(cx - bw*sx/2)
    ry = int(cy - bh*sy/2)
    rw = int(bw * sx)
    rh = int(bh * sy)
    pa.drawRect(rx, ry, rw, rh)
    if self._bc_match and time.time()-self._bc_match[2] < 2.0:
        rect, color, t0 = self._bc_match
        pen_m = QPen(color, 2)
        pa.setPen(pen_m)
        x,y,w,h = rect
        pa.drawRect(int(x*sx), int(y*sy), int(w*sx), int(h*sy))
    elif self._bc_match:
        self._bc_match = None
elif self.action_selector.current_action() == 'inspect':
    iw, ih = self._insp_w, self._insp_h
    sx = pix.width()/self._last_frame_sz[0]
    sy = pix.height()/self._last_frame_sz[1]
    pen_i = QPen(Qt.GlobalColor.yellow, 2)
    pa.setPen(pen_i)
    rx = int(cx - iw*sx/2)
    ry = int(cy - ih*sy/2)
    rw = int(iw*sx); rh = int(ih*sy)
    pa.drawRect(rx, ry, rw, rh)
    if self._insp_match and time.time()-self._insp_match[2]<2.0:
        rect,color,t0 = self._insp_match
        pen_m = QPen(color,2); pa.setPen(pen_m)
        x,y,w,h = rect
        pa.drawRect(int(x*sx), int(y*sy), int(w*sx), int(h*sy))
    elif self._insp_match:
        self._insp_match=None

```

```

pa.end()
qp = QPixmap(pix.size())
qp.fill(Qt.GlobalColor.transparent)
painter = QPainter(qp); painter.drawPixmap(0,0,pix); painter.drawPixmap(0,0,p); painter.end()
self.lbl_video.setPixmap(qp)

# ----- overlay helpers -----
def _on_fid_params(self, win:int, rad:int):
    self._fid_window = win
    self._fid_radius = rad
def _on_fid_match(self, sim:float, ok:bool, x:int, y:int, w:int, h:int):
    from PyQt6.QtGui import QColor
    self._match_info = ((x, y, w, h),
                        QColor(Qt.GlobalColor.green if ok else Qt.GlobalColor.red),
                        time.time())

# ----- barcode helpers -----
def _set_barcode_roi(self, w:int, h:int):
    self._bc_w, self._bc_h = w, h
def _on_bc_test(self, ok:bool, x:int, y:int, w:int, h:int):
    from PyQt6.QtGui import QColor
    self._bc_match = ((x,y,w,h),
                    QColor(Qt.GlobalColor.green if ok else Qt.GlobalColor.red),
                    time.time())

# ----- inspeção helpers -----
def _set_insp_roi(self,w:int,h:int):
    self._insp_w=w; self._insp_h=h
def _on_insp_region(self,img):
    # último retângulo verde (sempre OK por definição)
    from PyQt6.QtGui import QColor
    self._insp_match=((0,0,self._insp_w,self._insp_h),
                    QColor(Qt.GlobalColor.green),
                    time.time())

def _build_ui(self):
    grid = QGridLayout(self)
    grid.setContentsMargins(0, 0, 0, 0)

    # proporções
    grid.setColumnStretch(0, 15)
    grid.setColumnStretch(1, 70)
    grid.setColumnStretch(2, 15)
    grid.setRowStretch(0, 7)
    grid.setRowStretch(1, 3)

    # ----- COLUNA ESQUERDA (posições) -----
    v_left = QVBoxLayout()

    aux = self.ctrl.create_auxiliary_controls()
    v_left.addWidget(aux)

```

```

v_left.addStretch()

# ----- SELETOR DE AÇÃO -----
self.action_selector = DotActionSelectorWidget()
v_left.addWidget(self.action_selector)

# ----- STACK DE CONFIGURAÇÕES -----
self._config_stack = QStackedWidget()
# página 0 – DOTS
self.dots_view = DotPatternsWidget()
self._config_stack.addWidget(self.dots_view)
# página 1 – BARCODE (widget real)
self.barcode_cfg = BarcodeConfigWidget(self.ctrl)
self._config_stack.addWidget(self.barcode_cfg)
# página 2 – INSPEÇÃO (widget real)
self.inspect_cfg = InspectionConfigWidget(self.ctrl)
self._config_stack.addWidget(self.inspect_cfg)
# página 3 – FIDUCIAL (widget real – precisa do controller)
self.fiducial_cfg = FiducialConfigWidget(self.ctrl)
self._config_stack.addWidget(self.fiducial_cfg) # index 3

# ----- overlay barcode -----
self._bc_w = 400; self._bc_h = 150
self._bc_match = None # (rect,color,t0)
self.barcode_cfg.parametersChanged.connect(
    lambda w,h: self._set_barcode_roi(w,h))
self.barcode_cfg.testFinished.connect(self._on_bc_test)

# ----- overlay fiducial -----
self._fid_window = 50
self._fid_radius = 80
self._match_info = None # (rect, color, t0)
self.fiducial_cfg.parametersChanged.connect(self._on_fid_params)
self.fiducial_cfg.matchTested.connect(self._on_fid_match)
# ----- overlay inspeção -----
self._insp_w=400; self._insp_h=300
self._insp_match=None
self.inspect_cfg.parametersChanged.connect(
    lambda w,h: self._set_insp_roi(w,h))
self.inspect_cfg.regionCaptured.connect(self._on_insp_region)

v_left.addWidget(self._config_stack, 1) # ocupa o restante

# conecta mudança de ação
self.action_selector.actionChanged.connect(self._show_action_config)
# seleção default: Dot
self.action_selector._group.buttons()[1].setChecked(True) # 'dot'
self._show_action_config('dot')

```

```

w_left = QWidget(); w_left.setLayout(v_left)
grid.addWidget(w_left, 0, 0)

# ----- COLUNA CENTRAL (preview câmera) -----

self.lbl_video = ClickableLabel("Sem vídeo")
self.lbl_video.setAlignment(Qt.AlignmentFlag.AlignCenter)
# Use system palette instead of fixed dark colors, and a native frame
self.lbl_video.setAutoFillBackground(True)
self.lbl_video.setPalette(self.palette())
self.lbl_video.setFrameShape(QFrame.Shape.Box)
self.lbl_video.setFrameShadow(QFrame.Shadow.Sunken)
self.lbl_video.setLineWidth(1)
self.lbl_video.setMidLineWidth(0)
self.lbl_video.setStyleSheet("") # clear any custom CSS
# não deixa o label pedir altura maior que a célula
# policy fixa: será redimensionado manualmente em _fix_video_size(),
# evitando que "vaze" para a coluna da direita
self.lbl_video.setSizePolicy(
    QSizePolicy.Policy.Fixed,
    QSizePolicy.Policy.Fixed)
# mantém o label FIXO (já dimensionado por _fix_video_size) e
# o coloca sempre no centro da célula (h & v).
grid.addWidget(self.lbl_video, 0, 1,
               alignment=Qt.AlignmentFlag.AlignCenter)
# clique → mover cabeça
self.lbl_video.clicked.connect(self._on_video_click)

# ----- COLUNA DIREITA (controles) -----
v_right = QVBoxLayout()

self.mov_widget = MovementControlsWidget() # padrão completo
v_right.addWidget(self.mov_widget)

# limites provenientes do controlador
self._limits = self.ctrl.table_limits[self.mesa]

# Posições de inspeção (X,Y,Z apenas, com validação)
self.inspect_widget = InspectionPositionsWidget(
    get_current_position=self._get_current_position,
    validate_position=self._validate_position,
    get_action_context=self._current_action_context
)

# -----
# Botões EDITAR e REMOVER – tratativa personalizada
# • Remove o handler padrão de remoção (sem confirmação)
# • Conecta nosso slot com caixa de diálogo
# -----
self.inspect_widget.btn_edit.clicked.connect(self._on_edit_clicked)
try:

```

```

        # desconecta handler automático configurado no widget
        self.inspect_widget.btn_remove.clicked.disconnect()
except TypeError:
    pass
self.inspect_widget.btn_remove.clicked.connect(self._on_remove_clicked)

# ■■■■■■ duplo-clique leva a cabeça até a posição ■■■■■■
self.inspect_widget.list_widget.itemDoubleClicked.connect(
    self._on_position_double_clicked)
# qualquer mudança de seleção (clique simples, setas, PgUp, ...)
# bloqueia o botão até que outro duplo-clique seja feito
self.inspect_widget.list_widget.itemSelectionChanged.connect(
    lambda: [self.inspect_widget.btn_edit.setEnabled(False),
             self.inspect_widget.btn_remove.setEnabled(False)])

v_right.addWidget(self.inspect_widget)

# --- backend que salva 3 eixos + extensão dedicada -----
backend = TablePositionsBackend(self.inspect_widget, self.y_axis)
file_filter = f"Programa Mesa {self.mesa} (*.{self.mesa})"
self.prog_widget = ProgramIOWidget(
    backend,
    file_filter=file_filter,
    default_suffix=f".{self.mesa}",
    default_dir=self.ctrl.projects_dir
)
v_right.addWidget(self.prog_widget)
# remove botões Salvar/Carregar da interface desta aba
self.prog_widget.hide()

self.seq_widget = SequenceControlWidget(
    motion=self.ctrl._plc_motion_backend, camera=None)
self.seq_widget.fidMatch.connect(self._on_fid_match)
self.seq_widget.fidClear.connect(lambda: setattr(self, "_match_info", None))
v_right.addWidget(self.seq_widget)

# ----- BARCODE em execução -----
self.seq_widget.bcMatch.connect(self._on_bc_runtime)
self.seq_widget.bcClear.connect(lambda: setattr(self, "_bc_match", None))

# ----- realça linha que está sendo executada -----
self.seq_widget.progressIdx.connect(self._on_exec_progress)

v_right.addStretch()
w_right = QWidget(); w_right.setLayout(v_right)
grid.addWidget(w_right, 0, 2)

# ----- LINHA INFERIOR -----
left_sp = QWidget(); right_sp = QWidget()
for s in (left_sp, right_sp):
    s.setSizePolicy(QSizePolicy.Policy.Expanding,

```

QSizePolicy.Policy.Preferred)

```
bottom = QFrame()
# Use system palette instead of fixed light background and custom border
bottom.setAutoFillBackground(True)
bottom.setPalette(self.palette())
bottom.setFrameShape(QFrame.Shape.StyledPanel)
bottom.setFrameShadow(QFrame.Shadow.Raised)
bottom.setLineWidth(1)
bottom.setMidLineWidth(0)
# garante que a linha 1 tenha "peso" para competir com o vídeo
bottom.setSizePolicy(QSizePolicy.Policy.Expanding,
                    QSizePolicy.Policy.Expanding)

grid.addWidget(left_sp, 1, 0)
grid.addWidget(bottom, 1, 1)
grid.addWidget(right_sp, 1, 2)

# ----- SINAIS -----
self.mov_widget.stepMoveRequested.connect(self._on_step_move)
self.mov_widget.jogStart.connect(self._on_jog_start)
self.mov_widget.jogStop.connect(self._on_jog_stop)
self.mov_widget.goToZeroRequested.connect(self._go_center)

# inserir posição dispara renumeração do gráfico
self.inspect_widget.positionInserted.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))
self.inspect_widget.positionAdded.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))
# NOVO: criação imediata das pastas / imagens
self.inspect_widget.positionAdded.connect(self._on_position_added)
self.inspect_widget.positionRemoved.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))

self.prog_widget.fileLoaded.connect(
    lambda _: self.seq_widget.set_positions(self._to_model()))

# logs
self.prog_widget.fileLoaded.connect(
    lambda f: self.ctrl.log(f"[Mesa {self.mesa}] Programa carregado: {f}"))
self.prog_widget.fileSaved.connect(
    lambda f: self.ctrl.log(f"[Mesa {self.mesa}] Programa salvo: {f}"))

# ----- GRÁFICO -----
# sempre que houver mudança nos pontos ou carregamento de um
# programa, envia nova lista ao MesaPlotWidget da mesa.
for sig in (self.inspect_widget.positionAdded,
            self.inspect_widget.positionRemoved,
            self.prog_widget.fileLoaded):
    sig.connect(lambda _=None: self._update_plot())
```

```

QTimer.singleShot(0, self._update_plot) # primeira vez

# ===== PERSONALIZAÇÃO UI =====
self._adapt_widgets_for_single_y()

# -----
# Seleciona na lista a linha que o runner acabou de concluir
# -----
def _on_exec_progress(self, idx: int):
    """
    Recebe 1-based `idx` do SequenceRunnerThread e realça a linha
    correspondente (idx-1) no QListWidget.
    """
    row = idx - 1
    lw = self.inspect_widget.list_widget
    if 0 <= row < lw.count():
        lw.setCurrentRow(row)
        lw.scrollToItem(lw.item(row))

# -----
def _on_edit_clicked(self):
    """
    Salva edição garantindo que o deslocamento seja
    SUBTRAÍDO antes de gravar as coordenadas base.
    """
    cur_item = self.inspect_widget.list_widget.currentItem()
    if cur_item is None:
        return
    row = self.inspect_widget.list_widget.row(cur_item)
    positions = self.inspect_widget.positions()
    if row >= len(positions):
        return
    # ----- 1. nova AÇÃO -----
    new_meta = self._current_action_context()
    if new_meta is None:
        QMessageBox.warning(self, "Ação inválida",
                            "Configure a ação antes de salvar.")
        return

    # ----- 2. nova POSIÇÃO -----
    # Usa os spinBoxes (última posição comandada) para X, Y, Z
    x = float(self.ctrl.pulsos_spin_X.value())
    if self.y_axis == 'Y1':
        y1 = float(self.ctrl.pulsos_spin_Y1.value())
        y2 = 0.0
    else:
        y2 = float(self.ctrl.pulsos_spin_Y2.value())
        y1 = 0.0
    z = float(self.ctrl.pulsos_spin_Z.value())
    # Subtrai o último offset de fiducial (que foi aplicado para pré-visualização)
    dx_last, dy_last = getattr(self.ctrl._plc_motion_backend, "_last_offset", (0, 0))

```

```

if dx_last or dy_last:
    x -= dx_last
    if self.y_axis == 'Y1':
        y1 -= dy_last
    else:
        y2 -= dy_last
# Limpa o offset dinâmico interno (não altera os spinBoxes nem a posição real)
self.ctrl._plc_motion_backend.apply_dynamic_offset(0, 0)

# ----- Override de Z para 'dot' (conforme pattern) ---
# ■■■■ Override de Z ao editar para ação 'dot':
#   usa a altura do padrão em vez do Z atual
if new_meta.get("action") == "dot":
    z = float(new_meta.get("dot_height", z))

# valida dentro da área da mesa
if callable(self._validate_position):
    err = self._validate_position(x, y2, y1, z)
    if err:
        QMessageBox.warning(self, "Fora dos limites", err)
    return

# ----- 3. aplica ao objeto -----
pos      = positions[row]    # objeto existente
pos.x, pos.y1, pos.y2, pos.z = x, y1, y2, z
pos.camera_params = new_meta
setattr(pos, "meta", new_meta)    # retro-compat.

# ----- 4. refresca lista -----
self.inspect_widget._renumber_items()
self.inspect_widget._renumber_items()
self.inspect_widget.btn_edit.setEnabled(False)
# notifica sequence widget
self.seq_widget.set_positions(self._to_model())
self.ctrl.log(f"■■■ Posição {pos.name} atualizada: "
              f"X={x:.0f} Y={'Y1' if self.y_axis=='Y1' else 'Y2'}="
              f"y1 if self.y_axis=='Y1' else y2:.0f} Z={z:.0f}")

# ----- barcode recebido DURANTE A EXECUÇÃO -----
def _on_bc_runtime(self, ok: bool, x:int, y:int, w:int, h:int, text:str):
    from PyQt6.QtGui import QColor
    color = Qt.GlobalColor.green if ok else Qt.GlobalColor.red
    self._bc_match = ((x, y, w, h), QColor(color), time.time())
    if ok:
        self.ctrl.log(f"■■■ Barcode={text}")

# =====
# DUAS LINHAS → duplo-clique na lista "Posições de Inspeção"
# =====
def _on_position_double_clicked(self, item):
    """

```

- mini-sequência = [todos fiduciais] + [ponto clicado]
- runner aplica `apply_dynamic_offset()` internamente;
- câmera chega EXATAMENTE ao ponto corrigido;
- botão “Editar” é habilitado se a leitura dos fiduciais tiver sucesso.

return

```
if self.y_axis == 'Y1':
```

```

        return InspectionPosition(
            name=p.name,
            x=p.x,
            y1=p.y1,
            y2=None,      # evita "Y2 = 0" que disparava eixo errado
            z=p.z,
            camera_params=cam)
    else:      # Mesa 2
        return InspectionPosition(
            name=p.name,
            x=p.x,
            y2=p.y2,
            y1=None,
            z=p.z,
            camera_params=cam)

positions_model = [_to_ip(p) for p in mini_list]

# bloqueia botões enquanto corre
self.inspect_widget.btn_edit.setEnabled(False)
self.inspect_widget.btn_remove.setEnabled(False)

# -----
# garante que CameraManager possua método .capture(params)
# (SequenceRunnerThread passa um dict ou None)
# -----
if not hasattr(self.ctrl.camera_manager, "capture"):
    def _cam_capture(_params=None, cm=self.ctrl.camera_manager):
        """
        Substituto mínimo para CameraManager.capture expected by
        SequenceRunnerThread. Ignora quaisquer parâmetros e devolve
        o frame BGR ou None.
        """
        ok, frame = (cm._cap.read() if getattr(cm, "_cap", None)
                     else (False, None))
        return frame if ok else None

    setattr(self.ctrl.camera_manager, "capture", _cam_capture)

# -----
# 1) Desliga aplicação do offset câmera↔nozzle
# (somente CAMERA deve chegar ao ponto, não o nozzle)
# 2) Runner em modo VIEW (apply_mode=False)
# -----
if hasattr(self.ctrl._plc_motion_backend, "set_offset_mode"):
    self.ctrl._plc_motion_backend.set_offset_mode(False)

# 3) dispara runner em thread próprio (apply_mode=False garante
# que offset fixo câmera↔nozzle NÃO seja somado)
# -----
self._preview_runner = SequenceRunnerThread(

```

```

        motion=self.ctrl._plc_motion_backend,
        camera=self.ctrl.camera_manager,
        positions=positions_model,
        apply_mode=False)
# SequenceRunnerThread.finished NÃO envia argumentos
# → callback simplificado
self._preview_runner.finished.connect(
    lambda it=item: self._preview_finished(it))
# Se ocorrer erro → aborta edição
self._preview_runner.error.connect(
    lambda msg, it=item: self._preview_failed(msg, it))
# ----- FEEDBACK VISUAL (mesma UX do runner principal) -----
self._preview_runner.fidMatch.connect(self._on_fid_match)
self._preview_runner.fidClear.connect(
    lambda: setattr(self, "_match_info", None))
# opcional: barcode (não afeta ponto editado)
self._preview_runner.bcMatch.connect(self._on_bc_runtime)
self._preview_runner.bcClear.connect(
    lambda: setattr(self, "_bc_match", None))
self._preview_runner.start()

# -----
def _preview_finished(self, item):
    """
    Callback quando micro-runner concluiu.
    Se OK:
        – habilita Editar/Remover;
        – carrega meta nos widgets;
        – mantém offset dinâmico ativo para o usuário.
    Caso erro (fiducial não encontrado, etc.), aborta edição.
    """
    # offset dinâmico obtido ANTES do runner resetar (ver backend)
    dx, dy = self.ctrl._plc_motion_backend._last_offset
    self.ctrl.log(f"■ Pré-visualização concluída – ΔX={dx} ΔY={dy}")

    # habilita botões e carrega meta
    self.inspect_widget.btn_edit.setEnabled(True)
    self.inspect_widget.btn_remove.setEnabled(True)
    row = self.inspect_widget.list_widget.row(item)
    pos = self.inspect_widget.positions()[row]
    self._load_meta_to_widgets(pos.meta or {})

    # garante seleção visual (pode ter sido alterada durante runner)
    self.inspect_widget.list_widget.setCurrentItem(item)

# -----
# Falha na leitura dos fiduciais → aborta edição
# -----
def _preview_failed(self, msg: str, item):
    QMessageBox.warning(self, "Fiducial",
        f"Falha na leitura dos fiduciais:\n{msg}")

```

```

# limpa offset dinâmico
self.ctrl._plc_motion_backend.apply_dynamic_offset(0, 0)
# garante que botões permaneçam desativados
self.inspect_widget.btn_edit.setEnabled(False)
self.inspect_widget.btn_remove.setEnabled(False)
# devolve seleção visual ao item (sem edição)
self.inspect_widget.list_widget.setCurrentItem(item)

# -----
# Remover posição selecionada
# -----
def _on_remove_clicked(self):
    item = self.inspect_widget.list_widget.currentItem()
    if item is None:
        return
    row = self.inspect_widget.list_widget.row(item)
    try:
        pos = self.inspect_widget.positions()[row]
    except IndexError:
        return

    ret = QMessageBox.question(
        self, "Remover posição",
        f"Deseja remover a posição {pos.name} ?",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.Cancel,
        QMessageBox.StandardButton.Cancel)
    if ret != QMessageBox.StandardButton.Yes:
        return

    removed = self.inspect_widget.remove_selected()
    if removed:
        # renumeração e refresh já são feitos pelo widget
        self.seq_widget.set_positions(self._to_model())
        self.ctrl.log(f"■ Posição {removed.name} removida.")
    # desabilita botões até novo duplo-clique
    self.inspect_widget.btn_edit.setEnabled(False)
    self.inspect_widget.btn_remove.setEnabled(False)

# -----
def _load_meta_to_widgets(self, meta: dict):
    """Preenche action_selector & widgets com o meta selecionado."""
    act = meta.get("action", "dot")
    self.action_selector.select_action(act)
    if act == "fiducial":
        self.fiducial_cfg.load_from_meta(meta)
    elif act == "barcode":
        self.barcode_cfg.load_from_meta(meta)
    elif act == "inspect":
        self.inspect_cfg.load_from_meta(meta)
    elif act == "dot":
        self.dots_view.select_pattern_by_id(meta.get("dot_id"))

```

```

# -----
# NOVO – cria estrutura e salva imagens no ato da adição
# -----
def _on_position_added(self, pos):
    """
    Executado logo após o usuário clicar “Adicionar posição atual”.
    Cria as pastas da região / posição mecânica / w* e salva:
        – imagem da região completa (regioes/regiao_<n>/<...>.png)
        – ROI azul de cada janela (.../w*/referencia/referencia.png)
        – JSON mínimo (.../<posicao>_w*.json)
    """

    if not getattr(self.ctrl, "prog_mgr", None):
        # programa ainda não criado (deveria existir)
        return
    meta = pos.meta or {}
    if meta.get("action") != "inspect":
        return
    try:
        # ----- Nomes base -----
        regio_idx = self.inspect_widget.positions().index(pos) + 1
        regio_name = f"regiao_{regio_idx}"
        # Posição mecânica usa o nome dado pelo ROI editor
        componentes = meta.get("componentes", {})
        region_png = meta.get("_region_png")
        if region_png:
            np_img = cv2.imdecode(np.frombuffer(region_png, np.uint8),
                                     cv2.IMREAD_COLOR)
            if np_img is not None and np_img.size:
                self.ctrl.prog_mgr.save_region(
                    regio_name, Image.fromarray(
                        cv2.cvtColor(np_img, cv2.COLOR_BGR2RGB)))
        # ----- w* e janelas -----
        for posicao_nome, comp_data in componentes.items():
            inspecoes = comp_data.get("inspecoes", [])
            for insp in inspecoes:
                # crop ROI azul da imagem da região
                if region_png is None:
                    continue
                # As coordenadas já vêm em pixels absolutos
                x, y = insp.get("posicao", (0, 0))
                w, h = insp.get("tamanho", (0, 0))
                if w == 0 or h == 0:
                    continue
                roi_bgr = np_img[y:y+h, x:x+w].copy()
                if roi_bgr.size == 0:
                    continue
                roi_pil = Image.fromarray(
                    cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2RGB))
                w_nome = insp.get("nome") or insp.get("window_name") \
                    or f"w{inspecoes.index(insp)+1}"

```

```

        # salva estrutura + arquivos
        self.ctrl.prog_mgr.save_blue_reference(
            regiao_name,
            posicao_nome,
            w_nome,
            roi_pil,
            meta=insp    # JSON simples
        )
    except Exception as exc:
        self.ctrl.log(f"■ Erro ao criar estrutura da posição: {exc}")
    # -----
    # NOVO – limpa visores 'Região' e 'ROI' após adicionar INSPEÇÃO
    # -----
    try:
        if self.inspect_cfg:
            self.inspect_cfg.clear_views()
    except Exception as exc:
        self.ctrl.log(f"■ Erro ao limpar visores de inspeção: {exc}")

# -----
# Clique no vídeo → mover cabeça
# -----
def _on_video_click(self, px: float, py: float):
    if not hasattr(self, "_last_frame_sz"):
        return
    # ----- 1. Conversão pixel → mm -----
    # -----
    # Determina retângulo efetivo da imagem dentro do QLabel
    # -----
    disp_w = self.lbl_video.width()
    disp_h = self.lbl_video.height()
    orig_w, orig_h = self._last_frame_sz
    # escala aplicada pelo QPixmap.scaled(KeepAspectRatio)
    scale = min(disp_w / orig_w, disp_h / orig_h)
    pix_w = orig_w * scale
    pix_h = orig_h * scale
    offset_x = (disp_w - pix_w) / 2
    offset_y = (disp_h - pix_h) / 2
    # ignora clique fora da imagem
    if not (offset_x <= px <= offset_x + pix_w and
            offset_y <= py <= offset_y + pix_h):
        return
    # coordenada relativa ao centro da imagem, em pixels
    dx_pix = (px - offset_x - pix_w/2) / scale
    dy_pix = (py - offset_y - pix_h/2) / scale
    # conserta orientação Y
    dy_pix *= self._PIXEL_TO_Y_SIGN
    # ----- 4. Escala MM/PX – modelo óptico -----
    z_cur = self.ctrl.current_positions['Z']
    c    = self.ctrl._fov_coeffs
    w_mm = c["aX"] * z_cur + c["bX"]

```

```

h_mm = c["aY"] * z_cur + c["bY"]
mmpp_x = w_mm / orig_w
mmpp_y = h_mm / orig_h

dx_mm = dx_pix * mmpp_x
dy_mm = dy_pix * mmpp_y
# ----- 2. Converte para pulsos ----
dx_pulses = self.ctrl.pulses_from_mm('X', dx_mm)
dy_pulses = self.ctrl.pulses_from_mm(self.y_axis, dy_mm)

# ----- 3. Calcula alvo absoluto ---
cur_x = self.ctrl.current_positions['X']
cur_y = self.ctrl.current_positions[self.y_axis]
tgt_x = cur_x + dx_pulses
tgt_y = cur_y + dy_pulses

# limita dentro da área de trabalho da mesa
x_min, x_max = self._limits['x']
y_min, y_max = self._limits['y']
tgt_x = max(x_min, min(x_max, tgt_x))
tgt_y = max(y_min, min(y_max, tgt_y))

# deslocamentos corrigidos
dx_corr = tgt_x - cur_x
dy_corr = tgt_y - cur_y

# evita comandos nulos / redundantes
if dx_corr:
    self.ctrl.move_relative('X', int(dx_corr))
if dy_corr:
    self.ctrl.move_relative(self.y_axis, int(dy_corr))

self.ctrl.log(f"■ Clique: ΔX={dx_corr} ΔY={dy_corr} pulsos "
             f"(destino X={tgt_x}, Y={tgt_y})")

# ----- helpers ação -----
def _current_action_context(self) -> dict | None:
    key = self.action_selector.current_action()
    if not key:
        return None
    if key == 'fiducial':
        # precisa haver template capturado
        tmpl = self.fiducial_cfg.template_image()
        if tmpl is None:
            return None
        # Usa diretamente os módulos importados no topo
        _, buf = cv2.imencode('.png', tmpl)
        b64 = base64.b64encode(buf).decode('ascii')
        return {
            'action': 'fiducial',
            'window': self.fiducial_cfg.spin_window.value(),

```

```

        'radius': self.fiducial_cfg.spin_radius.value(),
        'threshold': self.fiducial_cfg.spin_thresh.value(),
        'template_png_b64': b64
    }
elif key == 'barcode':
    w,h = self.barcode_cfg.roi_size()
    return {'action':'barcode', 'width':w, 'height':h}
elif key == 'inspect':
    # 1) Dados completos (posições mecânicas + janelas azuis)
    aux_data = self.inspect_cfg.get_cached_auxiliary_data()
    comp_dict = aux_data.get('componentes', {})

    # -----
    # SANITIZAÇÃO:
    #   • mantemos SOMENTE coordenadas e dimensões das janelas;
    #   • cada janela ('w1' ...) recebe sua própria “similaridade”;
    #   • campo 'nome' é salvo para uso futuro.
    # -----
    sim_min = float(self.inspect_cfg.spin_sim.value()) / 100.0

def _clean_inspecao(idx: int, insp: dict) -> dict:
    return {
        'nome': f'w{idx+1}',
        'posicao': tuple(insp.get('posicao', (0, 0))),
        'tamanho': tuple(insp.get('tamanho', (0, 0))),
        'similaridade': sim_min
    }

serializable_comp = {}

for nome, dados in comp_dict.items():
    insp_list = dados.get('inspecoes', [])
    serializable_comp[nome] = dict(
        posicao = tuple(dados.get('posicao', (0, 0))),
        dimensoes = tuple(dados.get('dimensoes', (0, 0))),
        inspecoes = [_clean_inspecao(i, insp)
                     for i, insp in enumerate(insp_list)]
    )

return {
    'action': 'inspect',
    'componentes': serializable_comp,
    # -----
    # ROI COMPLETA (PNG *bytes*, NÃO base64)
    #   • É apenas um payload temporário para o backend
    #   salvar os .png na árvore de pastas.
    #   • Será removido antes do JSON final, portanto não
    #   aparece no arquivo *.m1 / *.m2.
    # -----
    '_region_png': (lambda _img=self.inspect_cfg
                    ._cached_auxiliary_data

```

```

        .get('region_image'):
        (None if _img is None else
         cv2.imencode('.png', _img)[1].tobytes()))()
    }
elif key == 'dot':
    pat = self.dots_view.current_pattern()
    if pat is None:
        return None    # impede adicionar sem seleção
    return {
        'action':    'dot',
        'dot_id':    pat.id,
        'dot_name':  pat.name,
        'dot_qty':   pat.qty,
        'dot_height': pat.height,
        # freq default será lida da aba Config Registradores
        'dot_freq_hz': self.ctrl.findChild(QSpinBox,
                                           "cfg_spin_D24000_FREQ").value()
    }
else:
    return {'action': key}

# -----
# Exibe página conforme ação escolhida
# -----
def _show_action_config(self, action_key: str):
    index_map = {
        'dot':    0,
        'barcode': 1,
        'inspect': 2,
        'fiducial': 3
    }
    idx = index_map.get(action_key, 0)
    self._config_stack.setCurrentIndex(idx)

# -----
# Atualiza gráfico correspondente à mesa
# -----
def _update_plot(self):
    if not hasattr(self.ctrl, "mesa_plot_widgets"):
        return
    pts = []
    for p in self.inspect_widget.positions():
        y_val = p.y1 if self.y_axis == 'Y1' else p.y2
        pts.append((p.x, y_val))
    self.ctrl.mesa_plot_widgets[self.mesa].update_points(pts)

# ----- pode ser chamado externamente -----
def set_limits(self, limits: dict[str, tuple[int, int]]):
    """Atualiza área de trabalho desta aba."""
    self._limits = limits

```

```

# -----
# valida ponto em relação à área de trabalho desta mesa
# -----
def _validate_position(self, x, y2, y1, z) -> str | None:
    # escolhe o Y físico desta mesa
    y_val = y1 if self.y_axis == 'Y1' else y2

    x_ok = self._limits['x'][0] <= x <= self._limits['x'][1]
    y_ok = self._limits['y'][0] <= y_val <= self._limits['y'][1]

    if not x_ok or not y_ok:
        return (f"Coordenada fora dos limites da Mesa {self.mesa}.\n"
                f"Permitido X:{self._limits['x']} "
                f"Y:{self._limits['y']}\n"
                f"Recebido X={x:.0f} Y={y_val:.0f}")
    return None

# -----

def _adapt_widgets_for_single_y(self):
    """
    Oculta/renomeia elementos de Y1 ou Y2 para que cada aba
    mostre apenas um eixo 'Y'.
    """
    # --- PositionStatusWidget -----
    # --- trata rótulos e valores de Y -----
    # --- Oculta linha Y que não pertence à mesa -----
    if self.y_axis == 'Y1':
        self.mov_widget.lbl_pos_y2.hide()
        self.mov_widget.lbl_pos_y1.setText("0")      # rename
    else:
        self.mov_widget.lbl_pos_y1.hide()
        self.mov_widget.lbl_pos_y2.setText("0")

    # --- MovementControlsWidget (oculta e realinha) -----
    g: QGridLayout = self.mov_widget.grid          # grid de botões

    if self.y_axis == 'Y1':
        # Oculta botões Y2 (btn_up / btn_down) e renomeia Y1
        for b in ('btn_up', 'btn_down'):
            getattr(self.mov_widget, b).hide()
        self.mov_widget.btn_y1_up.setText("Y+")
        self.mov_widget.btn_y1_down.setText("Y-")

        # Reposiciona Y+ / Y- da mesa na coluna do STOP (col 1)
        g.removeWidget(self.mov_widget.btn_y1_up)
        g.removeWidget(self.mov_widget.btn_y1_down)
        g.addWidget(self.mov_widget.btn_y1_up, 0, 1)
        g.addWidget(self.mov_widget.btn_y1_down, 2, 1)
    else:
        # Oculta botões Y1 e renomeia Y2

```

```

self.mov_widget.btn_y1_up.hide()
self.mov_widget.btn_y1_down.hide()
self.mov_widget.btn_up.setText("Y+")
self.mov_widget.btn_down.setText("Y-")

# Reposiciona Y+ / Y- padrão para coluna do STOP
g.removeWidget(self.mov_widget.btn_up)
g.removeWidget(self.mov_widget.btn_down)
g.addWidget(self.mov_widget.btn_up, 0, 1)
g.addWidget(self.mov_widget.btn_down, 2, 1)

# -----
def _get_current_position(self):
    """Retorna posição atual só com o eixo Y físico desta mesa."""
    if self.y_axis == 'Y1':
        return {
            'x': self.ctrl.current_positions['X'],
            'y1': self.ctrl.current_positions['Y1'],
            'y2': 0,
            'z': self.ctrl.current_positions['Z']
        }
    else: # mesa 2 → usa Y2
        return {
            'x': self.ctrl.current_positions['X'],
            'y2': self.ctrl.current_positions['Y2'],
            'y1': 0,
            'z': self.ctrl.current_positions['Z']
        }

# ----- callbacks movimento -----
def _on_step_move(self, axis, dist, feed):
    if axis == 'Y1' or axis == 'Y2':
        axis = self.y_axis # força usar o eixo da mesa
        self.ctrl.move_relative(axis, int(dist))

def _on_jog_start(self, axis, direction, feed):
    # converte Y lógico → eixo físico da mesa
    if axis in ('Y1', 'Y2'):
        axis = self.y_axis

    self._jog_active_axis = axis # memoriza
    self.ctrl.jog_start(axis, '+' if direction > 0 else '-')

def _on_jog_stop(self):
    if self._jog_active_axis:
        self.ctrl.jog_stop(self._jog_active_axis)
        self._jog_active_axis = None

# ----- homing / center -----
def _go_center(self):
    """Leva o cabeçote ao homing virtual configurado p/ esta mesa."""

```

```
self.ctrl.goto_virtual_home(self.mesa)
```

```
# -----
def _to_model(self):
    out = []
    for p in self.inspect_widget.positions():
        # -----
        # GUARDA META COMPLETA NO InspectionPosition
        #
        # • Precisamos do campo dot_qty no runner para escrever
        #   D24100, portanto não pode ser descartado aqui.
        # • camera_params agora recebe **todo** o dict meta
        #   (se existir) preservando "action", "dot_qty", etc.
        # -----
        cam_params = (p.meta.copy()          # dict completo
                      if isinstance(p.meta, dict)
                      else None)
        # Salva somente X, Y (desta mesa) e Z
        if self.y_axis == 'Y1':
            out.append(InspectionPosition(name=p.name,
                                           x=p.x,
                                           y1=p.y1, y2=None,
                                           z=p.z,
                                           camera_params=cam_params))
        else:
            out.append(InspectionPosition(name=p.name,
                                           x=p.x,
                                           y2=p.y2, y1=None,
                                           z=p.z,
                                           camera_params=cam_params))

    return out
```

File: dot_view_height_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
from PyQt6.QtWidgets import (
    QDialog, QVBoxLayout, QHBoxLayout,
    QLabel, QSpinBox, QPushButton
)
from PyQt6.QtCore import Qt

class DotViewHeightDialog(QDialog):
    """
    Diálogo para configurar a altura de visualização de pontos 'dot'.
    """
    def __init__(self, parent=None):
        super().__init__(parent, Qt.WindowType.Window)
        self.setWindowTitle("Altura de Visualização Dot")
        self.setWindowModality(Qt.WindowModality.NonModal)
```

```

self._ctrl = parent
self._build_ui()

def _build_ui(self):
    v = QVBoxLayout(self)
    # linha de edição
    h = QHBoxLayout()
    lbl = QLabel("Altura de visualização (Z):")
    self.spin = QSpinBox()
    # defina limites compatíveis com seu sistema (pulsos ou mm)
    self.spin.setRange(0, 100000)
    h.addWidget(lbl)
    h.addWidget(self.spin)
    v.addLayout(h)

    # botão Salvar
    btn_save = QPushButton("Salvar")
    btn_save.clicked.connect(self._on_save)
    v.addWidget(btn_save, alignment=Qt.AlignmentFlag.AlignRight)

def _on_save(self):
    # persiste no settings.json
    val = self.spin.value()
    s = self._ctrl.settings
    s.data["dot_view_height"] = val
    s.save()
    self._ctrl.log(f"■ Altura de visualização de Dot salva: Z={val}")
    self.close()

```

File: inspection_positions_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

# inspection_positions_widget.py
#
# Widget reutilizável para registrar posições de inspeção
# (c) 2025 – livre para uso sob MIT License

from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, List, Dict, Optional

from PyQt6.QtCore import Qt, pyqtSignal
from PyQt6.QtWidgets import (
    QWidget, QListWidget, QListWidgetItem, QPushButton, QVBoxLayout,
    QHBoxLayout, QLabel, QMessageBox
)

# -----
# Camada de modelo

```

```

# -----
@dataclass
class InspectionPosition:
    name: str
    x: float
    y2: float
    y1: float
    z: float = 0.0
    meta: Dict[str, object] | None = None          # espaço para parâmetros extras

    def __str__(self) -> str:
        return f"{self.name} (X={self.x:.2f}, Y2={self.y2:.2f}, Y1={self.y1:.2f}, Z={self.z:.2f})"

class InspectionPositionListModel:
    """
    Classe mínima para guardar a lista de posições; não depende de Qt.
    """

    def __init__(self) -> None:
        self._positions: List[InspectionPosition] = []

    # ---- API pública -----
    def add(self, pos: InspectionPosition) -> None:
        self._positions.append(pos)

    def remove(self, index: int) -> Optional[InspectionPosition]:
        if 0 <= index < len(self._positions):
            return self._positions.pop(index)
        return None

    def clear(self) -> None:
        self._positions.clear()

    def __iter__(self):
        return iter(self._positions)

    def __len__(self):
        return len(self._positions)

    def __getitem__(self, idx):
        return self._positions[idx]

    def insert(self, idx: int, pos: InspectionPosition) -> None:
        """Insere pos antes de idx (como list.insert)."""
        if idx < 0:
            idx = 0
        elif idx > len(self._positions):
            idx = len(self._positions)
        self._positions.insert(idx, pos)

```

```
def replace(self, idx: int, pos: InspectionPosition) -> None:
    """Substitui posição existente – usado pelo botão Editar."""
    if 0 <= idx < len(self._positions):
        self._positions[idx] = pos
```

```
# -----
# Camada de interface
# -----
```

```
class InspectionPositionsWidget(QWidget):
    """
```

Widget que exibe e gerencia as “Posições de Inspeção”.

Parâmetros do construtor:

```
get_current_position: Callable[[], Dict[str,float]]
    Função/callable que devolve
    {'x':float,'y2':float,'y1':float,'z':float}.
    Pode ser None; nesse caso somente add_position(...) externo
    poderá alimentar a lista.
```

Sinais:

```
positionAdded(InspectionPosition)
positionRemoved(InspectionPosition)
"""
```

```
positionAdded = pyqtSignal(object) # InspectionPosition
positionRemoved = pyqtSignal(object) # InspectionPosition
positionInserted = pyqtSignal(object) # InspectionPosition
positionEdited = pyqtSignal(object)
```

```
def __init__(self,
    get_current_position: Callable[[], Dict[str, float]] | None = None,
    validate_position: Callable[[float,float,float,float], str | None] | None = None,
    get_action_context: Callable[[], Dict[str,object] | None] | None = None,
    parent=None) -> None:
    super().__init__(parent)
    self._get_current_position = get_current_position
    self._validate_position = validate_position
    self._get_action_context = get_action_context
    self._model = InspectionPositionListModel()
    self._build_ui()
```

```
# -----
# API pública para outras partes do programa
# -----
```

```
def add_position(self, pos: InspectionPosition) -> None:
    """Adiciona já com objeto pronto (útil para carregamento de arquivo)."""
    self._model.add(pos)
    self._append_item_to_list(pos)
    self.positionAdded.emit(pos)
```

```

def remove_selected(self) -> Optional[InspectionPosition]:
    """Remove a posição atualmente destacada."""
    item = self.list_widget.currentItem()
    if not item:
        return None
    row = self.list_widget.row(item)
    removed = self._model.remove(row)
    if removed:
        self.list_widget.takeItem(row)
        self._renumber_items()
        self.positionRemoved.emit(removed)
    return removed

def clear(self) -> None:
    self.list_widget.clear()
    self._model.clear()

def positions(self) -> List[InspectionPosition]:
    """Devolve cópia da lista interna."""
    return list(self._model)

# -----
# Construção UI
# -----

def _build_ui(self) -> None:
    v = QVBoxLayout(self)

    title = QLabel("Posições de Inspeção")
    title.setStyleSheet("font-weight:bold;")
    v.addWidget(title)

    # listagem
    self.list_widget = QListWidget()
    v.addWidget(self.list_widget, 1)

    # botões
    h = QHBoxLayout()
    self.btn_add = QPushButton("Adicionar posição atual")
    self.btn_insert = QPushButton("Inserir nova posição")
    self.btn_edit = QPushButton("Editar posição atual")
    self.btn_edit.setEnabled(False) # só libera após duplo-click
    self.btn_remove = QPushButton("Remover posição")
    self.btn_remove.setEnabled(False)
    h.addWidget(self.btn_add)
    h.addWidget(self.btn_insert)
    h.addWidget(self.btn_edit)
    h.addWidget(self.btn_remove)
    v.addLayout(h)

    # conexões
    self.btn_add.clicked.connect(self._on_add_clicked)

```

```
self.btn_insert.clicked.connect(self._on_insert_clicked)
self.btn_remove.clicked.connect(self.remove_selected)
```

```
# -----
```

```
# Slots privados
```

```
# -----
```

```
def _on_add_clicked(self) -> None:
```

```
    """
```

```
    Obtém a posição atual através do callable injetado e solicita
    ao usuário um nome. Gera o objeto InspectionPosition e adiciona
    ao modelo/lista.
```

```
    """
```

```
if not callable(self._get_current_position):
```

```
    QMessageBox.warning(self, "Indisponível",
                        "Função get_current_position não foi fornecida.")
    return
```

```
try:
```

```
    pos_dict = self._get_current_position() or {}
    x = float(pos_dict.get("x", 0.0))
    y2 = float(pos_dict.get("y2", 0.0))
    y1 = float(pos_dict.get("y1", 0.0))
    z = float(pos_dict.get("z", 0.0))
```

```
except Exception as exc:
```

```
    QMessageBox.critical(self, "Erro",
                        f"Não foi possível obter a posição atual:\n{exc}")
    return
```

```
# --- validação opcional -----
```

```
if callable(self._validate_position):
```

```
    err = self._validate_position(x, y2, y1, z)
    if err:
        QMessageBox.warning(self, "Fora dos limites", err)
    return
```

```
# Nome numérico sequencial: 1, 2, 3...
```

```
name = str(len(self._model) + 1)
```

```
# ----- anexa dados de ação -----
```

```
if callable(self._get_action_context):
```

```
    meta = self._get_action_context()
    if meta is None:
        QMessageBox.warning(self, "Ação não definida",
                        "Selecione uma ação antes de adicionar o ponto.")
    return
```

```
else:
```

```
    meta = None
```

```
# somente para 'dot' usamos a altura do padrão em vez do Z atual
```

```
if meta and meta.get("action") == "dot":
```

```
    z = float(meta.get("dot_height", z))
```

```
new_pos = InspectionPosition(name, x, y2, y1, z, meta=meta)
```

```
self._model.add(new_pos)
self._append_item_to_list(new_pos)
self.positionAdded.emit(new_pos)
```

```
def _append_item_to_list(self, pos: InspectionPosition) -> None:
```

```
    txt = (f'{{pos.name}}. "
        f'X={{pos.x:.2f}} Y2={{pos.y2:.2f}} '
        f'Y1={{pos.y1:.2f}} Z={{pos.z:.2f}}")
    if pos.meta and pos.meta.get("action") == "dot":
        txt += f' [dot {{pos.meta.get('dot_qty',1)}}]"
    item = QListWidgetItem(txt)
    self.list_widget.addItem(item)
```

```
# -----
```

```
# Inserir posição EM CIMA do item selecionado
```

```
# -----
```

```
def _on_insert_clicked(self):
```

```
    if not callable(self._get_current_position):
        QMessageBox.warning(self, "Indisponível",
            "Função get_current_position não foi fornecida.")
        return
    cur_item = self.list_widget.currentItem()
    if cur_item is None:
        QMessageBox.warning(self, "Selecione",
            "Selecione a linha onde a nova posição será inserida.")
        return
    row = self.list_widget.row(cur_item)
```

```
# reaproveita a mesma lógica de _on_add_clicked
```

```
# → obtém posição e meta atuais
```

```
if callable(self._get_action_context):
```

```
    meta = self._get_action_context()
    if meta is None:
        QMessageBox.warning(self, "Ação não definida",
            "Selecione uma ação antes de inserir.")
        return
```

```
else:
```

```
    meta = None
```

```
pos_dict = self._get_current_position() or {}
```

```
x = float(pos_dict.get("x", 0))
```

```
y2 = float(pos_dict.get("y2", 0))
```

```
y1 = float(pos_dict.get("y1", 0))
```

```
z = float(pos_dict.get("z", 0))
```

```
# somente para 'dot' usamos a altura do padrão em vez do Z atual
```

```
if meta and meta.get("action") == "dot":
```

```
    z = float(meta.get("dot_height", z))
```

```
new_pos = InspectionPosition(str(row+1), x, y2, y1, z, meta=meta)
```

```
# insere no modelo e list widget
```

```
self._model.insert(row, new_pos)
```

```
self.list_widget.insertItem(row, QListWidgetItem())
```

```
self._renumber_items()
```

```

self.positionInserted.emit(new_pos)

# -----
def _renumber_items(self):
    """Ajusta o campo name e o texto exibido após inserção/remoção."""
    for idx, pos in enumerate(self._model):
        pos.name = str(idx + 1)
        lw_item = self.list_widget.item(idx)
        lw_item.setText(
            f"{pos.name}. X={pos.x:.2f} Y2={pos.y2:.2f} "
            f"Y1={pos.y1:.2f} Z={pos.z:.2f}" +
            (f" [dot {pos.meta.get('dot_qty')}]"
             if pos.meta and pos.meta.get('action') == 'dot' else "")
        )

# -----
# Test-drive rápido do widget isolado
# -----
if __name__ == "__main__":
    import sys
    import random
    from PyQt6.QtWidgets import QApplication

    # Simula um motion controller retornando coordenadas random
    def _mock_current_position():
        return {"x": random.uniform(0, 100),
                "y": random.uniform(0, 100),
                "z": random.uniform(0, 50)}

    app = QApplication(sys.argv)
    w = InspectionPositionsWidget(get_current_position=_mock_current_position)
    w.resize(400, 300)
    w.show()

    # Conecta sinais a prints
    w.positionAdded.connect(lambda p: print("ADDED:", p))
    w.positionRemoved.connect(lambda p: print("REMOVED:", p))

    sys.exit(app.exec())

```

File: dot_patterns_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

from PyQt6.QtWidgets import (
    QDialog, QVBoxLayout, QHBoxLayout, QTreeWidget, QTreeWidgetItem,
    QPushButton, QLineEdit, QSpinBox, QMessageBox
)
from PyQt6.QtCore import Qt

```

```
from dot_patterns import DotPatternManager
```

```
class DotPatternsDialog(QDialog):
```

```
    """
```

```
    Diálogo não-modal (always-on-top) para gerir padrões de Dot.
```

```
    """
```

```
    def __init__(self, parent=None):
```

```
        super().__init__(parent, Qt.WindowType.Window)
```

```
        self.setWindowModality(Qt.WindowModality.NonModal)
```

```
        self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)
```

```
        self.setWindowTitle("Padrões de Dot")
```

```
        self.man = DotPatternManager()
```

```
        self._build_ui()
```

```
        self.man.patternsChanged.connect(self._refresh)
```

```
        self._refresh()
```

```
    def _build_ui(self):
```

```
        v = QVBoxLayout(self)
```

```
        self.tree = QTreeWidget()
```

```
        # adiciona coluna de altura de aplicação
```

```
        self.tree.setHeaderLabels([
```

```
            "ID", "Nome", "Qtd. gotas", "Altura de aplicação"
```

```
        ])
```

```
        self.tree.setColumnWidth(1, 160)
```

```
        self.tree.setColumnWidth(3, 120)
```

```
        v.addWidget(self.tree, 1)
```

```
        # ---- formulário adicionar -----
```

```
        h = QHBoxLayout()
```

```
        self.ed_name = QLineEdit()
```

```
        self.ed_name.setPlaceholderText("Nome")
```

```
        self.sp_qty = QSpinBox()
```

```
        self.sp_qty.setRange(1, 1000)
```

```
        self.sp_qty.setValue(1)
```

```
        # campo de altura de aplicação
```

```
        self.sp_height = QSpinBox()
```

```
        self.sp_height.setRange(0, 10000)
```

```
        self.sp_height.setValue(0)
```

```
        btn_add = QPushButton("Adicionar")
```

```
        btn_del = QPushButton("Excluir")
```

```
        h.addWidget(self.ed_name)
```

```
        h.addWidget(self.sp_qty)
```

```
        h.addWidget(self.sp_height)
```

```
        h.addWidget(btn_add)
```

```
        h.addWidget(btn_del)
```

```
        v.addLayout(h)
```

```
        btn_close = QPushButton("Fechar"); btn_close.clicked.connect(self.hide)
```

```
        v.addWidget(btn_close, 0, Qt.AlignmentFlag.AlignRight)
```

```
        # ---- sinais -----
```

```
        btn_add.clicked.connect(self._add)
```

```
        btn_del.clicked.connect(self._delete)
```

```

# ----- slots -----
def _refresh(self):
    self.tree.clear()
    for p in self.man.patterns():
        QTreeWidgetItem(self.tree,
                        [str(p.id),
                        p.name,
                        str(p.qty),
                        str(p.height)])

def _add(self):
    name = self.ed_name.text().strip()
    qty = self.sp_qty.value()
    height = self.sp_height.value()
    if not name:
        QMessageBox.warning(self, "Nome vazio", "Digite um nome.")
        return
    self.man.add(name, qty, height)
    self.ed_name.clear()
    self.sp_qty.setValue(1)
    self.sp_height.setValue(0)

def _delete(self):
    item = self.tree.currentItem()
    if not item: return
    dot_id = int(item.text(0))
    if QMessageBox.question(self, "Confirma",
        f"Excluir dot #{dot_id} ({item.text(1)})?",
        QMessageBox.StandardButton.Yes |
        QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No) \
        != QMessageBox.StandardButton.Yes:
        return
    self.man.remove(dot_id)

```

File: dot_patterns_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

from PyQt6.QtWidgets import QTreeWidgetItem, QTreeWidgetItem, QGroupBox, QVBoxLayout
from PyQt6.QtCore import Qt
from dot_patterns import DotPatternManager

```

```

class DotPatternsWidget(QGroupBox):

```

```

    """

```

Widget compacto somente-leitura para mostrar os dots existentes.

```

    """

```

```

    def __init__(self, parent=None):

```

```

super().__init__("Padrões de Dot", parent)
self.tree = QTreeWidget()
self.tree.setHeaderLabels(["ID", "Nome", "Qtd", "Altura"])
# opcional: ajustar largura da nova coluna
self.tree.setColumnWidth(3, 80)
v = QVBoxLayout(self); v.addWidget(self.tree)
self.man = DotPatternManager()
self.man.patternsChanged.connect(self._refresh)
self._refresh()

# -----
# API pública: padrão selecionado
# -----
def current_pattern(self):
    """
    Devolve o DotPattern selecionado ou None.
    """
    item = self.tree.currentItem()
    if not item:
        return None
    try:
        dot_id = int(item.text(0))
    except ValueError:
        return None
    for p in self.man.patterns():
        if p.id == dot_id:
            return p
    return None

def _refresh(self):
    self.tree.clear()
    for p in self.man.patterns():
        QTreeWidgetItem(self.tree,
            [str(p.id),
             p.name,
             str(p.qty),
             str(p.height)])

# =====
# Seleciona o padrão cujo `pat_id` coincide com o informado.
# Percorre o QTreeWidget e marca o item correspondente.
# =====
def select_pattern_by_id(self, pat_id: int | str) -> None:
    try:
        pat_id = int(pat_id)
    except Exception:
        return

    tree = self.tree
    for i in range(tree.topLevelItemCount()):
        it = tree.topLevelItem(i)

```

```

if int(it.text(0)) == pat_id:
    tree.setCurrentItem(it)
    tree.scrollToItem(it)
    break

```

File: dot_patterns.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

from __future__ import annotations
import json, typing as _t
from pathlib import Path
from dataclasses import dataclass, asdict
from PyQt6.QtCore import QObject, pyqtSignal

```

```

_FILE = Path(__file__).with_name("dot_patterns.json")

```

```

@dataclass

```

```

class DotPattern:

```

```

    id: int
    name: str
    qty: int = 1
    height: int = 0 # altura de aplicação

```

```

class DotPatternManager(QObject):

```

```

    """

```

Singleton simples que mantém a lista de DotPattern e a persiste em dot_patterns.json. Qualquer mudança emite patternsChanged().

```

    """

```

```

    patternsChanged = pyqtSignal()
    _instance: "DotPatternManager | None" = None
    _patterns: list[DotPattern] = []

```

```

    def __new__(cls):

```

```

        if cls._instance is None:
            cls._instance = super().__new__(cls)
            super(DotPatternManager, cls._instance).__init__()
            cls._instance._patterns = []
            cls._instance._load()
        return cls._instance

```

```

    # -----

```

```

    def _load(self):

```

```

        if _FILE.exists():
            try:
                raw = json.loads(_FILE.read_text("utf-8"))
                self._patterns = [DotPattern(**d) for d in raw]
            except Exception:
                self._patterns = []

```

```

    def _save(self):

```

```

        _FILE.write_text(json.dumps([asdict(p) for p in self._patterns],
                                     indent=2), encoding="utf-8")

# ----- API pública -----
def patterns(self) -> list[DotPattern]:
    return list(self._patterns)

def add(self, name: str, qty: int, height: int = 0) -> DotPattern:
    next_id = (max((p.id for p in self._patterns), default=0)) + 1
    p = DotPattern(next_id, name, qty, height)
    self._patterns.append(p)
    self._save(); self.patternsChanged.emit()
    return p

def remove(self, dot_id:int):
    self._patterns = [p for p in self._patterns if p.id != dot_id]
    self._save(); self.patternsChanged.emit()

```

File: barcode_scanner.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# barcode_scanner.py
```

```
"""
```

Leitura de códigos de barras / QR Codes em imagens estáticas.

Requisitos:

```
pip install opencv-python pyzbar numpy Pillow
```

```
# pyzbar é opcional: se faltar, o módulo continua lendo QR Codes
```

```
# via OpenCV.QRCodeDetector, mas não lerá barcodes lineares.
```

API principal:

```
from barcode_scanner import BarcodeScanner, ScanResult
```

```
scanner = BarcodeScanner() # opções default
```

```
results = scanner.scan(pil_image,
                       roi=(x, y, w, h)) # lista[ScanResult]
```

```
if results:
```

```
    print(results[0].data, results[0].type)
```

```
# imagem com marcações (para depuração / relatório)
```

```
marked = scanner.draw_codes(pil_image, results)
```

```
"""
```

```
from __future__ import annotations
```

```
import cv2
```

```
import numpy as np
```

```
from dataclasses import dataclass
```

```
from typing import List, Sequence, Tuple, Optional, Any
```

```

try:
    from pyzbar.pyzbar import decode as _pyzbar_decode
    _PYZBAR_OK = True
except ImportError:
    # pyzbar é opcional
    _PYZBAR_OK = False

try:
    from PIL import Image
    _PIL_OK = True
except ImportError:
    _PIL_OK = False

# -----
# Estruturas de dados públicas
# -----
Point = Tuple[int, int]      # (x, y)

@dataclass
class ScanResult:
    """Resultado de um único código decodificado."""
    data: str          # conteúdo (texto)
    type: str          # 'CODE128', 'EAN13', 'QRCODE', ...
    polygon: List[Point] | None # pontos do contorno ou None

# -----
# Funções utilitárias privadas
# -----
def _to_bgr(image: Any) -> np.ndarray:
    """
    Converte `image` para ndarray BGR:
    - aceita ndarray BGR / RGB / Gray
    - aceita PIL.Image
    """
    if isinstance(image, np.ndarray):
        arr = image
        if arr.ndim == 2:
            # grayscale
            return cv2.cvtColor(arr, cv2.COLOR_GRAY2BGR)
        if arr.shape[2] == 3:
            # BGR ou RGB
            # heurística simples: média canal B < média canal R → RGB
            return (cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
                    if arr[..., 0].mean() < arr[..., 2].mean()
                    else arr.copy())
        raise ValueError("formato ndarray não suportado")
    if _PIL_OK and isinstance(image, Image.Image):
        img_rgb = image.convert("RGB")
        return cv2.cvtColor(np.array(img_rgb), cv2.COLOR_RGB2BGR)
    raise TypeError("tipo de imagem não suportado")

```

```

def _apply_preprocessing(
    bgr: np.ndarray,
    *,
    clahe_clip: float = 2.0,
    clahe_grid: int = 8,
    blur_sigma: int = 0,
    adaptive_thresh: bool = True
) -> np.ndarray:
    """
    Pipeline leve para melhorar contraste / ruído em barcodes.
    Ajuste os parâmetros conforme a necessidade.
    """
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)

    # CLAHE (equalização adaptativa)
    if clahe_clip > 0:
        clahe = cv2.createCLAHE(clipLimit=clahe_clip,
                                tileGridSize=(clahe_grid, clahe_grid))
        gray = clahe.apply(gray)

    # Blur gaussiano opcional
    if blur_sigma > 0:
        gray = cv2.GaussianBlur(gray, (5, 5), blur_sigma)

    # Binarização adaptativa (melhora contraste)
    if adaptive_thresh:
        gray = cv2.adaptiveThreshold(
            gray, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY,
            21, 10
        )

    return gray

# -----
# Classe principal
# -----
class BarcodeScanner:
    """
    Detector / decodificador de códigos de barras e QR Codes.
    Todos os parâmetros têm valor-padrão seguro; ajuste se necessário.
    """
    def __init__(
        self,
        *,
        clahe_clip: float = 2.0,
        clahe_grid: int = 8,
        blur_sigma: int = 0,

```

```

    adaptive_thresh: bool = True
):
    self.clahe_clip    = clahe_clip
    self.clahe_grid    = clahe_grid
    self.blur_sigma    = blur_sigma
    self.adaptive_thresh = adaptive_thresh
    self._qr_detector  = cv2.QRCodeDetector()

# ----- API pública -----
def scan(
    self,
    image: Any,
    *,
    roi: Optional[Tuple[int, int, int, int]] = None, # (x,y,w,h)
    preprocess: bool = True
) -> List[ScanResult]:
    """
    Decodifica códigos de barras / QR na imagem inteira
    ou dentro de uma ROI (x, y, w, h) em pixels.

    Retorna lista de ScanResult; vazia se nada encontrado.
    """
    bgr = _to_bgr(image)
    if roi is not None:
        x, y, w, h = map(int, roi)
        bgr = bgr[max(0, y):y + h, max(0, x):x + w] # clamp

    proc = (_apply_preprocessing(bgr,
                                clahe_clip=self.clahe_clip,
                                clahe_grid=self.clahe_grid,
                                blur_sigma=self.blur_sigma,
                                adaptive_thresh=self.adaptive_thresh)
            if preprocess else
            cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY))

    results: List[ScanResult] = []

    # -- pyzbar (QR + barcodes lineares) -----
    if _PYZBAR_OK:
        for obj in _pyzbar_decode(proc):
            poly = [(p.x, p.y) for p in obj.polygon] if obj.polygon else None
            results.append(ScanResult(
                data=obj.data.decode("utf-8", errors="ignore"),
                type=obj.type.upper(),
                polygon=poly
            ))

    # -- OpenCV QRCodeDetector (apenas QR) -----
    data, points, _ = self._qr_detector.detectAndDecode(proc)
    if data:
        poly = None

```

```

if points is not None:
    # Garante um array Nx2, mesmo vindo como Nx1x2
    pts = np.array(points)
    if pts.ndim == 3 and pts.shape[1] == 1:
        pts = pts[:, 0, :]
    # achata qualquer forma residual e extrai pares (x,y)
    pts = pts.reshape(-1, 2)
    poly = []
    for x_f, y_f in pts:
        try:
            poly.append((int(x_f), int(y_f)))
        except Exception:
            print("Erro ao converter pontos QR:", x_f, y_f)
            continue
    # evita duplicar se o pyzbar já leu o mesmo QR
    if not any(r.data == data for r in results):
        results.append(ScanResult(
            data=data,
            type="QRCODE",
            polygon=poly
        ))

```

```

return results

```

```

def draw_codes(
    self,
    image: Any,
    codes: Sequence[ScanResult],
    *,
    copy: bool = True
) -> "Image.Image":
    """
    Desenha contornos e legendas dos códigos detectados sobre a imagem.
    Devolve objeto PIL.Image. Se `copy=False`, sobrescreve a própria
    imagem (se for PIL); caso contrário, opera sobre uma cópia.
    """
    if not _PIL_OK:
        raise RuntimeError("Pillow não instalado – draw_codes indisponível")

    pil = image.copy() if copy and _PIL_OK else image
    if not isinstance(pil, Image.Image):
        pil = Image.fromarray(_to_bgr(image)[..., ::-1]) # BGR → RGB

    import PIL.ImageDraw as ImageDraw
    import PIL.ImageFont as ImageFont

    draw = ImageDraw.Draw(pil)
    try:
        font = ImageFont.truetype("arial.ttf", 14)
    except Exception:
        font = ImageFont.load_default()

```

```

for idx, code in enumerate(codes, 1):
    if code.polygon and len(code.polygon) >= 4:
        draw.line(code.polygon + [code.polygon[0]],
                  fill="red", width=3)
        x, y = code.polygon[0]
    else:
        x = y = 10
    label = f"{idx}: {code.type}"
    draw.text((x, y - 15), label, fill="yellow", font=font)
    draw.text((x, y - 30), code.data[:30] + ("..." if len(code.data) > 30 else ""),
              fill="cyan", font=font)

return pil

```

```

# -----
# Execução de teste rápida (python barcode_scanner.py <imagem>)
# -----
if __name__ == "__main__":
    import sys, pathlib, os
    if len(sys.argv) < 2:
        print("Uso: python barcode_scanner.py <imagem.[png|jpg|bmp|etc]> "
              "[x y w h]")
        sys.exit(1)

    img_path = pathlib.Path(sys.argv[1])
    if not img_path.exists():
        print("Arquivo não encontrado:", img_path)
        sys.exit(2)

    roi = None
    if len(sys.argv) == 6:                # ROI passada na linha
        roi = tuple(int(v) for v in sys.argv[2:6])

    from PIL import Image
    pil = Image.open(img_path)

    scanner = BarcodeScanner()
    codes = scanner.scan(pil, roi=roi)

    if not codes:
        print("Nenhum código encontrado.")
    else:
        print(f"{len(codes)} código(s) encontrado(s):")
        for c in codes:
            print(f" - {c.type:<8} {c.data}")

    out = scanner.draw_codes(pil, codes)
    out_name = img_path.with_name(img_path.stem + "_marked.png")
    out.save(out_name)

```

```
print("Imagem marcada salva em:", out_name)
```

File: safety_check.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

safety_check.py

Checklist de Segurança – Interface PyQt6 para verificar acionamento de cortinas.

Memórias do CLP:

- M23: Cortina Mesa 2
- M24: Cortina Mesa 1

Ao abrir:

- Indicadores iniciam em OFF.
- Usuário aciona manualmente as cortinas.
- A cada poll, lê-se M23/M24 via Modbus e atualiza-se o indicador.
- Botão 'Resetar' volta ambos para OFF.

.....

```
import sys
```

```
from PyQt6.QtWidgets import (  
    QApplication, QWidget, QVBoxLayout, QHBoxLayout,  
    QLabel, QPushButton, QMessageBox, QFrame  
)
```

```
from PyQt6.QtCore import QTimer, Qt
```

```
from pymodbus.client import ModbusTcpClient
```

```
class SafetyCheckApp(QWidget):
```

```
    def __init__(self,  
        plc_host: str = "192.168.1.5",  
        plc_port: int = 502,  
        poll_interval: int = 500 # ms  
    ):
```

```
        super().__init__()
```

```
        self.setWindowTitle("Checklist de Segurança")
```

```
        self.resize(320, 150)
```

```
# Conecta ao CLP via Modbus TCP
```

```
self.plc = ModbusTcpClient(plc_host, port=plc_port)
```

```
if not self.plc.connect():
```

```
    QMessageBox.critical(  
        self, "Erro de Conexão",  
        f"Não foi possível conectar ao CLP em {plc_host}:{plc_port}"  
    )
```

```
# Monta UI com LEDs e descrições
```

```
vbox = QVBoxLayout(self)
```

```
# M23 – Cortina Mesa 2
```

```

h_m23 = QHBoxLayout()
self.led_m23 = QFrame(self)
self.led_m23.setFixedSize(16, 16)
self.led_m23.setStyleSheet(self._led_off_style())
h_m23.addWidget(self.led_m23)
self.lbl_m23 = QLabel("Cortina Mesa 2 (M23)", self)
h_m23.addWidget(self.lbl_m23)
# Indicador em tempo real M23
self.rt_led_m23 = QFrame(self)
self.rt_led_m23.setFixedSize(16, 16)
self.rt_led_m23.setStyleSheet(self._led_off_style())
h_m23.addWidget(self.rt_led_m23)
vbox.addLayout(h_m23)

# M24 – Cortina Mesa 1
h_m24 = QHBoxLayout()
self.led_m24 = QFrame(self)
self.led_m24.setFixedSize(16, 16)
self.led_m24.setStyleSheet(self._led_off_style())
h_m24.addWidget(self.led_m24)
self.lbl_m24 = QLabel("Cortina Mesa 1 (M24)", self)
h_m24.addWidget(self.lbl_m24)
# Indicador em tempo real M24
self.rt_led_m24 = QFrame(self)
self.rt_led_m24.setFixedSize(16, 16)
self.rt_led_m24.setStyleSheet(self._led_off_style())
h_m24.addWidget(self.rt_led_m24)
vbox.addLayout(h_m24)

# M25 – Porta de Segurança / E-Stop
h_m25 = QHBoxLayout()
self.led_m25 = QFrame(self)
self.led_m25.setFixedSize(16, 16)
self.led_m25.setStyleSheet(self._led_off_style())
h_m25.addWidget(self.led_m25)
self.lbl_m25 = QLabel("Porta de Segurança/E-Stop (M25)", self)
h_m25.addWidget(self.lbl_m25)
# Indicador em tempo real M25
self.rt_led_m25 = QFrame(self)
self.rt_led_m25.setFixedSize(16, 16)
self.rt_led_m25.setStyleSheet(self._led_off_style())
h_m25.addWidget(self.rt_led_m25)
vbox.addLayout(h_m25)
# Flags para manter indicador acionado após primeiro disparo
self.m23_ok = False
self.m24_ok = False
self.m25_ok = False

# Botões de controle
hbox = QHBoxLayout()
btn_reset = QPushButton("Resetar", self)

```

```
btn_reset.clicked.connect(self.reset_indicators)
btn_quit = QPushButton("Sair", self)
btn_quit.clicked.connect(self.close)
hbox.addWidget(btn_reset)
hbox.addWidget(btn_quit)
vbox.addLayout(hbox)
```

```
# Timer para polling periódico
self.timer = QTimer(self)
self.timer.timeout.connect(self.update_states)
self.timer.start(poll_interval)
```

```
def _on_style(self) -> str:
    # não utilizado para LEDs
    return ""
```

```
def _off_style(self) -> str:
    # não utilizado para LEDs
    return ""
```

```
def _led_on_style(self) -> str:
    return "QFrame { background-color: green; border-radius: 8px; }"
```

```
def _led_off_style(self) -> str:
    return "QFrame { background-color: red; border-radius: 8px; }"
```

```
def update_states(self):
    """
    Lê M23 e M24 (coils 23/24) do CLP e atualiza os indicadores.
    """
    for mem, label, nome in (
        (23, self.lbl_m23, "Cortina Mesa 2"),
        (24, self.lbl_m24, "Cortina Mesa 1"),
        (25, self.lbl_m25, "Porta de Segurança/E-Stop"),
    ):
        try:
            res = self.plc.read_coils(mem, count=1)
            # atualiza indicador em tempo real
            rt_led = getattr(self, f"rt_led_m{mem}")
            if not res.isError() and res.bits[0]:
                rt_led.setStyleSheet(self._led_on_style())
            else:
                rt_led.setStyleSheet(self._led_off_style())
            # se disparou ao menos uma vez, fixa OK
            if not res.isError() and res.bits[0]:
                if mem == 23:
                    self.m23_ok = True
                elif mem == 24:
                    self.m24_ok = True
                else: # mem == 25
                    self.m25_ok = True
```

```

# exibe conforme flag persistente
if mem == 23:
    flag = self.m23_ok
elif mem == 24:
    flag = self.m24_ok
else:
    flag = self.m25_ok
# atualiza cor do LED permanente conforme flag
led = getattr(self, f"led_m{mem}")
if flag:
    led.setStyleSheet(self._led_on_style())
else:
    led.setStyleSheet(self._led_off_style())
except Exception:
    # em caso de erro, zera indicador em tempo real
    getattr(self, f"rt_led_m{mem}").setStyleSheet(self._led_off_style())
    label.setText(f"{nome} (M{mem}): ERRO")
    label.setStyleSheet(self._off_style())

```

```
def reset_indicators(self):
```

```
    """
```

```
    Volta ambos os indicadores para estado OFF.
```

```
    """
```

```
    # reseta flags e mostra todos em OFF
```

```
    self.m23_ok = False
```

```
    self.m24_ok = False
```

```
    self.m25_ok = False
```

```
    # reseta LEDs para estado OFF (vermelho)
```

```
    self.led_m23.setStyleSheet(self._led_off_style())
```

```
    self.led_m24.setStyleSheet(self._led_off_style())
```

```
    self.led_m25.setStyleSheet(self._led_off_style())
```

```
def closeEvent(self, event):
```

```
    # Fecha conexão Modbus ao sair
```

```
    try:
```

```
        self.plc.close()
```

```
    except Exception as e:
```

```
        print(f'Erro ao fechar conexão Modbus: {e}')

```

```
    event.accept()
```

```
if __name__ == "__main__":
```

```
    app = QApplication(sys.argv)
```

```
    win = SafetyCheckApp()
```

```
    win.show()
```

```
    sys.exit(app.exec())
```

File: mesa_plot_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

"""

mesa_plot_widget.py

Widget (QGraphicsView) que desenha a área de trabalho de uma mesa,
os pontos carregados no programa e a posição atual da cabeça.

Usa apenas Qt (sem dependências externas).

"""

```
from typing import List, Tuple, Optional
```

```
from PyQt6.QtWidgets import QGraphicsView, QGraphicsScene, QGraphicsEllipseItem, \
    QGraphicsRectItem, QGraphicsSimpleTextItem
```

```
from PyQt6.QtGui import QPen, QBrush, QColor, QPainter, QPalette
```

```
from PyQt6.QtCore import QRectF, Qt, QSizeF
```

```
class MesaPlotWidget(QGraphicsView):
```

```
    def __init__(self,
```

```
        limits_xy: dict[str, Tuple[int, int]],
```

```
        *,
```

```
        steps_per_mm: dict[str, float],
```

```
        y_axis: str,
```

```
        title: str = "",
```

```
        parent=None):
```

```
    super().__init__(parent)
```

```
    # ----- NOVO: unidades em milímetros -----
```

```
    self._steps = steps_per_mm          # {'X':..., 'Y1':..., 'Y2':..., 'Z':...}
```

```
    self._y_axis = y_axis                # 'Y1' ou 'Y2'
```

```
    # Converte limites de pulsos → milímetros
```

```
    x0_mm = limits_xy['x'][0] / self._steps['X']
```

```
    x1_mm = limits_xy['x'][1] / self._steps['X']
```

```
    y0_mm = limits_xy['y'][0] / self._steps[self._y_axis]
```

```
    y1_mm = limits_xy['y'][1] / self._steps[self._y_axis]
```

```
    # -----
```

```
    # Mantemos a proporção REAL da área de trabalho
```

```
    # (sem forçar o retângulo a virar um quadrado).
```

```
    # -----
```

```
    self._scale_y = 1.0          # compatibilidade interna
```

```
    self._lim = {
```

```
        'x': (x0_mm, x1_mm),
```

```
        'y': (y0_mm, y1_mm)
```

```
    }
```

```
    self._scene = QGraphicsScene(self)
```

```
    self.setScene(self._scene)
```

```
    # Qt 6 ⇒ enum está em QPainter.RenderHint
```

```
    self.setRenderHints(
```

```
        self.renderHints()
```

```
        | QPainter.RenderHint.Antialiasing
```

```
        | QPainter.RenderHint.TextAntialiasing
```

```

)
# elementos estáticos
self._build_static(title)

# barras de rolagem sempre ocultas
self.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
self.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
# dinâmicos
self._point_items: List[QGraphicsEllipseItem] = []
self._head_item: Optional[QGraphicsEllipseItem] = None

# -----
# Construção do retângulo e título
# -----
def _build_static(self, title:str):
    self._scene.clear()
    # Atenção: usamos limites COM y já escalonado
    x0,x1 = self._lim['x']; y0,y1 = self._lim['y']
    w = x1-x0; h = y1-y0
    # retângulo origem (0,0). Deixamos margem 5 %
    margin = 0.05*max(w,h)
    rect = QRectF(x0-margin, y0-margin, w+2*margin, h+2*margin)
    self._work_rect = rect
    self.setSceneRect(rect)
    # adapt grid frame to system theme
    # use ColorRole.Dark instead of deprecated QPalette.Dark
    pen = QPen(self.palette().color(QPalette.ColorRole.Dark)); pen.setWidth(2)
    brush = QBrush(Qt.BrushStyle.NoBrush)
    self._scene.addRect(QRectF(x0, y0, w, h), pen, brush)
    # título
    # ----- T Í T U L O -----
    tit = QGraphicsSimpleTextItem(title)
    # Não sofre espelhamento / zoom
    tit.setFlag(
        QGraphicsSimpleTextItem.GraphicsItemFlag.ItemIgnoresTransformations)
    # 20 mm (aprox.) abaixo da borda inferior
    tit.setPos(x0, y0 - 20)
    self._scene.addItem(tit)

    # invert-Y para ficar origem em baixo-esquerda
    self.scale(1, -1)
    # força ajuste inicial
    self._fit()

# -----
# Interface pública
# -----
# ----- helper pulsos → mm -----
def _p2mm(self, axis: str, pulses: float) -> float:
    return pulses / self._steps.get(axis, 1.0)

```

```

def update_points(self, points: List[Tuple[float,float]]):
    """Lista [(x,y), ...] em pulsos."""
    # remove antigos
    for it in self._point_items:
        self._scene.removeItem(it)
    self._point_items.clear()
    # -----
    # 1) Corrige ESPELHAMENTO X (imagem tipo "página virada")
    # 2) Aplica factor _scale_y para manter proporção 1:1
    # -----
    x_min, x_max = self._lim['x']
    # adiciona
    # adapt point color to system highlight
    # adapt point color to system highlight role
    pen = QPen(self.palette().color(QPalette.ColorRole.Highlight)); pen.setWidth(0)
    brush = QBrush(self.palette().color(QPalette.ColorRole.Highlight))
    r_pix = 6 # diâmetro em pixels (fixo)
    for (x_raw, y_raw) in points:
        # ----- conversão pulsos → mm + ajustes -----
        x_mm = self._p2mm('X', x_raw)
        y_mm = self._p2mm(self._y_axis, y_raw)

        # espelhamento X
        x_plot = x_min + x_max - x_mm
        y_plot = y_mm
        e = self._scene.addEllipse(
            x_plot, y_plot, 0, 0, pen, brush) # placeholder
        # tamanho fixo – ignora transformações (pixels na tela)
        e.setRect(-r_pix/2, -r_pix/2, r_pix, r_pix)
        e.setPos(x_plot, y_plot)
        e.setFlag(QGraphicsEllipseItem.GraphicsItemFlag.ItemIgnoresTransformations)
        self._point_items.append(e)

def update_head_position(self, x: int, y: int):
    """Atualiza / cria círculo vermelho da cabeça."""
    r_pix = 10
    # conversões para mm
    x_mm = self._p2mm('X', x)
    y_mm = self._p2mm(self._y_axis, y)

    # espelhamento X
    x_min, x_max = self._lim['x']
    x_plot = x_min + x_max - x_mm
    y_plot = y_mm

    # ----- visibilidade só dentro da área útil -----
    in_x = self._lim['x'][0] <= x_mm <= self._lim['x'][1]
    in_y = self._lim['y'][0] <= y_mm <= self._lim['y'][1]
    if not (in_x and in_y):
        if self._head_item is not None:
            self._head_item.setVisible(False)

```

```
return # ignora pontos fora da área
```

```
if self._head_item is None:
```

```
    # adapt head color to system highlighted-text role
```

```
    pen = QPen(self.palette().color(QPalette.ColorRole.HighlightedText)); pen.setWidth(0)
```

```
    brush = QBrush(self.palette().color(QPalette.ColorRole.HighlightedText))
```

```
    self._head_item = self._scene.addEllipse(  
        -r_pix/2, -r_pix/2, r_pix, r_pix, pen, brush)
```

```
    self._head_item.setFlag(  
        QGraphicsEllipseItem.GraphicsItemFlag.ItemIgnoresTransformations)
```

```
    self._head_item.setPos(x_plot, y_plot)
```

```
    # caso estivesse oculto e volte para a área útil
```

```
    self._head_item.setVisible(True)
```

```
    self._head_item.setPos(x_plot, y_plot)
```

```
# -----
```

```
# Ajusta automaticamente o zoom
```

```
# -----
```

```
def resizeEvent(self, ev):
```

```
    super().resizeEvent(ev)
```

```
    self._fit()
```

```
def _fit(self):
```

```
    """Ajusta o retângulo de trabalho para caber na view."""
```

```
    if hasattr(self, "_work_rect"):
```

```
        self.fitInView(self._work_rect, Qt.AspectRatioMode.KeepAspectRatio)
```

File: camera_manager.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# camera_manager.py
```

```
import cv2, sys
```

```
from PyQt6.QtCore import QObject, pyqtSignal, QTimer
```

```
from PyQt6.QtGui import QImage, QPixmap
```

```
class CameraManager(QObject):
```

```
    frameReady = pyqtSignal(QImage)
```

```
def __init__(self, cam_index: int = 0, parent=None):
```

```
    super().__init__(parent)
```

```
    self._idx = cam_index
```

```
    self._cap = None
```

```
    self._timer = QTimer(self)
```

```
    self._timer.timeout.connect(self._grab)
```

```
    self.open(self._idx)
```

```
    # coeficientes foco = a·Z+b (definidos pelo diálogo)
```

```
    self._a = 0.0
```

```
    self._b = 0.0
```

```
    self._auto_enabled = True
```

```

# -----
def open(self, idx: int):
    self.close()
    self._idx = idx
    self._cap = cv2.VideoCapture(idx, cv2.CAP_DSHOW if sys.platform.startswith("win") else 0)
    if self._cap.isOpened():
        self._timer.start(30)    # ~33 fps máx
        # tenta deixar o foco em modo manual
        self._cap.set(cv2.CAP_PROP_AUTOFOCUS, 0)
    else:
        self._timer.stop()

# ----- foco manual -----
def set_focus(self, value: int):
    """Define foco manual. Range expandido 0-1023; converte se driver
    limitar a 0-255. Desliga AF toda vez para garantir efeito."""
    if self._cap is None:
        return
    # força AF = OFF
    self._cap.set(cv2.CAP_PROP_AUTOFOCUS, 0)
    # descobre faixa aceita
    ok = self._cap.set(cv2.CAP_PROP_FOCUS, int(value))
    if not ok:
        # talvez range 0-255
        val255 = int(value*255/1023)
        self._cap.set(cv2.CAP_PROP_FOCUS, val255)

# chamada periódica pelo controlador
def auto_focus(self, z_pulses: int):
    if not self._auto_enabled:
        return
    if self._a == 0.0 and self._b == 0.0:
        return    # não calibrado
    foc = int(self._a * z_pulses + self._b)
    foc = max(0, min(foc, 1023))
    self.set_focus(foc)

def load_calibration(self, a: float, b: float):
    self._a, self._b = a, b

# ----- liga / desliga auto-focus -----
@property
def auto_enabled(self) -> bool:
    return self._auto_enabled

def enable_auto_focus(self, enabled: bool):
    self._auto_enabled = enabled

def save_calibration(self):
    return {"a": self._a, "b": self._b}

def close(self):

```

```

self._timer.stop()
if self._cap is not None:
    self._cap.release()
    self._cap = None

# -----
def _grab(self):
    if self._cap is None: return
    ok, frame = self._cap.read()
    if not ok: return
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    h, w, _ = frame.shape
    img = QImage(frame.data, w, h, QImage.Format.Format_RGB888)
    self.frameReady.emit(img)

```

File: hook-pyzbar.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
from PyInstaller.utils.hooks import collect_dynamic_libs
```

```

binaries = collect_dynamic_libs('pyzbar')
hiddenimports = ['pyzbar.pyzbar']

```

File: inspection_logger.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

import os
import json
import datetime
import requests
import logging
from typing import Optional, Tuple, Dict, Any, Union

```

```

# module level logger
logger = logging.getLogger(__name__)

```

```
class InspectionLogger:
```

```
    """
```

Classe responsável por gerenciar todas as operações relacionadas a logs e envio de dados durante a inspeção visual.

```
    """
```

```

def __init__(self,
    parent=None,
    config_manager=None,
    *,
    api_url: Optional[str] = None,
    json_posto: str = "",

```

```
        json_usuario: str = "",
        json_linha: str = ",");
"""
```

Inicializa o logger de inspeção.

Parâmetros opcionais (injeção direta ou via config_manager):

```
api_url: URL para envio SFCS
json_posto, json_usuario, json_linha
log_dir, images_dir
http_timeout (segundos)
session (requests.Session)
logger_ (logging.Logger)
"""
```

```
self.parent = parent
self.config_manager = config_manager
```

```
# runtime■configurable fields
```

```
self.api_url: str = ""
self.json_posto: str = ""
self.json_usuario: str = ""
self.json_linha: str = ""
```

```
# directories
```

```
self.log_dir: Optional[str] = None
self.images_dir: Optional[str] = None
```

```
# HTTP/session/logger
```

```
self.http_timeout: float = 5.0
self._session: Optional[requests.Session] = None
self._logger: logging.Logger = logger
```

```
# load from config_manager if provided
```

```
if config_manager:
```

```
    self.api_url = config_manager.get_config('api_url', self.api_url)
    self.json_posto = config_manager.get_config('json_posto', self.json_posto)
    self.json_usuario = config_manager.get_config('json_usuario', self.json_usuario)
    self.json_linha = config_manager.get_config('json_linha', self.json_linha)
```

```
# else: leave defaults or override via setters/env
```

```
    self.json_posto = config_manager.get_config('json_posto', "")
    self.json_usuario = config_manager.get_config('json_usuario', "")
    self.json_linha = config_manager.get_config('json_linha', "")
```

```
else:
```

```
    self.api_url = ""
```

```
    self.json_posto = ""
    self.json_usuario = ""
    self.json_linha = ""
```

```
# nível mínimo para o root logger (padrão INFO)
```

```
logging.basicConfig(level=logging.INFO)
```

```
# suprime logs verbosos do pymodbus (ex.: "Frame advanced")
logging.getLogger('pymodbus.logging').setLevel(logging.WARNING)
# opcional: também silencia todos os logs do pymodbus
logging.getLogger('pymodbus').setLevel(logging.WARNING)
```

```
def set_log_directory(self, log_dir):
    """Define o diretório para armazenar logs"""
    self.log_dir = log_dir
    os.makedirs(log_dir, exist_ok=True)
```

```
def set_images_directory(self, images_dir):
    """Define o diretório para armazenar imagens"""
    self.images_dir = images_dir
    os.makedirs(images_dir, exist_ok=True)
```

```
def generate_log_json(self, components, codigo_barras=None, codigo_externo=None):
    """
```

Gera um arquivo JSON estruturado com os resultados da inspeção.

Args:

components: Dicionário com os componentes inspecionados
codigo_barras: Informações do código de barras detectado
codigo_externo: Código externo, se fornecido pelo usuário

Returns:

Tuple com os dados do log, o caminho para o arquivo detalhado e o caminho para o arquivo SFCS

```
"""
if not self.log_dir:
    logging.error("Diretório de log não definido")
    return None, None, None
```

```
# Preparar informações de código
codigo_info = {
    "valor": "Não Identificado",
    "tipo": "Desconhecido",
    "encontrado": False
}
```

```
# Usar código externo se disponível
```

```
if codigo_externo:
    codigo_info = {
        "valor": codigo_externo,
        "tipo": "EXTERNO",
        "encontrado": True
    }
```

```
# Senão, usar código de barras se disponível
```

```
elif codigo_barras and isinstance(codigo_barras, dict) and "valor" in codigo_barras and codigo_barras["valor"]:
    codigo_info = codigo_barras
```

```
try:
```

```
    # Criar pasta baseada em timestamp e código
```

```

timestamp_str = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
if codigo_info["encontrado"] and codigo_info["valor"].strip():
    folder_name = f"{timestamp_str}_{codigo_info['valor']}"
else:
    folder_name = timestamp_str

destino = os.path.join(self.log_dir, folder_name)
os.makedirs(destino, exist_ok=True)

if hasattr(self.parent, 'imagem_map') and self.parent.imagem_map is not None:
    try:
        image_path = os.path.join(destino, "placa.png")
        # Garantir formato PNG original de alta qualidade
        self.parent.imagem_map.save(image_path, format="PNG", compress_level=0)
        logging.info("Imagem da placa completa salva em: %s", image_path)
    except Exception as e:
        logging.error(f"Erro ao salvar imagem da placa: {str(e)}")

# Formato de data/hora para os logs
data_hora_formatada = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# Criar estrutura de dados do log detalhado (formato original)
log_data = {
    "posto": self.json_posto,
    "usuario": self.json_usuario,
    "data_hora": data_hora_formatada,
    "quantidade_componentes": len([c for c in components.values() if isinstance(c, dict)]),
    "codigo_barras": codigo_info,
    "componentes": {}
}

# Adicionar informações de cada componente ao log detalhado
for comp, info in components.items():
    if isinstance(info, dict):
        componente_log = {
            "status": "Aprovado" if info.get("status", False) else "Reprovado",
            "similaridade_minima": info.get("similaridade_minima"),
            "posicao_componente": info.get("posicao"),          # [x, y] absolutos (px)
            "dimensoes_componente": info.get("dimensoes"),    # [w, h] absolutos (px)
            "similaridade": info.get("similaridade"),
            "janela_azul": info.get("janela_azul_config"),
            "janelas_vermelhas": []
        }

        if "inspecoes" in info:
            for i, janela in enumerate(info["inspecoes"]):
                janela_log = {
                    "habilitada": janela.get("habilitada", True),
                    "posicao": janela.get("posicao"),
                    "tamanho": janela.get("tamanho"),
                    "threshold": janela.get("threshold", 128),

```

```

        "cor_pixel": janela.get("cor_pixel"),
        "percentual_minimo": janela.get("percentual_minimo"),
        "resultado": janela.get("resultado", 0),
        "status": "Aprovado" if janela.get("status", False) else "Reprovado"
    }
    componente_log["janelas_vermelhas"].append(janela_log)

    log_data["componentes"][comp] = componente_log

# Salvar arquivo JSON detalhado (formato original)
log_filename = f"{folder_name}.json"
log_path = os.path.join(destino, log_filename)

with open(log_path, 'w', encoding='utf-8') as log_file:
    json.dump(log_data, log_file, indent=4, ensure_ascii=False)

# NOVO: Criar estrutura de dados do log simplificado (SFCS)
sfcs_data = {
    "posto": self.json_posto,
    "usuario": self.json_usuario,
    "data_hora": data_hora_formatada,
    "quantidade_componentes": len([c for c in components.values() if isinstance(c, dict)]),
    # Extrair apenas o valor do código de barras para o formato simplificado
    "codigo_barras": codigo_info.get("valor", "Não Identificado"),
    "componentes": {}
}

# Adicionar informações simplificadas de cada componente (1=aprovado, 0=reprovado)
for comp, info in components.items():
    if isinstance(info, dict):
        sfcs_data["componentes"][comp] = 1 if info.get("status", False) else 0

# Salvar arquivo JSON simplificado (SFCS)
sfcs_filename = f"{timestamp_str}_SFCS.json"
sfcs_path = os.path.join(destino, sfcs_filename)

with open(sfcs_path, 'w', encoding='utf-8') as sfcs_file:
    json.dump(sfcs_data, sfcs_file, indent=4, ensure_ascii=False)

logging.info(f"Log de inspeção detalhado salvo em: {log_path}")
logging.info(f"Log de inspeção SFCS salvo em: {sfcs_path}")

return log_data, log_path, sfcs_path

except Exception as e:
    logging.error(f"Erro durante a geração do log: {str(e)}")
    return None, None, None

def send_sfcs_data(self, sfcs_path):
    """
    Envia os dados do arquivo SFCS para a API e exclui o arquivo em caso de sucesso.

```

Args:

sfcs_path: Caminho para o arquivo SFCS JSON

Returns:

Boolean: True se o envio foi bem-sucedido, False caso contrário

"""

if not sfcs_path or not os.path.exists(sfcs_path):

logging.error(f"Arquivo SFCS não encontrado: {sfcs_path}")

return False

if not self.api_url:

logging.error("URL da API não configurada")

return False

try:

Carregar arquivo JSON

with open(sfcs_path, 'r') as file:

sfcs_data = json.load(file)

Enviar para a API

logging.info(f"Enviando dados SFCS para: {self.api_url}")

response = requests.post(self.api_url, json=sfcs_data)

response.raise_for_status() # Lança exceção para códigos de status 4xx/5xx

Excluir o arquivo após envio bem-sucedido

os.remove(sfcs_path)

logging.info(f"Dados SFCS enviados com sucesso e arquivo excluído: {sfcs_path}")

return True

except requests.exceptions.RequestException as e:

logging.error(f"Erro ao enviar dados SFCS: {str(e)}")

return False

except Exception as e:

logging.error(f"Erro ao processar envio de dados SFCS: {str(e)}")

return False

def send_data_http(self, log_path):

"""

Envia os dados do log para um servidor via HTTP.

Args:

log_path: Caminho para o arquivo JSON de log

Returns:

Tuple (success, response): Indica sucesso e retorna a resposta HTTP se houver

"""

if not log_path or not os.path.exists(log_path):

logging.error(f"Arquivo de log não encontrado: {log_path}")

return False, None

```
if not self.api_url:
    logging.error("URL da API não configurada")
    return False, None
```

```
try:
```

```
    # Carregar arquivo JSON
    with open(log_path, 'r') as file:
        log_data = json.load(file)
```

```
    # Enviar para a API
    response = requests.post(self.api_url, json=log_data)
    response.raise_for_status() # Lança exceção para códigos de status 4xx/5xx
```

```
    logging.info("Dados enviados com sucesso para a API")
    return True, response
```

```
except requests.exceptions.RequestException as e:
```

```
    logging.error(f"Erro ao enviar dados: {str(e)}")
    return False, None
```

```
except Exception as e:
```

```
    logging.error(f"Erro ao processar envio de dados: {str(e)}")
    return False, None
```

```
def send_sfcs_data_from_dict(self, sfcs_data: Dict[str, Any]) -> bool:
```

```
    """
```

Envia diretamente um dicionário SFCS para a API configurada,
sem gravar arquivo intermediário.

```
    """
```

```
if not self.api_url:
```

```
    self._logger.error("API URL not configured")
    return False
```

```
sess = self._session or requests
```

```
try:
```

```
    resp = sess.post(self.api_url,
                     json=sfcs_data,
                     timeout=self.http_timeout,
                     headers={"User-Agent": f"{__name__}/inspection_logger"})
    resp.raise_for_status()
    self._logger.info("SFCS dict sent successfully")
    return True
```

```
except requests.RequestException as e:
```

```
    self._logger.error("Error sending SFCS dict: %s", e)
    return False
```

```
def set_context(self,
```

```
    *, posto: Optional[str] = None,
    usuario: Optional[str] = None,
    linha: Optional[str] = None):
```

```
    """
```

Atualiza valores JSON de posto, usuário e linha
sem reinstanciar o logger.

```
"""
```

```
if posto is not None:
    self.json_posto = posto
if usuario is not None:
    self.json_usuario = usuario
if linha is not None:
    self.json_linha = linha
```

```
__all__ = ["InspectionLogger"]
```

File: settings_manager.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
# settings_manager.py
import json, typing as _t
from pathlib import Path

class SettingsManager:
    _FILE = Path(__file__).with_name("settings.json")

    def __init__(self):
        self.data = {  # defaults
            "table_limits": {
                "1": {"x": [0, 25000], "y": [0, 18000]},
                "2": {"x": [0, 25000], "y": [0, 18000]}
            },
            "camera_index": 0,
            "focus_calibration": {"a": 0.0, "b": 0.0},
            # ----- steps/mm por eixo -----
            "axis_calibration": {
                "X": 1.0,
                "Y2": 1.0,
                "Y1": 1.0,
                "Z": 1.0
            },
            # ----- homing virtual de cada mesa -----
            "virtual_home": {
                "1": {"x": 0, "y": 0},
                "2": {"x": 0, "y": 0}
            },
            # ----- escala câmera (mm por pixel @ Z-ref = 0) -----
            "camera_mm_per_pixel": 0.05,
            "camera_fov": {  # NOVO – campo de visão
                "z0_z_pulses": 0,
                "z0_width_mm": 61.0,
                "z0_height_mm": 45.0,
                "z1_z_pulses": 700,
                "z1_width_mm": 30.5,
                "z1_height_mm": 22.5
```

```

    },
    # ----- OFFSET CÂMERA → NOZZLE -----
    # Guardado em PULSOS (mesma unidade dos motores).
    "camera_nozzle_offset": {"x": 0, "y": 0},

    # ----- NOVO: pasta-raiz dos projetos -----
    # será algo como C:\QualquerLugar\Projetos
    "projects_dir": str((Path.home() / "Projetos").resolve())
}
self.load()

# -----
def load(self):
    if self._FILE.exists():
        try:
            self.data.update(json.loads(self._FILE.read_text("utf-8")))
        except Exception:
            print(f"Error loading settings file: {Exception}")
            self.data = {} # mantém defaults

def get_config(self, key: str, default=None):
    """
    Retorna o valor de configuração associado a `key`,
    ou `default` se a chave não existir.
    """
    return self.data.get(key, default)

def set_config(self, key: str, value):
    """
    Define o valor de configuração para `key` e persiste imediatamente.
    """
    self.data[key] = value
    self.save()

def save(self):
    try:
        self._FILE.write_text(json.dumps(self.data, indent=2), encoding="utf-8")
    except Exception:
        print(f"Error saving settings file: {Exception}")

# conveniência
@property
def table_limits(self) -> dict[int, dict[str, _t.Any]]:
    return {int(k): v for k, v in self.data["table_limits"].items()}

@table_limits.setter
def table_limits(self, v):
    self.data["table_limits"] = {str(k): v[k] for k in v}

@property
def camera_index(self) -> int:

```

```

return int(self.data.get("camera_index", 0))

@camera_index.setter
def camera_index(self, idx: int):
    self.data["camera_index"] = int(idx)

# ----- foco -----
@property
def focus_coefs(self):
    d = self.data.get("focus_calibration", {"a":0.0,"b":0.0})
    return d.get("a",0.0), d.get("b",0.0)

@focus_coefs.setter
def focus_coefs(self, ab):
    self.data["focus_calibration"] = {"a":ab[0], "b":ab[1]}

# ----- steps/mm -----
@property
def axis_steps(self) -> dict[str,float]:
    return {k: float(v) for k,v in self.data.get("axis_calibration", {}).items()}
@axis_steps.setter
def axis_steps(self, d: dict[str,float]):
    self.data["axis_calibration"] = {k: float(v) for k,v in d.items()}

# ----- homing virtual -----
@property
def virtual_home(self) -> dict[int, dict[str,int]]:
    vh = self.data.get("virtual_home", {"1":{"x":0,"y":0},"2":{"x":0,"y":0}})
    return {int(k): {"x":int(v["x"]), "y":int(v["y"])} for k,v in vh.items()}
@virtual_home.setter
def virtual_home(self, d: dict[int,dict[str,int]]):
    self.data["virtual_home"] = {str(k): {"x":int(v["x"]), "y":int(v["y"])}
                                for k,v in d.items()}

# ----- câmera -----
@property
def camera_mm_per_pixel(self) -> float:
    return float(self.data.get("camera_mm_per_pixel", 0.05))
@camera_mm_per_pixel.setter
def camera_mm_per_pixel(self, v: float):
    self.data["camera_mm_per_pixel"] = float(v)

# ----- campo-de-visão -----
@property
def camera_fov(self) -> dict:
    return self.data.get("camera_fov", {})
@camera_fov.setter
def camera_fov(self, d: dict):
    self.data["camera_fov"] = d

# ----- OFFSET CÂMERA↔NOZZLE -----
@property

```

```

def cam_noz_offset(self) -> dict[str, int]:
    off = self.data.get("camera_nozzle_offset", {"x": 0, "y": 0})
    return {"x": int(off.get("x", 0)), "y": int(off.get("y", 0))}

@cam_noz_offset.setter
def cam_noz_offset(self, off: dict[str, int]):
    self.data["camera_nozzle_offset"] = {"x": int(off["x"]), "y": int(off["y"])}

# ----- PASTA «PROJETOS» -----
@property
def projects_dir(self) -> Path:
    return Path(self.data.get("projects_dir",
                              str(Path.home() / "Projetos"))).resolve()

@projects_dir.setter
def projects_dir(self, p: Path | str):
    self.data["projects_dir"] = str(Path(p).resolve())

```

File: roi_window_editor.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

# roi_window_editor.py – v3 (22/Jul/2025)
#
# Novidades v3 – Janelas-dependentes (“filhos”)
# -----
# • Qualquer janela pode ser marcada como PAI (tipicamente a posição mecânica)
# • add_window(..., parent=item_pai) → cria filho
# • editor.focus_on_parent(item|uid|None) → mostra só os filhos do pai ativo
# • serialização inclui:
#   { "id": <int>, "parent": <int|null>, ... }
# • Métodos utilitários:
#   editor.current_parent() → item ou None
#   editor.all_children(parent) → lista[ResizableRectItem]
#
# Compatível com v2 – APIs antigas não mudaram.
# -----

from __future__ import annotations
import json, os, itertools
from typing import Callable, Dict, Optional, List, Any, Union
from PyQt6.QtCore import Qt, QRectF, QPointF, QPoint, QObject, QEvent, pyqtSignal
from PyQt6.QtGui import QPen, QBrush
from PyQt6.QtWidgets import (
    QGraphicsRectItem, QGraphicsItem, QGraphicsView, QGraphicsScene,
    QInputDialog
)

HANDLE = 6 # px

```

```

# =====
# Item individual (janela)
# =====

class ResizableRectItem(QGraphicsRectItem):
    """
    Retângulo arrastável/redimensionável.

    Novo v3:
    - id          → identificador único inteiro
    - parent_uid  → id do item-pai (ou None)
    - is_child()  → True se pertence a um pai
    """
    _id_counter = itertools.count(1)    # gerador global

    pen_normal   = QPen(Qt.GlobalColor.green, 2)
    pen_selected = QPen(Qt.GlobalColor.blue, 2, Qt.PenStyle.DashLine)
    handle_brush = QBrush(Qt.GlobalColor.white)
    handle_pen   = QPen(Qt.GlobalColor.black, 1)

    def __init__(
        self,
        x: float, y: float, w: float, h: float,
        *,
        pen: Optional[QPen] = None,
        change_callback: Optional[Callable[['ResizableRectItem'], None]] = None,
        deletable: bool = True,
        name: Optional[str] = None,
        meta: Optional[Dict[str, Any]] = None,
        parent_uid: Optional[int] = None,
        parent: Optional[QGraphicsItem] = None,
        editable: bool = True          # NOVO: janelas "somente-seleção" no visor principal
    ):
        super().__init__(x, y, w, h, parent)
        self.editable: bool = editable
        self.id: int = next(self._id_counter)
        self.parent_uid: Optional[int] = parent_uid
        # -----
        # ItemIsMovable + redimensionamento só se editable=True
        flags = (
            QGraphicsItem.GraphicsItemFlag.ItemIsSelectable |
            QGraphicsItem.GraphicsItemFlag.ItemSendsGeometryChanges
        )
        if self.editable:
            flags |= QGraphicsItem.GraphicsItemFlag.ItemIsMovable
        self.setFlags(flags)
        self.setAcceptHoverEvents(self.editable)
        self._active_handle: Optional[str] = None
        self._pen_custom = pen
        self.change_callback = change_callback
        self.deletable = deletable

```

```

# ----- metadados livres -----
self.name: Optional[str] = name
self.meta: Dict[str, Any] = meta or {}
if self.name:
    self.setToolTip(self.name)
# callback inicial
if callable(self.change_callback):
    self.change_callback(self)

# -----
# Conveniências
# -----
def is_child(self) -> bool:
    return self.parent_uid is not None

# -----
# Helpers internos (redimensionamento)
# -----
def _all_handles(self) -> Dict[str, QRectF]:
    r = self.rect()
    x, y, w, h = r.x(), r.y(), r.width(), r.height()
    s = HANDLE
    cx, cy = x + w/2.0, y + h/2.0
    return {
        'tl': QRectF(x - s/2, y - s/2, s, s),
        'tm': QRectF(cx - s/2, y - s/2, s, s),
        'tr': QRectF(x + w - s/2, y - s/2, s, s),
        'mr': QRectF(x + w - s/2, cy - s/2, s, s),
        'br': QRectF(x + w - s/2, y + h - s/2, s, s),
        'bm': QRectF(cx - s/2, y + h - s/2, s, s),
        'bl': QRectF(x - s/2, y + h - s/2, s, s),
        'ml': QRectF(x - s/2, cy - s/2, s, s),
    }

@staticmethod
def _cursor_for(handle: Optional[str]) -> Qt.CursorShape:
    return {
        'tl': Qt.CursorShape.SizeFDiagCursor,
        'br': Qt.CursorShape.SizeFDiagCursor,
        'tr': Qt.CursorShape.SizeBDiagCursor,
        'bl': Qt.CursorShape.SizeBDiagCursor,
        'ml': Qt.CursorShape.SizeHorCursor,
        'mr': Qt.CursorShape.SizeHorCursor,
        'tm': Qt.CursorShape.SizeVerCursor,
        'bm': Qt.CursorShape.SizeVerCursor,
    }.get(handle, Qt.CursorShape.ArrowCursor)

def _handle_at(self, pos: QPointF) -> Optional[str]:
    for name, rect in self._all_handles().items():
        if rect.contains(pos):
            return name

```

```

return None

# ----- eventos -----
def hoverMoveEvent(self, ev):
    if self.editable:
        self.setCursor(self._cursor_for(self._handle_at(ev.pos())))
    super().hoverMoveEvent(ev)

def mousePressEvent(self, ev):
    if self.editable and ev.button() == Qt.MouseButton.LeftButton:
        self._active_handle = self._handle_at(ev.pos())
    super().mousePressEvent(ev)

def mouseMoveEvent(self, ev):
    if self.editable and self._active_handle:
        self._resize(ev.pos())
        if callable(self.change_callback):
            self.change_callback(self)
    else:
        super().mouseMoveEvent(ev)

def mouseReleaseEvent(self, ev):
    self._active_handle = None
    if callable(self.change_callback):
        self.change_callback(self)
    super().mouseReleaseEvent(ev)

# ----- mudanças de posição -----
def itemChange(self, change, value):
    if (change == QGraphicsItem.GraphicsItemChange.ItemPositionHasChanged
        and callable(self.change_callback)):
        self.change_callback(self)
    return super().itemChange(change, value)

# ----- redimensionamento -----
def _resize(self, pos: QPointF):
    r = self.rect()
    x, y, w, h = r.x(), r.y(), r.width(), r.height()
    px, py = pos.x(), pos.y()

    if 'l' in self._active_handle:
        new_x = min(px, x + w - 1)
        w += x - new_x
        x = new_x
    if 'r' in self._active_handle:
        w = max(1, px - x)
    if 't' in self._active_handle:
        new_y = min(py, y + h - 1)
        h += y - new_y
        y = new_y
    if 'b' in self._active_handle:

```

```
h = max(1, py - y)
```

```
self.prepareGeometryChange()
self.setRect(QRectF(x, y, w, h))
```

```
# ----- desenho -----
```

```
def paint(self, painter, option, widget=None):
    pen = self._pen_custom or self.pen_normal
    if self.isSelected():
        pen = self.pen_selected
    painter.setPen(pen)
    painter.drawRect(self.rect())
    # Alças de redimensionamento apenas se editável
    if self.isSelected() and self.editable:
        painter.setBrush(self.handle_brush)
        painter.setPen(self.handle_pen)
        for rect in self._all_handles().values():
            painter.drawRect(rect)
```

```
# ----- serialização -----
```

```
def to_dict(self) -> Dict[str, Any]:
    r = self.rect()
    return dict(
        id=self.id,
        parent=self.parent_uid,
        x=r.x(), y=r.y(), w=r.width(), h=r.height(),
        name=self.name,
        meta=self.meta
    )
```

```
# =====
```

```
# Editor / Gerenciador
```

```
# =====
```

```
class ROIWindowEditor(QObject):
```

```
    """
```

Gerenciador universal de janelas redimensionáveis.

Novidades v3 (janelas-filhas):

- `add_window(..., parent=None)` → cria dependência hierárquica.
- `focus_on_parent(pai|uid|None)`
 - `None` → mostra todas as janelas
 - `ResizableItem` → mostra apenas seus filhos
- `current_parent()` → retorna pai ativo
- `all_children(pai)` → lista de filhos

```
    """
```

```
windowAdded = pyqtSignal(object) # ResizableRectItem
```

```
windowRemoved = pyqtSignal(object)
```

```
windowChanged = pyqtSignal(object)
```

```

# ----- init -----
def __init__(self,
    view: QGraphicsView,
    *,
    ask_name: bool = False,
    autosave_path: Optional[str] = None):

    super().__init__(view)
    self.view = view
    self._ask_name = ask_name
    self.autosave_path = autosave_path
    if self.view.scene() is None:
        self.view.setScene(QGraphicsScene(self.view))

    self._drawing = False
    self._drawing_parent: Optional[ResizableRectItem] = None
    self._tmp_rect: Optional[ResizableRectItem] = None
    self._start_pos: QPointF | None = None
    self.windows: List[ResizableRectItem] = []

    # parent currently focused (None = todos visíveis)
    self._parent_focus_uid: Optional[int] = None

    # event filter (viewport para mouse, scene para teclado)
    view.viewport().installEventFilter(self)
    self.view.scene().installEventFilter(self)

# -----
# PREVENÇÃO DE ACESSO APÓS DESTRUIÇÃO DO QGraphicsView
# -----
# Quando a janela é encerrada o QGraphicsView é destruído antes
# do ROIWindowEditor. Mantemos uma flag para ignorar eventos
# subsequentes e evitar "wrapped C/C++ object ... has been deleted".
self._view_deleted = False
self.view.destroyed.connect(self._on_view_destroyed)

# ----- API PÚBLICA -----
# ----- modo desenho -----
def start_drawing(self, *,
    pen: QPen = QPen(Qt.GlobalColor.red, 2),
    parent: Optional[ResizableRectItem] = None):
    """Habilita desenho; janelas receberão `parent` (opcional)."""
    self._drawing_pen = pen
    self._drawing_parent = parent
    self._drawing = True
    self.view.setCursor(Qt.CursorShape.CrossCursor)

def stop_drawing(self):
    self._drawing = False
    self._drawing_parent = None
    self.view.setCursor(Qt.CursorShape.ArrowCursor)

```

```

if self._tmp_rect and self._tmp_rect.scene():
    self.view.scene().removeItem(self._tmp_rect)
self._tmp_rect = None

```

---- criação programática -----

```

def add_window(self,
    x: float, y: float, w: float, h: float,
    *,
    pen: Optional[QPen] = None,
    deletable: bool = True,
    name: Optional[str] = None,
    meta: Optional[Dict[str, Any]] = None,
    change_cb: Optional[Callable[[ResizableRectItem], None]] = None,
    parent: Optional[ResizableRectItem] = None,
    editable: bool = True      # NOVO
) -> ResizableRectItem:

```

```

item_parent_uid = parent.id if parent else None

```

```

item = ResizableRectItem(
    x, y, w, h,
    pen=pen,
    deletable=deletable,
    name=name,
    meta=meta,
    parent_uid=item_parent_uid,
    parent=parent,          # hierarquia Qt
    change_callback=self._wrap_change_cb(change_cb),
    editable=editable
)

```

CORREÇÃO: Sempre adiciona à cena se não há pai Qt Graphics

(mesmo que tenha parent_uid para associação lógica)

if parent is None:

```

    self.view.scene().addItem(item)

```

Se tem pai Qt Graphics, não precisa addItem (já vinculado automaticamente)

```

self.windows.append(item)

```

```

self.windowAdded.emit(item)

```

```

self._autosave()

```

```

return item

```

----- foco / filtragem -----

```

def focus_on_parent(self,
    parent: Optional[Union[ResizableRectItem, int]]):
    """

```

Define a janela-pai “ativa”.

- None → todas as janelas visíveis
- item / id → exibe APENAS os filhos desse pai

Janelas sem pai (top-level) permanecem sempre visíveis.

"""

```

if isinstance(parent, ResizableRectItem):

```

```

    uid = parent.id

```

```

else:

```

```

        uid = parent
    self._parent_focus_uid = uid
    # PROTEÇÃO: Remove itens inválidos da lista antes de usar
    valid_windows = []
    # DEBUGGING: Log da operação de foco
    if uid is None:
        print(f"■ Focando em: TODOS os itens")
    else:
        print(f"■ Focando em pai UID: {uid}")
    for w in self.windows:
        try:
            # Testa se objeto Qt ainda existe
            _ = w.scene()
            valid_windows.append(w)
            # Se chegou aqui, objeto é válido - aplica visibilidade
            if w.parent_uid is None:
                w.setVisible(True)      # sempre visível
                print(f" ■ Item sem pai sempre visível: {getattr(w, 'name', w.id)}")
            else:
                should_be_visible = (w.parent_uid == uid)
                w.setVisible(should_be_visible)
                visibility_status = "VISÍVEL" if should_be_visible else "OCULTO"
                print(f" {'■' if should_be_visible else '■'} Item {getattr(w, 'name', w.id)} (pai: {w.parent_uid}): {visibility_status}")
        except RuntimeError:
            print(f" ■■ Item inválido removido da lista")
            # Objeto foi deletado - ignora silenciosamente
            pass
    self.windows = valid_windows

```

```

def show_all_comparison_windows(self):
    """
    Mostra todas as janelas de comparação (que têm parent_uid),
    mantendo posições mecânicas (sem parent_uid) sempre visíveis.
    """

```

```

    for w in self.windows:
        # Posições mecânicas (sem pai) sempre visíveis
        # Janelas de comparação (com pai) sempre visíveis também
        w.setVisible(True)

```

```

def current_parent(self) -> Optional[ResizableRectItem]:
    """Retorna o item-pai atualmente em foco (ou None)."""
    if self._parent_focus_uid is None:
        return None
    for w in self.windows:
        if w.id == self._parent_focus_uid:
            return w
    return None

```

```

def all_children(self, parent: ResizableRectItem) -> List[ResizableRectItem]:
    """Lista todos os filhos imediatos do item informado."""
    return [w for w in self.windows if w.parent_uid == parent.id]

```

```

def get_window_by_id(self, window_id: int) -> Optional[ResizableRectItem]:
    """Retorna janela pelo ID"""
    for w in self.windows:
        if hasattr(w, 'id') and w.id == window_id:
            return w
    return None

# ----- EVENT FILTER -----
def eventFilter(self, obj, ev):
    # --- tecla Delete (escopo da cena) -----
    # -----
    # Evita acessar self.view depois que o objeto C++ foi
    # destruído (fase de shutdown da aplicação).
    # -----
    if self._view_deleted:
        return False

    # viewport pode não existir mais nestas alturas; tenta acessar
    # com proteção.
    try:
        viewport = self.view.viewport()
    except RuntimeError:
        return False
    if ev.type() == QEvent.Type.KeyPress and ev.key() == Qt.Key.Key_Delete:
        self._delete_selected()
        return True

    # --- desenho (viewport) -----
    if obj is viewport and self._drawing:
        if ev.type() == QEvent.Type.MouseButtonPress and ev.button() == Qt.MouseButton.LeftButton:
            self._start_pos = self.view.mapToScene(QPoint(int(ev.position().x()),
                                                            int(ev.position().y())))
            self._tmp_rect = ResizableRectItem(
                self._start_pos.x(), self._start_pos.y(), 0, 0,
                pen=self._drawing_pen, deletable=True,
                parent=self._drawing_parent, # pai Qt Graphics (pode ser None)
                parent_uid=self._drawing_parent.id if self._drawing_parent else None
            )
            # CORREÇÃO: Sempre adiciona à cena durante desenho
            if self._drawing_parent is None:
                self.view.scene().addItem(self._tmp_rect)
            else:
                # Mesmo com pai lógico, se pai Qt Graphics for de outra cena, adiciona à cena atual
                if self._drawing_parent.scene() != self.view.scene():
                    self.view.scene().addItem(self._tmp_rect)
                    self._tmp_rect.parent = None # Remove pai Qt Graphics para evitar conflitos
            self.windows.append(self._tmp_rect)
            return True

    if ev.type() == QEvent.Type.MouseMove and self._tmp_rect and self._start_pos:

```

```

cur = self.view.mapToScene(QPoint(int(ev.position().x()),
                                int(ev.position().y())))
x = min(self._start_pos.x(), cur.x())
y = min(self._start_pos.y(), cur.y())
w = abs(cur.x() - self._start_pos.x())
h = abs(cur.y() - self._start_pos.y())
self._tmp_rect.setRect(x, y, w, h)
return True

```

```

if ev.type() == QEvent.Type.MouseButtonRelease and self._tmp_rect and self._start_pos:
    valid = (self._tmp_rect.rect().width() > 2
             and self._tmp_rect.rect().height() > 2)

```

```

# nome opcional somente para janelas TOP-LEVEL
if valid and self._ask_name and self._drawing_parent is None:
    name, ok = QInputDialog.getText(
        self.view, "Nome da Janela", "Digite um nome (opcional):"
    )
    if ok:
        name = name.strip()
        if name:
            self._tmp_rect.name = name
            self._tmp_rect.setToolTip(name)

```

```

if not valid:
    # descarta
    if self._tmp_rect in self.windows:
        self.windows.remove(self._tmp_rect)
    if self._tmp_rect.scene():
        self._tmp_rect.scene().removeItem(self._tmp_rect)
else:
    # finaliza
    self._tmp_rect.change_callback = self._wrap_change_cb(
        self._tmp_rect.change_callback)
    self.windowAdded.emit(self._tmp_rect)
    self._autosave()
self._tmp_rect = None
self._start_pos = None
if not self._drawing:
    self.view.setCursor(Qt.CursorShape.ArrowCursor)
return True
return False

```

```

# -----
# Callback disparado quando o QGraphicsView for destruído
# -----

```

```

def _on_view_destroyed(self, *_):

```

```

    """

```

Marca flag para que o eventFilter ignore qualquer evento
pendente após a destruição do QGraphicsView.

```

    """

```

```
self._view_deleted = True
```

```
# ----- SERIALIZAÇÃO -----
```

```
def to_dict_list(self) -> List[Dict[str, Any]]:  
    return [w.to_dict() for w in self.windows]
```

```
def from_dict_list(self, data: List[Dict[str, Any]], *, clear=True):
```

```
    """
```

```
    Restaura hierarquia em 2 passagens para garantir que  
    itens-pai existam antes dos filhos.
```

```
    """
```

```
    if clear:
```

```
        self._clear_all()
```

```
    # 1ª passagem – cria todos os TOP LEVEL
```

```
    id_map: Dict[int, ResizableRectItem] = {}
```

```
    for d in data:
```

```
        if d.get("parent") is not None:
```

```
            continue
```

```
        it = self.add_window(  
            d["x"], d["y"], d["w"], d["h"],
```

```
            name=d.get("name"), meta=d.get("meta", {}),
```

```
            parent=None
```

```
        )
```

```
        it.id = d["id"]          # preserva UID
```

```
        id_map[it.id] = it
```

```
    # 2ª passagem – cria filhos
```

```
    for d in data:
```

```
        if d.get("parent") is None:
```

```
            continue
```

```
        parent_it = id_map.get(d["parent"])
```

```
        if parent_it is None:      # órfão → ignora
```

```
            continue
```

```
        it = self.add_window(  
            d["x"], d["y"], d["w"], d["h"],
```

```
            name=d.get("name"), meta=d.get("meta", {}),
```

```
            parent=parent_it
```

```
        )
```

```
        it.id = d["id"]
```

```
        id_map[it.id] = it
```

```
self.view.viewport().update()
```

```
self._autosave()
```

```
def save_json(self, path: str):
```

```
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
```

```
    with open(path, "w", encoding="utf-8") as f:
```

```
        json.dump(self.to_dict_list(), f, indent=2)
```

```
def load_json(self, path: str, *, clear=True):
```

```
    if not os.path.isfile(path):
```

```

        return False
    try:
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
            if isinstance(data, list):
                self.from_dict_list(data, clear=clear)
            return True
    except Exception:
        pass
    return False

```

----- INTERNOS -----

```
def _wrap_change_cb(self, user_cb):
```

```
    """
```

```
    Garante callback + autosave + sinal.
```

```
    """
```

```
    def _cb(item):
```

```
        if callable(user_cb):
```

```
            user_cb(item)
```

```
        self.windowChanged.emit(item)
```

```
        self._autosave()
```

```
    return _cb
```

```
def _autosave(self):
```

```
    if self.autosave_path:
```

```
        try:
```

```
            self.save_json(self.autosave_path)
```

```
        except Exception:
```

```
            pass
```

```
def _delete_selected(self):
```

```
    scene = self.view.scene()
```

```
    for it in scene.selectedItems():
```

```
        if isinstance(it, ResizableRectItem) and it.deletable:
```

```
            # remove também os FILHOS ligados a este item
```

```
            for child in list(self.all_children(it)):
```

```
                if child.scene():
```

```
                    child.scene().removeItem(child)
```

```
                if child in self.windows:
```

```
                    self.windows.remove(child)
```

```
                self.windowRemoved.emit(child)
```

```
            scene.removeItem(it)
```

```
            if it in self.windows:
```

```
                self.windows.remove(it)
```

```
            self.windowRemoved.emit(it)
```

```
    scene.clearSelection()
```

```
    self._autosave()
```

```
def _clear_all(self):
```

```
    scene = self.view.scene()
```

```
    for w in list(self.windows):
```

```

        if w.scene() is scene:
            scene.removeItem(w)
    self.windows.clear()

```

File: inspection_config_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

from PyQt6.QtWidgets import (
    QGroupBox, QVBoxLayout, QFormLayout, QSpinBox, QPushButton,
    QLabel, QHBoxLayout, QSizePolicy, QDialog,
    QGraphicsView, QGraphicsScene, QWidget, QTreeWidget, QTreeWidgetItem
)
from roi_window_editor import ROIWindowEditor, ResizableRectItem
from selectable_rect_item import SelectableResizableRectItem
from PyQt6.QtCore import Qt, pyqtSignal, QTimer
from PyQt6.QtGui import QApplication
from PyQt6.QtGui import QPixmap, QPainter, QImage, QPen, QColor
import cv2
import os

# =====
# CONFIGURAÇÃO – larguras mínimas dos visores da janela auxiliar
# • altere aqui se desejar outros valores
# =====

MIN_REGION_WIDTH = 800    # visor principal (esquerda)
MIN_REGION_HEIGHT = 600   # altura do visor principal (4:3)
MIN_ROI_WIDTH = 350       # visor ROI      (direita)
MIN_ROI_HEIGHT = 262      # altura do ROI (4:3)
# MIN_REGION_WIDTH = 480   # visor principal (esquerda)
# MIN_REGION_HEIGHT = 360  # altura do visor principal (4:3)
# MIN_ROI_WIDTH = 240      # visor ROI      (direita)
# MIN_ROI_HEIGHT = 180     # altura do ROI (4:3)

# -----
# Diálogo auxiliar em 2 colunas (80 % região | 20 % controles)
# -----

class _AuxDialog(QDialog):
    def __init__(self,
                 pix_region,          # QPixmap da região capturada
                 ctrl_widget,         # InspectionConfigWidget (controles)
                 update_callback=None, # Função callback para atualização
                 parent=None):
        super().__init__(parent)
        self.setWindowTitle("Auxiliar de Inspeção")
        self.setWindowModality(Qt.WindowModality.NonModal)
        self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)
        self._main_widget = ctrl_widget # Referência ao widget principal
        self._update_callback = update_callback # Callback para atualização direta

```

```

# ----- COLUNA ESQUERDA (imagem + editor ROI) -----
# Usa QGraphicsView para poder desenhar janelas via ROIWindowEditor
# -----
# Imagem original (alta resolução) recebida do widget principal
# -----
self._orig_pix = pix_region or QPixmap() # cache como QPixmap

self.view_region = QGraphicsView()
self.view_region.setMinimumWidth(MIN_REGION_WIDTH)
self.view_region.setMinimumHeight(MIN_REGION_HEIGHT)
self.view_region.setScene(QGraphicsScene(self.view_region))
self.view_region.setRenderHints(
    QPainter.RenderHint.SmoothPixmapTransform |
    QPainter.RenderHint.Antialiasing
)
self._pix_item = self.view_region.scene().addPixmap(self._orig_pix)
# Posições mecânicas — pede nome ao concluir
_aux_dir = (os.path.join(ctrl_widget._c.prog_mgr.path, "arquivos_auxiliares")
             if hasattr(ctrl_widget, '_c') and hasattr(ctrl_widget._c, 'prog_mgr')
             and ctrl_widget._c.prog_mgr else None)
if _aux_dir:
    os.makedirs(_aux_dir, exist_ok=True)
self.region_editor = ROIWindowEditor(
    self.view_region,
    ask_name=True,
    autosave_path=(os.path.join(_aux_dir, "mechanical_positions.json")
                   if _aux_dir else None)
)
# ----- mantém cópia BGR da imagem para recortes -----
# numpy BGR correspondente – usado para recortes exatos
self._orig_bgr = None
if not self._orig_pix.isNull():
    self._orig_bgr = self._qpix_to_bgr(self._orig_pix)
# caso o widget principal possua buffer original, usa-o (mais fiel)
if getattr(parent, "_last_roi_bgr", None) is not None:
    self._orig_bgr = parent._last_roi_bgr.copy()

# ----- layout principal -----
h = QHBoxLayout(self)
h.addWidget(self.view_region, 4) # 80 %

# ----- coluna direita (controles + botões extras) ----
right_box = QWidget()
vright = QVBoxLayout(right_box); vright.setContentsMargins(0, 0, 0, 0)
vright.addWidget(ctrl_widget, 1)

# NOVOS BOTÕES – exclusivos desta janela
self.btn_mech = QPushButton("Posição mecanica")
self.btn_mech.setCheckable(True)
self.btn_cmp = QPushButton("Comparação de imagem")

```

```

self.btn_cmp.setCheckable(True)
# ----- TREEVIEW – lista de posições mecânicas -----
self.tree = QTreeWidget()
self.tree.setHeaderHidden(True)
vright.addWidget(self.tree, 2)

vright.addWidget(self.btn_mech)
vright.addWidget(self.btn_cmp)
vright.addStretch()

h.addWidget(right_box, 1)      # 20 %

# Garante que o preview interno e o botão auxiliar não apareçam
for attr in ("lbl_region", "btn_aux"):
    if hasattr(ctrl_widget, attr) and getattr(ctrl_widget, attr):
        getattr(ctrl_widget, attr).setVisible(False)

# ----- ROI (thumbnail) – substitui QLabel por View ---
roi_label = getattr(ctrl_widget, "lbl_roi", None)
if roi_label:
    roi_pix = roi_label.pixmap() or QPixmap()
    self.view_roi = QGraphicsView()
    self.view_roi.setMinimumWidth(MIN_ROI_WIDTH)
    self.view_roi.setMinimumHeight(MIN_ROI_HEIGHT)
    self.view_roi.setScene(QGraphicsScene(self.view_roi))
    self.view_roi.setRenderHints(
        QPainter.RenderHint.SmoothPixmapTransform |
        QPainter.RenderHint.Antialiasing
    )
    self.view_roi.scene().addPixmap(roi_pix)
    _aux_dir2 = _aux_dir      # mesma pasta calculada acima
    if _aux_dir2:
        os.makedirs(_aux_dir2, exist_ok=True)
    self.roi_editor = ROIWindowEditor(
        self.view_roi,
        autosave_path=(os.path.join(_aux_dir2, "comparison_windows.json")
                        if _aux_dir2 else None)
    )
    parent_lay = roi_label.parentWidget().layout()
    idx = parent_lay.indexOf(roi_label)
    parent_lay.removeWidget(roi_label)
    roi_label.setParent(None)
    parent_lay.insertWidget(idx, self.view_roi)
else:
    self.view_roi = None
    self.roi_editor = None
    self._roi_pix_item = None

# ----- sinais botões -----

```

```

self.btn_mech.toggled.connect(
    lambda st: self._toggle_editor("mech", st))
self.btn_cmp.toggled.connect(
    lambda st: self._toggle_editor("cmp", st))

# ----- sincronização Tree ■ Scene -----
self.region_editor.windowAdded.connect(self._on_window_added)
self.region_editor.windowRemoved.connect(self._on_window_removed)
self.region_editor.windowChanged.connect(self._on_window_changed)

# -----
# Quando o usuário seleciona um retângulo (posição mecânica),
# o conteúdo interno é recortado da imagem original e exibido
# no visor "Região ROI" à direita, mantendo resolução e escala.
# -----
self.view_region.scene().selectionChanged.connect(
    self._update_roi_from_selection)

# -----
# Sempre que o usuário clicar em "Definir região" no painel direito,
# o sinal regionCaptured(img_bgr) será emitido. Conectamos para que
# o novo ROI substitua imediatamente a imagem exibida na coluna
# esquerda ("Região de Inspeção").
# -----
if hasattr(ctrl_widget, "regionCaptured"):
    ctrl_widget.regionCaptured.connect(self._on_region_captured)
    # Atualiza também o visor da JANELA PRINCIPAL
    if parent is not None and hasattr(parent, "_apply_external_roi"):
        ctrl_widget.regionCaptured.connect(parent._apply_external_roi)

# ROI capturado na janela PRINCIPAL deve reflectir aqui
if parent is not None and hasattr(parent, "regionCaptured"):
    parent.regionCaptured.connect(self._on_region_captured)

# Flags para evitar loops infinitos de sinais
self._updating_tree_selection = False
self._updating_scene_selection = False
self._scene_selection_blocked = False
# NOVO SISTEMA: Armazena janelas de comparação nos dados dos componentes
# Exatamente como no gerador de projetos
self.inspecao_items = {} # {nome_componente: [items]}

# Inicializa dicionário de componentes (essencial!)
self.componentes = {}
# Flags de controle
self._criando_item = False

# Configuração da TreeView para permitir seleção individual
self.tree.setSelectionMode(QTreeWidget.SelectionMode.SingleSelection)

```

```

# ----- NOVA SINCRONIZAÇÃO com ROIWindowEditor -----
self.view_region.scene().selectionChanged.connect(self._on_position_selection_changed)
self.tree.itemSelectionChanged.connect(self._on_tree_selection_changed)
self.tree.itemClicked.connect(self._on_tree_item_clicked)

# EVENT-FILTER para logs de clique
self.view_region.viewport().installEventFilter(self)
if self.view_roi:
    self.view_roi.viewport().installEventFilter(self)
    self.roi_editor.windowAdded.connect(self._on_comparison_window_added)
    self.roi_editor.windowRemoved.connect(self._on_comparison_window_removed)
# Variável para guardar o pai das janelas de comparação
self._comparison_parent_uid = None

# NOVO: Carrega dados do cache ao inicializar
self.load_cached_data()

def _update_mechanical_positions_in_cache(self):
    """Atualiza as coordenadas das posições mecânicas no cache com as posições atuais"""
    for window in self.region_editor.windows:
        if isinstance(window, ResizableRectItem):
            window_name = getattr(window, 'name', None)
            if window_name and window_name in self.componentes:
                # CORREÇÃO: Captura posição atual da janela no editor
                pos_rect = window.sceneBoundingRect()
                atual_x = int(pos_rect.x())
                atual_y = int(pos_rect.y())
                atual_w = int(pos_rect.width())
                atual_h = int(pos_rect.height())

                # Atualiza no cache
                self.componentes[window_name]['posicao'] = (atual_x, atual_y)
                self.componentes[window_name]['dimensoes'] = (atual_w, atual_h)

                self.log(f"■ Posição '{window_name}' atualizada no cache: pos=({atual_x},{atual_y}), dims=({atual_w},{atual_h})")

def _on_window_changed(self, item):
    """Callback chamado quando janela é modificada - salva automaticamente"""
    # CORREÇÃO: Atualiza dados no cache se for posição mecânica
    if isinstance(item, ResizableRectItem):
        window_name = getattr(item, 'name', None)
        if window_name and window_name in self.componentes:
            pos_rect = item.sceneBoundingRect()
            self.componentes[window_name]['posicao'] = (int(pos_rect.x()), int(pos_rect.y()))
            self.componentes[window_name]['dimensoes'] = (int(pos_rect.width()), int(pos_rect.height()))
            self.log(f"■ Posição mecânica '{window_name}' atualizada em tempo real: pos={self.componentes[window_name]}")

    self.log(f"■ Janela modificada: {self._describe_item(item)}")

# ----- helpers internos -----
def _update_pix(self):

```

```

"""Escala novamente a partir da imagem original (evita cascata)."""
if self._orig_pix.isNull():
    return
if not self._orig_pix.isNull():
    self._pix_item.setPixmap(self._orig_pix)
    self.view_region.fitInView(self._pix_item,
                               Qt.AspectRatioMode.KeepAspectRatio)

def _on_position_selection_changed(self):
    """
    Chamado quando seleção muda no visor de posições mecânicas.

    """
    selected_items = self.view_region.scene().selectedItems()
    if not selected_items:
        # Nenhuma seleção - mostra todas as janelas de comparação
        if self.roi_editor:
            self.roi_editor.focus_on_parent(None)
        return

    # Pega primeiro item selecionado (posição mecânica)
    selected_item = selected_items[0]
    if isinstance(selected_item, ResizableRectItem):
        # Foca nas janelas filhas desta posição mecânica
        # Atualiza ROI e redesenha janelas

        # Atualiza ROI visual
        self._update_roi_from_selection()

        # Seleciona na TreeView
        self._select_tree_node_for_item(selected_item)

    self.log(f"■ Posição selecionada: {getattr(selected_item, 'name', '?')}")

# ===== TREEVIEW Sync =====
def _on_window_added(self, item):
    """Insere posição mecânica na árvore"""
    name = getattr(item, 'name', f'Posição {item.id}' if hasattr(item, 'id') else 'Posição mecânica')
    node = QTreeWidgetItem([name])
    node.setData(0, Qt.ItemDataRole.UserRole, item)
    self.tree.addTopLevelItem(node)
    self.log(f"■ Posição mecânica adicionada à tree: {name}")

def _on_comparison_window_added(self, roi_item):
    """
    NOVA ABORDAGEM: Converte janela do ROIWindowEditor em dados do componente
    e redesenha usando o sistema do gerador de projetos

    """
    # Identifica a posição mecânica pai
    parent_item = None

```

```

if self._comparison_parent_uid is not None:
    for window in self.region_editor.windows:
        if hasattr(window, 'id') and window.id == self._comparison_parent_uid:
            parent_item = window
            break

if parent_item is None:
    # Fallback: usa posição selecionada
    selected_items = self.view_region.scene().selectedItems()
    for scene_item in selected_items:
        if isinstance(scene_item, ResizableRectItem) and scene_item in self.region_editor.windows:
            parent_item = scene_item
            break

if parent_item is None:
    self.log("■■■ Janela de comparação criada sem pai definido")
    return

# Encontra nó pai na TreeView
parent_node = None
parent_name = getattr(parent_item, 'name', f'ID:{parent_item.id}')
for i in range(self.tree.topLevelItemCount()):
    node = self.tree.topLevelItem(i)
    if node.data(0, Qt.ItemDataRole.UserRole) is parent_item:
        parent_node = node
        parent_name = node.text(0)
        break

if parent_node is None:
    self.log("■■■ Nó pai não encontrado na TreeView")
    return

# NOVA LÓGICA: Converte para dados do componente (como no gerador)
# -----
# NOVO: guarda POSIÇÃO ABSOLUTA dentro da imagem da REGIÃO
# -----
roi_rect = roi_item.rect()          # rel. à ROI
par_scene = parent_item.sceneBoundingRect() # abs. REGIÃO
par_x, par_y = int(par_scene.x()), int(par_scene.y())

# Garante entrada e ACTUALIZA coordenadas da posição mecânica
comp_entry = self.componentes.setdefault(
    parent_name,
    {'posicao': (par_x, par_y),
     'dimensoes': (int(par_scene.width()), int(par_scene.height())),
     'inspecoes': []}
)
comp_entry['posicao'] = (par_x, par_y)
comp_entry['dimensoes'] = (int(par_scene.width()), int(par_scene.height()))

# Coordenadas ABSOLUTAS da janela azul

```

```

x_orig = par_x + int(roi_rect.x())
y_orig = par_y + int(roi_rect.y())
w_orig = int(roi_rect.width())
h_orig = int(roi_rect.height())
# Conta janelas existentes para gerar nome
existing_count = len(self.componentes[parent_name]['inspecoes'])
comp_name = f"w{existing_count + 1}"

# Salva referência bidirecional
inspecao_data = {
    'posicao': (x_orig, y_orig),
    'tamanho': (w_orig, h_orig),
    'threshold': 100, # valor padrão
    'cor_pixel': 'branco',
    'percentual_minimo': 50.0,
    'roi_editor_item': roi_item, # referência para sincronização
    'tree_node': None,          # será preenchido abaixo
    'blue_window': None        # será preenchido no redesenho
}

self.componentes[parent_name]['inspecoes'].append(inspecao_data)

# CORREÇÃO: Remove janela vermelha do ROIWindowEditor após salvar dados
if roi_item in self.roi_editor.windows:
    self.roi_editor.windows.remove(roi_item)
    if roi_item.scene():
        roi_item.scene().removeItem(roi_item)

# FORÇA redesenho imediato para mostrar janela azul
self._redesenhar_janelas_inspecao(parent_name)

# DEBUGGING: Log detalhado da associação
self.log(f"■ Dados salvos {comp_name} → pai {parent_name}")
# Cria subnó na TreeView
comp_node = QTreeWidgetItem([comp_name])
# CORREÇÃO CRÍTICA: Associa imediatamente a um placeholder que será substituído
placeholder_data = {
    'type': 'comparison_placeholder',
    'component_name': parent_name,
    'window_name': comp_name,
    'inspecao_ref': inspecao_data
}

comp_node.setData(0, Qt.ItemDataRole.UserRole, placeholder_data)

# Log detalhado da associação
self.log(f"■ Criando nó TreeView '{comp_name}' com placeholder")

# Salva referência bidirecional
inspecao_data['tree_node'] = comp_node
parent_node.addChild(comp_node)
parent_node.setExpanded(True)

```

```

# NOVO: Força atualização visual da TreeView
self.tree.update()
self.tree.repaint()

# NOVO: Torna o item explicitamente selecionável
comp_node.setFlags(
    comp_node.flags() |
    Qt.ItemFlag.ItemIsSelectable |
    Qt.ItemFlag.ItemIsEnabled
)

self.log(f"■ Janela de comparação adicionada: {comp_name}")

# Limpa referência do pai após usar
self._comparison_parent_uid = None

def _select_tree_node_for_item(self, item):
    """Seleciona o nó da TreeView correspondente ao item"""
    for i in range(self.tree.topLevelItemCount()):
        node = self.tree.topLevelItem(i)
        if node.data(0, Qt.ItemDataRole.UserRole) is item:
            self.tree.setCurrentItem(node)
            return

# -----
# LOG (agora com supressão de mensagens muito frequentes)
# -----
_SUPPRESS_PATTERNS = (
    "Tentando carregar cache:",
    "Widget principal - inicializando auxiliar:",
    "Aguardando inicialização completa",
    "■ Aguardando inicialização completa"
)

def log(self, message: str):
    """
    Helper para logging. Mensagens que contenham qualquer um dos
    fragmentos listados em `_SUPPRESS_PATTERNS` são ignoradas
    para evitar poluição do log.
    """
    for pat in self._SUPPRESS_PATTERNS:
        if pat in message:
            return # suprime

    if hasattr(self, '_c') and hasattr(self._c, 'log'):
        self._c.log(message)
    elif hasattr(self, 'parent') and hasattr(self.parent(), 'log'):
        self.parent().log(message)

```

else:

```
print(f"[AuxDialog] {message}")
```

```
# =====
```

```
# NOVO SISTEMA: CARREGAMENTO E TRANSFERÊNCIA DE DADOS
```

```
# =====
```

```
def load_cached_data(self):
```

```
    """Carrega dados do cache do widget principal"""
```

```
    if not hasattr(self._main_widget, 'get_cached_auxiliary_data'):
```

```
        self.log("Widget principal não suporta cache")
```

```
        return
```

```
    cached_data = self._main_widget.get_cached_auxiliary_data()
```

```
    componentes_cache = cached_data.get('componentes', {})
```

```
    # DEBUGGING DETALHADO das coordenadas
```

```
    if not hasattr(self, 'componentes'):
```

```
        self.componentes = {}
```

```
        self.log("■■■ Dicionário componentes não existia - inicializado")
```

```
    if not hasattr(self, 'region_editor') or self.region_editor is None:
```

```
        self.log("■ region_editor não disponível - cancelando reconstrução")
```

```
        return
```

```
    self.log(f"■ Tentando carregar cache: {len(componentes_cache)} componentes disponíveis")
```

```
    for nome, dados in componentes_cache.items():
```

```
        pos = dados.get('posicao', (0, 0))
```

```
        dims = dados.get('dimensoes', (100, 100))
```

```
        inspecoes = dados.get('inspecoes', [])
```

```
        self.log(f"■ Cache '{nome}': pos={pos}, dims={dims}, {len(inspecoes)} inspeções")
```

```
    # ALERTA se posição for (0,0)
```

```
    if pos == (0, 0) and dims == (100, 100):
```

```
        self.log(f"■■■ POSIÇÃO SUSPEITA para '{nome}': pode estar em posição padrão")
```

```
    # DEBUGGING: Mostra estado do widget principal
```

```
    self.log(f"■ Widget principal - inicializando auxiliar: {getattr(self._main_widget, '_initializing_auxiliary', 'N/A')}")
```

```
    # NOVO: Se está inicializando e não tem dados, aguarda um pouco
```

```
    if not componentes_cache and hasattr(self._main_widget, '_initializing_auxiliary'):
```

```
        self.log("■ Aguardando inicialização completa...")
```

```
        QTimer.singleShot(100, self.load_cached_data) # Tenta novamente em 100ms
```

```
        return
```

```
    # DEBUGGING: Log detalhado do que está sendo carregado
```

```
    self.log(f"■ Tentando carregar cache: {len(componentes_cache)} componentes disponíveis")
```

```
    for nome, dados in componentes_cache.items():
```

```
        pos = dados.get('posicao', (0, 0))
```

```
        dims = dados.get('dimensoes', (100, 100))
```

```
        inspecoes = dados.get('inspecoes', [])
```

```
self.log(f"■ Cache '{nome}': pos={pos}, dims={dims}, {len(inspecoes)} inspeções")
```

```
if not componentes_cache:
```

```
    self.log("Nenhum dado em cache para carregar")
```

```
    return
```

```
# Carrega componentes do cache
```

```
self.componentes.update(componentes_cache)
```

```
self.log(f"■ Carregados {len(componentes_cache)} componentes do cache")
```

```
self.log(f"■ Total de componentes após carregar: {len(self.componentes)}")
```

```
# Reconstroi a interface baseada nos dados carregados
```

```
self.rebuild_interface_from_cache()
```

```
def rebuild_interface_from_cache(self):
```

```
    """Reconstroi TreeView e editores baseado nos componentes em cache"""
```

```
    # Limpa TreeView atual
```

```
    self.tree.clear()
```

```
    # CORREÇÃO: Limpa editor antes de recriar
```

```
    if hasattr(self.region_editor, 'windows'):
```

```
        for window in list(self.region_editor.windows):
```

```
            try:
```

```
                if window.scene():
```

```
                    window.scene().removeItem(window)
```

```
            except RuntimeError:
```

```
                pass
```

```
        self.region_editor.windows.clear()
```

```
# Adiciona posições mecânicas ao region_editor e TreeView
```

```
for comp_name, comp_data in self.componentes.items():
```

```
    pos = comp_data.get('posicao', (0, 0))
```

```
    dims = comp_data.get('dimensoes', (100, 100))
```

```
self.log(f"■ Recriando '{comp_name}': pos={pos}, dims={dims}")
```

```
# CORREÇÃO CRÍTICA: Cria o item no region_editor usando coordenadas do cache
```

```
item = self.region_editor.add_window(
```

```
    x=pos[0], y=pos[1],
```

```
    w=dims[0], h=dims[1],
```

```
    name=comp_name,
```

```
    deletable=True,
```

```
    editable=False
```

```
)
```

```
# Adiciona à TreeView
```

```
node = QTreeWidgetItem([comp_name])
```

```
node.setData(0, Qt.ItemDataRole.UserRole, item)
```

```
self.tree.addTopLevelItem(node)
```

```
# Adiciona janelas de inspeção como subnós
```

```
for i, inspecao in enumerate(comp_data.get('inspecoes', []), 1):
```

```
    comp_node = QTreeWidgetItem([f"w{i}"])
```

```

# Cria placeholder para a janela de comparação
placeholder_data = {
    'type': 'comparison_placeholder',
    'component_name': comp_name,
    'window_name': f'w{i}',
    'inspecao_ref': inspecao
}
comp_node.setData(0, Qt.ItemDataRole.UserRole, placeholder_data)
inspecao['tree_node'] = comp_node
node.addChild(comp_node)
node.setExpanded(True)

```

```

self.log(f"■ Posição '{comp_name}' restaurada com {len(comp_data.get('inspecoes', []))} janelas")

```

```

def closeEvent(self, event):
    """Atualiza posições e chama callback direto"""
    try:
        # Atualiza posições mecânicas no cache
        self._update_mechanical_positions_in_cache()

        # Chama callback direto se disponível
        if self._update_callback:
            self._update_callback(self.componentes, self._orig_bgr)
            print(f"[DEBUG] ■ Callback direto executado com sucesso")

    except Exception as e:
        print(f"[ERROR] Erro no closeEvent: {e}")
        super().closeEvent(event)

```

```

# =====
# EVENT-FILTER (cliques nos visores) + logs auxiliares
# =====

def eventFilter(self, obj, ev):
    from PyQt6.QtCore import QEvent
    if ev.type() == QEvent.Type.MouseButtonPress:
        if obj is self.view_region.viewport():
            pos = self.view_region.mapToScene(int(ev.position().x()),
                                                int(ev.position().y()))
            it = self.view_region.scene().itemAt(pos, self.view_region.transform())
            self._log_click("vis. ESQ", it, pos)
        elif self.view_roi and obj is self.view_roi.viewport():
            pos = self.view_roi.mapToScene(int(ev.position().x()),
                                             int(ev.position().y()))
            it = self.view_roi.scene().itemAt(pos, self.view_roi.transform())
            self._log_click("vis. DIR", it, pos)
        # NOVA FUNCIONALIDADE: Detecta clique em janela azul para sincronizar TreeView
        if (hasattr(it, 'item_type') and it.item_type == 'inspection'):
            self._handle_blue_window_click(it)
            # NOVO: Força seleção visual da janela clicada
            try:

```

```

        self.view_roi.scene().clearSelection()
        it.setSelected(True)
        self.view_roi.scene().update()
        self.view_roi.viewport().update()
    except RuntimeError:
        self.log("■■■ Erro ao selecionar janela azul clicada")
        pass
    return super().eventFilter(obj, ev)

```

```
def _handle_blue_window_click(self, blue_window):
```

```
    """Manipula clique em janela azul para sincronizar com TreeView"""
```

```
    self.log(f"■■■ Clique na janela azul: {getattr(blue_window, 'inspecao_name', '?')}")
```

```
    # NOVO: Força seleção visual imediata
```

```
    try:
```

```
        self.view_roi.scene().clearSelection()
```

```
        blue_window.setSelected(True)
```

```
        self.view_roi.scene().update()
```

```
        self.view_roi.viewport().update()
```

```
    except RuntimeError:
```

```
        self.log("■■■ Erro ao selecionar janela azul clicada")
```

```
        pass
```

```
    # Encontra o nó correspondente na TreeView
```

```
    tree_node = getattr(blue_window, 'tree_node', None)
```

```
    if tree_node:
```

```
        # Evita loops infinitos
```

```
        self._updating_tree_selection = True
```

```
        try:
```

```
            self.tree.setCurrentItem(tree_node)
```

```
            # NOVO: Força expansão do nó pai para visibilidade
```

```
            if tree_node.parent():
```

```
                tree_node.parent().setExpanded(True)
```

```
            self.log(f"■■■ TreeView sincronizada: {tree_node.text(0)} selecionado")
```

```
        # Também garante que a posição mecânica pai seja selecionada
```

```
        self._sync_parent_selection_from_comparison(blue_window)
```

```
        finally:
```

```
            self._updating_tree_selection = False
```

```
    else:
```

```
        self.log("■■■ Nó da TreeView não encontrado para janela azul")
```

```
def _sync_parent_selection_from_comparison(self, blue_window):
```

```
    """Garante que a posição mecânica pai seja selecionada quando janela azul é clicada"""
```

```
    # Usa método centralizado
```

```
    self._ensure_parent_position_selected(blue_window)
```

```
# ----- helpers interno -----
```

```
def _describe_item(self, it) -> str:
```

```
    if it is None:
```

```

        return "área vazia"
    if hasattr(it, "comparison_name"):
        return f"Cmp \"{it.comparison_name}\""
    if isinstance(it, ResizableRectItem):
        nm = getattr(it, "name", "")
        return f"Posição \"{nm or '?'}\""
    return str(it)

def _log_click(self, visor, item, pos):
    desc = self._describe_item(item)
    self.log(f"███ Click {visor}: {desc} @({pos.x():.0f},{pos.y():.0f})")

# -----
#     LOG para clique na TREEVIEW
# -----

def _on_tree_item_clicked(self, item: QTreeWidgetItem, col: int):
    data = item.data(0, Qt.ItemDataRole.UserRole)
    if isinstance(data, dict) and data.get('type') == 'comparison_placeholder':
        kind = f"Placeholder w* ({data.get('window_name')})"
    elif hasattr(data, 'item_type') and data.item_type == 'inspection':
        kind = f"Janela azul ({getattr(data, 'inspecao_name', '?')})"
    elif isinstance(data, ResizableRectItem):
        kind = "Posição mecânica"
    else:
        kind = f"Desconhecido ({type(data)})"
    self.log(f"███ Click Tree → {kind}: {item.text(0)}")

def _on_window_removed(self, item):
    """Remove posição mecânica da tree"""
    for i in range(self.tree.topLevelItemCount()):
        n = self.tree.topLevelItem(i)
        if n.data(0, Qt.ItemDataRole.UserRole) is item:
            self.tree.takeTopLevelItem(i)
            self.log(f"███ Posição mecânica removida da tree")
    return

def _safe_select_tree_node(self, node):
    """Seleciona nó na TreeView de forma segura"""
    if node is None:
        return
    try:
        self._updating_tree_selection = True
        if not node.isSelected():
            self.tree.setCurrentItem(node)
    finally:
        self._updating_tree_selection = False

def _select_tree_node(self, node):
    """Helper para selecionar um nó na tree"""
    self._safe_select_tree_node(node)

```

```

def _on_tree_selection_changed(self):
    """Seleciona retângulo na cena ao clicar na tree"""
    # Evita loops infinitos
    if self._updating_tree_selection:
        return
    nodes = self.tree.selectedItems()
    if not nodes:
        # Limpa seleção das cenas de forma segura
        try:
            self.view_region.scene().clearSelection()
            if self.view_roi:
                self.view_roi.scene().clearSelection()
        except RuntimeError:
            pass
        # Mostra todas as janelas
        if self.roi_editor:
            self.roi_editor.focus_on_parent(None)
        return
    node = nodes[0]
    item = node.data(0, Qt.ItemDataRole.UserRole)

    if item is None:
        return

    # CORREÇÃO: Distingue entre posição mecânica e janela de comparação
    is_mechanical_position = False
    is_comparison_window = False
    comparison_placeholder = False

    if isinstance(item, ResizableRectItem):
        # Verifica se é posição mecânica (está na cena esquerda)
        if item in self.region_editor.windows:
            is_mechanical_position = True
    elif isinstance(item, dict) and item.get('type') == 'comparison_placeholder':
        # NOVO: Placeholder para janelas de comparação (antes do redesenho)
        comparison_placeholder = True
        self.log(f"■ Detectado placeholder para: {item.get('window_name')}")
    elif hasattr(item, 'item_type') and item.item_type == 'inspection':
        # Janela azul já redesenhada
        is_comparison_window = True

    # NOVA LÓGICA: Trata placeholder como janela de comparação
    if comparison_placeholder:
        # CORREÇÃO: Converte placeholder MAS MANTÉM seleção no item w*
        self._handle_comparison_placeholder_selection(node, item, keep_selection=True)
        return
        # CORREÇÃO: Janela azul de comparação (SelectableResizableRectItem)
        is_comparison_window = True
    elif isinstance(item, ResizableRectItem) and hasattr(item, 'parent_uid'):
        is_comparison_window = True

```

```

if is_mechanical_position:
    # Seleciona na cena principal
    try:
        self.view_region.scene().clearSelection()
    except RuntimeError:
        pass
    item.setSelected(True)
    self.view_region.centerOn(item)

    # SEMPRE chama redesenho, mesmo se componente não tem janelas
    if self.roi_editor:
        # Mostra todas as janelas, não apenas as filhas
        self.roi_editor.focus_on_parent(None)

    # Atualiza ROI visual
    self._update_roi_from_selection()

    self.log(f"■ Seleção tree → posição: {node.text(0)}")

```

```

elif is_comparison_window:
    # NOVO: Selecionou janela de comparação (w*) - sincronização completa
    self.log(f"■ Seleção tree → janela comparação: {node.text(0)}")

```

```

# 1. CRÍTICO: Seleciona a janela azul no visor direito e centraliza
try:

```

```

    if self.view_roi:
        self.view_roi.scene().clearSelection()
        item.setSelected(True)
        self.view_roi.centerOn(item)
        # Força atualização visual da seleção
        self.view_roi.scene().update()
        self.view_roi.viewport().update()
        self.log(f"■ Janela azul selecionada e centralizada")

```

```

except RuntimeError:
    self.log("■■ Erro ao selecionar janela azul")

```

```

# 2. Garante que posição mecânica pai seja selecionada no visor esquerdo
self._ensure_parent_position_selected(item)

```

```

else:
    self.log(f"■■ Tipo de item desconhecido selecionado: {type(item)}")

```

```

def _ensure_parent_position_selected(self, comparison_item):

```

```

    """Garante que a posição mecânica pai esteja selecionada no visor esquerdo"""

```

```

    component_name = getattr(comparison_item, 'componente', None)

```

```

    if not component_name:

```

```

        return

```

```

# Encontra a posição mecânica pai

```

```

for window in self.region_editor.windows:

```

```

    if getattr(window, 'name', None) == component_name:

```

```

try:
    # Seleciona no visor esquerdo apenas se necessário
    current_selection = self.view_region.scene().selectedItems()
    if not current_selection or current_selection[0] != window:
        self.view_region.scene().clearSelection()
        window.setSelected(True)
        self.view_region.centerOn(window)
        # Força atualização do ROI
        self._update_roi_from_selection()
        self.log(f"■ Posição pai '{component_name}' selecionada automaticamente")
    break
except RuntimeError:
    self.log("■■ Erro ao selecionar posição mecânica pai")
    break

```

```

def _handle_comparison_placeholder_selection(self, node, placeholder_data, keep_selection=False):
    """Manipula seleção de placeholder de janela de comparação"""
    component_name = placeholder_data.get('component_name')
    window_name = placeholder_data.get('window_name')

```

```

    self.log(f"■ Seleção tree → janela comparação: {window_name} (via placeholder)")

```

```

# 1. Força redesenho para criar as janelas azuis se necessário

```

```

if component_name and component_name in self.componentes:

```

```

    self._redesenhar_janelas_inspecao(component_name)

```

```

# 2. Encontra a janela azul correspondente após redesenho

```

```

target_window = None

```

```

if component_name in self.inspecao_items:

```

```

    for blue_window in self.inspecao_items[component_name]:

```

```

        if hasattr(blue_window, 'inspecao_name') and blue_window.inspecao_name == window_name:

```

```

            target_window = blue_window

```

```

            break

```

```

# 3. CRÍTICO: Seleciona e centraliza a janela azul com feedback visual

```

```

if target_window:

```

```

    try:

```

```

        if self.view_roi:

```

```

            self.view_roi.scene().clearSelection()

```

```

            target_window.setSelected(True)

```

```

            self.view_roi.centerOn(target_window)

```

```

            # NOVO: Força feedback visual da seleção

```

```

            self.view_roi.scene().update()

```

```

            self.view_roi.viewport().update()

```

```

            self.log(f"■ Janela azul {window_name} selecionada via placeholder")

```

```

        except RuntimeError:

```

```

            self.log("■■ Erro ao selecionar janela azul via placeholder")

```

```

# 4. Garante posição pai selecionada

```

```

if target_window:

```

```

    self._ensure_parent_position_selected(target_window)

```

```

# NOVO: Se keep_selection=True, volta seleção TreeView para o item w*
if keep_selection:
    QTimer.singleShot(100, lambda: self._force_tree_selection(node))

def _force_tree_selection(self, node):
    """Força seleção de um nó específico na TreeView"""
    try:
        self._updating_tree_selection = True
        self.tree.setCurrentItem(node)
        self.log(f"■ TreeView forçada para: {node.text(0)}")
    finally:
        self._updating_tree_selection = False

# ----- slot: novo ROI vindo do painel -----
def _on_region_captured(self, img_bgr):
    """
    Recebe numpy BGR do widget de controle e actualiza o visor grande
    sem efeito "cascata".
    """
    # troca cache original (pix + bgr) e reajusta view
    self._orig_bgr = img_bgr.copy() if img_bgr is not None else None
    self._orig_pix = InspectionConfigWidget._bgr_to_pixmap(img_bgr)
    self._update_pix()

# ----- novo: recorte da posição mecânica -----
def _update_roi_from_selection(self):
    """
    Recorta a área da Posição Mecânica selecionada e mostra
    na view_roi preservando a resolução original.
    """
    if self.view_roi is None or self._orig_bgr is None:
        return
    sel = [it for it in self.view_region.scene().selectedItems()
            if isinstance(it, ResizableRectItem)]
    if not sel:
        return
    item = sel[0]
    # usa retângulo do item já mapeado p/ cena (inclui posição)
    r_scene = item.mapRectToScene(item.rect())
    x, y, w, h = map(int, [r_scene.x(), r_scene.y(),
                           r_scene.width(), r_scene.height()])
    h_img, w_img, _ = self._orig_bgr.shape
    # limita dentro da imagem
    x = max(0, min(x, w_img - 1))
    y = max(0, min(y, h_img - 1))
    w = max(1, min(w, w_img - x))
    h = max(1, min(h, h_img - y))
    roi_bgr = self._orig_bgr[y:y + h, x:x + w].copy()
    if roi_bgr.size == 0:
        return

```

```
# Atualiza o visor ROI
```

```
roi_px = InspectionConfigWidget._bgr_to_pixmap(roi_bgr) # qualidade máx.
```

```
# CORREÇÃO: Remove apenas o pixmap anterior, mantém janelas de comparação
```

```
scene = self.view_roi.scene()
```

```
# Remove apenas o item de imagem anterior (se existir)
```

```
if hasattr(self, '_roi_pix_item') and self._roi_pix_item:
```

```
    scene.removeItem(self._roi_pix_item)
```

```
# Adiciona nova imagem
```

```
self._roi_pix_item = scene.addPixmap(roi_px)
```

```
self.view_roi.fitInView(self._roi_pix_item,
```

```
                        Qt.AspectRatioMode.KeepAspectRatio)
```

```
# CORREÇÃO: Mantém todas as janelas sempre visíveis
```

```
if self.roi_editor and isinstance(item, ResizableRectItem):
```

```
    # Encontra o nome do componente
```

```
    component_name = getattr(item, 'name', None)
```

```
    # DEBUGGING: Log da operação
```

```
    self.log(f"■ Atualizando ROI para componente: {component_name}")
```

```
    # FORÇA redesenho mesmo se componente não tem janelas (para limpar cena)
```

```
    if component_name:
```

```
        # POSIÇÃO *REAL* da posição mecânica
```

```
        par_scene = item.sceneBoundingRect()
```

```
        pos_tuple = (int(par_scene.x()), int(par_scene.y()))
```

```
        dim_tuple = (int(par_scene.width()), int(par_scene.height()))
```

```
        comp = self.componentes.setdefault(
```

```
            component_name,
```

```
            {'posicao': pos_tuple,
```

```
            'dimensoes': dim_tuple,
```

```
            'inspecoes': []}
```

```
        )
```

```
        # mantém sempre actualizados
```

```
        comp['posicao'] = pos_tuple
```

```
        comp['dimensoes'] = dim_tuple
```

```
        # redesenha (ou limpa) janelas do componente
```

```
        self._redesenhar_janelas_inspecao(component_name)
```

```
def _redesenhar_janelas_inspecao(self, componente):
```

```
    """
```

```
    NOVA IMPLEMENTAÇÃO: Identica ao gerador de projetos
```

```
    Redesenha todas as janelas de comparação para o componente especificado
```

```
    """
```

```
# Verifica se componentes foi inicializado
```

```
if not hasattr(self, 'componentes'):
```

```
    self.componentes = {}
```

```

    return
if componente not in self.componentes:
    return

# DEBUGGING: Log do componente sendo redesenhado
inspecoes_count = len(self.componentes[componente].get('inspecoes', []))
self.log(f"■ Iniciando redesenho para '{componente}' ({inspecoes_count} janelas)")

if not self.view_roi or not self.view_roi.scene():
    self.log(f"■■ Não foi possível redesenhar - visor ROI não disponível")
    return

scene = self.view_roi.scene()

# CORREÇÃO: Remove TODAS as janelas de inspeção da cena (de todos os componentes)
# Isso garante que apenas as janelas do componente atual sejam exibidas
for comp_name in list(self.inspecao_items.keys()):
    for item in self.inspecao_items[comp_name]:
        if item.scene():
            scene.removeItem(item)
    self.inspecao_items[comp_name] = []

self.log(f"■ Cena ROI limpa - todas as janelas antigas removidas")

# Garante que o componente atual existe no dicionário
if componente not in self.inspecao_items:
    self.inspecao_items[componente] = []

# Obtém dimensões da cena para escala
scene_rect = scene.sceneRect()
scene_width = scene_rect.width()
scene_height = scene_rect.height()

if scene_width == 0 or scene_height == 0:
    return

# Redesenha cada janela de inspeção salva
# Usa apenas cor azul para todas as janelas de comparação
cor_azul = Qt.GlobalColor.blue

for i, inspecao in enumerate(self.componentes[componente].get('inspecoes', [])):
    # -----
    # 1. Posição absoluta (salva no dicionário)
    x_abs, y_abs = inspecao['posicao']
    w_orig, h_orig = inspecao['tamanho']
    # 2. Deslocamento da ROI mecânica (ponto-pai)
    pai_x, pai_y = self.componentes[componente]['posicao']
    # 3. Converte para COORDENADA RELATIVA à imagem exibida
    graphicsView_x = x_abs - pai_x
    graphicsView_y = y_abs - pai_y

```

```

# DEBUG
self.log(f"■ Redesenhando w[i+1]: abs=({x_abs},{y_abs}) "
        f"→ rel=({graphicsView_x},{graphicsView_y}) "
        f"tam=({w_orig},{h_orig})")
graphicsView_w = w_orig
graphicsView_h = h_orig

# CORREÇÃO CRÍTICA: Callback que salva alterações do usuário
def _on_insp_change(item,
                    d=inspecao,
                    comp=componente,
                    off_x=pai_x,
                    off_y=pai_y):
    """Atualiza dados quando item é modificado"""
    # CORREÇÃO: Permite salvar alterações do usuário mesmo durante redesenho
    # Só bloqueia durante a criação inicial do item
    if hasattr(self, '_criando_item') and self._criando_item:
        return
    br = item.sceneBoundingRect()
    # Converte de volta para absoluta somando o offset da ROI
    new_pos = (int(br.x()) + off_x,
               int(br.y()) + off_y)
    new_tam = (int(br.width()), int(br.height()))
    d['posicao'] = new_pos
    d['tamanho'] = new_tam
    # NOVO: Também atualiza originais para manter consistência
    d['original_posicao'] = new_pos
    d['original_tamanho'] = new_tam
    self.log(f"■ Janela {comp} atualizada: pos={d['posicao']}, tam={d['tamanho']}")
    # Marca que está criando item (evita callback durante criação)
    self._criando_item = True

# Cria item visual

rect_item = SelectableResizableRectItem(
    graphicsView_x, graphicsView_y,
    graphicsView_w, graphicsView_h,
    pen=QPen(cor_azul, 2),
    change_callback=_on_insp_change
)

# Item criado - libera callbacks
self._criando_item = False

# Metadados para identificação
rect_item.item_type = 'inspection'
rect_item.componente = componente
rect_item.inspecao_ref = inspecao
rect_item.deletavel = True

# NOVO: Estabelece associação bidirecional TreeView ↔ Janela Azul

```

```

inspecao['blue_window'] = rect_item
if inspecao.get('tree_node'):
    # CORREÇÃO: Substitui placeholder pela janela azul real
    inspecao['tree_node'].setData(0, Qt.ItemDataRole.UserRole, rect_item)
    self.log(f"■ Nó '{inspecao['tree_node'].text(0)}' associado à janela azul (substitui placeholder)")
    # CRÍTICO: Associação bidirecional completa
    rect_item.tree_node = inspecao['tree_node']
    rect_item.inspecao_name = f"w{i+1}"

rect_item.component_name = componente

# NOVO: Metadados extras para debugging
rect_item.original_inspecao_data = inspecao

# DEBUGGING: Confirma associação bidirecional
self.log(f"■ Associação bidirecional completa: TreeView({inspecao['tree_node'].text(0)}) ↔ Janela({rect_item.insp

scene.addItem(rect_item)
self.inspecao_items[componente].append(rect_item)

# Força atualização da visualização
self.view_roi.viewport().update()

# DEBUGGING: Log do resultado final
total_items_scene = len([item for item in scene.items()
                        if hasattr(item, 'item_type') and item.item_type == 'inspection'])
self.log(f"■ Cena ROI agora tem {total_items_scene} janelas de inspeção visíveis")

self.log(f"■ Redesenhadas {len(self.componentes[componente].get('inspecoes', []))} janelas para {componente}")

# ===== NOVO: SCENE EVENT FILTER PARA JANELAS AZUIS =====
def sceneEventFilter(self, watched, event):
    """Event filter específico para janelas azuis"""
    try:
        from PyQt6.QtCore import QEvent

        # Só processa cliques do mouse em janelas azuis
        if (hasattr(watched, 'item_type') and
            watched.item_type == 'inspection' and
            event.type() == QEvent.Type.GraphicsSceneMousePress):

            self.log(f"■ Scene Event Filter: clique em {getattr(watched, 'inspecao_name', '?')}")

            # Chama handler específico
            self._handle_blue_window_click(watched)

            # Propaga o evento normalmente
            return False

    except Exception as e:
        self.log(f"■ Erro no scene event filter: {e}")

```

```

# Propaga evento para handlers normais
return False

# mantém proporção 4:3 na imagem grande
def resizeEvent(self, ev):
    super().resizeEvent(ev)
    self._fit_views()

# NOVO MÉTODO: Remove janela dos dados do componente
def _on_comparison_window_removed(self, roi_item):
    """Remove janela de comparação dos dados e da TreeView"""
    # Encontra o componente e remove a janela dos dados
    if not hasattr(self, 'componentes'):
        self.componentes = {}
        return
    # DEBUGGING: Log antes da busca
    self.log(f"■ Procurando janela removida entre {len(self.componentes)} componentes")

    removed = False
    for comp_name, comp_data in self.componentes.items():
        inspecoes = comp_data.get('inspecoes', [])
        for i, insp in enumerate(inspecoes):
            if insp.get('roi_editor_item') is roi_item:
                # Remove dos dados
                inspecoes.pop(i)
                self.log(f"■ Janela removida dos dados de {comp_name}")
                # Redesenha as janelas restantes
                self._redesenhar_janelas_inspecao(comp_name)
                break
        if removed:
            break

    # Remove também janela do ROIWindowEditor se ainda estiver lá
    if roi_item in self.roi_editor.windows:
        self.roi_editor.windows.remove(roi_item)

    # Remove da TreeView
    for i in range(self.tree.topLevelItemCount()):
        parent_node = self.tree.topLevelItem(i)
        for j in range(parent_node.childCount()):
            child_node = parent_node.child(j)
            if child_node.data(0, Qt.ItemDataRole.UserRole) is roi_item:
                parent_node.removeChild(child_node)
                self.log(f"■ Janela removida da TreeView")
                return

    # ----- ajuste automático das views -----
    def _fit_views(self):
        if hasattr(self, "_pix_item") and self._pix_item:
            self.view_region.fitInView(self._pix_item,

```

```

        Qt.AspectRatioMode.KeepAspectRatio)
if hasattr(self, "view_roi") and self.view_roi:
    self.view_roi.fitInView(
        self.view_roi.scene().itemsBoundingRect(),
        Qt.AspectRatioMode.KeepAspectRatio)

# -----
# Conversões utilitárias
# -----
@staticmethod
def _qpix_to_bgr(px: QPixmap):
    """Converte QPixmap → numpy BGR preservando resolução."""
    if px.isNull():
        return None
    img = px.toImage().convertToFormat(QImage.Format.Format_RGB888)
    w, h = img.width(), img.height()
    ptr = img.bits().asstring(w * h * 3)
    import numpy as np
    arr = np.frombuffer(ptr, np.uint8).reshape((h, w, 3))
    return cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)

# ----- habilita / desabilita editores -----
def _toggle_editor(self, which: str, enabled: bool):
    if which == "mech":
        if enabled:
            self.region_editor.start_drawing()
            self.btn_cmp.setChecked(False)
        else:
            self.region_editor.stop_drawing()
    elif which == "cmp" and self.roi_editor is not None:
        if enabled:
            # Determina o item pai baseado na seleção atual
            parent_item = None
            parent_uid = None
            selected_items = self.view_region.scene().selectedItems()
            if selected_items:
                for item in selected_items:
                    if isinstance(item, ResizableRectItem):
                        parent_item = item
                        parent_uid = item.id
                        break

            # Inicia desenho SEM pai Qt Graphics (evita problemas entre cenas)
            # Mas salva o parent_uid para associação lógica
            self._comparison_parent_uid = parent_uid
            # CORREÇÃO: Garante que desenho acontece APENAS no visor direito
            if self.roi_editor and self.view_roi:
                # Define pen azul para ROIWindowEditor (evita janelas vermelhas)
                from PyQt6.QtGui import QPen
                blue_pen = QPen(Qt.GlobalColor.blue, 2)
                # Força foco no visor direito antes de iniciar desenho

```

```

        self.view_roi.setFocus()
        self.roi_editor.start_drawing(pen=blue_pen)
    else:
        self.log("■■ Visor direito não disponível para desenho")

```

```

        self.btn_mech.setChecked(False)
    else:
        self.roi_editor.stop_drawing()

```

```

# -----
# QLabel com proporção fixa (default = 4:3)
# -----

```

```

class AspectRatioLabel(QLabel):

```

```

    """

```

```

    QLabel que mantém width : height = 4 : 3
    (ou outro aspecto definido no construtor).
    Usa o mecanismo height-for-width do Qt ■ sem precisar de
    resizeEvent personalizado no widget contêiner.
    """

```

```

    def __init__(self, *args, aspect_ratio: float = 4/3, **kwargs):
        super().__init__(*args, **kwargs)
        self._aspect_ratio = aspect_ratio    # width / height
        self.setSizePolicy(QSizePolicy.Policy.Expanding,
                           QSizePolicy.Policy.Expanding)

```

```

# ---- height-for-width -----

```

```

    def hasHeightForWidth(self) -> bool:
        return True

```

```

    def heightForWidth(self, w: int) -> int:
        return int(w / self._aspect_ratio)

```

```

# sizeHint garante 4:3 quando o layout consulta o widget

```

```

    def sizeHint(self):
        hint = super().sizeHint()
        hint.setHeight(self.heightForWidth(hint.width()))
        return hint

```

```

# -----

```

```

# InspectionConfigWidget

```

```

# • show_region      → exibe/omite o visor grande na própria coluna
# • show_aux_button  → exibe/omite o botão "Aux. de inspeção"
# • show_size_controls → exibe/omite campos Largura / Altura
# • show_region_button → exibe/omite o botão "Definir região"
# -----

```

```

class InspectionConfigWidget(QGroupBox):

```

```

    """

```

```

    UI da Ação 'Inspeção'.
    """

```

```

    parametersChanged = pyqtSignal(int, int)    # width, height

```

```
regionCaptured = pyqtSignal(object)      # img_bgr
```

```
def __init__(self, controller,
              parent=None,
              *,
              show_region: bool = True,
              show_aux_button: bool = True,
              show_size_controls: bool = True,
              show_region_button: bool = True):
    super().__init__("Config. Inspeção", parent)
    self._c = controller
    self._show_region = show_region
    self._show_aux_button = show_aux_button
    self._template = None
    self._last_roi_bgr = None
    self._show_size_controls = show_size_controls
    self._show_region_button = show_region_button
    self._build_ui()
    # Cache simples para dados da janela auxiliar
    self._cached_auxiliary_data = {
        'componentes': {},      # posições mecânicas + janelas
        'region_image': None    # imagem da região (BGR)
    }
    # Flag para controlar limpeza de cache durante inicialização
    self._initializing_auxiliary = False
```

```
# =====
# NOVO – carregar parâmetros vindos de um ponto existente
# =====
```

```
def load_from_meta(self, meta: dict):
```

```
    """
```

Preenche largura, altura e similaridade mínima de acordo com o dicionário salvo no ponto selecionado.

A estrutura completa de 'componentes' também é carregada no cache interno para que as janelas apareçam automaticamente no painel.

```
    """
```

```
# --- ROI principal (usa primeiro componente, se houver) ----
```

```
comp_dict = meta.get("componentes", {})
```

```
if comp_dict:
```

```
    # pega dimensões do primeiro componente para sugerir ROI
```

```
    first = next(iter(comp_dict.values()))
```

```
    w, h = first.get("dimensoes", (self.spin_w.value(),
                                   self.spin_h.value()))
```

```
    self.spin_w.setValue(int(w))
```

```
    self.spin_h.setValue(int(h))
```

```
# --- similaridade mínima -----
```

```
sim = meta.get("similaridade", None)
```

```
if sim is not None:
```

```
    self.spin_sim.setValue(int(float(sim)*100))
```

```

# --- cache dos componentes / janelas -----
if comp_dict:
    self._cached_auxiliary_data['componentes'] = comp_dict
    # força reconstrução imediata dos retângulos
    self.update_main_viewer()

# ----- construção -----
def _build_ui(self):
    v = QVBoxLayout(self)

    form = QFormLayout()
    self._ratio = 4/3
    self._internal = False # evita recursão ao sincronizar
    self.spin_w = QSpinBox(); self.spin_w.setRange(12, 4000); self.spin_w.setValue(400)
    self.spin_h = QSpinBox(); self.spin_h.setRange(9, 3000); self.spin_h.setValue(int(400/ self._ratio))
    if self._show_size_controls:
        form.addRow("Largura (px):", self.spin_w)
        form.addRow("Altura (px):", self.spin_h)
    v.addLayout(form)

    # botão capturar região
    self.btn_region = QPushButton("Definir região")
    if self._show_region_button:
        v.addWidget(self.btn_region)

# -----
# NOVO VISOR PRINCIPAL (QGraphicsView + ROIWindowEditor)
# -----
self.view_region = QGraphicsView()
self.view_region.setRenderHints(
    QPainter.RenderHint.SmoothPixmapTransform |
    QPainter.RenderHint.Antialiasing
)
self.view_region.setSizePolicy(QSizePolicy.Policy.Expanding,
                               QSizePolicy.Policy.Expanding)
scene = QGraphicsScene(self.view_region)
self.view_region.setScene(scene)
# item que exibirá o QPixmap da região
self._region_pix_item = scene.addPixmap(QPixmap())
# editor que torna as janelas vermelhas clicáveis
self.region_editor = ROIWindowEditor(self.view_region)

# Mantém o mesmo nome ("lbl_region") para compatibilidade externa
self.lbl_region = self.view_region

# -----
# Sempre que o usuário selecionar uma Posição Mecânica
# no visor "Região", recortamos a área correspondente
# e a exibimos no pequeno visor 'ROI' aqui do painel
# -----

```

```

self.view_region.scene().selectionChanged.connect(
    self._update_roi_from_selection_main)

if self._show_region:
    v.addWidget(self.view_region)
else:
    self.view_region.setVisible(False)

# garante 4:3 e largura ≤ 15 % da tela já na criação
self._adjust_region_size()

# visor pequeno (ROI mecânico)
# visor ROI menor (metade da largura, 4:3, centrado)
self.lbl_roi = QLabel("ROI")
self._style_label(self.lbl_roi)
self.lbl_roi.setAlignment(
    Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignVCenter)
# largura controlada manualmente → Policy.Fixed evita "esticar"
self.lbl_roi.setSizePolicy(QSizePolicy.Policy.Fixed,
                           QSizePolicy.Policy.Fixed)
v.addWidget(self.lbl_roi)
# Ajusta aspecto inicial do visor ROI
self._update_roi_aspect()

# similaridade + salvar
hsim = QHBoxLayout()
self.spin_sim = QSpinBox(); self.spin_sim.setRange(50, 100); self.spin_sim.setValue(95)
hsim.addWidget(QLabel("Similaridade mínima %:"))
hsim.addWidget(self.spin_sim)
self.btn_save = QPushButton("Salvar")
hsim.addWidget(self.btn_save)
hsim.addStretch()
v.addLayout(hsim)

# linha de botões
h = QHBoxLayout()
self.btn_inspect = QPushButton("Inspeccionar")
self.btn_addimg = QPushButton("Add imagem")
h.addWidget(self.btn_inspect); h.addWidget(self.btn_addimg)
v.addLayout(h)

# botão auxiliar (opcional)
if self._show_aux_button:
    self.btn_aux = QPushButton("Aux. de inspeção")
    v.addWidget(self.btn_aux)
else:
    self.btn_aux = None # atributo presente para checagem externa
v.addStretch()

# sinais

```

```

if self._show_size_controls:
    self.spin_w.valueChanged.connect(self._emit_params)
    self.spin_h.valueChanged.connect(self._emit_params)
    # mantém 4:3
    self.spin_w.valueChanged.connect(lambda w: self._sync_hw('w', w))
    self.spin_h.valueChanged.connect(lambda h: self._sync_hw('h', h))
if self._show_region_button:
    self.btn_region.clicked.connect(self._capture_region)
# botão "Aux. de inspeção" só existe se show_aux_button=True
if self.btn_aux is not None:
    self.btn_aux.clicked.connect(self._open_aux)

# -----
# NOVO: coloca imagem na cena principal
# -----
def _set_main_pixmap(self, img_bgr):
    """Atualiza o QGraphicsPixmapItem da região principal."""
    if img_bgr is None:
        self._region_pix_item.setPixmap(QPixmap())
        return
    h, w = img_bgr.shape[:2]
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    qimg = QImage(img_rgb.data, w, h, 3 * w, QImage.Format.Format_RGB888)
    self._region_pix_item.setPixmap(QPixmap.fromImage(qimg))
    # mantém 4:3
    self.view_region.fitInView(self._region_pix_item,
                               Qt.AspectRatioMode.KeepAspectRatio)

# -----
# AJUSTE 4:3 (imagem "Região") + limite 15 % largura da tela
# -----
def _adjust_region_size(self):
    """
    Mantém o visor 'Região' com proporção 4:3, limitando sua largura
    a, no máximo, 15 % da largura do monitor principal.
    """
    if not hasattr(self, "view_region"):
        return

    # Largura máxima baseada em 15 % da largura da tela
    scr = QApplication.primaryScreen()
    if scr is None:
        return
    max_w = int(scr.size().width() * 0.18)

    # Largura realmente disponível no layout da coluna
    col_w = max(10, self.width())
    tgt_w = min(col_w, max_w)
    tgt_h = int(tgt_w * 3 / 4) # 4:3

    # Define dimensões fixas

```

```

self.view_region.setFixedWidth(tgt_w)
self.view_region.setFixedHeight(tgt_h)

# Re-ajusta a imagem para caber
if hasattr(self, "_region_pix_item") and self._region_pix_item:
    self.view_region.fitInView(self._region_pix_item,
                               Qt.AspectRatioMode.KeepAspectRatio)

# ----- tamanho fixo 4:3 -----

def _update_roi_aspect(self):
    """
    Ajusta SOMENTE o visor ROI.
    Passa a ser completamente independente do visor 'Região'.
    """
    col_w = self.width() or 1          # largura real da coluna
    roi_w = max(10, int(col_w * 0.5))  # 50 % da coluna
    self.lbl_roi.setFixedWidth(roi_w)   # largura fixa
    self.lbl_roi.setFixedHeight(int(roi_w * 3 / 4)) # mantém 4:3

def _style_label(self, lab: QLabel):
    lab.setAlignment(Qt.AlignmentFlag.AlignCenter)
    lab.setStyleSheet("QLabel { background:#263238; color:#CFD8DC; }")

# ----- ratio keeper -----
def _sync_hw(self, changed, val):
    if self._internal: return
    self._internal = True
    if changed == 'w':
        self.spin_h.setValue(int(val / self._ratio))
    else:
        self.spin_w.setValue(int(val * self._ratio))
    self._internal = False

# ----- -- slots -----
def _emit_params(self):
    self.parametersChanged.emit(self.spin_w.value(), self.spin_h.value())

def _grab_frame(self):
    cm = getattr(self._c, "camera_manager", None)
    if not cm or not getattr(cm, "_cap", None): return None
    ok, frame = cm._cap.read()
    return frame if ok else None

# =====
# Sincronização com outras janelas
# =====
def _apply_external_roi(self, img_bgr):
    """
    Recebe um ROI capturado em OUTRO InspectionConfigWidget

```

(ex.: o widget embutido na janela Auxiliar) e apenas actualiza o visor + buffer interno, SEM re-emitir regionCaptured (assim evitamos loops infinitos de sinal).

"""

if img_bgr is None:

return

self._last_roi_bgr = img_bgr

self._set_main_pixmap(img_bgr)

def _capture_region(self):

frame = self._grab_frame()

if frame is None:

return

h_img, w_img, _ = frame.shape

ww = self.spin_w.value(); hh = self.spin_h.value()

cx, cy = w_img//2, h_img//2

x0 = max(0, cx-ww//2); y0 = max(0, cy-hh//2)

roi = frame[y0:y0 + hh, x0:x0 + ww].copy()

self._last_roi_bgr = roi # guarda original p/ alta qualidade

self._set_main_pixmap(roi)

self.regionCaptured.emit(roi)

Limpa cache quando nova região é definida

self.clear_auxiliary_cache()

=====

NOVO – limpeza completa dos visores Região / ROI

=====

def clear_views(self):

"""

Apaga as imagens exibidas nos visores 'Região' e 'ROI' e remove todas as janelas (vermelhas/azuis) desenhadas nesses visores.

Usado quando o utilizador adiciona um ponto de INSPEÇÃO para iniciar a próxima configuração “do zero”.

"""

----- limpa imagem da REGIÃO -----

self._set_main_pixmap(None)

remove retângulos do editor da Região

for it in list(self.region_editor.windows):

try:

if it.scene():

self.region_editor.view.scene().removeItem(it)

except RuntimeError:

pass

self.region_editor.windows.clear()

----- limpa ROI -----

self.lbl_roi.clear()

if hasattr(self, "_roi_pix_item") and self._roi_pix_item:

try:

if self.view_roi and self._roi_pix_item.scene():

self.view_roi.scene().removeItem(self._roi_pix_item)

except RuntimeError:

```

        pass
        self._roi_pix_item = None
# remove janelas de comparação
if hasattr(self, "roi_editor") and self.roi_editor:
    for it in list(self.roi_editor.windows):
        try:
            if it.scene():
                it.scene().removeItem(it)
        except RuntimeError:
            pass
    self.roi_editor.windows.clear()
# zera caches visuais
self._last_roi_bgr = None
self._cached_auxiliary_data['region_image'] = None

def _show_pixmap(self, label: QLabel, img_bgr, *, draw_border=False):
    print(f"[DEBUG] === _SHOW_PIXMAP INICIADO ===")
    print(f"[DEBUG] Label: {label.objectName() if hasattr(label, 'objectName') else 'unnamed'}")
    print(f"[DEBUG] Imagem: {img_bgr.shape if img_bgr is not None else 'None'}")
    print(f"[DEBUG] Draw border: {draw_border}")
    if img_bgr is None:
        print(f"[DEBUG] ■ Imagem é None - abortando")
        return
    h,w,_ = img_bgr.shape
    print(f"[DEBUG] Dimensões da imagem: {w}x{h}")
    if draw_border:
        print(f"[DEBUG] Desenhando borda amarela...")
        img_bgr = cv2.rectangle(img_bgr.copy(), (0,0), (w-1,h-1),
                                (0,255,255), 2)
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    print(f"[DEBUG] Imagem convertida para RGB")
    from PyQt6.QtGui import QImage, QPixmap
    qimg = QImage(img_rgb.data, w, h, 3*w, QImage.Format.Format_RGB888)
    print(f"[DEBUG] QImage criada: {qimg.width()}x{qimg.height()}")
    px = QPixmap.fromImage(qimg).scaled(
        label.width(), label.height(),
        Qt.AspectRatioMode.KeepAspectRatio,
        Qt.TransformationMode.SmoothTransformation)
    print(f"[DEBUG] QPixmap escalado para: {px.width()}x{px.height()}")
    print(f"[DEBUG] Label size: {label.width()}x{label.height()}")

    label.setPixmap(px)
    print(f"[DEBUG] ■ QPixmap definido no label")

# NOVO: Força atualização visual
label.update()
label.repaint()
print(f"[DEBUG] ■ Update e repaint forçados")

def _open_aux(self):
    # NOVA ABORDAGEM: Callback direto

```

```

def update_callback(componentes, region_image):
    """Callback que é chamado diretamente quando auxiliar fecha"""
    print(f"[DEBUG] ■ Callback direto recebido: {len(componentes)} componentes")
    self._update_main_with_data(componentes, region_image)
# 1) widget de controle (lado direito)
# no painel à direita omitimos o visor da região E o botão auxiliar
ctrl_w = InspectionConfigWidget(
    self._c,
    show_region=False,
    show_aux_button=False,
    show_size_controls=False,    # ■ remove Largura / Altura
    show_region_button=False    # ■ remove "Definir região"
)
# CORREÇÃO CRÍTICA: Bloqueia limpeza de cache durante inicialização da auxiliar
self._initializing_auxiliary = True
ctrl_w.spin_w.setValue(self.spin_w.value())
ctrl_w.spin_h.setValue(self.spin_h.value())
ctrl_w.spin_sim.setValue(self.spin_sim.value())
# Passa referência do cache para o widget de controle
ctrl_w._cached_auxiliary_data = self._cached_auxiliary_data
ctrl_w._last_roi_bgr = self._last_roi_bgr
# 2) obtém ROI em resolução total, se disponível
if self._last_roi_bgr is not None:
    pix = self._bgr_to_pixmap(self._last_roi_bgr)
    # CORREÇÃO: Não sobrescreve cache existente durante inicialização
    if self._cached_auxiliary_data.get('region_image') is None:
        self._cached_auxiliary_data['region_image'] = self._last_roi_bgr.copy()
    else:
        pix = self._bgr_to_pixmap(self._last_roi_bgr) if self._last_roi_bgr is not None else QPixmap()
        # Não força None se já há dados em cache
        if self._cached_auxiliary_data.get('region_image') is None:
            self._cached_auxiliary_data['region_image'] = None
# 3) cria diálogo e mostra
dlg = _AuxDialog(pix, ctrl_w, update_callback, self) # <-- callback direto

# CORREÇÃO: Libera bloqueio após auxiliar estar totalmente inicializada
self._initializing_auxiliary = False

dlg.resize(900, 600)
dlg.show()

```

```

def _update_main_with_data(self, componentes, region_image):
    """Método simplificado para atualizar visor principal com dados da auxiliar"""
    print(f"[DEBUG] ■ Atualizando visor principal diretamente: {len(componentes)} componentes")

    # Atualiza cache interno
    self._cached_auxiliary_data['componentes'] = componentes.copy() if componentes else {}
    if region_image is not None:
        self._cached_auxiliary_data['region_image'] = region_image.copy()
        self._last_roi_bgr = region_image.copy()
        print(f"[DEBUG] Imagem atualizada: {region_image.shape}")

```

```

# Chama atualização imediata - SEM delay
try:
    self.update_main_viewer()
    print(f"[DEBUG] ■ Visor principal atualizado com sucesso")
except Exception as e:
    print(f"[DEBUG] ■ Erro na atualização: {e}")

# =====
# NOVO SISTEMA: CACHE SIMPLES PARA DADOS AUXILIARES
# =====

def clear_auxiliary_cache(self):
    """Limpa o cache de dados auxiliares"""
    # CORREÇÃO: Não limpa cache se estiver inicializando auxiliar
    if hasattr(self, '_initializing_auxiliary') and self._initializing_auxiliary:
        print(f"[DEBUG] Limpeza de cache bloqueada - inicializando auxiliar")
        return
    old_count = len(self._cached_auxiliary_data.get('componentes', {}))
    self._cached_auxiliary_data = {
        'componentes': {},
        'region_image': None
    }
    print(f"[DEBUG] Cache auxiliar limpo - {old_count} componentes removidos")

    # CORREÇÃO: Também limpa o visor principal se necessário
    if hasattr(self, 'lbl_region') and hasattr(self, '_last_roi_bgr') and self._last_roi_bgr is not None:
        # Reexibe apenas a imagem-base
        self._set_main_pixmap(self._last_roi_bgr)
        print(f"[DEBUG] Visor principal limpo")

def get_cached_auxiliary_data(self):
    """Retorna dados em cache para a auxiliar"""
    return self._cached_auxiliary_data.copy()

def update_main_viewer(self):
    """Atualiza o visor principal com as posições mecânicas do cache"""
    print(f"[DEBUG] ■ UPDATE_MAIN_VIEWER chamado automaticamente")
    componentes = self._cached_auxiliary_data.get('componentes', {})

    if not componentes:
        print(f"[DEBUG] ■■ Nenhum componente no cache para exibir")
        return

    print(f"[DEBUG] Atualizando visor principal com {len(componentes)} posições")

    # DEBUG: Log detalhado dos componentes
    for nome, dados in componentes.items():
        pos = dados.get('posicao', (0, 0))
        dims = dados.get('dimensoes', (100, 100))
        insp_count = len(dados.get('inspecoes', []))

```

```

print(f"[DEBUG] Componente '{nome}': pos={pos}, dims={dims}, {insp_count} inspeções")
for i, insp in enumerate(dados.get('inspecoes', [])):
    print(f"[DEBUG] Inspeção {i+1}: pos={insp.get('posicao', (0,0))}, tam={insp.get('tamanho', (0,0))}")

# CORREÇÃO: Verifica se há imagem base disponível
if not hasattr(self, '_last_roi_bgr') or self._last_roi_bgr is None:
    print(f"[DEBUG] ■■ Nenhuma imagem base disponível (_last_roi_bgr é None)")
    # Tenta usar imagem do cache se existir
    cached_image = self._cached_auxiliary_data.get('region_image')
    if cached_image is not None:
        self._last_roi_bgr = cached_image.copy()
        print(f"[DEBUG] ■ Imagem restaurada do cache: {self._last_roi_bgr.shape}")

# lbl_region hoje é QGraphicsView; só usa o pixmap se
# ainda for um QLabel legado.
elif isinstance(getattr(self, 'lbl_region', None), QLabel) \
    and self.lbl_region.pixmap():
    # Fallback: usa imagem do label se existir
    print(f"[DEBUG] Tentando usar imagem do label como base")
    try:
        # Converte pixmap para BGR
        pixmap = self.lbl_region.pixmap()
        self._last_roi_bgr = self._qpixmap_to_bgr(pixmap)
        if self._last_roi_bgr is None:
            print(f"[DEBUG] ■ Falha ao converter pixmap para BGR")
            return
        print(f"[DEBUG] ■ Imagem convertida: {self._last_roi_bgr.shape}")
    except Exception as e:
        print(f"[DEBUG] ■ Erro ao converter pixmap: {e}")
        return
else:
    # NOVO: Tenta criar imagem padrão se não houver nenhuma
    print(f"[DEBUG] ■ Criando imagem padrão para desenho...")
    try:
        import numpy as np
        # Cria imagem padrão 400x300 (cinza escuro)
        default_image = np.full((300, 400, 3), (64, 64, 64), dtype=np.uint8)
        self._last_roi_bgr = default_image
        self._cached_auxiliary_data['region_image'] = default_image.copy()
        print(f"[DEBUG] ■ Imagem padrão criada: {default_image.shape}")
    except Exception as e:
        print(f"[DEBUG] ■ Erro ao criar imagem padrão: {e}")
        return

# Garante que o cache tem a imagem atual
if self._cached_auxiliary_data.get('region_image') is None and self._last_roi_bgr is not None:
    self._cached_auxiliary_data['region_image'] = self._last_roi_bgr.copy()
    print(f"[DEBUG] ■ Cache atualizado com imagem atual")

# Log das dimensões da imagem base
h, w = self._last_roi_bgr.shape[:2]

```

```

print(f"[DEBUG] Imagem base: {w}x{h} pixels")

# Log dos dados dos componentes
for nome, dados in componentes.items():
    pos = dados.get('posicao', (0, 0))
    dims = dados.get('dimensoes', (100, 100))

    inspecoes = dados.get('inspecoes', [])
    print(f"[DEBUG] Componente '{nome}': pos={pos}, dims={dims}, {len(inspecoes)} inspeções")

# Exibe APENAS a imagem, sem desenhar retângulos nela
# (os retângulos agora serão itens clicáveis sobrepostos na cena)
self._set_main_pixmap(self._last_roi_bgr)

# -----
# NOVO: cria/actualiza retângulos clicáveis para cada posição
# -----
self._refresh_clickable_position_items(componentes)
print(f"[DEBUG] Visor principal atualizado com itens clicáveis")

# -----
# NOVO – retângulos vermelhos clicáveis no visor principal
# -----
def _refresh_clickable_position_items(self, componentes):
    """
    Remove janelas antigas e cria novas janelas vermelhas clicáveis
    correspondentes às posições mecânicas do cache.
    """
    # 1) remove janelas existentes
    for win in list(self.region_editor.windows):
        try:
            if win.scene():
                win.scene().removeItem(win)
        except RuntimeError:
            pass
    self.region_editor.windows.clear()

    # 2) recria uma janela para cada componente
    red_pen = QPen(Qt.GlobalColor.red, 2)
    for nome, dados in componentes.items():
        pos_x, pos_y = dados.get('posicao', (0, 0))
        dim_w, dim_h = dados.get('dimensoes', (100, 100))
        self.region_editor.add_window(
            x=pos_x, y=pos_y,
            w=dim_w, h=dim_h,
            pen=red_pen,
            name=nome,
            deletable=False,    # protegidas de deleção acidental
            editable=False     # protegidas de deleção acidental
        )

```

```

def _qixmap_to_bgr(self, pixmap):
    """Converte QPixmap para numpy BGR"""
    try:
        if pixmap.isNull():
            return None
        image = pixmap.toImage().convertToFormat QImage.Format.Format_RGB888
        w, h = image.width(), image.height()
        ptr = image.bits().asstring(w * h * 3)
        import numpy as np
        arr = np.frombuffer(ptr, np.uint8).reshape((h, w, 3))
        return cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
    except Exception as e:
        print(f"[DEBUG] Erro na conversão QPixmap->BGR: {e}")
        return None

```

```

# -----
# Utilitário estático – converte numpy BGR → QPixmap sem downscale
# -----

```

```

@staticmethod
def _bgr_to_pixmap(img_bgr):
    if img_bgr is None:
        return QPixmap()
    h, w, _ = img_bgr.shape
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    from PyQt6.QtGui import QImage, QPixmap
    qimg = QImage(img_rgb.data, w, h, 3 * w, QImage.Format.Format_RGB888)
    return QPixmap.fromImage(qimg)

```

```

# exportação
def roi_size(self):
    return self.spin_w.value(), self.spin_h.value()

```

```

def resizeEvent(self, ev):
    super().resizeEvent(ev)
    # Visor ROI (pequeno) continua com ajuste já existente
    self._update_roi_aspect()

    # Novo: mantém visor 'Região' em 4:3 e ≤ 15 % da tela
    self._adjust_region_size()

    # Re-encaixa a imagem na área disponível
    if hasattr(self, "view_region"):
        self.view_region.fitInView(self._region_pix_item,
                                    Qt.AspectRatioMode.KeepAspectRatio)

```

```

# -----
# NOVO MÉTODO – mostra ROI da posição mecânica selecionada
# -----

```

```

def _update_roi_from_selection_main(self):
    """
    Exibe, no visor 'ROI' deste painel, o conteúdo da

```

Posição Mecânica (ResizableRectItem) atualmente selecionada no visor principal ('Região').

''''''

É necessário ter imagem de referência

if self._last_roi_bgr is None:

return

Há item selecionado?

sel_items = [it for it in self.view_region.scene().selectedItems()

if isinstance(it, ResizableRectItem)]

if not sel_items:

return

item = sel_items[0]

Converte retângulo do item para coordenadas de cena

r_scene = item.mapRectToScene(item.rect())

x, y, w, h = map(int, [r_scene.x(), r_scene.y(),
r_scene.width(), r_scene.height()])

h_img, w_img, _ = self._last_roi_bgr.shape

Garante que o recorte está dentro da imagem

x = max(0, min(x, w_img - 1))

y = max(0, min(y, h_img - 1))

w = max(1, min(w, w_img - x))

h = max(1, min(h, h_img - y))

roi_bgr = self._last_roi_bgr[y:y + h, x:x + w].copy()

if roi_bgr.size == 0:

return

desenha as "janelas azuis" (comparações de imagem)

pertencentes a esta posição mecânica, se existirem.

component_name = getattr(item, 'name', None)

if component_name:

roi_bgr = self._draw_comparison_windows(
roi_bgr,
component_name,
roi_origin=(x, y) # origem ABS da ROI
)

Mostra no visor 'ROI'

self._show_pixmap(self.lbl_roi, roi_bgr)

Desenha retângulos AZUIS das janelas de comparação

sobre a imagem do ROI recortado.

```

def _draw_comparison_windows(self,
                            img_bgr,
                            component_name,
                            roi_origin=(0, 0)):
    """
    Sobre põe à imagem 'img_bgr' (numpy BGR) os retângulos das
    janelas de comparação (cor azul) pertencentes ao componente
    'component_name'. As coordenadas já estão no mesmo espaço
    do ROI, portanto basta desenhar diretamente.
    """
    comp_dict = self._cached_auxiliary_data.get('componentes', {})
    comp_data = comp_dict.get(component_name)
    if not comp_data:
        return img_bgr

    inspecoes = comp_data.get('inspecoes', [])
    if not inspecoes:
        return img_bgr

    h_img, w_img = img_bgr.shape[:2]
    for insp in inspecoes:
        # Converte coordenadas ABS → relativas à ROI recortada
        (x_abs, y_abs) = insp.get('posicao', (0, 0))
        x = x_abs - roi_origin[0]
        y = y_abs - roi_origin[1]
        (w, h) = insp.get('tamanho', (0, 0))
        # garante que o retângulo fica dentro da imagem
        x = max(0, min(x, w_img - 1))
        y = max(0, min(y, h_img - 1))
        w = max(1, min(w, w_img - x))
        h = max(1, min(h, h_img - y))
        # desenha: cor BGR -> azul (255,0,0)
        import cv2
        cv2.rectangle(img_bgr,
                      (int(x), int(y)),
                      (int(x + w), int(y + h)),
                      (255, 0, 0), 2)
    return img_bgr

```

File: table_positions_backend.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

table_positions_backend.py

"""

Backend de persistência específico das mesas.

Grava apenas X, Y (físico da mesa) e Z em JSON.

Extensões Mesa 1 → *.m1

Mesa 2 → *.m2

"""

```

from pathlib import Path
from typing import List

from program_io_widget      import ProgramStorageBackend
from inspection_positions_widget import InspectionPositionsWidget, InspectionPosition
from adhesive_program_manager import AdhesiveProgramManager
import os, copy, json, cv2, numpy as np
from PIL import Image

```

```

class TablePositionsBackend(ProgramStorageBackend):
    def __init__(self,
                  widget: InspectionPositionsWidget,
                  y_axis: str):          # 'Y1' ou 'Y2'
        self._w = widget
        assert y_axis in ('Y1', 'Y2')
        self._y_axis = y_axis

# ----- salvar -----
def save_to_file(self, filename: str) -> bool:
    data: List[dict] = []
    for p in self._w.positions():
        y = p.y1 if self._y_axis == 'Y1' else p.y2
        data.append({"name": p.name,
                     "x":  p.x,
                     "y":  y,
                     "z":  p.z,
                     "meta": p.meta or {}})

# -----
# EXTRA 1 ■ gera estrutura de pastas + salva imagens (.png)
# -----
try:
    prog_name = Path(filename).stem
    base_dir  = Path(filename).parent
    mgr = AdhesiveProgramManager(base_dir)
    try:
        mgr.create_program(prog_name, overwrite=False)
    except FileExistsError:
        mgr.model_dir = Path(base_dir) / "modelos" / prog_name
    # garante pasta de arquivos auxiliares
    (mgr.model_dir / "arquivos_auxiliares").mkdir(parents=True, exist_ok=True)

    # percorre pontos para salvar imagens
    for idx, p in enumerate(self._w.positions(), 1):
        meta = p.meta or {}
        if meta.get("action") != "inspect":
            continue

# ----- imagem da REGIÃO -----
    regiao_name = f"regiao_{idx}"

```

```

region_bytes = meta.get("_region_png")
if not region_bytes:
    continue
np_img = cv2.imdecode(np.frombuffer(region_bytes, np.uint8),
                      cv2.IMREAD_COLOR)
if np_img is None:
    continue
region_pil = Image.fromarray(cv2.cvtColor(np_img,
                                           cv2.COLOR_BGR2RGB))
mgr.save_region(regiao_name, region_pil)
# ----- janelas w* -----
componentes = meta.get("componentes", {})
for posicao_nome, comp_data in componentes.items():
    for insp in comp_data.get("inspecoes", []):
        # As coordenadas já estão em pixels absolutos
        x, y = map(int, insp.get("posicao", (0, 0)))
        w, h = map(int, insp.get("tamanho", (0, 0)))

        # Ignora valores vazios ou negativos
        if w <= 0 or h <= 0:
            continue

        # Garante que o recorte não ultrapasse a borda da imagem
        img_h, img_w = np_img.shape[:2]
        x = max(0, min(x, img_w - 1))
        y = max(0, min(y, img_h - 1))
        w = min(w, img_w - x)
        h = min(h, img_h - y)

        roi = np_img[y:y + h, x:x + w]
        if roi.size == 0:
            continue

        roi_pil = Image.fromarray(
            cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
        w_nome = insp.get("nome", "w?")
        meta_js = {
            "janela_azul": (x, y, w, h),
            "similaridade": insp.get("similaridade", 0.9)
        }
        mgr.save_blue_reference(
            regiao_name,
            posicao_nome,
            w_nome,
            roi_pil,
            meta_js)

except Exception as exc:
    print("[WARN] Falha ao salvar imagens de inspeção:", exc)

# -----

```

```
# EXTRA 2 ■ remove chaves temporárias antes de serializar JSON
```

```
# -----
```

```
json_ready: List[dict] = []
for p in self._w.positions():
    y = p.y1 if self._y_axis == 'Y1' else p.y2
    m = copy.deepcopy(p.meta or {})
    if m.get("action") == "inspect":
        # descarta payload binário antes do JSON
        m.pop("_region_png", None)
    json_ready.append({
        "name": p.name,
        "x": p.x,
        "y": y,
        "z": p.z,
        "meta": m
    })
```

```
Path(filename).write_text(json.dumps(json_ready, indent=2), 'utf-8')
```

```
# -----
```

```
# EXTRA 3 ■ apenas garante árvore vazia quando não havia w*
```

```
# -----
```

```
try:
    # Nome do programa = nome do arquivo sem extensão
    prog_name = Path(filename).stem
    # Raiz = mesma pasta do *.m? (mantém simples)
    base_dir = Path(filename).parent
    mgr = AdhesiveProgramManager(base_dir)
    # Se já existir não apaga – somente garante estrutura
    try:
        mgr.create_program(prog_name, overwrite=False)
    except FileExistsError:
        mgr.model_dir = Path(base_dir) / "modelos" / prog_name
```

```
for idx, p in enumerate(self._w.positions(), 1):
    meta = p.meta or {}
    if meta.get("action") != "inspect":
        continue
    componentes : dict = meta.get("componentes", {})
    if not componentes:
        continue
```

```
# Cada posição de inspeção recebe uma pasta "regiao_<n>"
regiao = f"regiao_{idx}"
```

```
for comp_name, comp_data in componentes.items():
    inspecoes = comp_data.get("inspecoes", [])
    # Cria sub-pastas w1, w2...
    for i in range(max(1, len(inspecoes))):
        w_name = f"w{i+1}"
        mgr.ensure_position_dirs(regiao, comp_name, w_name)
```

```
except Exception as exc:
```

```

# não falha a gravação do programa se a estrutura de pastas
# não puder ser criada (por exemplo, permissão)
print("[WARN] Falha ao preparar pasta de inspeção:", exc)

return True

# ----- carregar -----
def load_from_file(self, filename: str) -> bool:
    raw = json.loads(Path(filename).read_text('utf-8'))
    if not isinstance(raw, list):
        raise ValueError("formato inválido – esperada lista JSON")

    self._w.clear()
    for d in raw:
        y_val = float(d.get("y", 0))
        pos = InspectionPosition(
            name=str(d.get("name", "")),
            x=float(d.get("x", 0)),
            y1=y_val if self._y_axis == 'Y1' else 0.0,
            y2=y_val if self._y_axis == 'Y2' else 0.0,
            z=float(d.get("z", 0)),
            meta=d.get("meta")
        )
        self._w.add_position(pos)
    return True

```

File: helpers.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

helpers.py – utilitários de alinhamento fiducial

.....

```
import base64
```

```
import time
```

```
from typing import Tuple, List
```

```
import cv2
```

```
import numpy as np
```

```
from PyQt6.QtWidgets import QApplication
```

```
# -----
```

```
# LEITURA DE FIDUCIAIS (edição off-line)
```

```
# -----
```

```
def read_fiducials(ctrl,
    positions,
    camera,
    current_offset: Tuple[int, int] = (0, 0)) -> Tuple[int, int]:

```

.....

Percorre a lista `positions` procurando aqueles cujo
meta['action'] == 'fiducial'.

Para cada fiducial:

1. Move X/Y SEM aplicar offset algum;
2. Captura frame da câmera;
3. Faz template-matching;
4. Converte $\Delta px \rightarrow \Delta \text{pulsos}$ via `ctrl.pulsos_from_pixels()`;

Retorna (dx_total, dy_total) em pulsos.

Caso não haja fiducial $\rightarrow$ devolve (0,0).

.....

```
fid_pos = [p for p in positions
            if (getattr(p, "meta", {}) or {}).get("action") == "fiducial"]
if not fid_pos:
    return 0, 0
```

```
dx_sum = dy_sum = 0
```

```
ox_prev, oy_prev = current_offset
```

```
matched_cnt = 0
```

```
for p in fid_pos:
```

```
    # -----
```

```
    # Determina qual eixo Y físico está ativo nesta execução.
```

```
    # PlateFlowManager define ctrl._active_plate_y antes de
```

```
    # disparar SequenceRunnerThread, evitando a dependência
```

```
    # da aba visível (que causava AttributeError quando
```

```
    # currentWidget() era AxesControlTab).
```

```
    # -----
```

```
    axis_y = getattr(ctrl, "_active_plate_y", "Y1")
```

```
    # ----- MOVE X,Y,Z (usa Z do próprio fiducial) -----
```

```
    ctrl.pulsos_spin_X.setValue(int(p.x + ox_prev))
```

```
    getattr(ctrl, f'pulsos_spin_{axis_y}').setValue(
```

```
        int((p.y1 if axis_y == 'Y1' else p.y2) + oy_prev))
```

```
    ctrl.pulsos_spin_Z.setValue(int(p.z))
```

```
    # Comando de movimento (X e Y) -----
```

```
    ctrl.move_axis_absolute("X")
```

```
    ctrl.move_axis_absolute(axis_y)
```

```
    ctrl.move_axis_absolute("Z")
```

```
    # -----
```

```
    # Aguarda CHEGAR ao alvo sem depender de wait_for_idle()
```

```
    # (que usa internamente _targets do backend e aqui estamos
```

```
    # chamando o controlador diretamente).
```

```
    # Critério simples:
```

```
    # • lê posição física a cada 100 ms;
```

```
    # • quando |pos-alvo|  $\leq 2$  pulsos para ambos os eixos
```

```
    #   durante 3 leituras consecutivas  $\rightarrow$  considera parado;
```

```
    # • abandona após 5 s para não travar a edição.
```

```
    # -----
```

```
    tgt_x = int(p.x + ox_prev)
```

```
    tgt_y = int((p.y1 if axis_y == 'Y1' else p.y2) + oy_prev)
```

```

tgt_z = int(p.z)
stable_cnt = 0
t0 = time.time()
while time.time() - t0 < 5.0:
    # atualiza dict current_positions
    if hasattr(ctrl, "read_current_positions"):
        ctrl.read_current_positions()
        cur_x = ctrl.current_positions.get('X', tgt_x+9999)
        cur_y = ctrl.current_positions.get(axis_y, tgt_y+9999)
        cur_z = ctrl.current_positions.get('Z', tgt_z+9999)
        if (abs(cur_x - tgt_x) <= 2 and
            abs(cur_y - tgt_y) <= 2 and
            abs(cur_z - tgt_z) <= 1):
            stable_cnt += 1
            if stable_cnt >= 3:
                break
    else:
        stable_cnt = 0
    time.sleep(0.1)
    QApplication.processEvents()
else:
    # timeout: apenas avisa no log e segue (evita travar GUI)
    ctrl.log("■■■ Timeout esperando parada durante leitura de fiducial")

```

```

time.sleep(0.1)          # estabiliza 100 ms
ok, frame = camera._cap.read()
if not ok:
    continue
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

meta = p.meta
tmpl_png = base64.b64decode(meta['template_png_b64'])
tmpl = cv2.imdecode(np.frombuffer(tmpl_png, np.uint8),
                    cv2.IMREAD_GRAYSCALE)
res = cv2.matchTemplate(gray, tmpl, cv2.TM_CCOEFF_NORMED)
_, sim, _, loc = cv2.minMaxLoc(res)
if sim*100 < meta.get('threshold', 70):
    continue
h, w = tmpl.shape
cx_t = loc[0] + w//2
cy_t = loc[1] + h//2
cx_i, cy_i = gray.shape[1]//2, gray.shape[0]//2
dx_pix = cx_t - cx_i
dy_pix = cy_t - cy_i
z_cur = ctrl.current_positions['Z']
dx_p, dy_p = ctrl.pulses_from_pixels(dx_pix, dy_pix, z_cur, axis_y)
dx_sum += dx_p
dy_sum += dy_p
matched_cnt += 1

```

```
# -----
# Offset = MÉDIA dos deslocamentos medidos nos n fiduciais
# (soma duplicaria a correção quando existem 2 ou mais pontos)
# -----
if matched_cnt == 0:
    raise ValueError("Nenhum fiducial pôde ser lido (similaridade abaixo do limiar).")

dx_avg = int(round(dx_sum / matched_cnt))
dy_avg = int(round(dy_sum / matched_cnt))
return dx_avg, dy_avg
```

File: dot_action_selector_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
"""
```

```
dot_action_selector_widget.py
```

```
-----
```

Widget compacto contendo quatro botões toggle que representam as ações disponíveis para cada ponto do programa:

- Fiducial • Aplicar Dot
- Leitura Barcode • Inspeção Visual

Somente um pode ficar selecionado de cada vez.

```
"""
```

```
from PyQt6.QtWidgets import (
    QGroupBox, QHBoxLayout, QPushButton, QButtonGroup
)
from PyQt6.QtCore import pyqtSignal
```

```
class DotActionSelectorWidget(QGroupBox):
```

```
    """
```

```
    Grupo exclusivo de seleção de ação.
```

```
    Sinal:
```

```
        actionChanged(str) – id interno: 'fiducial' | 'dot' | 'barcode' | 'inspect'
```

```
    """
```

```
    actionChanged = pyqtSignal(str)
```

```
    _ACTIONS = [
```

```
        ("fiducial", "Fiducial"),
```

```
        ("dot", "Dot"),
```

```
        ("barcode", "Barcode"),
```

```
        ("inspect", "Inspeção")
```

```
    ]
```

```
# =====
```

```
# NOVO – permite selecionar programaticamente a ação
```

```
# =====
```

```
def select_action(self, key: str) -> None:
```

```
    """
```

```
    Marca o botão correspondente a `key`
    e emite actionChanged(key).
```

```
    """
```

```

for btn in self._group.buttons():
    # propriedade foi gravada no __init__ como "actionKey"
    if btn.property("actionKey") == key:
        btn.setChecked(True)
        # garante que o sinal seja disparado mesmo
        # quando a seleção veio de fora (duplo-clique)
        self.actionChanged.emit(key)
        break

def __init__(self, parent=None):
    super().__init__("Ação Seleccionada", parent)
    h = QHBoxLayout(self)
    self._group = QButtonGroup(self)
    self._group.setExclusive(True)
    for idx,(key, label) in enumerate(self._ACTIONS):
        btn = QPushButton(label)
        btn.setCheckable(True)
        self._group.addButton(btn, idx)
        btn.setProperty("actionKey", key)
        h.addWidget(btn)
    # conecta
    self._group.idToggled.connect(self._on_toggled)
    # seleção inicial – nenhum
    # -----
    def _on_toggled(self, _id:int, checked:bool):
        if not checked:
            return
        btn = self._group.button(_id)
        key = btn.property("actionKey")
        self.actionChanged.emit(key)
    # -----
    def current_action(self) -> str | None:
        """Devolve a string da ação seleccionada ou None."""
        btn = self._group.checkedButton()
        return btn.property("actionKey") if btn else None

```

File: fiducial_config_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

"""

fiducial_config_widget.py

Configuração de captura / teste de fiducial.

Depende do OpenCV já usado pelo CameraManager.

"""

import cv2, numpy as np

from PyQt6.QtWidgets import (

QGroupBox, QVBoxLayout, QFormLayout, QSpinBox, QPushButton, QLabel,
 QHBoxLayout, QMessageBox, QSizePolicy

)

```
from PyQt6.QtCore import Qt, pyqtSignal
```

```
class FiducialConfigWidget(QGroupBox):
```

```
    """
```

Widget contendo:

- Janela (px) – lado do quadrado central
- Raio de busca (px) – $\pm$ pixels ao redor
- Similaridade mínima (%) – 70...100
- Botões Capturar / Testar
- Label resultado

Guarda internamente o template (numpy array BGR) no atributo `self._template`.

```
    """
```

```
parametersChanged = pyqtSignal(int, int)      # window, radius
```

```
matchTested       = pyqtSignal(float, bool, int, int, int, int)
```

```
def __init__(self, controller, parent=None):
```

```
    super().__init__("Config. Fiducial", parent)
```

```
    self._c = controller
```

```
    self._template = None      # imagem capturada
```

```
    self._build_ui()
```

```
# -----
```

```
def _build_ui(self):
```

```
    v = QVBoxLayout(self)
```

```
    form = QFormLayout()
```

```
    self.spin_window = QSpinBox(); self.spin_window.setRange(5, 500); self.spin_window.setValue(50)
```

```
    self.spin_radius = QSpinBox(); self.spin_radius.setRange(0, 1000); self.spin_radius.setValue(80)
```

```
    self.spin_thresh = QSpinBox(); self.spin_thresh.setRange(50, 100); self.spin_thresh.setValue(80)
```

```
    form.addRow("Janela (px):", self.spin_window)
```

```
    form.addRow("Raio busca (px):", self.spin_radius)
```

```
    form.addRow("Similaridade mínima (%):", self.spin_thresh)
```

```
    v.addLayout(form)
```

```
    h = QHBoxLayout()
```

```
    btn_cap = QPushButton("Capturar Fiducial")
```

```
    btn_test = QPushButton("Testar")
```

```
    h.addWidget(btn_cap); h.addWidget(btn_test)
```

```
    v.addLayout(h)
```

```
# ----- visores de imagens (template | match) -----
```

```
img_row = QHBoxLayout()
```

```
self.lbl_tmpl = QLabel("Template")
```

```
self.lbl_match = QLabel("Match")
```

```
for lab in (self.lbl_tmpl, self.lbl_match):
```

```
    lab.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
    lab.setSizePolicy(QSizePolicy.Policy.Preferred,  
                     QSizePolicy.Policy.Preferred)
```

```
    lab.setStyleSheet("QLabel { background:#263238; color:#CFD8DC; }")
```

```
img_row.addWidget(self.lbl_tmpl); img_row.addWidget(self.lbl_match)
```

```

v.addLayout(img_row)

self.lbl_result = QLabel("")
self.lbl_result.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.lbl_result.setStyleSheet("QLabel { background:#ECEFF1; padding:4px; }")
v.addWidget(self.lbl_result)
v.addStretch()

# sinais
btn_cap.clicked.connect(self._on_capture)
btn_test.clicked.connect(self._on_test)

# emite alteração de parâmetros
self.spin_window.valueChanged.connect(
    lambda _: self.parametersChanged.emit(self.spin_window.value(),
                                           self.spin_radius.value()))
self.spin_radius.valueChanged.connect(
    lambda _: self.parametersChanged.emit(self.spin_window.value(),
                                           self.spin_radius.value()))

# ----- NOVO -----
def load_from_meta(self, meta: dict):
    self.spin_window.setValue(int(meta.get("window", 50)))
    self.spin_radius.setValue(int(meta.get("radius", 80)))
    self.spin_thresh.setValue(int(meta.get("threshold", 70)))
    # template já está contido em meta; não há necessidade de
    # repor a imagem aqui.

# -----
def _grab_frame(self):
    """Obtém o último frame RGB da CameraManager."""
    if not hasattr(self._c, "camera_manager"): return None
    cm = self._c.camera_manager
    # usa método interno _cap para pegar diretamente
    cap = getattr(cm, "_cap", None)
    if cap is None or not cap.isOpened():
        return None
    ok, frame = cap.read()
    return frame if ok else None

def _on_capture(self):
    frame = self._grab_frame()
    if frame is None:
        QMessageBox.warning(self, "Câmera", "Frame indisponível.")
        return
    h, w, _ = frame.shape
    sz = self.spin_window.value()
    cx, cy = w // 2, h // 2
    x0 = max(0, cx - sz // 2)
    y0 = max(0, cy - sz // 2)
    roi = frame[y0:y0 + sz, x0:x0 + sz].copy()

```

```

if roi.size == 0:
    QMessageBox.warning(self, "Erro", "ROI vazia.")
    return
self._template_bgr = roi      # salvo p/ persistir depois
self._template = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
self.lbl_result.setText(f"Template capturado ({sz}x{sz}px)")
self._show_pixmap(self.lbl_tmpl, roi)

def _on_test(self):
    if self._template is None:
        QMessageBox.information(self, "Template", "Capture o fiducial primeiro.")
        return
    frame = self._grab_frame()
    if frame is None:
        QMessageBox.warning(self, "Câmera", "Frame indisponível.")
        return
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    res = cv2.matchTemplate(gray, self._template, cv2.TM_CCOEFF_NORMED)
    min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)
    similarity = max_val * 100
    th = self.spin_thresh.value()

    # ----- verifica raio de busca -----
    win = self.spin_window.value()
    rad = self.spin_radius.value()
    cx, cy = gray.shape[1] // 2, gray.shape[0] // 2
    tx, ty = max_loc          # canto superior-esquerdo
    mx = tx + self._template.shape[1] // 2
    my = ty + self._template.shape[0] // 2
    in_radius = (abs(mx - cx) <= rad) and (abs(my - cy) <= rad)
    # prepara sub-imagem do local encontrado para mostrar
    tx, ty = max_loc
    w0, h0 = self._template.shape[1], self._template.shape[0]
    sub = frame[ty:ty+h0, tx:tx+w0].copy()
    self._last_match_bgr = sub      # salvo p/ persistir depois
    ok_match = (similarity >= th) and in_radius

    if not ok_match:
        self.lbl_result.setStyleSheet("QLabel { background:#FFCDD2; }")
        # mensagem: mostra se falhou por limite ou por similaridade
        msg = (f"Fora do raio ( $\Delta x = \{mx - cx\}$   $\Delta y = \{my - cy\}$ )"
              if similarity >= th else
              f"{similarity:.1f}% (<{th}%)")
        self.lbl_result.setText(f"Falhou: {msg}")
        self.matchTested.emit(similarity, False,
                               tx, ty,
                               self._template.shape[1],
                               self._template.shape[0])
    else:
        cx, cy = gray.shape[1]//2, gray.shape[0]//2

```

```

dx = tx + self._template.shape[1]//2 - cx
dy = ty + self._template.shape[0]//2 - cy
self.lbl_result.setStyleSheet("QLabel { background:#C8E6C9; }")
self.lbl_result.setText(f"OK {similarity:.1f}% Δx={dx} Δy={dy}")
self.matchTested.emit(similarity, True,
                        tx, ty,
                        self._template.shape[1],
                        self._template.shape[0])

# mostra sub-imagem com borda colorida
brd_color = (0,255,0) if ok_match else (0,0,255)
sub_brd = cv2.copyMakeBorder(sub, 2,2,2,2, cv2.BORDER_CONSTANT, value=brd_color)
self._show_pixmap(self.lbl_match, sub_brd)

# -----
# utilitário para mostrar numpy BGR em QLabel mantendo largura
# -----
def _show_pixmap(self, label: QLabel, img_bgr):
    h, w, _ = img_bgr.shape
    # converte para QPixmap
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    from PyQt6.QtGui import QImage, QPixmap
    qimg = QImage(
        img_rgb.data, w, h, 3 * w,          # bytesPerLine = 3*w
        QImage.Format.Format_RGB888
    )
    px = QPixmap.fromImage(qimg)
    # largura = largura do botão Capturar (aprox.)
    btn_width = label.parent().findChild(QPushButton, "Capturar Fiducial")
    max_w = btn_width.width() if btn_width else 120
    px = px.scaledToWidth(max_w, Qt.TransformationMode.SmoothTransformation)
    label.setPixmap(px)

# -----
# Métodos auxiliares para exportar imagens (futuro save)
# -----
def template_image(self):
    """Devolve numpy array BGR do template ou None."""
    return getattr(self, "_template_bgr", None)

def last_match_image(self):
    """Devolve numpy array BGR do último match ou None."""
    return getattr(self, "_last_match_bgr", None)

```

File: barcode_config_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

barcode_config_widget.py

Configuração de leitura de código de barras / QR.

.....

```
from PyQt6.QtWidgets import (
    QGroupBox, QVBoxLayout, QFormLayout, QSpinBox, QPushButton,
    QLabel, QMessageBox
)
from PyQt6.QtCore import Qt, pyqtSignal
import cv2
from barcode_scanner import BarcodeScanner

class BarcodeConfigWidget(QGroupBox):
    parametersChanged = pyqtSignal(int, int) # width, height
    testFinished = pyqtSignal(bool, int, int, int, int) # ok,x,y,w,h
    def __init__(self, controller, parent=None):
        super().__init__("Config. Barcode", parent)
        self._c = controller
        self._scanner = BarcodeScanner()
        self._build_ui()

    def _build_ui(self):
        v = QVBoxLayout(self)
        form = QFormLayout()
        self.spin_w = QSpinBox(); self.spin_w.setRange(20, 2000); self.spin_w.setValue(400)
        self.spin_h = QSpinBox(); self.spin_h.setRange(20, 2000); self.spin_h.setValue(150)
        form.addRow("Largura (px):", self.spin_w)
        form.addRow("Altura (px):", self.spin_h)
        v.addLayout(form)
        btn_test = QPushButton("Testar Leitura")
        v.addWidget(btn_test)
        self.lbl_result = QLabel("—")
        self.lbl_result.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.lbl_result.setStyleSheet("QLabel { background:gray; padding:4px; }")
        v.addWidget(self.lbl_result)
        v.addStretch()
        # sinais
        self.spin_w.valueChanged.connect(
            lambda _: self.parametersChanged.emit(self.spin_w.value(), self.spin_h.value()))
        self.spin_h.valueChanged.connect(
            lambda _: self.parametersChanged.emit(self.spin_w.value(), self.spin_h.value()))
        btn_test.clicked.connect(self._on_test)

    # ----- helpers -----
    def _grab_frame(self):
        cm = getattr(self._c, "camera_manager", None)
        if not cm or not getattr(cm, "_cap", None): return None
        ok, frame = cm._cap.read()
        return frame if ok else None

    def _on_test(self):
        frame = self._grab_frame()
```

```

if frame is None:
    QMessageBox.warning(self, "Câmara", "Frame indisponível.")
    return
h_img, w_img, _ = frame.shape
ww = self.spin_w.value(); hh = self.spin_h.value()
cx, cy = w_img // 2, h_img // 2
x0 = max(0, cx - ww // 2); y0 = max(0, cy - hh // 2)
roi = (x0, y0, ww, hh)
results = self._scanner.scan(frame, roi=roi)
if results:
    r = results[0]
    # cor da letra preta
    self.lbl_result.setStyleSheet("QLabel { background:#C8E6C9; padding:4px; color: black; }")
    self.lbl_result.setText(f"{r.type}: {r.data}")
    self.testFinished.emit(True, x0, y0, ww, hh)
else:
    self.lbl_result.setStyleSheet("QLabel { background:#FFCDD2; padding:4px; color: black; }")
    self.lbl_result.setText("Nenhum código encontrado")
    self.testFinished.emit(False, x0, y0, ww, hh)

# exportação p/ meta-dados
def roi_size(self):
    return self.spin_w.value(), self.spin_h.value()

# ----- NOVO: carregar valores salvos -----
def load_from_meta(self, meta: dict):
    w = int(meta.get("width", 400))
    h = int(meta.get("height", 150))
    self.spin_w.setValue(w)
    self.spin_h.setValue(h)

```

File: program_converter_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

.....

Janela não-modal que converte um arquivo *.m1 ■ *.m2 para a mesa atualmente selecionada.

Fluxo para o operador

1. Estar na aba Mesa 1 ou Mesa 2.
2. Menu Programa ■ Converter programa da outra mesa...
3. Clicar "Selecionar programa...".
 - se estiver na Mesa 1 só aparecem *.m2;
 - se Mesa 2, apenas *.m1.
4. Posicionar a câmara sobre o Fiducial 1 e clicar "Definir Fiducial 1"; fazer o mesmo para Fiducial 2.
5. "Converter" aplica translação + pequena correção de escala (proporcional entre os 2 fiduciais).
6. "Salvar" grava o novo arquivo com sufixo correto na pasta do projecto actual ou onde o utilizador escolher.

```

"""
from __future__ import annotations
from pathlib import Path
import json, math, typing as _t

from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QLabel, QFileDialog, QMessageBox
)
from PyQt6.QtCore import Qt, pyqtSignal

class ProgramConverterDialog(QWidget):
    """
    Dialogo perene (não modal, sempre-on-top) que converte
    programas entre as mesas.
    """

    # sinal opcional: novo programa carregado na aba
    programConverted = pyqtSignal(list)    # lista[dict] já convertida

    def __init__(self, ctrl, mesa_tab):
        """
        ctrl    = MultiAxisMotorController (janela principal)
        mesa_tab = TableProgramTab actualmente visível
        """
        super().__init__(parent=None) # ← sem parent → janela top-level

        # Torna-a uma janela independente sempre no topo
        flags = (Qt.WindowType.Window |
                 Qt.WindowType.WindowStaysOnTopHint |
                 Qt.WindowType.CustomizeWindowHint |
                 Qt.WindowType.WindowCloseButtonHint)
        self.setWindowFlags(flags)

        self.setWindowTitle(f'Conversor de Programa – Mesa {mesa_tab.mesa}')
        self.ctrl    = ctrl
        self.mesa_tab = mesa_tab
        self._file_in : Path | None = None
        self._prog_raw : list[dict] | None = None
        self._fid_old : list[tuple[float,float]] = [] # coordenadas no arquivo
        self._fid_new : list[tuple[int,int]] = [] # pulsos medidos
        self._build_ui()

    # -----
    def _build_ui(self):
        v = QVBoxLayout(self)

        self.lbl_file = QLabel("Nenhum programa seleccionado")
        self.lbl_f1 = QLabel("Fiducial 1: —")
        self.lbl_f2 = QLabel("Fiducial 2: —")

```

```

btn_sel = QPushButton("Selecionar programa...")
btn_f1 = QPushButton("Definir Fiducial 1")
btn_f2 = QPushButton("Definir Fiducial 2")
btn_conv = QPushButton("Converter")
btn_save = QPushButton("Salvar")

for b in (btn_conv, btn_save):
    b.setEnabled(False)

self._btn_conv = btn_conv
self._btn_save = btn_save

v.addWidget(self.lbl_file)
v.addSpacing(10)

h1 = QHBoxLayout(); h1.addWidget(btn_sel); h1.addStretch()
v.addLayout(h1)

v.addSpacing(15)
v.addWidget(self.lbl_f1)
v.addWidget(self.lbl_f2)
h2 = QHBoxLayout(); h2.addWidget(btn_f1); h2.addWidget(btn_f2)
v.addLayout(h2)

v.addSpacing(15)
h3 = QHBoxLayout(); h3.addWidget(btn_conv); h3.addWidget(btn_save)
v.addLayout(h3)
v.addStretch()

# conexões -----
btn_sel.clicked.connect(self._select_program)
btn_f1.clicked.connect(lambda: self._capture_fid(1))
btn_f2.clicked.connect(lambda: self._capture_fid(2))
btn_conv.clicked.connect(self._convert)
btn_save.clicked.connect(self._save)

# -----
# Passo 1 – escolher o arquivo da outra mesa
# -----
def _select_program(self):
    # mesa actual
    mesa_atual = self.mesa_tab.mesa      # 1 ou 2
    other_mesa = 2 if mesa_atual == 1 else 1
    filter_str = f"Programa Mesa {other_mesa} (*.m{other_mesa})"
    fname, _ = QFileDialog.getOpenFileName(self,
                                           "Escolher programa da outra mesa",
                                           str(self.ctrl.projects_dir),
                                           filter_str)

    if not fname:
        return

```

```

self._file_in = Path(fname)
try:
    self._prog_raw = json.loads(Path(fname).read_text(encoding="utf-8"))
except Exception as exc:
    QMessageBox.critical(self, "Erro",
        f"Falha ao abrir:\n{exc}")

    return

```

```

# procura dois fiduciais no programa
fids = [(p["x"], p["y"]) for p in self._prog_raw
        if (p.get("meta") or {}).get("action") == "fiducial"]
if len(fids) < 2:
    QMessageBox.warning(self, "Fiduciais insuficientes",
        "O programa da outra mesa precisa ter "
        "pelo menos 2 pontos de fiducial.")
    self._prog_raw = None
    return
self._fid_old = fids[:2]    # usa os dois primeiros
self.lbl_file.setText(f"Programa: {self._file_in.name} "
        f"(fid1={self._fid_old[0]}, "
        f"fid2={self._fid_old[1]})")
self._check_ready()

```

```

# -----
# Passo 2 – capturar coords em pulsos do ponto sob a câmera
# -----

```

```

def _capture_fid(self, idx: int):
    pos = self.ctrl._get_current_position_dict()
    x_p = pos["x"]
    y_p = pos["y1"] if self.mesa_tab.y_axis == "Y1" else pos["y2"]
    if idx == 1:
        if len(self._fid_new) >= 1:
            self._fid_new[0] = (x_p, y_p)
        else:
            self._fid_new.insert(0, (x_p, y_p))
        self.lbl_f1.setText(f"Fiducial 1: X={x_p} Y={y_p}")
    else:
        if len(self._fid_new) == 0:
            self._fid_new.append((0, 0))    # placeholder
        if len(self._fid_new) >= 2:
            self._fid_new[1] = (x_p, y_p)
        else:
            self._fid_new.append((x_p, y_p))
        self.lbl_f2.setText(f"Fiducial 2: X={x_p} Y={y_p}")
    self._check_ready()

```

```

# -----
def _check_ready(self):
    ready = self._prog_raw is not None and len(self._fid_new) >= 2
    self._btn_conv.setEnabled(ready)

```

```

# -----
# Passo 3 – converter
# -----
def _convert(self):
    if not self._btn_conv.isEnabled():
        return
    f1_old, f2_old = self._fid_old
    f1_new, f2_new = self._fid_new

    # escala (pequenas variações de distância)
    dist_old = math.hypot(f2_old[0]-f1_old[0], f2_old[1]-f1_old[1])
    dist_new = math.hypot(f2_new[0]-f1_new[0], f2_new[1]-f1_new[1])
    scale = dist_new/dist_old if dist_old else 1.0

    self._prog_conv : list[dict] = []
    for p in self._prog_raw:
        dx = p["x"] - f1_old[0]
        dy = p["y"] - f1_old[1]
        new_x = f1_new[0] + dx*scale
        new_y = f1_new[1] + dy*scale
        # troca campo y1 / y2 conforme mesa destino
        if self.mesa_tab.y_axis == "Y1":
            y1, y2 = new_y, 0.0
        else:
            y1, y2 = 0.0, new_y
        self._prog_conv.append({
            "name": p["name"],
            "x": new_x,
            "y": new_y,
            "z": p["z"],
            "meta": p.get("meta", {})
        })
    QMessageBox.information(self, "Conversão",
        "Programa convertido.\n"
        "Clique “Salvar” para guardar.")
    self._btn_save.setEnabled(True)

    # carrega na aba actual
    self._load_into_current_tab()

def _load_into_current_tab(self):
    """Preenche a lista de posições da mesa destino."""
    if not self._prog_conv:
        return
    w = self.mesa_tab.inspect_widget
    w.blockSignals(True)
    try:
        w.clear()
        for d in self._prog_conv:
            y1 = d["y"] if self.mesa_tab.y_axis == "Y1" else 0.0
            y2 = d["y"] if self.mesa_tab.y_axis == "Y2" else 0.0

```

```

from sequence_control import InspectionPosition
cam_meta = d.get("meta", {}) # mantém meta original
pos = InspectionPosition(
    name=d["name"],
    x=d["x"],
    y1=y1, y2=y2,
    z=d["z"],
    camera_params=cam_meta) # <-- parâmetro correto

# ---- compatibilidade: vários widgets ainda usam .meta
setattr(pos, "meta", cam_meta)
w.add_position(pos)
finally:
    w.blockSignals(False)
# atualiza sequence widget
self.mesa_tab.seq_widget.set_positions(self.mesa_tab._to_model())

# -----
# Passo 4 – salvar
# -----
def _default_save_path(self) -> Path:
    base = self.ctrl.projects_dir
    orig = self._file_in.stem if self._file_in else "programa"
    new_suffix = ".m1" if self.mesa_tab.mesa == 1 else ".m2"
    return base / f"{orig}{new_suffix}"

def _save(self):
    if not self._prog_conv:
        return
    suggested = str(self._default_save_path())
    fname, _ = QFileDialog.getSaveFileName(self, "Salvar programa convertido",
                                           suggested,
                                           "Programas (*.m1 *.m2)")

    if not fname:
        return
    # força extensão certa
    if self.mesa_tab.mesa == 1 and not fname.endswith(".m1"):
        fname += ".m1"
    if self.mesa_tab.mesa == 2 and not fname.endswith(".m2"):
        fname += ".m2"
    try:
        Path(fname).write_text(json.dumps(self._prog_conv, indent=2),
                               encoding="utf-8")
    except Exception as exc:
        QMessageBox.critical(self, "Erro", str(exc))
        return
    QMessageBox.information(self, "Salvar",
                             f"Programa gravado em:\n{fname}")
# atualiza ProgramIOWidget da mesa (para permitir salvar de novo)
self.mesa_tab.prog_widget._last_file = fname

```

```
# -----
def closeEvent(self, ev):
    """Mantém a instância viva (não destrói)"""
    ev.ignore()
    self.hide()
```

File: alignment_dialog.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```
from PyQt6.QtWidgets import (
    QDialog, QVBoxLayout, QLabel, QPushButton, QHBoxLayout, QMessageBox
)
from PyQt6.QtCore import Qt, pyqtSignal
```

```
class AlignmentDialog(QDialog):
```

```
    """
```

Passo-a-passo para medir o deslocamento entre
o centro da CÂMERA e o centro do NOZZLE.

1. Leve o NOZZLE manualmente até um ponto de referência e clique
"Capturar bico".
2. Leve a CÂMERA (centro da imagem) ao MESMO ponto e clique
"Capturar câmera".

O diálogo calcula ΔX , ΔY e salva.

```
    """
```

```
offsetSaved = pyqtSignal(dict)    # {'x':dx, 'y':dy}
```

```
def __init__(self, ctrl, parent=None):
```

```
    super().__init__(parent, Qt.WindowType.Window)
    self.setWindowTitle("Alinhamento Nozzle ↔ Câmera")
    self.setWindowModality(Qt.WindowModality.NonModal)
    self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, True)
```

```
    self._c = ctrl
    self._noz_pos = None    # (x,y)
    self._cam_pos = None
```

```
    self._build_ui()
```

```
# ----- UI -----
```

```
def _build_ui(self):
```

```
    v = QVBoxLayout(self)
    v.addWidget(QLabel(
        "1. Posicione o BICO sobre um ponto fixo e clique em "
        "<b>Capturar bico</b>.\n"
        "2. Mova a CÂMERA até o MESMO ponto (use o clique na imagem) "
        "e clique em <b>Capturar câmera</b>.\n"
        "3. Clique <b>Salvar</b> para gravar o offset."))
```

```

# linha status
self.lbl_status = QLabel("ΔX = 0  ΔY = 0")
self.lbl_status.setAlignment(Qt.AlignmentFlag.AlignCenter)
v.addWidget(self.lbl_status)

h1 = QHBoxLayout()
self.btn_noz = QPushButton("Capturar bico")
self.btn_cam = QPushButton("Capturar câmera")
h1.addWidget(self.btn_noz)
h1.addWidget(self.btn_cam)
v.addLayout(h1)

h2 = QHBoxLayout()
btn_save = QPushButton("Salvar")
btn_cancel = QPushButton("Fechar")
h2.addStretch()
h2.addWidget(btn_save)
h2.addWidget(btn_cancel)
v.addLayout(h2)

# conexões
self.btn_noz.clicked.connect(self._capture_nozzle)
self.btn_cam.clicked.connect(self._capture_camera)
btn_save.clicked.connect(self._on_save)
btn_cancel.clicked.connect(self.close)

# ----- capturas -----
def _capture_nozzle(self):
    self._noz_pos = (self._c.current_positions['X'],
                    self._c.current_positions['Y1' # Y física da Mesa 1
                    if self._c.tab_widget.currentWidget() is self._c.mesa_tabs[1]
                    else 'Y2'])
    self._update_status()

def _capture_camera(self):
    """
    Posição da CÂMERA é simplesmente a posição atual dos eixos,
    pois o usuário deve mover o cabeçote de forma que o ponto de
    interesse fique no centro da imagem.
    """
    self._cam_pos = (self._c.current_positions['X'],
                    self._c.current_positions['Y1'
                    if self._c.tab_widget.currentWidget() is self._c.mesa_tabs[1]
                    else 'Y2'])
    self._update_status()

def _update_status(self):
    if self._noz_pos and self._cam_pos:
        dx = self._cam_pos[0] - self._noz_pos[0]
        dy = self._cam_pos[1] - self._noz_pos[1]

```

```

        self.lbl_status.setText(f"ΔX = {dx}  ΔY = {dy}")
    else:
        self.lbl_status.setText("ΔX = ?  ΔY = ?")

# ----- salvar -----
def _on_save(self):
    if not (self._noz_pos and self._cam_pos):
        QMessageBox.warning(self, "Incompleto",
                            "Capture as duas posições antes de salvar.")
        return
    dx = self._cam_pos[0] - self._noz_pos[0]
    dy = self._cam_pos[1] - self._noz_pos[1]
    self.offsetSaved.emit({"x": dx, "y": dy})
    self.close()

```

File: program_io_widget.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

from __future__ import annotations
from pathlib import Path
from typing import Protocol, runtime_checkable

from PyQt6.QtCore import pyqtSignal
from PyQt6.QtWidgets import (
    QWidget, QGroupBox, QVBoxLayout, QPushButton,
    QFileDialog, QMessageBox
)

# -----
# 1) Protocolo (interface) para o backend que realmente salva/carrega
# -----
@runtime_checkable
class ProgramStorageBackend(Protocol):
    """
    Interface mínima que o backend de persistência deve expor.
    – InspectionPositionManager do projeto antigo já satisfaz.
    """
    def save_to_file(self, filename: str) -> bool: ...
    def load_from_file(self, filename: str) -> bool: ...

# -----
# 2) Widget de UI
# -----
class ProgramIOWidget(QWidget):
    """
    Painel "Salvar / Carregar" pronto para ser acoplado a qualquer
    aplicação. Usa um backend que implemente ProgramStorageBackend.
    """

```

```

fileSaved = pyqtSignal(str) # Caminho salvo
fileLoaded = pyqtSignal(str) # Caminho carregado

def __init__(self,
              backend: ProgramStorageBackend,
              *,
              file_filter: str = "Arquivos JSON (*.json)",
              default_suffix: str = ".json",
              default_dir: str | Path = "",
              parent=None) -> None:
    super().__init__(parent)
    if not isinstance(backend, ProgramStorageBackend):
        raise TypeError(
            "backend não implementa ProgramStorageBackend (save_to_file / load_from_file)"
        )
    self._backend = backend
    self._file_filter = file_filter
    self._default_suf = default_suffix
    self._default_dir = str(default_dir) if default_dir else ""
    self._build_ui()

# -----
# construção da interface
# -----
def _build_ui(self):
    grp = QGroupBox("Salvar / Carregar")
    v = QVBoxLayout(grp)

    self.btn_save = QPushButton("Salvar Programa")
    self.btn_load = QPushButton("Carregar Programa")

    v.addWidget(self.btn_save)
    v.addWidget(self.btn_load)

    main = QVBoxLayout(self)
    main.addWidget(grp)
    main.addStretch()

# conexões
self.btn_save.clicked.connect(self._on_save_clicked)
self.btn_load.clicked.connect(self._on_load_clicked)

# -----
# slots privados
# -----
def _on_save_clicked(self):
    fname, _ = QFileDialog.getSaveFileName(
        self, "Salvar Programa", "", self._file_filter
    )
    if not fname:

```

```

        return
# garante extensão correta
if self._default_suf and not fname.lower().endswith(self._default_suf):
    fname += self._default_suf

```

```

ok = False
try:
    ok = self._backend.save_to_file(fname)
except Exception as exc:
    QMessageBox.critical(self, "Erro", f"Falha ao salvar:\n{exc}")
    return

```

```

if ok:
    self.fileSaved.emit(fname)
else:
    QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")

```

```

def _on_load_clicked(self):
    fname, _ = QFileDialog.getOpenFileName(
        self, "Carregar Programa",
        self._default_dir,
        self._file_filter
    )
    if not fname:
        return

```

```

ok = False
try:
    ok = self._backend.load_from_file(fname)
except Exception as exc:
    QMessageBox.critical(self, "Erro", f"Falha ao carregar:\n{exc}")
    return

```

```

if ok:
    self.fileLoaded.emit(fname)
else:
    QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")

```

```

# -----

```

```

# 3) Demonstração rápida

```

```

# -----

```

```

if __name__ == "__main__":
    import sys, json, tempfile
    from dataclasses import dataclass, asdict
    from PyQt6.QtWidgets import QApplication, QLabel

```

```

# Backend de exemplo -----

```

```

class _DummyBackend:
    """Exemplo simples que salva/carrega um dict em JSON."""
    def __init__(self):

```

```

self.data = {"positions": [], "sequences": []}

def save_to_file(self, filename: str) -> bool:
    Path(filename).write_text(json.dumps(self.data, indent=2),
                               encoding="utf-8")
    return True

def load_from_file(self, filename: str) -> bool:
    self.data = json.loads(Path(filename).read_text(encoding="utf-8"))
    return True

# Qt demo -----
app = QApplication(sys.argv)
backend = _DummyBackend()
widget = ProgramIOWidget(backend)
widget.resize(220, 120)

# Mostra em label onde salvou/carregou
info = QLabel()
def _saved(f): info.setText(f"Salvo: {Path(f).name}")
def _loaded(f): info.setText(f"Carregado: {Path(f).name}")
widget.fileSaved.connect(_saved)
widget.fileLoaded.connect(_loaded)

wrapper = QWidget()
lay = QVBoxLayout(wrapper); lay.addWidget(widget); lay.addWidget(info)
wrapper.show()
sys.exit(app.exec())

```

File: adhesive_program_manager.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

adhesive_program_manager.py

"""

AdhesiveProgramManager

Cria e gerencia a estrutura de diretórios/arquivos de um programa de inspeção da adesivadora.

Estrutura-alvo (ver txt original):

modelos/<nome_prog>/

■ logs/<datahora>_<codigo>/ (# sessões de produção)

■ ■ <datahora>_<codigo>.json (log consolidado)

■ ■ <datahora>_SFCS.json (opcional)

■ ■ regiao_<n>.png (snapshots)

■ regioes/

■ <regiao_X>/posicoes_mecanicas/

■ <posicao_mec_Y>/inspecoes/

■ w1/ ... w4/

```

■■■ correct/      (imagens OK)
■■■ incorrect/    (imagens NOK)
■■■ <posicao_wi>.json (meta)
■■■ <posicao_wi>.png (ROI azul)
■■■ referencia/referencia.png

```

```

"""

```

```

from __future__ import annotations
from dataclasses import dataclass, asdict
from pathlib import Path
from datetime import datetime
import json, shutil
from typing import List, Tuple, Dict, Any, Optional
from PIL import Image

# -----
# 1) Tipos de dados — opcionais, mas ajudam a manter contrato
# -----
@dataclass
class InspectionWindowInfo:
    posicao: Tuple[int, int] # x,y absolutos na ROI mecânica
    tamanho: Tuple[int, int] # w,h absolutos
    threshold: int = 128
    cor_pixel: str = 'branco' # 'branco'|'preto'
    percentual_minimo: float = .80 # 0-1
    # caminho para imagens corret/incorrect ficarão no disco

@dataclass
class MechanicalPositionInfo:
    """Informações persistidas no JSON '<posicao>_w?.json'."""
    janela_azul: Tuple[int,int,int,int] # x,y,w,h
    similaridade: float # 0-1
    inspecoes: List[InspectionWindowInfo] # w1...w4

# -----
# 2) Classe principal
# -----
class AdhesiveProgramManager:
    def __init__(self, base_dir: str | Path = ".") -> None:
        self.base_dir = Path(base_dir).resolve()
        self.model_dir: Path | None = None

# -----
# Propriedade de compatibilidade
# Várias partes do código legado ainda acessam `prog_mgr.path`.
# Para não precisar alterar todos os módulos, expomos `.path`
# como alias somente-leitura para `model_dir`.
# -----
@property
def path(self) -> Path | None:
    """

```

Alias para `model\_dir`.

Retorna None até `create\_program()` ser chamado.

"""

return self.model_dir

----- criação do PROGRAMA -----

def create_program(self, name: str, overwrite: bool = True) -> Path:

"""

Cria ou abre um programa:

modelos/<name> + sub-pastas vazias.

"""

self.model_dir = self.base_dir / "modelos" / name

if self.model_dir.exists() and not overwrite:

raise FileExistsError(f"Programa \"{name}\" já existe.")

limpa só se overwrite=True

if self.model_dir.exists() and overwrite:

shutil.rmtree(self.model_dir)

cria tronco principal

sub-pastas fixas

for sub in ("logs", "regioes", "arquivos_auxiliares"):

(self.model_dir / sub).mkdir(parents=True, exist_ok=True)

grava config.txt minimal (mesma linha de raciocínio do PM original)

cfg_txt = self.model_dir / "config.txt"

cfg_txt.write_text(f"model_name={name}\n"

f"created_at={datetime.now()}\n"

f"created_by=adhesive_program_manager\n",

encoding="utf-8")

return self.model_dir

----- regiões / posições -----

def ensure_position_dirs(self,

regiao: str,

posicao: str,

w: str) -> Path:

"""

Garante que .../regioes/<regiao>/posicoes_mecanicas/<posicao>/inspecoes/<w>/
exista e devolve esse Path.

"""

if not self.model_dir:

raise RuntimeError("create_program() ainda não foi chamado.")

target = (self.model_dir / "regioes" / regiao /

"posicoes_mecanicas" / posicao /

"inspecoes" / w)

sub-pastas fixas

for p in ("correct", "incorrect", "referencia"):

```

        (target / p).mkdir(parents=True, exist_ok=True)
    return target

# ----- salvar imagens e JSONs -----
def save_blue_reference(self,
                        regioao: str,
                        posicao: str,
                        w: str,
                        roi_pil: Image.Image,
                        meta: MechanicalPositionInfo | Dict[str,Any]):
    """
    Salva:
    • ROI azul (.../<posicao>_<w>.png)
    • JSON   (.../<posicao>_<w>.json)
    Ambos dentro de .../inspecoes/<w>/
    """
    dest = self.ensure_position_dirs(regiao, posicao, w)

    png_path = dest / f"{posicao}_{w}.png"
    json_path = dest / f"{posicao}_{w}.json"

    # imagem
    roi_pil.save(png_path, format="PNG", compress_level=0)

    # json
    if isinstance(meta, MechanicalPositionInfo):
        meta_to_dump = asdict(meta)
    else:
        meta_to_dump = meta
    json_path.write_text(json.dumps(meta_to_dump, indent=2), "utf-8")

    # referência global (janela azul) — sobrescreve
    ref_dir = dest / "referencia"
    roi_pil.save(ref_dir / "referencia.png", format="PNG", compress_level=0)

# -----
# NOVO: salva imagem completa da região logo que é criada
# -----
def save_region(self, regioao: str, img_pil: "Image.Image"):
    """
    Salva regioes/<regiao>/<regiao>.png (sobre-escreve).
    """
    if not self.model_dir:
        raise RuntimeError("create_program() ainda não foi chamado.")
    target_dir = self.model_dir / "regioes" / regioao
    target_dir.mkdir(parents=True, exist_ok=True)
    img_pil.save(target_dir / f"{regiao}.png",
                 format="PNG", compress_level=0)

# ----- LOG de execução -----
def new_log_session(self, codigo_lote: str) -> Path:

```

```
"""
```

```
Cria pasta logs/<datahora>_<codigo>/ e devolve o Path.
```

```
"""
```

```
if not self.model_dir:
```

```
    raise RuntimeError("create_program() ainda não foi chamado.")
```

```
ts = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```
dest = self.model_dir / "logs" / f"{ts}_{codigo_lote}"
```

```
dest.mkdir(parents=True, exist_ok=True)
```

```
return dest
```

```
def append_json_log(self, session_dir: Path, data: Dict[str,Any]):
```

```
    """
```

```
    Acrescenta entrada a <session>/*.json (cria se não existir).
```

```
    """
```

```
    main_json = session_dir / f"{session_dir.name}.json"
```

```
    entries : List[Dict] = []
```

```
    if main_json.exists():
```

```
        entries = json.loads(main_json.read_text("utf-8"))
```

```
        if not isinstance(entries, list):
```

```
            entries = [entries]
```

```
    entries.append(data)
```

```
    main_json.write_text(json.dumps(entries, indent=2), "utf-8")
```

```
def save_snapshot(self, session_dir: Path,
```

```
    regio_idx: int,
```

```
    img: Image.Image):
```

```
    """
```

```
    Guarda regio_<n>.png dentro da pasta da sessão.
```

```
    """
```

```
    snap_path = session_dir / f"regiao_{regio_idx}.png"
```

```
    img.save(snap_path, format="PNG", compress_level=0)
```

```
# -----
```

```
# 3) Exemplo de uso rápido
```

```
# -----
```

```
if __name__ == "__main__": # test-drive
```

```
    from PIL import Image, ImageDraw
```

```
    base = AdhesiveProgramManager(".")
```

```
    base.create_program("demo_prog", overwrite=True)
```

```
    # cria ROI fictícia só para teste
```

```
    roi = Image.new("RGB", (400, 300), "gray")
```

```
    d = ImageDraw.Draw(roi); d.rectangle([50,50,350,250], outline="blue", width=4)
```

```
    insp_info = MechanicalPositionInfo(
```

```
        janela_azul=(50, 50, 300, 200),
```

```
        similaridade=.97,
```

```
        inspecoes=[
```

```
            InspectionWindowInfo((60,60),(80,60),128,'preto',.9),
```

```

        InspectionWindowInfo((200,80),(60,60),100,'branco',.8)
    ]
)
base.save_blue_reference("regiao_1", "posicao_mecanica_A", "w1",
                        roi_pil=roi, meta=insp_info)

ses = base.new_log_session("L123")
base.append_json_log(ses, {"evento": "start", "t": datetime.now().isoformat()})
base.save_snapshot(ses, 1, roi)
print("Estrutura criada em:", base.model_dir)

```

File: selectable_rect_item.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/teste_clp

```

# selectable_rect_item.py
from PyQt6.QtWidgets import QGraphicsRectItem, QGraphicsItem
from PyQt6.QtGui import QPen, QBrush, QColor, QCursor
from PyQt6.QtCore import Qt, QRectF, QPointF

HANDLE_SIZE = 6      # px
HANDLE_BRUSH = QBrush(QColor(255, 255, 255))
HANDLE_PEN = QPen(Qt.GlobalColor.black, 1)

class SelectableResizableRectItem(QGraphicsRectItem):
    """
    • Seleccionável (clique) – borda fica tracejada
    • Arrastável – ItemIsMovable
    • Redimensionável – 8 alças nos cantos/lados
    """
    def __init__(self, x, y, w, h, pen=None, parent=None,
                 change_callback=None, deletavel: bool = True):
        super().__init__(x, y, w, h, parent)
        self.setFlags(
            QGraphicsItem.GraphicsItemFlag.ItemIsSelectable |
            QGraphicsItem.GraphicsItemFlag.ItemIsMovable |
            QGraphicsItem.GraphicsItemFlag.ItemSendsGeometryChanges
        )
        self.setAcceptHoverEvents(True)
        self.pen_default = pen or QPen(Qt.GlobalColor.green, 2)
        self.pen_selected = QPen(Qt.GlobalColor.blue, 2, Qt.PenStyle.DashLine)
        self.setPen(self.pen_default)
        self.change_callback = change_callback
        # --- nova propriedade comum -----
        # True → pode ser removido com a tecla Delete
        # False → protegido contra deleção pelo atalho
        self.deletavel = deletavel

        # alça atualmente ativa (None se não está redimensionando)
        if callable(self.change_callback):

```

```

        self.change_callback(self)

# -----
# HELPERS DE ALÇAS
# -----
def _handles(self):
    """
    Retorna dicionário {nome: QRectF} em coordenadas da cena
    nomes = 'tl','tm','tr','mr','br','bm','bl','ml' (top-left etc.)
    """

    r = self.rect()
    x, y, w, h = r.x(), r.y(), r.width(), r.height()
    s = HANDLE_SIZE
    cx, cy = x + w/2, y + h/2
    return {
        'tl': QRectF(x-s/2, y-s/2, s, s),
        'tm': QRectF(cx-s/2, y-s/2, s, s),
        'tr': QRectF(x+w-s/2, y-s/2, s, s),
        'mr': QRectF(x+w-s/2, cy-s/2, s, s),
        'br': QRectF(x+w-s/2, y+h-s/2, s, s),
        'bm': QRectF(cx-s/2, y+h-s/2, s, s),
        'bl': QRectF(x-s/2, y+h-s/2, s, s),
        'ml': QRectF(x-s/2, cy-s/2, s, s)
    }

def _handle_at(self, pos: QPointF):
    for name, rect in self._handles().items():
        if rect.contains(pos):
            return name
    return None

# -----
# EVENTOS mouse / hover
# -----
def hoverMoveEvent(self, ev):
    handle = self._handle_at(ev.pos())
    cursors = {
        'tl': Qt.CursorShape.SizeFDiagCursor, 'br': Qt.CursorShape.SizeFDiagCursor,
        'tr': Qt.CursorShape.SizeBDiagCursor, 'bl': Qt.CursorShape.SizeBDiagCursor,
        'ml': Qt.CursorShape.SizeHorCursor, 'mr': Qt.CursorShape.SizeHorCursor,
        'tm': Qt.CursorShape.SizeVerCursor, 'bm': Qt.CursorShape.SizeVerCursor,
    }
    self.setCursor(QCursor(cursors.get(handle, Qt.CursorShape.ArrowCursor)))
    super().hoverMoveEvent(ev)

def mousePressEvent(self, ev):
    self._active_handle = self._handle_at(ev.pos())
    super().mousePressEvent(ev)

def mouseMoveEvent(self, ev):
    if self._active_handle:

```

```

        self._resize(ev.pos())
    else:
        super().mouseMoveEvent(ev)

def mouseReleaseEvent(self, ev):
    self._active_handle = None
    super().mouseReleaseEvent(ev)

def itemChange(self, change, value):
    if (change == QGraphicsItem.GraphicsItemChange.ItemPositionHasChanged and
        callable(self.change_callback)):
        self.change_callback(self)
    return super().itemChange(change, value)

# -----
def _resize(self, pos: QPointF):
    rect = self.rect()
    x, y, w, h = rect.x(), rect.y(), rect.width(), rect.height()
    px, py = pos.x(), pos.y()

    if 'l' in self._active_handle:
        new_x = min(px, x+w-1)
        w += x - new_x
        x = new_x
    if 'r' in self._active_handle:
        w = max(1, px - x)
    if 't' in self._active_handle:
        new_y = min(py, y+h-1)
        h += y - new_y
        y = new_y
    if 'b' in self._active_handle:
        h = max(1, py - y)

    self.prepareGeometryChange()
    self.setRect(QRectF(x, y, w, h))
    # notifica alteração durante o redimensionamento
    if callable(self.change_callback):
        self.change_callback(self)

# -----
# DESENHO
# -----
def paint(self, painter, option, widget=None):
    # borda
    painter.setPen(self.pen_selected if self.isSelected() else self.pen_default)
    super().paint(painter, option, widget)
    # alças
    if self.isSelected():
        painter.setBrush(HANDLE_BRUSH)
        painter.setPen(HANDLE_PEN)
        for rect in self._handles().values():

```

```
painter.drawRect(rect)
```